



Programa Profesional en Inteligencia Artificial y Data
Science

Módulo 5. Procesamiento Real en Entornos Distribuidos

Índice

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- 1.1. Introducción y objetivos
- 1.2. Big data e IA en la nube
- 1.3. Hiperescalares como proveedores de big data e inteligencia artificial
- 1.4. Referencias bibliográficas

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- 2.1. Introducción y objetivos
- 2.2. Ingesta de datos batch
- 2.3. Ingesta de datos en streaming
- 2.4. Referencias bibliográficas

Tema 3. Data science cloud storage

- 3.1. Introducción y objetivos
- 3.2. Sistemas de almacenamiento: big data
- 3.3. Sistemas de almacenamiento: AWS
- 3.4. Data warehouse en la nube
- 3.5. Referencias bibliográficas

Tema 4. Computación distribuida. Computación paralela

- 4.1. Introducción y objetivos
- 4.2. Spark I
- 4.3. Spark II

4.4. Spark III

4.5. MapReduce

4.6. Referencias bibliográficas

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

5.1. Introducción y objetivos

5.2. Machine learning con SageMaker

5.3. MLOps con SageMaker

5.4. Referencias bibliográficas

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

1.1. Introducción y objetivos

Vamos a ver cómo el *cloud computing* y la inteligencia artificial (IA) tienen una **relación simbiótica** que potencia el desarrollo y la implementación de soluciones avanzadas en este ámbito.

Hoy en día, la mayor parte de las empresas y organizaciones, despliegan sus soluciones de *big data* e IA en *cloud*, por lo que estudiaremos las razones basadas en los **beneficios** que el *cloud computing* aporta a la IA, como la escalabilidad, la reducción de costes, el uso de computación distribuida, la facilitación del despliegue de soluciones o el fomento de la colaboración.

Además, el **cloud computing** ha revolucionado la forma en que las empresas gestionan y procesan grandes volúmenes de datos, lo que es fundamental para el ámbito del *big data*. Por ello, estudiaremos el papel fundamental que el *cloud* desempeña en el éxito del *big data* al proporcionar escalabilidad, flexibilidad, reducción de costes, acceso global, agilidad y capacidades de recuperación ante desastres que son esenciales para gestionar y procesar grandes volúmenes de datos de manera eficiente y efectiva.

Revisaremos la oferta y las capacidades de los principales **proveedores cloud** (los llamados hiperescalares: Microsoft Azure, Amazon Web Services —AWS— y Google Cloud Platform —GCP—) en el ámbito de *big data* e IA y describiremos sus principales servicios.

El proveedor por excelencia hoy en día es **AWS**. Por ello, dedicaremos parte de este tema a explorar los servicios que ofrece y cómo está organizado, de forma que tengamos una introducción de las tecnologías *big data* e IA que se estudiarán con profundidad más adelante.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

1.2. Big data e IA en la nube

Introducción

La simbiosis entre el *cloud computing* y la IA está transformando profundamente la manera en que las organizaciones y la sociedad en general interactúan con la tecnología. Esta **integración** no solo está impulsando innovaciones sin precedentes, sino que también está remodelando las bases de los negocios, la investigación y la vida cotidiana. Para entender plenamente esta transformación, es crucial reflexionar sobre cómo estas tecnologías se potencian mutuamente, los beneficios y desafíos que presentan, y su impacto a largo plazo.

La sinergia tecnológica

- ▶ **Acceso a recursos computacionales escalables.** La IA, especialmente el aprendizaje profundo, requiere una gran cantidad de poder computacional para entrenar modelos complejos y procesar grandes volúmenes de datos. El *cloud computing* proporciona acceso bajo demanda a recursos computacionales masivos, eliminando las limitaciones de la infraestructura local.

Ejemplo

Servicios como AWS SageMaker, Google AI Platform y Azure Machine Learning les permiten a los investigadores y a las empresas entrenar modelos de IA en clústeres de GPU y TPU sin necesidad de invertir en *hardware* costoso.

- ▶ **Almacenamiento y gestión de datos.** La IA depende en gran medida de grandes conjuntos de datos para entrenar modelos precisos. El *cloud computing* ofrece soluciones de almacenamiento escalables y económicas, facilitando la gestión y el acceso a datos masivos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Ejemplo

Amazon S3, Google Cloud Storage y Azure Blob Storage permiten almacenar y acceder a petabytes de datos de manera eficiente.

- ▶ **Desarrollo y despliegue de modelos de IA.** Las plataformas de *cloud computing* proporcionan herramientas integradas para el desarrollo, el entrenamiento, el despliegue y la gestión de modelos de IA. Estas herramientas simplifican el ciclo de vida del desarrollo de IA, desde la experimentación hasta la producción.

Ejemplo

Herramientas como TensorFlow, PyTorch y Keras están completamente integradas en plataformas de nube, lo que facilita su uso para desarrolladores y científicos de datos.

Impacto en la innovación y los negocios

- ▶ **Democratización de la IA.** La accesibilidad de los servicios de IA en la nube está democratizando el acceso a tecnologías avanzadas. Las pequeñas y medianas empresas, que anteriormente no podían permitirse la infraestructura necesaria, ahora pueden implementar soluciones de IA.

Ejemplo

Las *startups* pueden aprovechar servicios, como IBM Watson, para desarrollar aplicaciones de análisis de datos y procesamiento del lenguaje natural sin grandes inversiones iniciales.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Agilidad y velocidad de innovación.** La capacidad de escalar rápidamente los recursos permiten a las empresas innovar más rápidamente. Las organizaciones pueden iterar rápidamente en modelos de IA, probar nuevas ideas y desplegar soluciones con agilidad.

Ejemplo

Las empresas de tecnología financiera (*fintech*) utilizan la IA en la nube para desarrollar algoritmos de detección de fraude y optimización de inversiones iterando y mejorando continuamente sus modelos.

- ▶ **Eficiencia operativa.** La automatización impulsada por la IA en la nube está mejorando la eficiencia operativa en múltiples industrias. Desde la optimización de la cadena de suministro hasta la atención al cliente automatizada, la IA en la nube está transformando los procesos empresariales.

Ejemplo

Amazon utiliza IA para optimizar su cadena de suministro y logística, lo que mejora la precisión de las predicciones de demanda y reduce los costos operativos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Desafíos y consideraciones éticas

- ▶ **Seguridad y privacidad de los datos.** El manejo de grandes volúmenes de datos sensibles en la nube plantea desafíos significativos de seguridad y privacidad. Las organizaciones deben asegurar que los datos estén protegidos contra accesos no autorizados y cumplir con regulaciones como RGPD (Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos) y California Consumer Privacy Act of 2018 (CCPA) o Ley de Privacidad del Consumidor de California (California Privacy Protection Agency, 2024).

Reflexión: ¿Cómo pueden las empresas equilibrar la necesidad de grandes volúmenes de datos para entrenar modelos de IA con la obligación de proteger la privacidad de los usuarios?

- ▶ **Transparencia y sesgo algorítmico.** La IA puede inherente incorporar sesgos presentes en los datos de entrenamiento. La falta de transparencia en los algoritmos de IA puede llevar a decisiones injustas o discriminatorias.

Reflexión: ¿Cómo pueden los desarrolladores de IA y los proveedores de la nube garantizar que los modelos sean transparentes y justos, y qué mecanismos deben implementarse para mitigar el sesgo?

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- **Sostenibilidad y consumo energético.** El entrenamiento de modelos de IA en grandes clústeres de computación consume una cantidad significativa de energía, lo que plantea preocupaciones sobre la sostenibilidad ambiental.

Reflexión: ¿Cómo pueden las organizaciones equilibrar el avance de la IA con prácticas sostenibles y responsables minimizando el impacto ambiental de sus operaciones en la nube?

Futuro y conclusión

La **simbiosis** entre el *cloud computing* y la IA continuará evolucionando, impulsando innovaciones en áreas como la automatización, el análisis predictivo, la inteligencia de negocios y más. Esta relación simbiótica promete transformar industrias enteras y resolver problemas complejos a escala global. Sin embargo, también es fundamental abordar los desafíos éticos, de privacidad y sostenibilidad asociados con estas tecnologías.

A medida que avanzamos, es crucial que las organizaciones adopten un enfoque responsable y considerado, y que aseguren que la integración de la nube y la IA beneficie a la sociedad en su conjunto. La **clave del éxito** radica en la colaboración entre los tecnólogos, los reguladores y la sociedad para construir un futuro donde la tecnología no solo impulse la eficiencia y la innovación, sino también respete y proteja los valores humanos fundamentales.

Introducción al cloud computing

La computación en la nube o **cloud computing**, como es más comúnmente conocida, son una serie de servicios relacionados con la computación que se proporcionan a través de Internet. De hecho, la referencia a la nube proviene de la representación más habitual de la red de redes como una nube.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Es una **tecnología** que les permite a los usuarios acceder y utilizar recursos informáticos (como servidores, almacenamiento, bases de datos, redes, *software* y más) a través de Internet, en lugar de depender de una infraestructura física local. La computación en la nube ha transformado la manera en que las organizaciones y los individuos gestionan y utilizan la tecnología, ya que ofrece flexibilidad, escalabilidad y ahorro de costos significativos.

Cloud computing se basa en el uso de **servicios de computación a través de internet**, que proporcionan acceso bajo demanda a una red compartida de recursos informáticos configurables. Estos recursos pueden ser rápidamente provisionados y liberados con un esfuerzo mínimo de gestión o interacción con el proveedor del servicio.

Por tanto, concentra toda la complejidad subyacente de Internet (protocolos y estándares) necesaria para proporcionar servicios, como las páginas web, los buscadores o el correo electrónico. Algo que nos debería ir dando alguna pista de lo que van a significar estos servicios de computación en la nube es la **abstracción de la complejidad** de los sistemas, los protocolos y las tecnologías que existen por debajo y que interactúan para proporcionar toda una serie de servicios, no solo web en este caso, sino más variados, como veremos más adelante.

Cabe considerar también que Internet fue concebido desde el principio como una **red de redes** redundante y que pudiera hacer frente a los fallos. Esta característica permite que los recursos a los que da acceso se pueden escalar de forma masiva, otra de las características de los servicios de *cloud computing*.

Se suele decir que son servicios de computación en la nube, pero lo cierto es que van más allá de la mera computación. El **abanico de servicios** ofrecidos es muy diverso: almacenamiento, bases de datos, redes, *software*, servicios de analítica de datos, inteligencia artificial, servicios para Internet de las cosas (*Internet of Things* o IoT) o para aplicaciones móviles, entre muchos otros. Todos ellos se ofrecen con el

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

objetivo de proporcionar la oportunidad de innovar de forma más rápida, acceder a recursos flexibles y ofrecer economías de escala. Una de las principales ventajas, por ejemplo, es que generalmente el usuario paga solo por los servicios en la nube que usa, lo que ayuda a reducir los costes operacionales, tener una infraestructura más eficiente y escalar el sistema según las necesidades de cada momento. Es decir, es posible empezar con sistemas pequeños, para probar cómo funcionan, y escalarlos después al ritmo que sea necesario.

De forma más técnica, el *cloud computing* se refiere a aplicaciones y servicios que se ejecutan en una **red distribuida usando recursos virtualizados**, y a los que se accede mediante protocolos y estándares comunes de Internet y redes (de los cuales también hacen uso). Como hemos señalado, los recursos que emplean son virtuales y los detalles de los sistemas físicos se abstraen (esconden) del usuario. Por ejemplo, si accedemos a un servicio de computación, nos dará la sensación de que estamos conectados a un ordenador o servidor físico concreto, pero lo que realmente habrá por debajo es una colección de servidores cuya capacidad de cómputo se «agrega»; en este sentido, lo que se nos ofrece una «porción» de dicha capacidad de cómputo global mediante un servidor virtual: no es un equipo físico real, sino una parte de la capacidad de computación disponible, que se nos presenta dando la ilusión de ser un equipo real.

El *cloud computing* se ofrece generalmente de **forma comercial**, a través de un proveedor que nos proporciona servicios que teníamos previamente en nuestros propios ordenadores o servidores locales (lo que se conoce como *on-premises*), con la única diferencia de que estos ahora estarán alojados en sus servidores y accederemos a ellos a través de Internet. Como decíamos, el proveedor ofrece estos servicios bajo demanda, en lo que se conoce como **pay-per-use** (es decir, se paga solo por lo que se usa). Esto es una gran ventaja porque, si se necesita ampliar la infraestructura IT, esto es, si en cierto momento necesitamos más capacidad de cómputo, de almacenamiento o de tráfico de datos por un aumento en la demanda

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

de la aplicación que tenemos entre manos, no es necesario comprar todo el *hardware* para lo que puede ser un momento puntual, con la gran inversión que supone de antemano, tanto económica como de recursos para instalar, configurar y mantener dicha infraestructura.

Otra perspectiva de los servicios en la nube va más allá de albergarlos en servidores de terceros y acceder a ellos a través de Internet. En el mundo tecnológico, existen una serie de acciones que, a la vez que importantes, son también muy comunes para una gran cantidad de negocios y aplicaciones. Toda aplicación o servicio necesita almacenamiento, gestión de la identidad, redes o cierto grado de cómputo. Los servicios en la nube permiten eliminar los trabajos (y, por tanto, los recursos humanos y económicos involucrados) que conllevan este **mantenimiento**, los cuales, a pesar de su vital importancia, no aportan ningún elemento diferenciador en el negocio, sino que se trata sencillamente de elementos necesarios (en inglés, se suelen denominar *utilities*). De esta forma, los recursos pueden enfocarse en los aspectos que aportan valor en lugar de emplear el tiempo en gestionar estas otras tareas que son necesarias para toda aplicación, pero no diferenciadoras.

El *cloud computing* se basa en varios conceptos y tecnologías fundamentales que lo posibilitan. Estos conceptos aseguran que los recursos de computación puedan ser entregados de manera eficiente, escalable y segura a través de la nube. A continuación, se detallan los **conceptos clave** que hacen posible el *cloud computing*:

- ▶ **Virtualización.** Es la tecnología que permite crear versiones virtuales de recursos informáticos físicos, como servidores, almacenamiento y redes. Mediante la virtualización, un único recurso físico puede ser dividido en múltiples recursos virtuales, mejorando así la utilización de los recursos y proporcionando una mayor flexibilidad.
- **Máquinas virtuales (VM).** Emulan el *hardware* físico y permiten ejecutar múltiples sistemas operativos en un solo servidor físico.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- **Hypervisores.** Es un *software* que permite crear y gestionar máquinas virtuales. Por ejemplo, VMware ESXi, Microsoft Hyper-V y KVM.
- ▶ **Contenedores.** Son paquetes de *software* que agrupan una aplicación y sus dependencias en una unidad independiente y consistente. Los contenedores facilitan la portabilidad de aplicaciones entre diferentes entornos.
- **Docker.** Es una plataforma popular de contenedores que les permite a los desarrolladores empaquetar aplicaciones con todas sus dependencias.
- **Kubernetes.** Es un sistema de orquestación de contenedores que automatiza la implementación, escalado y gestión de aplicaciones en contenedores.
- ▶ **Infraestructura como código (IaC).** Es un enfoque para gestionar y aprovisionar recursos informáticos a través de archivos de configuración legibles por humanos, en lugar de procesos manuales. La IaC permite la automatización y repetibilidad en la creación y configuración de infraestructuras.
- **Herramientas IaC:** Terraform, AWS CloudFormation, Ansible y Puppet.
- ▶ **Redes definidas por *software* (SDN).** Separan el plano de control del plano de datos en redes, permitiendo una gestión más flexible y dinámica de los recursos de red. SDN facilita la configuración y gestión de redes de manera programática.
- **Controladores SDN:** OpenFlow y Cisco ACI.
- ▶ **Almacenamiento en la nube.** Permite almacenar datos de manera remota y acceder a ellos a través de Internet. Los proveedores de almacenamiento en la nube ofrecen soluciones escalables y gestionadas para almacenar grandes volúmenes de datos.
- **Servicios de almacenamiento en la nube:** Amazon S3, Google Cloud Storage y Azure Blob Storage.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Modelos de servicio.** El *cloud computing* se organiza en varios modelos de servicio, cada uno proporcionando diferentes niveles de control y gestión:
 - **Infrastructure as a service (IaaS).** Proporciona recursos de infraestructura virtualizados, como servidores, almacenamiento y redes.
 - **Platform as a service (PaaS).** Proporciona una plataforma gestionada que les permite a los desarrolladores crear, probar y desplegar aplicaciones.
 - **Software as a service (SaaS).** Proporciona aplicaciones completas a través de Internet y accesibles a través de navegadores web.
- ▶ **Escalabilidad y elasticidad.** La escalabilidad es la capacidad de aumentar o disminuir los recursos según las necesidades. La elasticidad se refiere a la capacidad de escalar recursos automáticamente en respuesta a las demandas fluctuantes. Estas características aseguran que los servicios en la nube puedan manejar cargas de trabajo variables de manera eficiente.
- ▶ **Seguridad y cumplimiento.** Son fundamentales para la adopción del *cloud computing*. Los proveedores de la nube implementan medidas avanzadas de seguridad para proteger los datos y asegurar la conformidad con regulaciones y normativas.
 - **Seguridad en la nube.** Encriptación de datos, autenticación multifactor, gestión de identidades y accesos (IAM).
 - **Cumplimiento.** Certificaciones y conformidad con estándares como GDPR, HIPAA y SOC 2.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Automatización y orquestación.** Se refieren al uso de herramientas y *scripts* para gestionar la infraestructura y las aplicaciones de manera automática. La orquestación coordina múltiples tareas automatizadas para garantizar que se realicen en el orden correcto.
- **Herramientas de orquestación:** Kubernetes y AWS Step Functions.
- ▶ **Multi-Tenancy** permite que múltiples clientes (o inquilinos) compartan la misma infraestructura física o aplicación, manteniendo la separación y seguridad de los datos de cada cliente. Esto mejora la utilización de recursos y reduce costos.
- ▶ **DevOps y CI/CD.** El DevOps es una práctica que unifica el desarrollo de *software* (Dev) y las operaciones de TI (Ops) para mejorar la eficiencia y la entrega continua. La CI/CD (integración continua/despliegue continuo) automatiza el proceso de desarrollo, prueba y despliegue de aplicaciones.
- **Herramientas DevOps y CI/CD:** Jenkins, GitLab CI, CircleCI y AWS CodePipeline.

El *cloud computing* ha revolucionado la tecnología de la información al ofrecer una **alternativa flexible, escalable y económica** a la infraestructura de TI tradicional. Con su capacidad para proporcionar acceso bajo demanda a una amplia gama de recursos y servicios, la computación en la nube les permite a las organizaciones innovar más rápidamente, adaptarse a las cambiantes condiciones del mercado y optimizar sus operaciones tecnológicas. A medida que la tecnología continúa evolucionando, la adopción de *cloud computing* seguirá creciendo, impulsando nuevas oportunidades y desafíos en el ámbito de la tecnología y los negocios

Ventajas del cloud computing en la aplicación del big data y la IA

Utilizar la computación en la nube para el *big data* y la IA ofrece una serie de ventajas significativas que pueden impulsar la eficiencia, la escalabilidad y la innovación en las organizaciones. Aquí se desarrollan algunos argumentos clave.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Escalabilidad flexible

La computación en la nube proporciona recursos informáticos **escalables y bajo demanda**. Esto significa que las organizaciones pueden aumentar o disminuir rápidamente la capacidad de procesamiento y almacenamiento según sea necesario para manejar conjuntos de datos variables en el *big data* y entrenar modelos de IA sin incurrir en costos significativos de infraestructura fija.

Acceso global

Proporciona acceso a **recursos y aplicaciones** desde cualquier lugar con conexión a Internet, lo que facilita el trabajo remoto y la colaboración global.

Continuidad del negocio y recuperación ante desastres

Ofrece soluciones de **respaldo y recuperación** ante desastres que son más económicas y fáciles de implementar que las soluciones tradicionales.

Actualizaciones automáticas

Los proveedores de servicios en la nube gestionan automáticamente las **actualizaciones y el mantenimiento** del *hardware* y el *software*, lo que asegura que los usuarios siempre tengan acceso a las versiones más recientes.

Reducción de costes de infraestructura

Al utilizar servicios en la nube, las organizaciones pueden evitar la necesidad de invertir en **infraestructura** física costosa para alojar sus aplicaciones de *big data* e IA. En lugar de adquirir y mantener servidores y equipos de almacenamiento, pueden pagar solo por los recursos que consumen, lo que puede resultar en ahorros significativos a largo plazo.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Acceso a recursos especializados

Las plataformas de computación en la nube ofrecen una amplia gama de **servicios y herramientas** especializadas para el procesamiento de *big data* y la implementación de modelos de IA. Esto incluye servicios de almacenamiento de datos escalables, herramientas de análisis y visualización, así como marcos de trabajo y servicios gestionados para entrenamiento y despliegue de modelos de IA.

Velocidad y agilidad

La computación en la nube les permite a las organizaciones implementar y escalar rápidamente soluciones de *big data* e IA, lo que les permite adaptarse ágilmente a las cambiantes necesidades del negocio y a los nuevos desafíos. Esto es especialmente importante en un entorno empresarial dinámico donde la **velocidad** de respuesta es crítica para mantener la competitividad.

Facilita la colaboración y la distribución

Con la computación en la nube, los equipos pueden colaborar de manera más efectiva en proyectos de *big data* e IA, ya que los datos y los recursos computacionales están disponibles de forma remota y se pueden compartir fácilmente entre ubicaciones geográficas dispersas. Esto fomenta la **colaboración** global y la **innovación** en equipo.

Seguridad y conformidad

Los proveedores de servicios en la nube invierten fuertemente en medidas de seguridad física y digital para proteger los datos de los clientes. Además, muchos proveedores ofrecen herramientas y servicios avanzados de **seguridad y conformidad** que ayudan a garantizar que los datos sensibles estén protegidos y que se cumplan los requisitos regulatorios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Coste

Hay dos aspectos sobre el coste económico de los sistemas que se ven favorecidos por el uso del *cloud computing*:

- ▶ **Economías de escala.** Es el más obvio y, aplicado al caso que nos ocupa, se traduce en que el coste de la infraestructura IT disminuye con su tamaño. Esto se debe a que los costes fijos se mantienen y se amortizan entre más unidades, no hay gran diferencia en gestionar cien o mil servidores, y se pueden obtener mejores ofertas al adquirir grandes volúmenes de estos; por tanto, el coste final es menor. Es decir, los grandes proveedores, como Microsoft, Amazon o Google, al adquirir equipos en grandes volúmenes, van a obtener mejores ofertas que una empresa de menor tamaño, que comprará equipos en un volumen mucho menor.
- ▶ **Tipos de gasto.** El segundo aspecto tiene que ver con el tipo de gasto que supone comprar infraestructura directamente frente a contratar servicios de computación en la nube. Cuando se adquieren equipos físicos, estos gastos se consideran «gastos de capital» (*capital expenses* o *CapEx*), es decir, dinero fijo invertido de antemano en elementos, como servidores, que van a reportar, potencialmente, beneficio con el tiempo (*return of investment*). Estos costes, que no son nada despreciables, hay que afrontarlos de antemano, sin saber si realmente se obtendrán esos beneficios esperados de la inversión, lo que supone un riesgo importante que se debe tener en cuenta.

Por otro lado, en los servicios de computación en la nube, al pagar según lo que se consume, se consideran **operacionales** (*operation costs* o *OpEx*), es decir, gastos del día a día para mantener el negocio en funcionamiento (como la luz, la línea telefónica o el alquiler de una oficina). Estos gastos se derivan del beneficio obtenido mes a mes y, en general, se prefieren frente a los gastos de capital, ya que son variables y no hay que afrontarlos de antemano. Dicho de otro modo, no es necesario hacer una importante inversión inicial sin tener siquiera asegurada su rentabilidad.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

La computación en la nube elimina los **gastos de capital** derivados de la compra de servidores, *racks*, redes, climatización, etc., es decir, de toda la infraestructura *hardware* y *software*. Y, con ello, la consiguiente necesidad de contratar a expertos que la pongan en funcionamiento y la gestionen. Además, evita que existan estos costes para cubrir aumentos de demanda puntuales que no se mantienen en el tiempo y que conllevan un derroche posterior de recursos. La compañía solo paga por el servicio que se consume en cada momento.

En resumen, el *cloud computing* proporciona una plataforma altamente escalable, rentable y flexible para el procesamiento de *big data* y la implementación de soluciones de IA. Al aprovechar estos servicios en la nube, las organizaciones pueden acelerar la innovación, mejorar la eficiencia operativa y obtener *insights* valiosos de sus datos de manera más rápida y efectiva.

Tipos de nube y servicios en la nube

Existen diferentes modelos mediante los que clasificar los sistemas de computación en la nube. Principalmente, podemos utilizar dos **tipos de clasificación**: por modelos de despliegue y por modelos de servicios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

La Figura 1 muestra la relación entre estos dos modelos y algunas características de la computación en la nube. Básicamente, en la base de la Figura 1 tenemos el **conjunto de recursos** (ya sean de computación, de almacenamiento o de cualquier otro tipo) que, mediante su virtualización, son compartidos por todos los usuarios de la nube. Dichos recursos se pueden ofrecer como **servicios de infraestructura, plataforma o software**, cuyas diferencias veremos más adelante. En un escalón por encima, se encuentra el **modelo de despliegue** de la nube que proporcionan estos servicios, que puede ser pública, privada e híbrida. Veremos también las diferencias entre estos modelos más adelante.

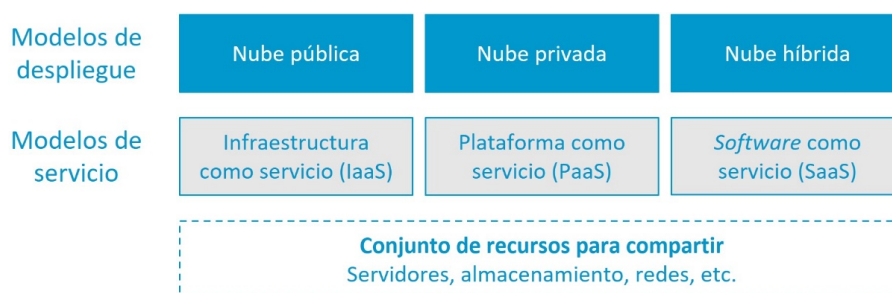


Figura 1. Relación de los modelos de nube. Fuente: elaboración propia.

Modelos de despliegue de nube

El modelo de despliegue se refiere a la **localización** donde se aloja la nube, es decir, dónde se despliega y quién puede usarla. Encontramos tres modalidades, que se desarrollarán a continuación.

Nube pública

Los recursos de computación son proporcionados por un tercero y se comparten entre **múltiples organizaciones**. Es ideal para organizaciones que buscan reducir costes y no tienen requisitos estrictos de seguridad. Las nubes públicas están alojadas y operadas por un proveedor externo de servicios en la nube, que proporciona sus recursos, como servidores y almacenamiento, a través de Internet.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Los servicios en la nube que ofrecen los grandes proveedores (AWS, GCP y Microsoft Azure) son ejemplos de nubes públicas. En estos casos, dichos proveedores se convierten en **propietarios y gestores** de todo el *hardware*, el *software* y la infraestructura auxiliar. Por su parte, los **usuarios** comparten todos estos elementos de almacenamiento, cómputo y red con otras organizaciones, y acceden a sus servicios y gestionan su cuenta a través de una página web. Cualquiera con una tarjeta de crédito puede contratar servicios de nube pública.

Nube privada

Los recursos de computación son utilizados exclusivamente por **una sola organización**, lo que ofrece un mayor control y seguridad. Es ideal para organizaciones con requisitos estrictos de conformidad y privacidad. Se refiere a que los recursos de computación en la red los usa exclusivamente una única empresa u organización.

Una nube privada puede estar físicamente en la propia compañía (*on-premises*) o con un proveedor de servicios *cloud* (*off-premises*). De igual forma, la nube puede estar gestionada por el servicio IT de la propia compañía, o bien esta puede decidir pagar un servicio adicional a un proveedor para que albergue y gestione la nube privada. De una u otra manera, todos los servicios e infraestructuras de una nube privada estarán conectados bajo una **red privada** y no a la red pública de Internet. En caso de estar albergada en otra empresa o gestionada por esta, su servicio de nube privada solo se obtiene mediante contratos específicos, bajo discusión con un representante de la compañía oferente. Por ejemplo, VMware Cloud y OpenStack.

Nube híbrida

Combina nubes públicas y privadas, lo que permite que los datos y las aplicaciones se compartan entre ellas. Ofrece la flexibilidad de la nube pública junto con la seguridad de la nube privada. Las **nubes híbridas** combinan una nube pública con

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

una privada mediante tecnologías que permiten compartir datos y aplicaciones entre ellas. De este modo, proporcionan a las compañías mayor flexibilidad y más opciones de despliegue, y ayudan a optimizar la infraestructura ya existente, así como la seguridad y conformidad con diferentes normas, como el Reglamento general de protección de datos (RGPD).

Una opción habitual es tener la aplicación a desplegar en una nube privada y aprovechar los servicios de nube pública para escalar cuando es necesario. Combina **infraestructura on-premises** (es decir, el centro de datos propio de la compañía o una nube privada) **con una nube pública**. Muchas organizaciones eligen esta opción debido a restricciones del negocio como cumplir normativas de gobierno y privacidad de datos para aprovechar la inversión de su propia infraestructura o para cumplir con los requisitos de baja latencia que necesitan.

La nube híbrida está evolucionando para incluir lo que se denominan **edge workloads o edge computing**. El *edge computing* posibilita llevar la computación más cerca de, por ejemplo, dispositivos IoT, donde los datos residen, de forma que la latencia es más baja porque el tiempo desde que el dispositivo envía los datos recabados hasta que se reciben en el sistema de procesado es menor. Por otro lado, la nube híbrida permite que, cuando la demanda de recursos de cómputo y procesado fluctúan (por ejemplo, incrementos de demanda por época de rebajas, Navidad o de declaración de impuestos...), sea posible escalar la infraestructura *on-premises* mediante el uso de la nube pública de un modo transparente y sin dar acceso a terceros al conjunto completo de datos. De esta forma, las empresas se benefician de la capacidad de escalado, pero mantienen los datos altamente sensibles dentro de sus centros de datos para cumplir con las regulaciones vigentes. Por ejemplo, Microsoft Azure Stack, AWS Outposts.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Multinube

Utiliza múltiples servicios de nube de diferentes proveedores para evitar la dependencia de un solo proveedor y aumentar la resiliencia y la flexibilidad. Por ejemplo, la integración de servicios de AWS, Azure y Google Cloud.

Modelos de servicios en la nube

Existen diferentes formas de ejecutar código en la nube. Por ejemplo, alguien que tiene en su empresa un servidor Windows puede contratar otro servidor Windows en la nube y ejecutar su código en él. Este es el ejemplo más sencillo de migrar un *software* a la nube. Pero existen más alternativas o modelos de servicios en la nube que definen los **diferentes niveles de responsabilidad** de los proveedores de *cloud computing* y de los usuarios. En el fondo, la nube se puede ver como la frontera entre la red, la gestión y las responsabilidades del proveedor de servicios y del cliente. Para hacer esta barrera más concreta y definir dónde se encuentra, los proveedores ofrecen diferentes servicios, que engloban más o menos responsabilidades, lo que da lugar a los diferentes modelos de servicio.

Los **modelos principales** son los siguientes:

- ▶ **Infraestructura como servicio** (*infrastructure as a service* o IaaS).
- ▶ **Plataforma como servicio** (*platform as a service* o PaaS).
- ▶ **Serverless y software como servicio** (*software as a service* o SaaS).

En ocasiones, estas modalidades son denominadas la «pila (*stack*)» o «modelo SPI» de la computación en la nube, ya que unas se construyen sobre otras. Existen modelos adicionales, como *storage as a service* o *identity as a service*, pero vamos a ver con algo más de detalle **function as a service (FaaS)**, también conocido como *serverless computing* o servicios *serverless*, dada su gran disponibilidad actualmente.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Infrastructure as a service (IaaS)

Proporciona **recursos de infraestructura virtualizados**, como servidores, almacenamiento y redes. Los usuarios pueden alquilar estos recursos y administrarlos como si fueran su propia infraestructura. Esta es la categoría más básica de servicios en la nube.

Con IaaS, el usuario «alquila» la infraestructura que necesita a un proveedor de servicios en la nube en una modalidad de **pago por consumo** (*pay-as-you-go* o *pay-per-use*): servidores, máquinas virtuales (VM), almacenamiento, redes, sistemas operativos, balanceadores de carga, *firewalls*, etc. En definitiva, todos los componentes que habría que comprar, instalar, configurar, etc., en un centro de datos propio. Toda esta infraestructura como servicio, sin embargo, no hace nada por sí misma, sino que es el usuario quien tendrá que instalar los programas que desee usar, el código del sistema que se va a desplegar, etc. Por ejemplo, AWS EC2, Microsoft Azure y GCP.

Platform as a service (PaaS)

Ofrece una plataforma y un entorno que permiten a los desarrolladores crear, probar y desplegar aplicaciones **sin gestionar la infraestructura subyacente**. Hace referencia a servicios de *cloud computing* que proporcionan un entorno bajo demanda para desarrollo, pruebas, despliegue y gestión de aplicaciones *software*.

PaaS está diseñado para facilitar a los **desarrolladores**, por ejemplo, la creación de una página web o aplicación móvil, sin preocuparse de preparar o gestionar la infraestructura de servidores, almacenamiento, red y bases de datos necesaria para el desarrollo. Por ejemplo, se puede diseñar una aplicación web sin más acciones que subir el código necesario y la configuración requerida, y el proveedor de PaaS se encargará de todo lo demás. El **usuario**, por su parte, ni tan siquiera sabe en qué *hardware* se está ejecutando su aplicación web; sencillamente habrá contratado un

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

nivel de servicio (SLA o *service level agreement*) que le asegurará cierto grado de disponibilidad y carga máxima, entre otros, pero no conocerá la infraestructura que hay por debajo para proporcionar dicha aplicación web. Por ejemplo, AWS Elastic Beanstalk, Google App Engine y Microsoft Azure App Services.

Computación *serverless* o *function as a service* (FaaS)

Este modelo se solapa, en cierta manera, con PaaS en el sentido de que se centra en construir funcionalidades de aplicaciones sin invertir tiempo continuamente en gestionar los servidores y la infraestructura requeridos. El proveedor de la nube se encarga de la configuración, planificación y provisión de los **recursos necesarios**, así como de la **gestión del servidor**. La principal diferencia se centra en que la arquitectura *serverless* está orientada a eventos, es decir, las funcionalidades se ejecutan solo cuando ocurre un evento o *trigger* específicos. No almacena datos y el entorno se detiene durante el tiempo en el que la función no se ejecuta, para no incurrir en gastos en ese intervalo.

Software as a service (SaaS)

El **software como servicio** es un modelo para proporcionar aplicaciones *software* a través de Internet, bajo demanda y generalmente bajo una suscripción. Con SaaS, los proveedores de *cloud computing* alojan y gestionan dichas aplicaciones y la infraestructura que hay por debajo, y se encargan de todo el mantenimiento, como actualizaciones de *software* y de seguridad. Los usuarios (no los desarrolladores) se conectan a las aplicaciones también a través de Internet, normalmente desde un navegador en su teléfono móvil, tableta u ordenador.

Se trata de aplicaciones cuyo código ya está desarrollado e instalado, y el usuario no puede cambiar nada de las **funcionalidades** proporcionadas, solo configurar ciertas opciones y utilizarlo. Algunos ejemplos de SaaS incluyen aplicaciones de conferencia *online* (Zoom), seguimiento de la jornada laboral, nóminas y contabilidad, Google Drive, Office365 o Salesforce, entre otras muchas.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

En la Figura 2, se muestran las **fronteras** que delimitan qué parte gestiona el proveedor de servicios de *cloud* y cuál el usuario en cada uno de los modelos descritos, respecto a un despliegue tradicional, es decir, a una aplicación que se ejecuta en los servidores alojados y gestionados por el propio usuario.

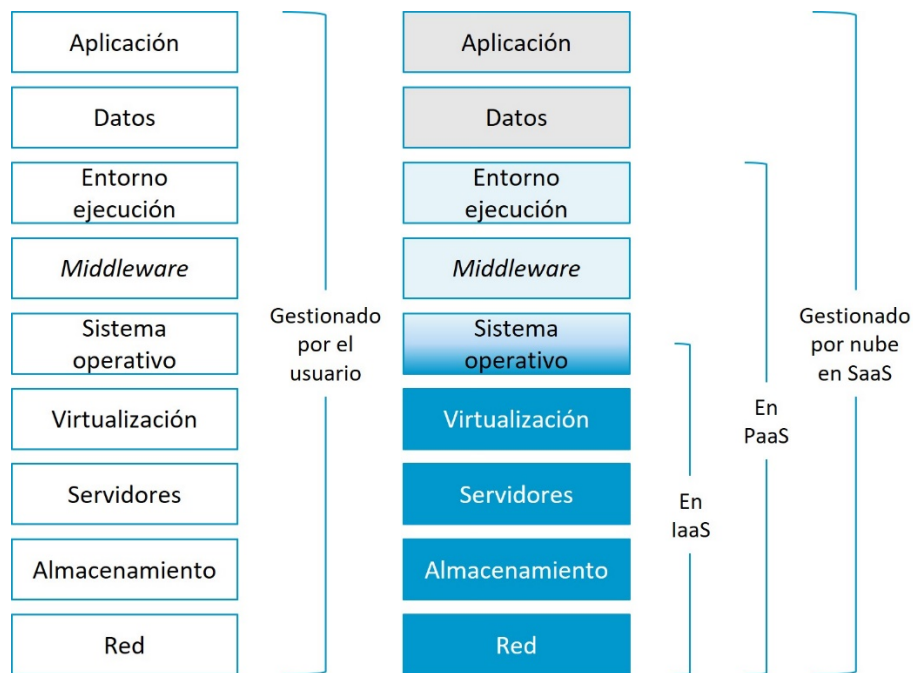


Figura 2. Gestión de recursos en un sistema *on-premises* (izquierda) y en un sistema en la nube, dependiendo del modelo de servicio (derecha). Fuente: elaboración propia.

Desafíos y consideraciones

A pesar de sus numerosos beneficios, *cloud computing* también presenta ciertos desafíos que deben ser considerados:

- **Seguridad y privacidad.** La migración a la nube plantea preocupaciones sobre la seguridad de los datos y la conformidad con regulaciones de privacidad. Es crucial asegurarse de que los proveedores de la nube cumplan con los estándares de seguridad y conformidad necesarios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Dependencia del proveedor.** La dependencia de un solo proveedor de nube puede crear problemas de *lock-in*, donde es difícil cambiar de proveedor debido a la integración profunda de servicios y datos.
- ▶ **Disponibilidad y confiabilidad.** Asegurarse de que los servicios en la nube sean altamente disponibles y confiables es esencial. Las interrupciones en los servicios de nube pueden afectar gravemente las operaciones empresariales.
- ▶ **Costos a largo plazo.** Aunque la nube puede reducir los costos iniciales, es importante gestionar y monitorear el uso para evitar costos inesperados a largo plazo.

Casos de uso de los servicios en la nube de big data e IA

El matrimonio entre *big data*, inteligencia artificial (IA) y *cloud computing* ha transformado radicalmente la forma en que las organizaciones gestionan, analizan y utilizan sus datos. Al aprovechar las capacidades escalables y flexibles de la computación en la nube, junto con las técnicas avanzadas de análisis de datos y aprendizaje automático, las empresas pueden desbloquear una serie de casos de uso innovadores y poderosos. Aquí hay una introducción a algunos de los **casos de uso** más destacados:

- ▶ **Análisis de datos escalable.** La computación en la nube proporciona recursos escalables para procesar grandes volúmenes de datos de manera eficiente. Con *big data* y IA, las organizaciones pueden analizar conjuntos de datos masivos para obtener *insights* valiosos, como patrones de comportamiento del cliente, tendencias de mercado o anomalías operativas.
- ▶ **Personalización y recomendaciones.** Las empresas pueden utilizar IA para analizar el comportamiento del usuario y los datos demográficos, y así ofrecer recomendaciones y experiencias personalizadas a sus clientes. Estos sistemas pueden adaptarse dinámicamente a medida que se recopilan más datos, todo ello aprovechando la capacidad de almacenamiento y procesamiento en la nube.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Predicción y pronóstico.** Con el poder de la computación en la nube y las técnicas avanzadas de IA, las organizaciones pueden desarrollar modelos predictivos precisos para prever resultados futuros en una amplia gama de áreas, desde la demanda del mercado hasta el mantenimiento predictivo de equipos.
- ▶ **Procesamiento de lenguaje natural (PLN).** Los servicios en la nube ofrecen potentes herramientas de PLN que permiten a las empresas analizar y comprender grandes cantidades de texto sin estructurar, como redes sociales, opiniones de clientes o documentos empresariales. Esto abre oportunidades para la extracción de información, la clasificación de documentos y la generación de contenido automatizado.
- ▶ **Automatización inteligente.** Integrando IA en los flujos de trabajo en la nube, las organizaciones pueden automatizar tareas repetitivas y mejorar la eficiencia operativa. Esto incluye desde la automatización de procesos empresariales hasta la optimización de la cadena de suministro y la gestión de recursos humanos.
- ▶ **Seguridad y detección de fraude.** Mediante el análisis continuo de datos en la nube y el uso de algoritmos avanzados de IA, las empresas pueden identificar patrones sospechosos y detectar actividades fraudulentas en tiempo real, lo que ayuda a proteger los activos y la reputación de la organización.

Estos son solo algunos ejemplos de cómo *big data*, IA y *cloud computing* se combinan para ofrecer soluciones innovadoras y transformadoras en una amplia gama de industrias y aplicaciones. La escalabilidad, la flexibilidad y el poder computacional de la nube son fundamentales para desbloquear todo el potencial de estos casos de uso.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

1.3. Hiperescalares como proveedores de big data e inteligencia artificial

Los principales proveedores cloud: los hiperescalares

Los **hiperescalares** son empresas que operan a una escala masiva en el sector de la computación en la nube proporcionando una amplia gama de servicios de infraestructura, plataforma y *software* a clientes de todo el mundo. Estas empresas han construido enormes centros de datos distribuidos globalmente y han desarrollado tecnologías avanzadas para ofrecer servicios de computación en la nube de manera eficiente y rentable. Algunos de los hiperescalares más conocidos incluyen AWS, Microsoft Azure, GCP, IBM Cloud y Alibaba Cloud.

Los hiperescalares ofrecen una serie de **características** distintivas que los diferencian de otros proveedores de servicios en la nube:

- ▶ **Escalabilidad global.** Los hiperescalares operan una red global de centros de datos que les permite ofrecer servicios de computación en la nube a escala global. Esto significa que pueden proporcionar recursos informáticos, almacenamiento y servicios a clientes en cualquier parte del mundo de manera rápida y eficiente.
- ▶ **Amplia gama de servicios.** Los hiperescalares ofrecen una amplia gama de servicios de computación en la nube, que van desde servicios básicos de infraestructura, como servidores virtuales y almacenamiento en la nube, hasta servicios avanzados de inteligencia artificial, *machine learning* (ML), análisis de datos y más.
- ▶ **Economías de escala.** Debido a su enorme tamaño y escala, los hiperescalares pueden lograr economías de escala significativas en la construcción y operación de centros de datos, lo que les permite ofrecer servicios a precios competitivos y mantener márgenes de beneficio sólidos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Innovación tecnológica.** Los hiperescalares están a la vanguardia de la innovación tecnológica en el campo de la computación en la nube. Constantemente desarrollan nuevas tecnologías y servicios para mejorar el rendimiento, la seguridad, la confiabilidad y la eficiencia de sus plataformas en la nube.
- ▶ **Seguridad y confiabilidad.** Los hiperescalares invierten fuertemente en medidas de seguridad física y digital para proteger los datos de sus clientes. Además, utilizan redundancia y replicación para garantizar la disponibilidad y la confiabilidad de sus servicios, incluso en caso de fallas de *hardware* o interrupciones en el servicio.

Podemos concluir que los hiperescalares son actores dominantes en el mercado de la computación en la nube que ofrecen una amplia gama de servicios a una escala masiva y desempeñando un papel fundamental en la transformación digital de las organizaciones en todo el mundo.

A continuación, daremos un repaso sobre los líderes en ofrecer servicios *cloud* para *big data* e IA: Microsoft Azure, Google GCP y Amazon AWS.

Microsoft Azure

Azure es la plataforma en la nube de Microsoft y ofrece una amplia gama de servicios de computación en la nube para ayudar a las organizaciones a construir, implementar y administrar aplicaciones y servicios de manera eficiente y escalable. Con una presencia global que abarca más de sesenta regiones en todo el mundo, Azure proporciona una infraestructura confiable y segura para respaldar las necesidades de las empresas, desde *startups* hasta grandes corporaciones.

Esta plataforma ofrece una variedad de **servicios** que incluyen almacenamiento, bases de datos, redes, análisis de datos, IA, IoT, desarrollo de aplicaciones, seguridad y mucho más. Estos servicios se pueden utilizar para una amplia gama de casos de uso, desde la migración de cargas de trabajo a la nube hasta la

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

construcción de aplicaciones empresariales modernas y la implementación de soluciones de inteligencia artificial.

Azure se distingue por su integración con las **tecnologías y herramientas de Microsoft**, lo que les permite a los clientes aprovechar sus inversiones existentes en productos como Windows Server, SQL Server, Active Directory y Visual Studio. Además, Azure es compatible con una amplia variedad de sistemas operativos, lenguajes de programación, *frameworks* y herramientas de desarrollo, lo que brinda a los clientes flexibilidad y opciones para construir y desplegar aplicaciones en su entorno preferido.

Además, ofrece opciones de **precios flexibles**, que incluyen modelos de pago por uso y descuentos por compromisos a largo plazo, lo que les permite a las organizaciones optimizar sus costos y maximizar el valor de su inversión en la nube.

¿Cuáles son las ventajas competitivas de Azure?

- ▶ **Escalabilidad y flexibilidad.** Escala automáticamente según las necesidades de tu negocio y paga solo por lo que utilizas.
- ▶ **Integración profunda.** Integración con herramientas y tecnologías de Microsoft, como Windows Server, SQL Server, Visual Studio y más.
- ▶ **Amplia gama de servicios.** Más de doscientos servicios para cubrir todas las necesidades de infraestructura y desarrollo.
- ▶ **Seguridad y confiabilidad.** Cumplimiento de los más altos estándares de seguridad y disponibilidad.
- ▶ **Innovación continua.** Constante lanzamiento de nuevos servicios y características para impulsar la innovación en la nube.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Principales servicios de Azure

Entre los principales servicios que la nube de Microsoft ofrece, se pueden destacar de manera categorizada los siguientes:

- ▶ **Cómputo:** Azure Virtual Machines, Azure Kubernetes Service y Azure Functions.
- ▶ **Almacenamiento:** Azure Blob Storage, Azure Files y Azure Disk Storage.
- ▶ **Bases de datos:** Azure SQL Database, Azure Cosmos DB y Azure Synapse Analytics.
- ▶ **Redes:** Azure Virtual Network, Azure Load Balancer y Azure VPN Gateway.
- ▶ **IA y ML:** Azure Machine Learning, Azure Cognitive Services y Azure Databricks.
- ▶ **IoT:** Azure IoT Hub, Azure IoT Central y Azure IoT Edge.
- ▶ **Seguridad:** Azure Active Directory, Azure Security Center y Azure Sentinel.
- ▶ **Desarrollo de aplicaciones:** Azure DevOps, Azure App Service y Azure Functions.

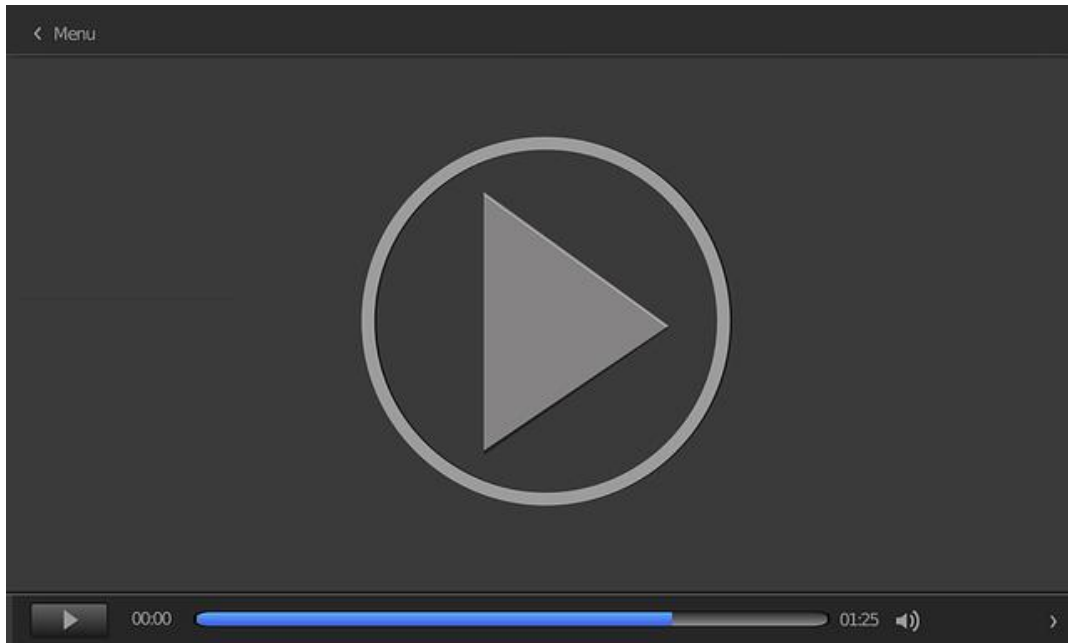
Casos de éxito de Azure

A continuación, se muestra una lista de casos de éxito de Azure en el ámbito del *big data* y de la IA.

- ▶ **Johnson Controls.** Esta empresa global de tecnología para edificios utilizó Azure para desarrollar soluciones de *big data* e IoT para optimizar la eficiencia energética y mejorar la gestión de sus sistemas de calefacción, ventilación y aire acondicionado en edificios comerciales e industriales. Con Azure, Johnson Controls pudo analizar grandes volúmenes de datos en tiempo real y proporcionar recomendaciones inteligentes para mejorar el rendimiento y reducir los costos operativos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre el proyecto Johnson Controls, te recomendamos ver el vídeo *Johnson Controls wins 2023 Microsoft Global ISV Partner of the Year for OpenBlue Building Solutions* (Johnson Controls, 2023).



Johnson Controls wins 2023 Microsoft Global ISV Partner of the Year for OpenBlue Building Solutions.

Accede al vídeo:

<https://www.youtube.com/embed/8qHLAZUIMXk>

- **ASOS.** Como una de las principales empresas de moda *online*, ASOS aprovechó Azure para impulsar su estrategia de personalización y recomendación. Utilizando soluciones de inteligencia artificial y análisis de *big data* en Azure, ASOS pudo analizar el comportamiento de compra de sus clientes y ofrecer recomendaciones de productos altamente relevantes y personalizadas, lo que resultó en un aumento significativo en las conversiones y la retención de clientes.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Microsoft. (2017, octubre 25). *Online retailer uses cloud database to deliver world-class shopping experiences*. <https://customers.microsoft.com/en-us/story/asos-retail-and-consumer-goods-azure>

- **Shell.** La multinacional energética Shell utilizó Azure para optimizar sus operaciones de exploración y producción de petróleo y gas. Con soluciones de *big data* e IA en Azure, Shell pudo analizar datos sísmicos, de perforación y de producción en tiempo real para identificar oportunidades de optimización, mejorar la seguridad y aumentar la eficiencia en sus operaciones en todo el mundo.

Microsoft. 2018, septiembre 24). *Shell invests in safety with Azure, AI, and machine vision to better protect customers and service champions*. <https://customers.microsoft.com/en-us/story/shell-mining-oil-gas-azure-databricks>

- **Maersk.** Como una de las mayores empresas de transporte marítimo del mundo, Maersk utilizó Azure para desarrollar soluciones de *big data* e IA que mejoraron la eficiencia operativa y redujeron los costos en su cadena de suministro global. Con Azure, Maersk pudo analizar grandes volúmenes de datos de seguimiento de contenedores, rutas marítimas, condiciones meteorológicas y más, lo que le permitió optimizar la planificación de rutas, minimizar los tiempos de inactividad de los buques y mejorar la visibilidad de la cadena de suministro.

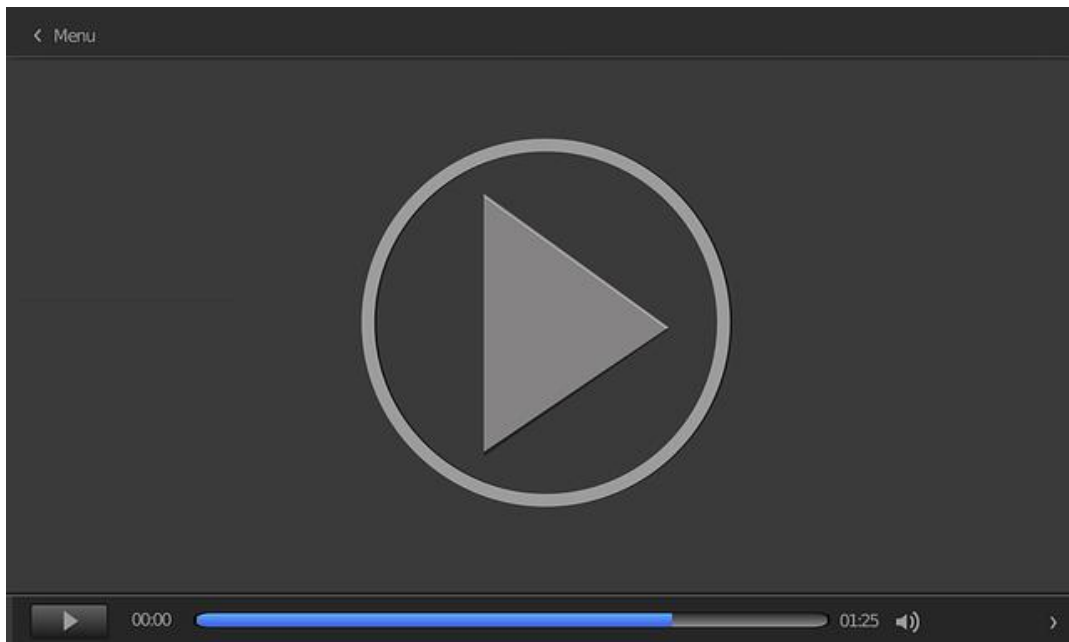
Microsoft. (2017, diciembre 21). *Global transport and logistics company goes digital to transform its operations*. <https://customers.microsoft.com/en-us/story/maersk-travel-transportation-microsoft-services>

- **BMW.** El fabricante de automóviles BMW utilizó Azure para desarrollar soluciones de *big data* e IA que mejoraron la experiencia del cliente y optimizaron la producción.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Con Azure, BMW pudo analizar datos de sensores en tiempo real de sus vehículos conectados para identificar patrones de uso, predecir fallos mecánicos y ofrecer servicios de mantenimiento predictivo a los propietarios de vehículos. Además, Azure ayudó a BMW a optimizar sus procesos de fabricación mediante el análisis de datos de sensores en las líneas de producción para identificar áreas de mejora y aumentar la eficiencia.

Si quieres saber más sobre cómo BMW utilizó Azure, te recomendamos ver el vídeo *Microsoft Intelligent Manufacturing Award 2021 – BMW* (Roland Berger, 2022).



Microsoft Intelligent Manufacturing Award 2021 – BMW.

Accede al vídeo:

<https://www.youtube.com/embed/bACcdP7kz5A>

Estos son solo algunos ejemplos de cómo las empresas líderes en diversos sectores están utilizando Azure para impulsar la innovación, mejorar la eficiencia operativa y ofrecer experiencias personalizadas a los clientes mediante el aprovechamiento de soluciones de *big data* e IA.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Servicios *big data* e IA de Azure

Azure Big Data, la plataforma en la nube de Microsoft, ofrece una amplia gama de **servicios especializados** en *big data* e IA, que permiten a las organizaciones construir, desplegar y escalar soluciones avanzadas de análisis de datos y aprendizaje automático.

A continuación, se expone una descripción general de algunos de los principales servicios que Azure ofrece en estas áreas.

- **Big data.** En la Figura 3 se muestra la amplia gama de servicios de *big data* e IA ofrecidos por esta plataforma.

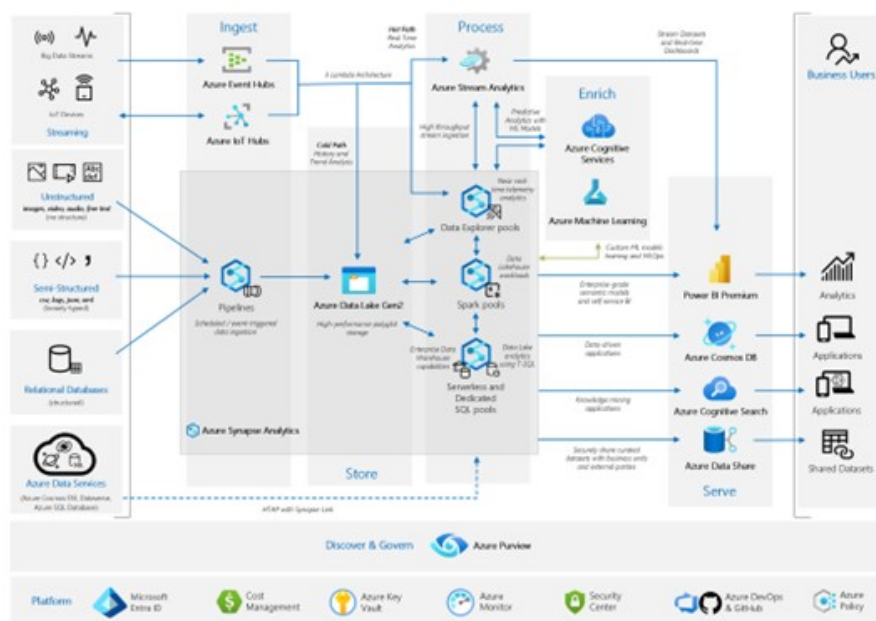


Figura 3. servicios de *big data* e IA. Fuente: Microsoft Learn, 2023.

Profundizando en los principales **servicios** de *big data* que aparecen en la Figura 3, tenemos los siguientes:

- **Azure Event Hubs.** Es un servicio de ingestión de datos en tiempo real y escalable que puede procesar millones de eventos por segundo. Permite a las organizaciones

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

recopilar y procesar datos de dispositivos IoT, aplicaciones móviles, registros de servidores, redes sociales y otros sistemas en tiempo real. Event Hubs integra fácilmente con otros servicios de Azure, como Azure Stream Analytics y Azure Functions, para realizar análisis en tiempo real y activar acciones automatizadas.

- ▶ **Azure IoT Hub.** Es un servicio gestionado de IoT que les permite a las organizaciones conectar, supervisar y administrar dispositivos IoT a escala. Proporciona capacidades de gestión de datos para dispositivos IoT, así como funciones de seguridad, control de acceso y administración de dispositivos. IoT Hub permite la integración con otros servicios de Azure, como Azure Stream Analytics y Azure Functions, para procesar y analizar datos de dispositivos IoT en tiempo real.
- ▶ **Azure Data Factory.** Es un servicio de orquestación de datos en la nube que les permite a las organizaciones crear, programar y administrar flujos de datos entre diversos orígenes y destinos. Permite la ingestión de datos en lotes y en tiempo real, así como la transformación y el enriquecimiento de datos durante el proceso de ingestión. Data Factory integra con una amplia gama de servicios de Azure y fuentes de datos externas.
- ▶ **Azure Data Lake Storage.** Es un servicio de almacenamiento de datos escalable y seguro que les permite a las organizaciones almacenar grandes volúmenes de datos estructurados y no estructurados. Proporciona capacidades de ingestión de datos para datos en lotes y en tiempo real, así como funciones avanzadas de análisis y procesamiento de datos utilizando herramientas como Azure Data Lake Analytics y Azure Databricks.
- ▶ **Azure Blob Storage.** Es un servicio de almacenamiento en la nube escalable y económico diseñado para almacenar grandes volúmenes de datos no estructurados, como archivos de registro, imágenes, vídeos y archivos de audio. Se puede utilizar como destino de datos para la ingestión de datos en lotes, así como para la ingestión de datos en tiempo real utilizando Azure Event Hubs, Azure IoT Hub y otros servicios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Azure Data Lake Storage.** Es un servicio de almacenamiento escalable y seguro diseñado para almacenar grandes volúmenes de datos sin estructurar. Permite el análisis de datos en tiempo real y el procesamiento distribuido de conjuntos de datos masivos.
- ▶ **Azure HDInsight.** Es un servicio completamente gestionado de Apache Hadoop, Spark, HBase y otros *frameworks* de *big data*. Proporciona capacidades de procesamiento distribuido y análisis de datos en clústeres escalables y seguros.
- ▶ **Azure Databricks.** Es una plataforma de análisis de *big data* basada en Apache Spark que simplifica el desarrollo, la implementación y la administración de *pipelines* de datos y modelos de ML.
- ▶ **Azure Synapse Analytics.** Es un servicio de análisis de datos que combina el almacenamiento de datos empresariales con la integración de *big data* y el análisis de datos en tiempo real. Permite realizar consultas SQL en datos estructurados y no estructurados a gran escala.

En el área de IA, la Figura 4 proporciona una visión de la oferta de esta plataforma.



Figura 4. Catálogo de Azure. Fuente: Rautenstrauch, 2023.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Entre su amplia oferta de IA, se destacan los siguientes **servicios**:

- ▶ **Azure Machine Learning.** Es un servicio de ML que les permite a los científicos de datos y desarrolladores construir, entrenar y desplegar modelos de ML en la nube y en el borde de la red.
- ▶ **Azure Cognitive Services.** Es una colección de servicios de IA preconstruidos que ofrecen capacidades de visión artificial, procesamiento de lenguaje natural, reconocimiento de voz, entre otros. Estos servicios permiten agregar inteligencia a las aplicaciones sin necesidad de experiencia en ML.
- ▶ **Azure Bot Service.** Es un servicio que les permite a los desarrolladores construir, conectar y desplegar chatbots inteligentes que interactúan de manera natural con los usuarios a través de múltiples canales.
- ▶ **Azure Computer Vision.** Es un servicio que proporciona capacidades avanzadas de reconocimiento visual, incluyendo etiquetado de imágenes, detección de objetos, reconocimiento facial y más.
- ▶ **Azure Speech Services.** Es un conjunto de servicios de IA que permiten la conversión de voz a texto, la traducción de voz en tiempo real y la generación de voz natural.

Estos son solo algunos ejemplos de la amplia gama de servicios de *big data* e IA que Azure ofrece. Con estas herramientas, las organizaciones pueden aprovechar la nube de Microsoft para obtener *insights* valiosos de sus datos y construir soluciones innovadoras impulsadas por la inteligencia artificial.

Google Cloud Platform (GCP)

El lema de la **plataforma GCP** es «transformando negocios con la nube», con este lema quieren transmitir el carácter innovador de esta nube. Pero ¿qué es GCP? GCP

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

es la plataforma en la nube de Google, que ofrece una amplia gama de servicios de infraestructura, análisis, ML, almacenamiento, bases de datos, desarrollo de aplicaciones, etc. Google Cloud Platform ofrece una solución completa y escalable para las necesidades de infraestructura y desarrollo de cualquier organización.

Con su enfoque en la innovación, escalabilidad, seguridad y precio competitivo, GCP es la **opción preferida** para empresas de todos los tamaños y sectores que buscan transformar sus negocios con la nube.

¿Cuáles son las principales ventajas competitivas de GCP?

Esta plataforma proporciona herramientas y servicios para ayudar a las organizaciones a construir, escalar y administrar aplicaciones y servicios de manera eficiente en la nube. Entre sus **características** fundamentales, se pueden enumerar las siguientes:

- ▶ **Innovación y tecnología avanzada.** Google es conocido por su enfoque en la innovación y la tecnología de vanguardia, lo que se refleja en GCP con servicios líderes en la industria.
- ▶ **Escalabilidad y rendimiento.** GCP ofrece una infraestructura altamente escalable y un rendimiento confiable, lo que les permite a las organizaciones escalar según la demanda y mantener una alta disponibilidad.
- ▶ **Integración con herramientas de Google,** como TensorFlow para ML, Kubernetes para contenedores y BigQuery para análisis de datos.
- ▶ **Seguridad y cumplimiento.** Compromiso con la seguridad y el cumplimiento de los más altos estándares de la industria para proteger los datos y la privacidad del cliente.
- ▶ **Precio competitivo.** Precios competitivos y opciones flexibles de pago por uso que ayudan a optimizar los costos y maximizar el valor.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Principales servicios de GCP

Entre los principales servicios que la nube de Google ofrece, se pueden destacar de manera categorizada los siguientes:

- ▶ **Cómputo:** Compute Engine, Kubernetes Engine y App Engine.
- ▶ **Almacenamiento:** Cloud Storage, Cloud SQL y Cloud Bigtable.
- ▶ **Bases de datos:** Cloud Firestore, Cloud Spanner y BigQuery.
- ▶ **Redes:** Virtual Private Cloud (VPC), Cloud Load Balancing y Cloud CDN.
- ▶ **IA y ML:** AI Platform, TensorFlow y AutoML.
- ▶ **IoT:** Cloud IoT Core, Edge TPU y IoT Edge.
- ▶ **Seguridad:** Identity and Access Management (IAM), Cloud Identity y Security Command Center.
- ▶ **Desarrollo de aplicaciones:** Cloud Functions, Cloud Run y Firebase.

Casos de éxito de GCP

A continuación, se muestra una lista de casos de éxito de GCP en *big data* e IA.

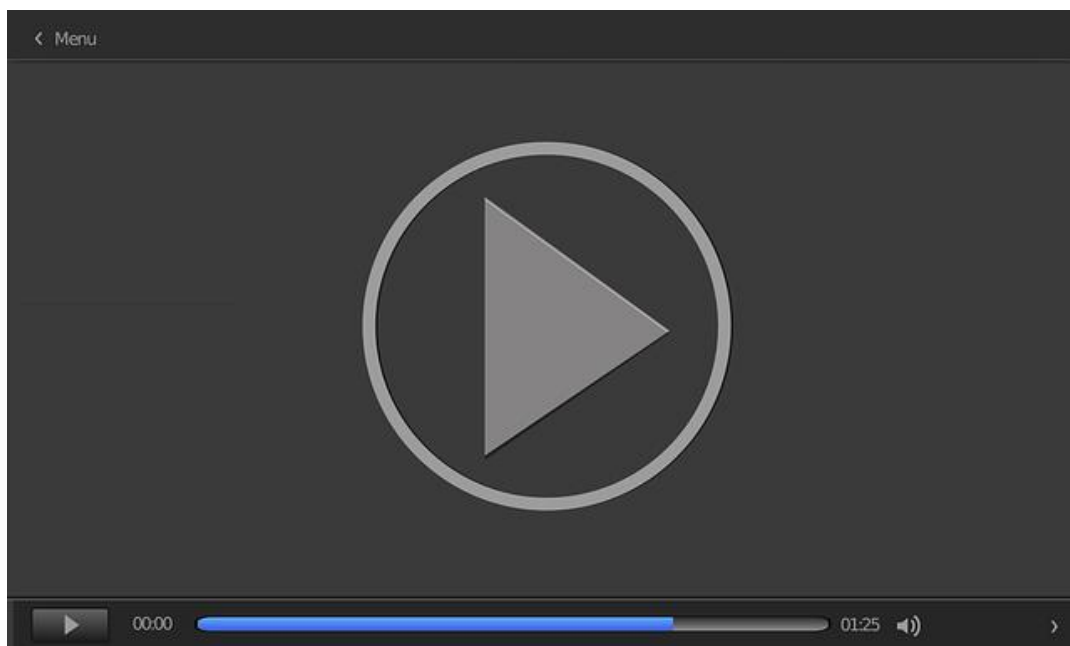
- ▶ **Snap Inc.** La empresa detrás de Snapchat utiliza GCP para manejar grandes volúmenes de datos generados por sus usuarios. GCP proporciona a Snap herramientas escalables de *big data* para procesar y analizar datos en tiempo real, lo que permite a la empresa ofrecer características innovadoras y personalizadas a sus usuarios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Google Cloud. (2022, mayo 31). *Snap Inc. adopts Google Cloud TPU for deep learning recommendation models*. <https://cloud.google.com/blog/products/ai-machine-learning/snap-inc-uses-google-cloud-tpu-for-deep-learning-recommendation-models>

- **Spotify.** La popular plataforma de *streaming* de música aprovecha GCP para almacenar y procesar grandes cantidades de datos de audio. GCP proporciona a Spotify la capacidad de analizar los gustos musicales de sus usuarios y ofrecer recomendaciones personalizadas, lo que mejora la experiencia del usuario y aumenta la retención.

Si quieres saber más sobre cómo Spotify utiliza la plataforma GCP, te recomendamos ver el vídeo *Spotify uses Google Cloud to unlock infinite capacity and faster innovation* (Google Cloud, 2020).



Spotify uses Google Cloud to unlock infinite capacity and faster innovation.

Accede al vídeo:

https://www.youtube.com/embed/55xgR_o4PGs

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

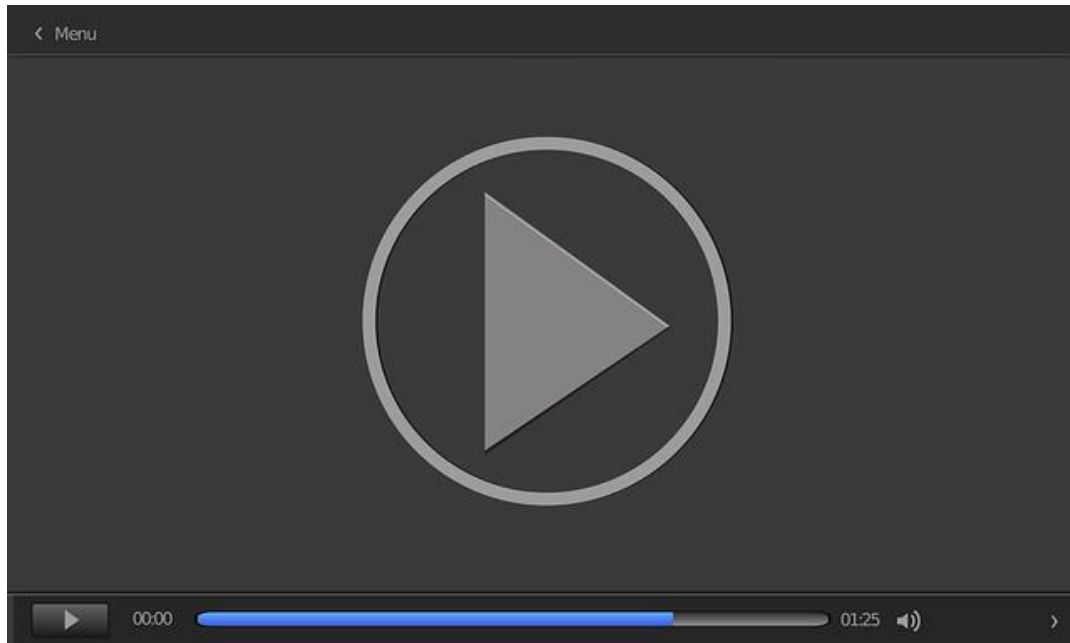
- ▶ **Niantic.** La empresa detrás del juego de realidad aumentada Pokémon GO utiliza GCP para gestionar la gran cantidad de datos generados por millones de jugadores en todo el mundo. GCP proporciona a Niantic la escalabilidad necesaria para manejar picos de tráfico repentinos y analizar datos en tiempo real para mejorar la jugabilidad y la experiencia del usuario.

Google Cloud. (2021, octubre 27). *How Pokémon GO scales to millions of requests?*
<https://cloud.google.com/blog/topics/developers-practitioners/how-pok%C3%A9mon-go-scales-millions-requests>

- ▶ **PayPal.** La plataforma de pagos en línea PayPal utiliza GCP para mejorar la seguridad y prevenir el fraude. GCP proporciona a PayPal herramientas avanzadas de análisis de *big data* e IA para detectar patrones de fraude y proteger las transacciones de los usuarios en tiempo real.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo PayPal utiliza la plataforma GCP, te recomendamos ver el vídeo *PayPal partners with Google Cloud* (Google Cloud, 2018).



PayPal partners with Google Cloud.

Accede al vídeo:

<https://www.youtube.com/embed/9jJ6xLOSS3c>

- **X (antes Twitter).** La red social X utiliza GCP para almacenar y analizar grandes volúmenes de datos generados por millones de tweets diarios. GCP proporciona a X la capacidad de procesar y analizar datos en tiempo real para identificar tendencias, mejorar la segmentación de anuncios y proporcionar recomendaciones personalizadas a los usuarios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Google Cloud. (2022, mayo 17). *Twitter takes data activation to new heights with Google Cloud*. <https://cloud.google.com/blog/products/data-analytics/how-twitter-modernized-its-data-processing-with-google-cloud>

- **HSBC.** El banco multinacional HSBC utiliza GCP para desarrollar soluciones de *big data* e IA que mejoran la experiencia del cliente y optimizan las operaciones bancarias. GCP proporciona a HSBC la capacidad de analizar grandes volúmenes de datos de transacciones financieras para detectar fraudes, identificar oportunidades de *upselling* y ofrecer servicios personalizados a sus clientes.

Google Cloud. (s. f.). *HSBC: Using Google Cloud to improve customer service at call centers*. <https://cloud.google.com/customers/hsbc>

Estos son solo algunos ejemplos de cómo empresas líderes están utilizando GCP para impulsar la innovación, mejorar la experiencia del cliente y optimizar sus operaciones mediante el aprovechamiento de soluciones de *big data* e IA.

Servicios de GCP de *big data* e IA

GCP ofrece una amplia gama de servicios y productos en el ámbito de *big data* e IA que permiten a las organizaciones gestionar, procesar y analizar grandes volúmenes de datos de manera eficiente, así como construir y desplegar modelos avanzados de IA.

A continuación, se expone una descripción general de algunos de los principales **servicios** que GCP ofrece en estas áreas:

- **Big data.** En la Figura 5 se muestra un resumen de los servicios de *big data* e IA ofrecidos por esta plataforma.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

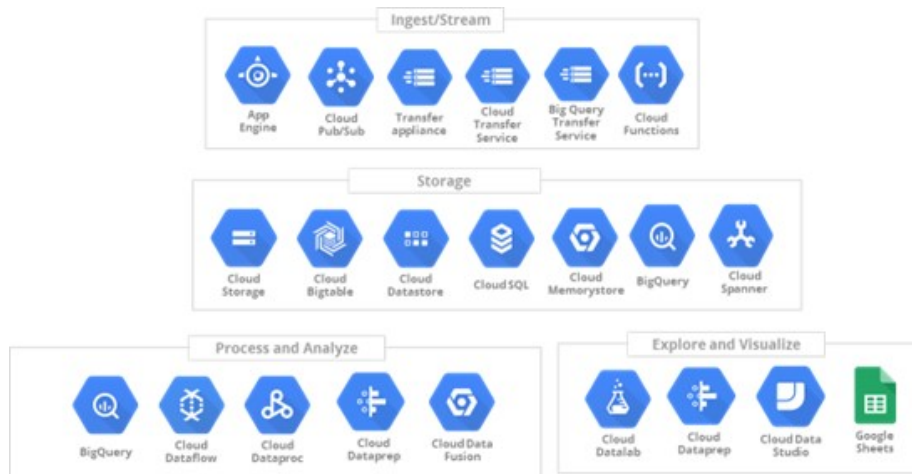


Figura 5. Servicios de *big data* e IA de GCP. Fuente: Srinivasan, 2019.

Si ahondamos en los principales **servicios** de *big data* que aparecen en la Figura 5, tendremos los siguientes:

- **BigQuery.** Es un almacén de datos completamente gestionado y altamente escalable que permite analizar conjuntos de datos grandes y complejos de manera rápida y eficiente mediante SQL estándar. Ofrece capacidades de procesamiento en paralelo y permite ejecutar consultas en tiempo real sobre petabytes de datos. BigQuery también ofrece integración con otras herramientas de Google Cloud, como Dataflow y TensorFlow.
- **Dataflow.** Es un servicio de procesamiento de datos en tiempo real y por lotes que permite crear, ejecutar y escalar *pipelines* de datos de manera sencilla. Utiliza un modelo de programación basado en Apache Beam para procesar datos de manera paralela y distribuida en entornos de nube y local. Dataflow es compatible con una amplia variedad de fuentes y destinos de datos, incluidos BigQuery, Cloud Storage y Pub/Sub.
- **Dataproc.** Es un servicio de procesamiento de datos gestionado que permite ejecutar clústeres de Hadoop y Spark en la nube de manera rápida y sencilla. Proporciona capacidades de escalado automático y administración de clústeres, lo

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

que les permite a las organizaciones procesar grandes volúmenes de datos de manera eficiente y rentable. Dataproc también ofrece integración con otros servicios de GCP, como BigQuery y Dataflow.

- **Pub/Sub.** Es un servicio de mensajería en tiempo real que les permite a las organizaciones enviar y recibir mensajes entre aplicaciones y servicios de manera segura y escalable. Utiliza un modelo de publicación/suscripción para transmitir datos de manera confiable y duradera. Pub/Sub es ideal para casos de uso de ingesta de datos en tiempo real, como el procesamiento de registros de eventos, telemetría de dispositivos IoT y más.

En el área de la IA, la plataforma GCP ofrece la plataforma **Google Cloud Vertex AI**, que es un servicio completamente gestionado que les permite a las organizaciones construir, entrenar y desplegar modelos de aprendizaje automático de manera rápida y sencilla en la nube de Google.

Vertex AI proporciona una **plataforma unificada** que integra una variedad de herramientas y servicios de Google Cloud para facilitar el desarrollo y la implementación de modelos de inteligencia artificial a escala.

La Figura 6 proporciona una visión de la oferta de la plataforma Google Vertex AI.



Figura 6. Catálogo de Google Vertex AI. Fuente: Inderpreet Singh, 2024.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Para el área de **unificación de servicios**, tenemos los siguiente:

- ▶ **Integración de servicios de aprendizaje automático.** Vertex AI integra varios servicios de aprendizaje automático de Google Cloud, incluidos AutoML, TensorFlow, AI Platform y más, en una plataforma unificada. Esto les permite a los usuarios acceder a una amplia gama de herramientas y capacidades para construir y desplegar modelos de inteligencia artificial de manera eficiente.

Dentro del **desarrollo de modelos** podemos destacar:

- ▶ **AutoML.** Vertex AI ofrece capacidades de AutoML para permitir a los usuarios entrenar modelos de aprendizaje automático personalizados sin necesidad de experiencia en ciencia de datos o aprendizaje automático. Los usuarios pueden utilizar AutoML Vision, AutoML Natural Language, AutoML Tables y otros servicios para construir modelos de clasificación, detección de objetos, análisis de sentimientos y más.
- ▶ **TensorFlow.** Vertex AI proporciona soporte para TensorFlow, la biblioteca de aprendizaje automático de código abierto de Google. Los usuarios pueden aprovechar TensorFlow para construir modelos de aprendizaje profundo personalizados utilizando una variedad de herramientas y API, incluyendo TensorFlow Extended (TFX) para el desarrollo de *pipelines* de datos y TensorFlow Serving para el despliegue de modelos en producción.

En cuanto a los servicios dedicados al **entrenamiento y evaluación de modelos**:

- ▶ **AI Platform Training.** Vertex AI ofrece capacidades de entrenamiento de modelos escalables y distribuidas a través de AI Platform Training. Los usuarios pueden entrenar modelos utilizando recursos de procesamiento distribuido y GPU en la nube de Google, lo que permite acelerar el proceso de entrenamiento y mejorar el rendimiento de los modelos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Evaluación de modelos.** Vertex AI proporciona herramientas para evaluar y validar modelos de aprendizaje automático, incluidas las métricas de evaluación, la visualización de resultados y la comparación de modelos. Esto les permite a los usuarios medir el rendimiento de los modelos en datos de prueba y ajustar los hiperparámetros para mejorar la precisión y la generalización del modelo.

Respecto a la **implementación y despliegue de modelos**, destacan los siguientes:

- ▶ **AI Platform Prediction.** Vertex AI ofrece capacidades de despliegue de modelos escalables y seguras a través de AI Platform Prediction. Los usuarios pueden implementar modelos entrenados en la nube de Google y exponerlos como servicios web RESTful para realizar inferencias en tiempo real. AI Platform Prediction ofrece integración con otros servicios de Google Cloud, como Cloud Functions y Cloud Run, para permitir la automatización y la escalabilidad de las inferencias.
- ▶ **Model Monitoring.** Vertex AI proporciona herramientas para monitorear el rendimiento y la salud de los modelos desplegados en producción. Los usuarios pueden supervisar métricas de rendimiento, detectar anomalías y recibir alertas en tiempo real para garantizar la fiabilidad y la disponibilidad de los modelos en producción.

Finalmente, para la **automatización y optimización** se ofrece lo siguiente:

- ▶ **Vertex AI MLOps.** Vertex AI ofrece capacidades de MLOps para permitir la automatización y la optimización del ciclo de vida de los modelos de aprendizaje automático. Esto incluye la automatización de flujos de trabajo de entrenamiento, evaluación y despliegue de modelos, así como la gestión de versiones y el control de calidad de los modelos en producción.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- **Dialogflow.** Dialogflow es un servicio de Google Cloud que les permite a los desarrolladores crear interfaces de conversación para aplicaciones y servicios utilizando inteligencia artificial y procesamiento de lenguaje natural. Proporciona capacidades avanzadas de reconocimiento de voz, comprensión del lenguaje natural y generación de respuestas para crear experiencias de usuario conversacionales y personalizadas. Dialogflow se utiliza comúnmente para construir chatbots, asistentes virtuales y aplicaciones de voz.

En resumen, **Google Cloud Vertex AI** es un servicio integral que les permite a las organizaciones construir, entrenar y desplegar modelos de aprendizaje automático de manera eficiente y escalable en la nube de Google. Con su enfoque en la unificación de servicios, desarrollo de modelos, entrenamiento y evaluación, implementación y despliegue, así como automatización y optimización, Vertex AI facilita el proceso de desarrollo y operación de modelos de inteligencia artificial en producción. Estos son solo algunos ejemplos de los principales servicios y productos de Google Cloud Platform en *big data* e IA.

GCP ofrece una amplia variedad de herramientas y servicios que permiten a las organizaciones aprovechar la potencia de la nube para gestionar y analizar datos a gran escala, así como construir y desplegar aplicaciones y servicios inteligentes impulsados por la IA.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Amazon Web Services (AWS)

AWS es la **plataforma líder** en servicios en la nube, ofrecida por Amazon.com. Proporciona una amplia gama de servicios de infraestructura informática, almacenamiento, bases de datos, análisis, inteligencia artificial, IoT, seguridad, desarrollo de aplicaciones y más. Ayuda a las organizaciones a escalar y crecer, eliminando la necesidad de invertir en infraestructura física costosa y ofreciendo flexibilidad, agilidad y escalabilidad.

AWS es, actualmente, la plataforma líder de servicios en la nube, la cual presta sus servicios a grandes empresas (Expedia, Shell), *startups* (Airbnb, Lyft), sector público (Coursera, FDA) y particulares. En la Figura 7 se puede observar la posición de cada **proveedor de servicios cloud**, según el cuadrante del año 2023 de la consultora Gartner; en él se aprecia cómo AWS tiene un papel dominante, seguido de cerca por Microsoft y Google Cloud Services.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Figure 1: Magic Quadrant for Strategic Cloud Platform Services



Figura 7. Cuadrante mágico de Gartner para servicios cloud de infraestructura (IaaS) y plataforma (PaaS), a fecha de agosto de 2020. Fuente: Amazon Web Service, s. f.-a.

AWS ofrece un gran abanico de servicios en la nube mediante una infraestructura global que proporciona acceso desde casi cualquier parte del mundo, alta disponibilidad y seguridad, y un esquema de pago por uso (*pay-as-you-go*). Entre los **servicios** que ofrece, podemos encontrar los comunes a todas las plataformas de *cloud computing*, como son el almacenamiento, el procesamiento, las redes y la seguridad; pero también otros más específicos, relacionados con sectores concretos, como la analítica de datos, el DevOps, las bases de datos, la inteligencia artificial, el IoT, el desarrollo móvil, entre muchos otros.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Además, al igual que otras plataformas de *cloud computing*, permite desplegar **modelos** tanto completamente en la nube como de forma híbrida, es decir, ofreciendo algunos servicios en sus propios servidores y otros en la nube, y dando posibilidad de migrar cuando el usuario decida hacerlo.

Además de los servicios que ofrece la propia plataforma AWS, cuenta con el denominado AWS Marketplace, un **catálogo digital** con numerosos recursos adicionales proporcionados por empresas independientes de Amazon, pero basadas en los servicios en la nube de AWS, como imágenes de máquinas virtuales, herramientas de BI, bases de datos, etc.

¿Cuáles son las ventajas competitivas de AWS?

- ▶ **Escalabilidad y flexibilidad.** Escala a medida que tu negocio crece y paga solo por lo que usas.
- ▶ **Amplia gama de servicios.** Más de doscientos servicios y productos para cubrir todas las necesidades de infraestructura y desarrollo.
- ▶ **Seguridad y confiabilidad.** Cumplimiento de los más altos estándares de seguridad y disponibilidad.
- ▶ **Facilidad de uso.** Interfaz intuitiva y herramientas de gestión para facilitar la administración de tus recursos.
- ▶ **Innovación continua.** Líder en innovación, con lanzamientos regulares de nuevos servicios y características.

Principales servicios de AWS

Entre los principales servicios que la nube de Amazon ofrece, se pueden destacar de manera categorizada los siguientes:

- ▶ **Cómputo:** Amazon EC2, Amazon ECS y AWS Lambda.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **Almacenamiento:** Amazon S3, Amazon EBS y Amazon Glacier.
- ▶ **Bases de datos:** Amazon RDS, Amazon DynamoDB y Amazon Redshift.
- ▶ **Redes:** Amazon VPC, Amazon Route 53 y AWS Direct Connect.
- ▶ **IA y ML:** Amazon SageMaker, AWS Deep Learning AMLs y Amazon Rekognition.
- ▶ **IoT:** AWS IoT Core, AWS IoT Greengrass y Amazon FreeRTOS.
- ▶ **Seguridad:** AWS IAM, AWS Key Management Service (KMS) y AWS WAF.
- ▶ **Desarrollo de aplicaciones:** AWS SDKs, AWS Amplify y AWS CloudFormation.

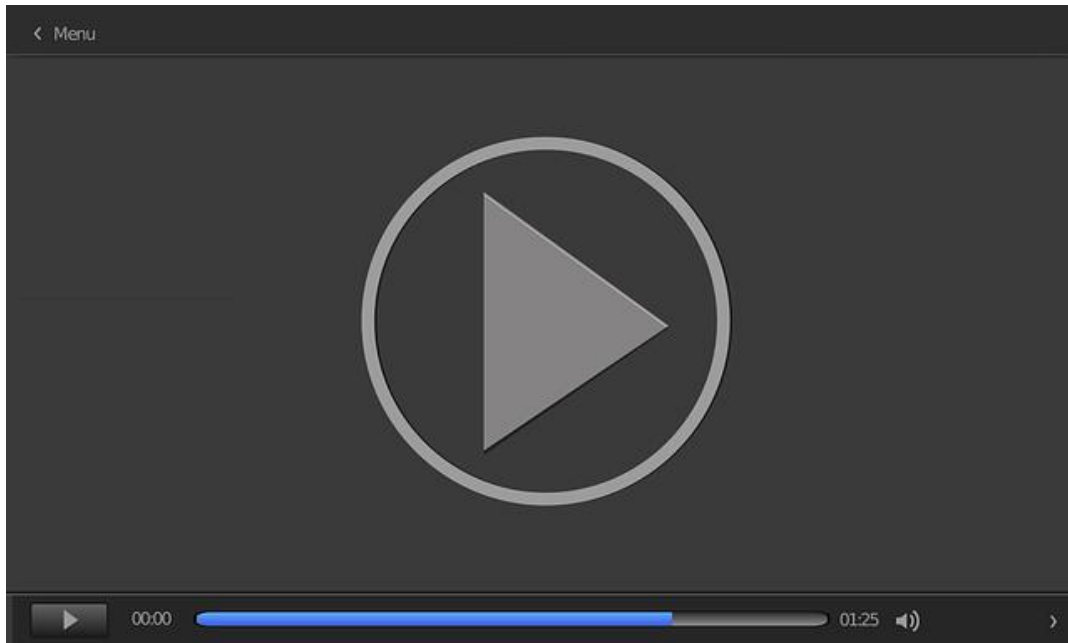
Casos de éxito de AWS

A continuación, se muestra una lista de casos de éxito de AWS en *big data* e IA:

- ▶ **Netflix.** La plataforma de *streaming* de vídeo más grande del mundo, Netflix, utiliza AWS para almacenar, procesar y analizar grandes volúmenes de datos de sus usuarios y contenido. AWS proporciona a Netflix la escalabilidad y las herramientas necesarias para recomendar contenido personalizado a sus usuarios, mejorar la experiencia del usuario y optimizar la entrega de contenido en tiempo real.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo Netflix utiliza la plataforma AWS, te recomendamos ver el vídeo *Netflix Delivers Billions of Hours of Content Globally by Running on AWS* (Amazon Web Services, 2015).



Netflix Delivers Billions of Hours of Content Globally by Running on AWS.

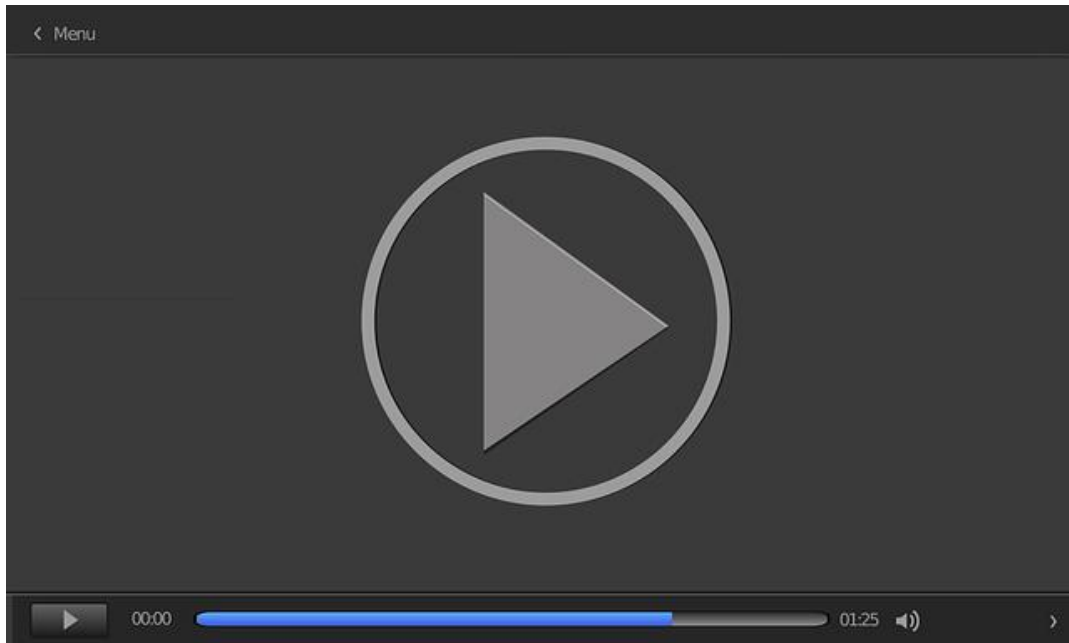
Accede al vídeo:

<https://www.youtube.com/embed/IQGHsBOZJBw>

- **Airbnb.** El popular servicio de alquiler de alojamientos Airbnb utiliza AWS para gestionar grandes volúmenes de datos generados por sus usuarios y anfitriones. AWS proporciona a Airbnb la capacidad de analizar datos en tiempo real para mejorar la segmentación de usuarios, ofrecer recomendaciones personalizadas y optimizar los precios de los alojamientos.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo Airbnb utiliza la plataforma AWS, te recomendamos ver el vídeo *AWS re:Invent 2018: Airbnb's Journey from Self-Managed Redis to ElastiCache for Redis (DAT319)* (Amazon Web Services, 2018a).



AWS re:Invent 2018: Airbnb's Journey from Self-Managed Redis to ElastiCache for Redis (DAT319).

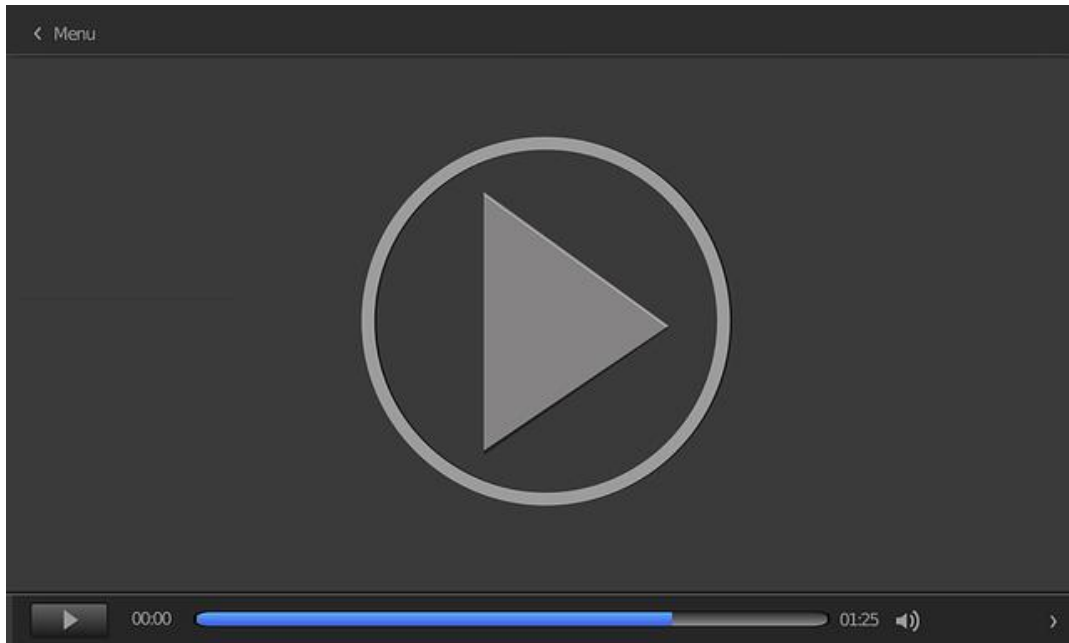
Accede al vídeo:

https://www.youtube.com/embed/eyd_8efUCwM

- **NASA.** La NASA utiliza AWS para almacenar y analizar grandes cantidades de datos generados por sus misiones espaciales y proyectos de investigación. AWS proporciona a la NASA la capacidad de procesar y analizar datos de telemetría, imágenes satelitales y simulaciones numéricas para obtener información valiosa sobre el espacio y la Tierra.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo Airbnb utiliza la plataforma AWS, te recomendamos ver el vídeo *AWS re:Invent 2017: #EarthonAWS: How NASA Is Using AWS (STG205)* (Amazon Web Services, 2017).



AWS re:Invent 2017: #EarthonAWS: How NASA Is Using AWS (STG205).

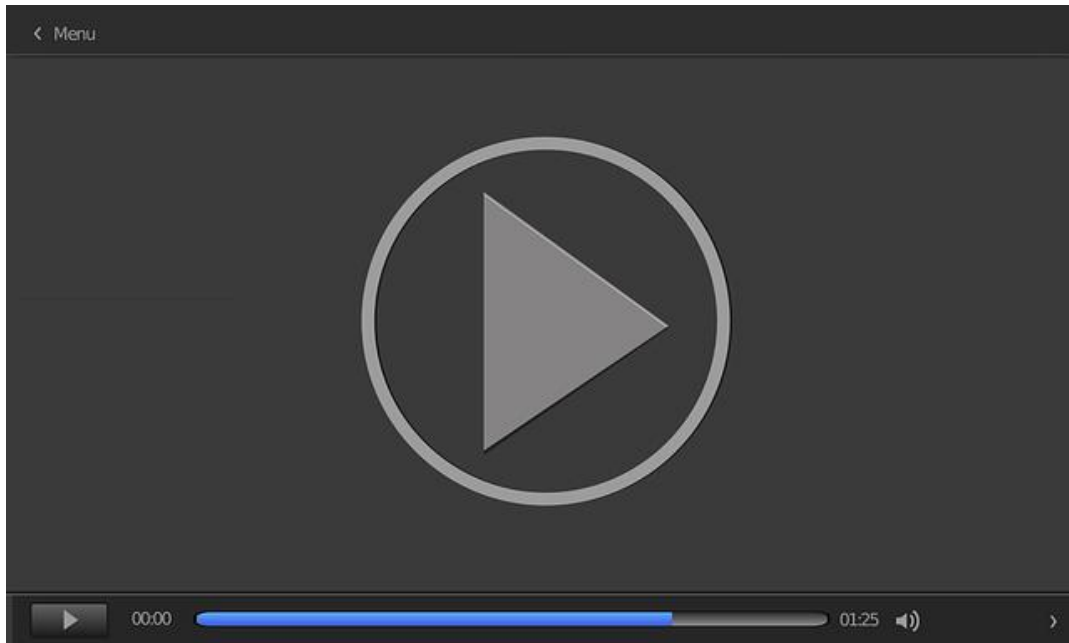
Accede al vídeo:

<https://www.youtube.com/embed/Sh7FB-tkYXM>

- **Pinterest.** La plataforma de descubrimiento visual Pinterest utiliza AWS para almacenar, procesar y analizar grandes volúmenes de datos generados por sus usuarios y contenido. AWS proporciona a Pinterest la capacidad de ofrecer recomendaciones personalizadas, mejorar la segmentación de anuncios y optimizar la experiencia del usuario en tiempo real.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo Pinterest utiliza la plataforma AWS, te recomendamos ver el vídeo *AWS re:Invent 2023 - Pinterest extends existing data lake with generative AI* | AWS Events (Amazon Web Services, 2024).



WS re:Invent 2023 - Pinterest extends existing data lake with generative AI | AWS Events.

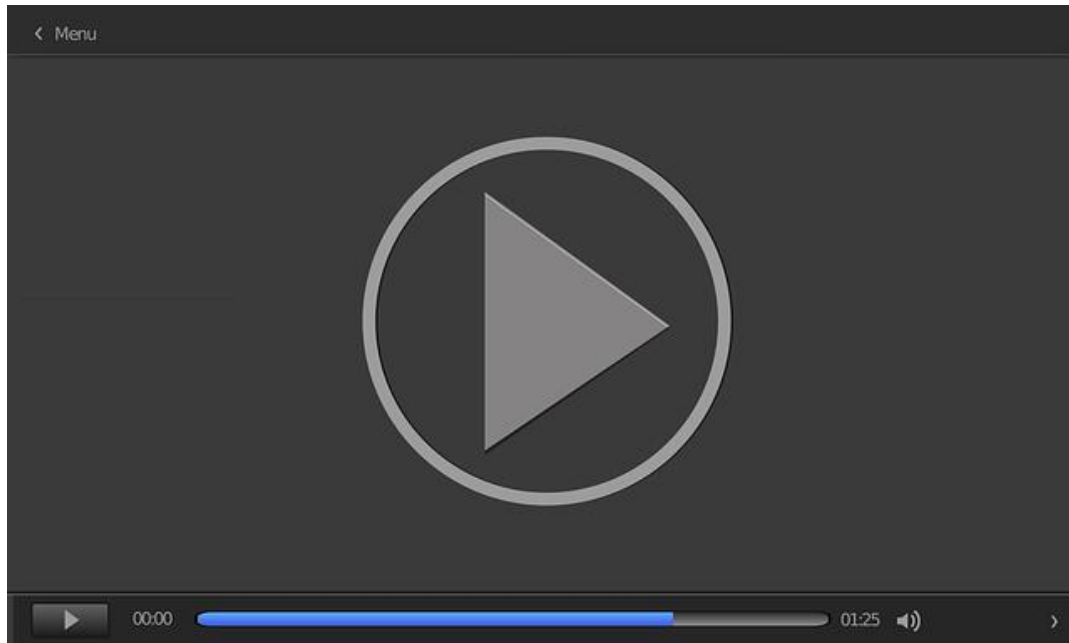
Accede al vídeo:

<https://www.youtube.com/embed/p6B7QX2osRU>

- **Capital One.** La empresa de servicios financieros Capital One utiliza AWS para desarrollar soluciones de *big data* e IA que mejoran la experiencia del cliente y optimizan las operaciones bancarias. AWS proporciona a Capital One la capacidad de analizar grandes volúmenes de datos de transacciones financieras para detectar fraudes, identificar oportunidades de *upselling* y ofrecer servicios personalizados a sus clientes.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo Capitol One utiliza la plataforma AWS, te recomendamos ver el vídeo *Capital One Reimagines Banking Using AWS* (Amazon Web Services, 2018b).



Capital One Reimagines Banking Using AWS.

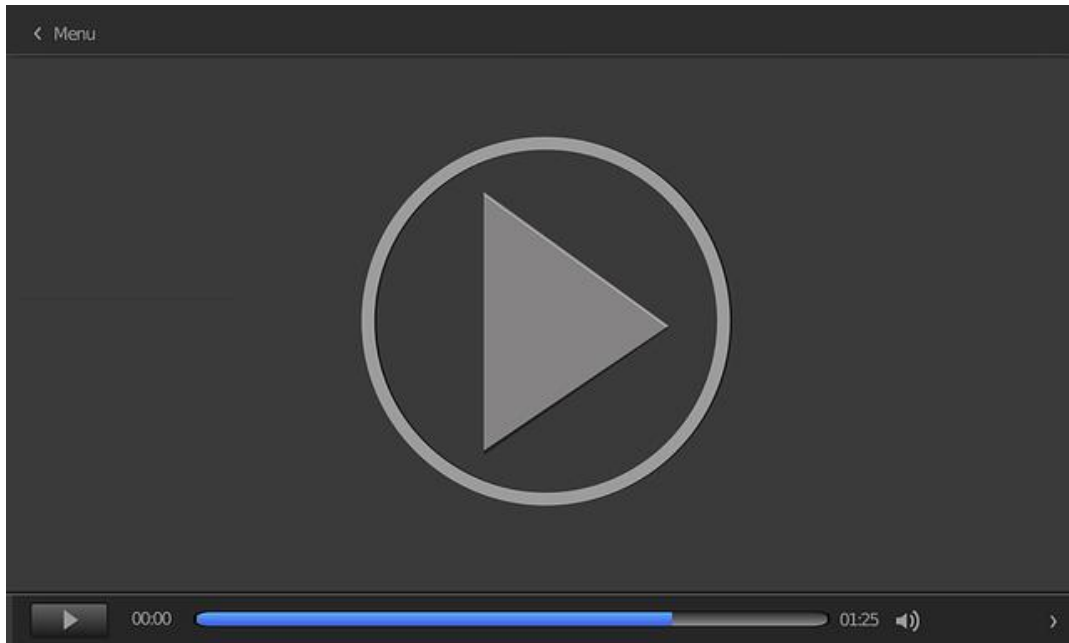
Accede al vídeo:

<https://www.youtube.com/embed/qU0HuWtzDC4>

- ▶ **General Electric (GE).** La empresa multinacional de tecnología e industria GE utiliza AWS para desarrollar soluciones de *big data* e IoT que mejoran la eficiencia operativa y optimizan el mantenimiento predictivo en sus productos y servicios. AWS proporciona a GE la capacidad de analizar grandes volúmenes de datos de sensores en tiempo real para identificar patrones de uso, predecir fallos mecánicos y mejorar la fiabilidad de sus productos y servicios.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Si quieres saber más sobre cómo General Electric utiliza la plataforma AWS, te recomendamos ver el vídeo *Accelerating Enterprise-Wide AI/ML Innovation: GE Healthcare and Autodesk* (Amazon Web Services, 2021).



Accelerating Enterprise-Wide AI/ML Innovation: GE Healthcare and Autodesk.

Accede al vídeo:

<https://www.youtube.com/embed/0vWIAOYkjRk>

Estos son solo algunos ejemplos de cómo empresas líderes están utilizando AWS para impulsar la innovación, mejorar la experiencia del cliente y optimizar sus operaciones mediante el aprovechamiento de soluciones de *big data* e IA.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Servicios *big data* e IA de AWS

AWS ofrece una amplia gama de servicios especializados en *big data* e IA. Estos servicios permiten a las organizaciones recopilar, almacenar, procesar y analizar grandes volúmenes de datos, así como construir y desplegar modelos avanzados de IA para una variedad de aplicaciones. A continuación, profundizaremos en ellos:

- **Big data.** En la Figura 8 se muestra la amplia gama de servicios de *big data* e IA ofrecidos por esta plataforma.

AWS Big Data Portfolio



Figura 8. Catálogo AWS *big data*. Fuente: Kyle Escosia, 2021.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

- ▶ **AWS Glue.** Es un servicio de ETL (*extract, transform, load*) completamente administrado que les permite a las organizaciones descubrir, preparar y cargar datos de manera eficiente para el análisis. Proporciona capacidades de catalogación de datos, generación de código ETL automático y ejecución de trabajos de transformación de datos a escala. Las características clave son las siguientes:
 - **Catalogación de datos.** AWS Glue ofrece un catálogo de datos centralizado que les permite a los usuarios descubrir y entender fácilmente los datos disponibles en la organización.
 - **Generación de código ETL automático.** AWS Glue puede generar automáticamente código ETL (*extract, transform, load*) para transformar y preparar datos para el análisis. Utiliza un motor de generación de código basado en la lógica de transformación definida por el usuario y los metadatos del catálogo de datos.
 - **Ejecución de trabajos de transformación de datos.** AWS Glue les permite a los usuarios ejecutar trabajos de transformación de datos en un entorno completamente administrado y escalable. Utiliza Apache Spark como motor de ejecución para procesar grandes volúmenes de datos de manera rápida y eficiente.
 - **Integración con otros servicios de AWS.** AWS Glue se integra estrechamente con otros servicios de AWS, incluidos S3, Redshift, Athena y más. Esto les permite a los usuarios cargar datos desde diversas fuentes, realizar transformaciones complejas y cargar los datos transformados en almacenes de datos y sistemas de análisis.
- ▶ **AWS Kinesis.** Es una plataforma de *streaming* de datos completamente administrada que les permite a las organizaciones recopilar, procesar y analizar datos en tiempo real. Proporciona capacidades para la ingesta de grandes volúmenes de datos en tiempo real, procesar datos en tiempo real y almacenar datos durante períodos prolongados. Las características clave son las siguientes:
 - **Ingesta de datos en tiempo real y procesamiento de datos en tiempo real.** AWS Kinesis les permite a los usuarios la ingesta de grandes volúmenes de datos en

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

tiempo real desde una variedad de fuentes, incluidos los dispositivos IoT, las aplicaciones móviles y los sistemas de registro. Proporciona capacidades para procesar flujos de datos en tiempo real y almacenar datos en *streams* duraderos.

- Además, AWS Kinesis se integra con **otros servicios de AWS**, como S3, Redshift, DynamoDB y más. Esto les permite a los usuarios procesar datos en tiempo real y almacenar datos en *streams* duraderos para su análisis y uso futuro.
- ▶ **Amazon S3 (Simple Storage Service).** Es un servicio de almacenamiento en la nube altamente escalable y duradero que les permite a las organizaciones almacenar grandes volúmenes de datos de manera segura y accesible desde cualquier lugar. Es ideal para almacenar datos estructurados, semiestructurados y no estructurados, incluidos archivos de registros, imágenes, vídeos y más. Las características clave son la durabilidad, la escalabilidad, el cifrado de datos y la integración con otros servicios de AWS, como Athena, Glue y Redshift.
- ▶ **Amazon EMR (Elastic MapReduce).** Es un servicio de procesamiento de datos distribuido basado en Apache Hadoop y Apache Spark que permite ejecutar y escalar fácilmente clústeres de procesamiento de datos en la nube. Las características clave son la escalabilidad automática, el soporte para una variedad de motores de procesamiento de datos (Hadoop, Spark, Presto, Hive) y la integración con otros servicios de AWS, como S3, DynamoDB y Redshift.
- ▶ **Amazon Redshift.** Es un servicio de almacenamiento de datos en la nube diseñado para la creación y análisis de *data warehouses* a escala petabyte. Permite que las organizaciones analicen grandes volúmenes de datos de manera rápida y eficiente utilizando SQL estándar. Las características clave son el alto rendimiento, la escalabilidad masiva, la integración con herramientas de análisis de datos, como Tableau, Looker y Power BI.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

En el área de la inteligencia artificial, la plataforma AWS ofrece los siguientes servicios:

- ▶ **Amazon SageMaker.** Es un servicio completamente gestionado que les permite a los científicos de datos y desarrolladores construir, entrenar y desplegar modelos de aprendizaje automático de manera rápida y sencilla.
- **Características clave.** Los servicios de entrenamiento y despliegue automático de modelos, la integración con marcos de aprendizaje automático, como TensorFlow y PyTorch, y la capacidad de escalar y gestionar flujos de trabajo de aprendizaje automático de extremo a extremo.
- ▶ **Amazon Rekognition.** Es un servicio de visión artificial que permite analizar imágenes y vídeos para detectar y reconocer objetos, rostros, texto y más.
- **Características clave.** Detección y reconocimiento de objetos y rostros en imágenes y vídeos, análisis de contenido visual en tiempo real, integración con otros servicios de AWS.
- ▶ **Amazon Comprehend.** Es un servicio de procesamiento de lenguaje natural (NLP) que permite analizar y extraer información de texto no estructurado.
- **Características clave.** Análisis de sentimientos, detección de entidades y frases clave, clasificación de documentos, integración con otros servicios de AWS.
- ▶ **Amazon BedRock.** Es un servicio totalmente administrado que ofrece una selección de modelos fundacionales (FM) de alto rendimiento de las principales empresas de IA, como AI21 Labs, Anthropic, Cohere, Meta, Mistral AI, Stability AI y Amazon, a través de una sola API, junto con un amplio conjunto de funciones necesarias para crear aplicaciones de IA generativa con seguridad, privacidad e IA responsables.
- **Características clave.** Con Amazon Bedrock, puede experimentar y evaluar con facilidad los mejores FM para su caso de uso, personalizarlos de forma privada con sus datos mediante técnicas como el ajuste y la generación aumentada de

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

recuperación (RAG), y crear agentes que ejecuten tareas utilizando los sistemas y orígenes de datos de su empresa. Dado que Amazon Bedrock no tiene servidores, no se tiene que administrar ninguna infraestructura y puede integrar e implementar de forma segura capacidades de IA generativa en sus aplicaciones mediante los servicios de AWS que ya conoce.

Estos son solo algunos ejemplos de los servicios de AWS en *big data* e IA. AWS ofrece una amplia gama de herramientas y servicios que permiten que las organizaciones aprovechen la potencia de la nube para gestionar datos a gran escala y construir soluciones avanzadas de inteligencia artificial para una variedad de aplicaciones.

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

1.4. Referencias bibliográficas

Amazon Web Services. (2017, noviembre 29). *AWS re:Invent 2017: #EarthonAWS: How NASA Is Using AWS (STG205)* [Vídeo]. YouTube. <https://youtu.be/Sh7FB-tkYXM>

Amazon Web Service. (2018a, noviembre 28). *AWS re:Invent 2018: Airbnb's Journey from Self-Managed Redis to ElastiCache for Redis (DAT319)* [Vídeo]. YouTube. https://youtu.be/eyd_8efUCwM

Amazon Web Service. (2018b, agosto 24). *Capital One Reimagines Banking Using AWS* [Vídeo]. YouTube. <https://youtu.be/qU0HuWtzDC4>

Amazon Web Services. (2021, abril 21). *Accelerating Enterprise-Wide AI/ML Innovation: GE Healthcare and Autodesk* [Vídeo]. YouTube. <https://youtu.be/0vWIAOYkjRk>

Amazon Web Services. (2024, abril 12). *AWS re:Invent 2023 - Pinterest extends existing data lake with generative AI | AWS Events* [Vídeo]. YouTube. <https://youtu.be/p6B7QX2osRU>

Amazon Web Service. (s. f.-a). *2024 Gartner Report. Magic Quadrant for Cloud AI Developer Services*. <https://pages.awscloud.com/GLOBAL-multi-DL-gartner-mq-cips-2020-learn.html?pg=WIAWS>

Amazon Web Services. (s. f.-b). *AWS Documentation*. https://docs.aws.amazon.com/es_es/

Amazon Web Services. (2015, febrero 27). *Netflix Delivers Billions of Hours of Content Globally by Running on AWS* [Vídeo]. YouTube. <https://youtu.be/IQGHsBOZJBw>

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Andersson, J. C. (2023). *Learning Microsoft Azure: Cloud Computing and Development Fundamentals*. O'Reilly Media, Inc.

Bhatia, J. y Chaudhary, K. (2023). *The Definitive Guide to Google Vertex AI: Accelerate your machine learning journey with Google Cloud Vertex AI and MLOps best practices*. Packt Publishing.

California Privacy Protection Agency. (2024, Agosto). *California Consumer Privacy Act of 2018*. https://coppa.ca.gov/regulations/pdf/coppa_act.pdf

Eagar, G. (2023). *Data Engineering with AWS - Second Edition*. Packt Publishing.

Google Cloud. (2018, septiembre 10). *PayPal partners with Google Cloud* [Video]. YouTube. <https://youtu.be/9jJ6xLOSS3c>

Google Cloud. (2020, julio 14). *Spotify uses Google Cloud to unlock infinite capacity and faster innovation* [Video]. YouTube. https://youtu.be/55xgR_o4PGs

Google Cloud. (s. f.). *Documentación de Google Cloud*. <https://cloud.google.com/docs?hl=es-419>

Inderpreet Singh. (2024, marzo 14). *Beyond the Hype: Choosing the Right AI Platform for You — Vertex AI vs. AWS SageMaker*. Medium. <https://medium.com/@inderpreet.khanduja/beyond-the-hype-choosing-the-right-ai-platform-for-you-vertex-ai-vs-aws-sagemaker-871beb47d909>

Johnson Controls. (2023, junio 27). *Johnson Controls wins 2023 Microsoft Global ISV Partner of the Year for OpenBlue Building Solutions* [Video]. YouTube. <https://youtu.be/8qHLAZUIMXk>

Kozlovski, S. (2018, abril 27). *A Thorough Introduction to Distributed Systems*. Free Code Camp. FreeCodeCamp. <https://www.freecodecamp.org/news/a-thorough-introduction-to-distributed-systems-3b91562c9b3c/>

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Krzyzanowski, P. (2018). *Distributed Systems*. 27. *Engineering Distributed Systems*.

Rutgers University. <https://people.cs.rutgers.edu/~pxk/417/notes/content/27-engineering-ds-slides.pdf>

Kyle Escosia. (2021, mayo 29). *Introduction to the AWS Big Data Portfolio* [Entrada de un foro]. AWS Community ASEAN. <https://dev.to/awscommunity-asean/introduction-to-the-aws-big-data-portfolio-2539>

Lee, R. (2018). *Big data, Cloud computing, Data Science & Engineering*. Springer.

Mahmood, Z., Puttini, R. y Erl, T. (2013). *Cloud computing: Concepts, Technology & Architecture*. Pearson.

Microsoft Learn. (2023, marzo 13). *What is a data mesh?* <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/scenarios/cloud-scale-analytics/architectures/what-is-data-mesh>

Microsoft Learn. (s. f.). *Documentación de Azure*. <https://learn.microsoft.com/es-es/azure/?product=popular>

Mr Anand M. (2024). *Microsoft Azure AI Fundamentals* [libro autopublicado].

Paganini, C. (2019, octubre 23). *Primer: Distributed Systems and Cloud Native Computing*. *The New Stack*. <https://thenewstack.io/primer-distributed-systems-and-cloud-native-computing/>

Rautenstrauch, R. (2023, septiembre 30). *Integración de OpenAI en el universo Azure*. Consultor365. <https://www.consultor365.com/ia/integracion-openai-universo-azure/>

Tema 1. Computación en la nube en la era del big data y la inteligencia artificial

Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos). *Diario Oficial de la Unión Europea*, de 27 de abril de 2016. <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=celex%3A32016R0679>

Roland Berger. (2022, junio 1). *Microsoft Intelligent Manufacturing Award 2021 – BMW* [Vídeo]. YouTube. <https://youtu.be/bACcdP7kz5A>

Sanjay, M., Kumar Tyagi, A., Pluri, V. y Garg, L. (2022). *Artificial Intelligence for Cloud and Edge Computing*. Springer.

Srinivasan, S. (2019, junio 27). *Data and Analytics on Google Cloud Platform*. Medium. <https://medium.com/@srivatsan88/data-and-analytics-on-google-cloud-platform-13bc92a4596f>

Subramanian, S. y Nathu, S. (2021). *AWS Certified Machine Learning: Specialty*. Editorial Sybes.

Universidad Internacional de Valencia. (2017, febrero 22). *Sistemas distribuidos, características y clasificación*. <https://www.universidadviu.com/es/actualidad/nuestros-expertos/sistemas-distribuidos-caracteristicas-y-clasificacion>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

2.1. Introducción y objetivos

Considerando la importancia del *cloud computing* en el desarrollo e implementación de soluciones de IA y *big data*, los **objetivos** que persigue este tema son los siguientes:

- ▶ Entender el ciclo de vida del dato en el *big data* y la IA.
- ▶ Identificar las etapas que componen este ciclo de vida.
- ▶ Profundizar en la etapa de ingesta de datos, explicando principales tipologías, paradigmas y desafío.
- ▶ Conocer el principal servicio de AWS para ingesta de datos *batch*: AWS Glue.
- ▶ Aprender los principales componentes de AWS Glue: Data Catalog, Glue Studio, Glue Notebooks y Glue Pipelines.
- ▶ Aprender el funcionamiento de AWS Glue Data Catalog para inferir esquemas y crear un catálogo de metadatos.
- ▶ Entender el funcionamiento de AWS Glue Studio para desarrollar procesos ETL.
- ▶ Conocer el funcionamiento de AWS Glue Notebooks para desarrollar procesos de ETL en la ingesta de datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

2.2. Ingesta de datos batch

Ciclo de vida del dato en el big data y la IA

El **ciclo de vida de los datos** en *big data* e IA es un proceso que abarca desde la recolección inicial de los datos hasta la implementación y la mejora continua de modelos de IA. Estas etapas suelen estar interconectadas y pueden variar según el contexto y los requisitos específicos del proyecto, pero, a continuación, hay una descripción general de las etapas comunes:

- ▶ **Adquisición de datos.** En esta etapa, se recopilan los datos de diversas fuentes, que pueden incluir bases de datos empresariales, archivos de registros, datos de sensores, redes sociales, transacciones en línea, entre otros. Es crucial asegurarse de que los datos sean relevantes, precisos, completos y estén disponibles en el formato adecuado para su procesamiento posterior.
- ▶ **Almacenamiento de datos.** Los datos recopilados se almacenan en un sistema de almacenamiento adecuado, que puede ser un sistema de archivos distribuido, como Hadoop Distributed File System (HDFS), un almacén de datos en la nube, como Amazon S3 o Google Cloud Storage, o una base de datos distribuida, como Apache Cassandra o MongoDB. Es importante diseñar un esquema de almacenamiento eficiente que permita el acceso rápido a los datos y pueda escalar según sea necesario.
- ▶ **Procesamiento de datos.** En esta etapa, los datos almacenados se procesan para su análisis y extracción de información relevante. Esto puede implicar la limpieza de datos para eliminar valores atípicos y datos incompletos, la transformación de datos para adaptarlos a un formato adecuado para el análisis y la agregación de datos para obtener resúmenes y estadísticas útiles. Se utilizan herramientas como Apache Spark, Apache Flink o Hadoop MapReduce para procesar grandes volúmenes de datos de manera eficiente.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Análisis de datos.** Una vez procesados, los datos se analizan para extraer información significativa y obtener *insights* que puedan ser útiles para la toma de decisiones. Esto puede incluir análisis descriptivos para comprender patrones y tendencias en los datos, análisis predictivos para hacer predicciones sobre eventos futuros, y análisis prescriptivos para recomendar acciones basadas en los datos. Se utilizan técnicas como minería de datos, aprendizaje automático y análisis estadístico para realizar este tipo de análisis.
- ▶ **Desarrollo de modelos de IA.** En esta etapa, se desarrollan y entrenan modelos de inteligencia artificial utilizando técnicas de aprendizaje automático y procesamiento de lenguaje natural. Esto puede implicar la selección y la preparación de características, la elección de algoritmos de aprendizaje automático adecuados, el entrenamiento de modelos con datos de entrenamiento y la evaluación del rendimiento del modelo utilizando datos de prueba. Se pueden utilizar herramientas, como TensorFlow, scikit-learn y PyTorch, para desarrollar y entrenar modelos de IA.
- ▶ **Implementación y despliegue.** Una vez que se han desarrollado y entrenado los modelos, se implementan y despliegan en entornos de producción para su uso en aplicaciones y sistemas en tiempo real. Esto puede implicar la integración de modelos en aplicaciones existentes, la exposición de modelos, como servicios web API o contenedores Docker, y la monitorización del rendimiento del modelo en producción para garantizar su eficacia y fiabilidad.
- ▶ **Monitorización y optimización.** Finalmente, se monitorean continuamente los modelos desplegados en producción para detectar cualquier degradación en el rendimiento y tomar medidas correctivas según sea necesario. Esto puede implicar la recopilación y el análisis de datos de entrada y salida del modelo, la evaluación del rendimiento del modelo en tiempo real y la reentrenamiento periódico del modelo con nuevos datos para mejorar su precisión y generalización.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

El ciclo de vida de los datos en *big data* e IA es un proceso iterativo y continuo, donde cada etapa alimenta a la siguiente y donde el aprendizaje y la mejora son fundamentales para el éxito a largo plazo.

La Figura 1 muestra los componentes básicos, los datos y el ciclo de vida de los datos asociado (el círculo interior), y las herramientas/infraestructuras que permiten el proceso de investigación para poder aplicar IA (el círculo exterior).

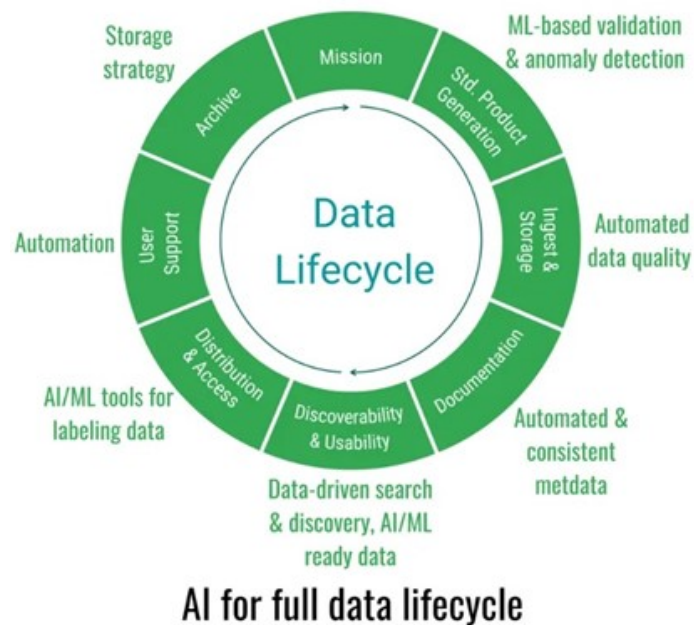


Figura 1. Ciclo de vida de los datos. Fuente: IMPACT Unofficial, 2022.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Ingesta de datos

La ingesta de datos es una etapa crítica en el proceso de IA que se refiere a la recopilación y adquisición de datos de diversas fuentes para su posterior análisis y procesamiento. A continuación, se muestra una descripción detallada de la ingesta de datos para IA:

- ▶ **Recopilación de datos.** La recopilación de datos implica la obtención de datos de diversas fuentes, como bases de datos empresariales, sistemas de archivos, dispositivos IoT, registros en línea, redes sociales y más. Estos datos pueden ser estructurados, semiestructurados o no estructurados, y pueden provenir de diferentes formatos y ubicaciones.
- ▶ **Extracción de datos.** Una vez recopilados, los datos deben ser extraídos de sus fuentes originales y transformados en un formato adecuado para su procesamiento posterior. Esto puede implicar la limpieza de datos para eliminar valores atípicos y datos incompletos, la normalización de datos para asegurar la coherencia y la consistencia, y la conversión de datos a un formato compatible con las herramientas de análisis y modelado de IA.
- ▶ **Preprocesamiento de datos.** El preprocesamiento de datos implica la preparación de los datos para su uso en modelos de IA. Esto puede implicar la selección de características relevantes, la codificación de variables categóricas, la normalización de datos numéricos y la división de los datos en conjuntos de entrenamiento, validación y prueba. El preprocesamiento es crucial para garantizar la calidad y la eficacia de los modelos de IA.
- ▶ **Integración de datos.** En muchos casos, los datos provienen de múltiples fuentes y deben integrarse para su análisis y modelado. Esto puede implicar la combinación de datos de diferentes bases de datos, sistemas de archivos o API, y la unificación de esquemas y formatos de datos. La integración de datos es importante para obtener una visión completa y coherente de la información que se utilizará en los modelos de IA.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Almacenamiento de datos.** Una vez que los datos han sido recopilados, extraídos, preprocesados e integrados deben ser almacenados en un sistema de almacenamiento adecuado para su acceso y uso posterior. Esto puede implicar el uso de sistemas de almacenamiento en la nube, bases de datos distribuidas, sistemas de archivos distribuidos o almacenes de datos especializados. El almacenamiento de datos debe ser escalable, seguro y accesible para garantizar un procesamiento eficiente y eficaz de los datos.
- ▶ **Gestión de metadatos.** La gestión de metadatos es importante para mantener un registro de los datos y proporcionar información sobre su origen, significado y calidad. Esto puede incluir la creación de catálogos de datos, la documentación de esquemas de datos, la trazabilidad de los datos y la gestión de versiones de los datos. La gestión de metadatos facilita la búsqueda, la comprensión y la utilización de los datos en proyectos de IA.
- ▶ **Automatización y orquestación.** La automatización y la orquestación son importantes para gestionar el flujo de datos de manera eficiente y escalable. Esto puede implicar el uso de herramientas de automatización para programar y ejecutar tareas de ingesta de datos de manera automatizada, y el uso de sistemas de orquestación para coordinar y gestionar flujos de datos complejos entre diferentes sistemas y servicios.

En resumen, la **ingesta de datos** es una etapa fundamental en el proceso de IA que implica la recopilación, extracción, preprocesamiento, integración, almacenamiento, gestión de metadatos y automatización de datos para su uso en modelos de IA. Una ingesta de datos eficiente y bien gestionada es crucial para el éxito de los proyectos de IA, como podemos ver en la Figura 2.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

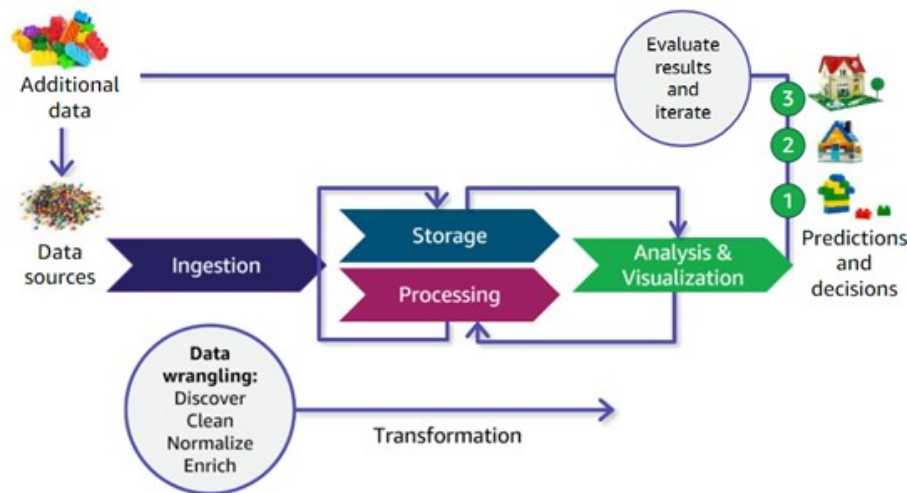


Figura 2. Ingesta de datos. Fuente: Aitor Medrano, s. f.

Tipologías de ingestión de datos

La ingestión de datos en el contexto de la informática y la IA se refiere al proceso de **recopilación y adquisición de datos** desde diversas fuentes para su posterior procesamiento y análisis. Hay varias tipologías de ingestión de datos, que se clasifican según la naturaleza de los datos y cómo se obtienen. A continuación, se mencionan algunas de las tipologías más comunes:

- **Ingesta *batch*.** Implica la recopilación y procesamiento de grandes volúmenes de datos en bloques o lotes periódicos. Los datos se recopilan durante un período de tiempo y se procesan en lotes, generalmente en intervalos programados. Esto es útil para casos de uso donde el análisis no necesita ser en tiempo real y se puede tolerar cierto retraso en la disponibilidad de los resultados.
- **Ingesta *streaming*.** Implica la recopilación y procesamiento de datos en tiempo real a medida que se generan. Los datos se transmiten continuamente desde las fuentes de origen a los sistemas de destino, donde se procesan y analizan en tiempo real. Esto es útil para casos de uso donde se requiere una respuesta rápida a los datos entrantes y se necesita un análisis en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

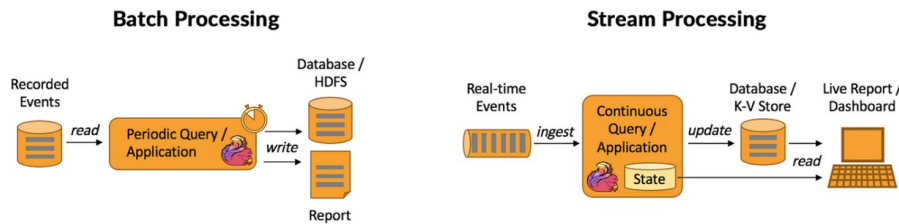


Figura 3. Ingesta *batch* y *streaming*. Fuente: Murat Sivri, 2022.

- ▶ **Ingesta incremental.** Implica la recopilación de solo los datos nuevos o actualizados desde la última ejecución. En lugar de procesar todos los datos cada vez, solo se procesan los datos que han cambiado desde la última ingesta. Esto es útil para reducir el tiempo y los recursos necesarios para procesar datos, especialmente en entornos con grandes volúmenes de datos que cambian con frecuencia.
- ▶ **Ingesta de datos estructurados.** Implica la recopilación de datos que siguen un formato predefinido y organizado, como las bases de datos relacionales, los archivos CSV o JSON con una estructura definida. Estos datos suelen ser fáciles de procesar y analizar debido a su organización y coherencia.
- ▶ **Ingesta de datos semiestructurados.** Implica la recopilación de datos que tienen una estructura parcialmente definida, como documentos HTML, archivos XML o registros de servidor. Estos datos pueden contener elementos que no siguen una estructura estricta y pueden requerir un procesamiento adicional para extraer información útil.
- ▶ **Ingesta de datos no estructurados.** Implica la recopilación de datos que no siguen ningún formato predefinido o estructura organizada, como imágenes, vídeos, archivos de audio o texto libre. Estos datos suelen ser más difíciles de procesar y analizar debido a su falta de estructura, pero pueden contener información valiosa que puede ser aprovechada con técnicas de IA como el procesamiento de lenguaje natural o la visión por computadora.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Ingesta de datos de alta velocidad.** La ingesta de datos de alta velocidad implica la recopilación de datos que se generan a una velocidad muy rápida, como datos de sensores, registros de eventos o datos de transmisión en tiempo real. Estos datos deben ser procesados y analizados rápidamente para mantenerse al día con la tasa de generación de datos.

Cada tipo de ingesta de datos tiene sus propias características y desafíos, y la elección del enfoque adecuado dependerá de los requisitos específicos del proyecto y las necesidades de análisis de datos. Es común que los sistemas de ingesta de datos utilicen una combinación de estos enfoques para manejar diferentes tipos de datos y casos de uso

Desafíos en la ingesta de datos para IA

La ingesta de datos en entornos de *big data* enfrenta desafíos específicos debido al gran volumen, la variedad y la velocidad de los datos involucrados. A continuación, se presenta una lista de los **principales desafíos** de la ingesta de datos en *big data*, junto con una breve descripción de cada uno:

- ▶ **Escalabilidad.** Manejar grandes volúmenes de datos de manera eficiente y escalable es fundamental en *big data*. Los sistemas de ingesta deben poder procesar y almacenar enormes cantidades de datos de manera rápida y efectiva.
- ▶ **Velocidad de ingesta.** En entornos de *big data*, la velocidad de ingesta puede ser crítica, especialmente en aplicaciones de tiempo real. Los sistemas deben ser capaces de capturar y procesar datos rápidamente para garantizar que la información sea relevante y útil en el momento adecuado.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Variedad de fuentes y formatos de datos.** Los datos en entornos de *big data* pueden provenir de una amplia variedad de fuentes, incluidas las bases de datos, los archivos de registro, las redes sociales, los sensores IoT, entre otros. Integrar datos de diferentes formatos y estructuras puede ser complicado y requerir procesos de transformación y normalización.
- ▶ **Latencia.** La latencia en el procesamiento y la entrega de datos puede afectar la eficiencia y la utilidad de la información en aplicaciones de *big data*. Minimizar la latencia en la ingesta de datos es importante para garantizar que los datos estén disponibles para análisis y procesamiento en tiempo real.
- ▶ **Consistencia y confiabilidad.** Garantizar la consistencia y la confiabilidad de los datos en entornos de *big data* puede ser un desafío. Los sistemas de ingesta deben ser capaces de manejar errores, duplicados y pérdida de datos de manera efectiva para garantizar la integridad de los datos.
- ▶ **Seguridad y cumplimiento normativo.** La seguridad de los datos y el cumplimiento de regulaciones son preocupaciones importantes en la ingesta de datos en entornos de *big data*. Proteger los datos sensibles y cumplir con los requisitos normativos puede ser un desafío técnico y legal.
- ▶ **Complejidad de la arquitectura.** Diseñar, implementar y mantener una arquitectura de ingesta de datos robusta y escalable en entornos de *big data* puede ser complicado. La infraestructura y los sistemas de ingesta deben ser capaces de escalar y adaptarse a medida que crecen los volúmenes de datos y las necesidades del negocio.
- ▶ **Monitorización y gestión.** Monitorear y gestionar el proceso de ingesta de datos en entornos de *big data* es fundamental para garantizar la integridad y la disponibilidad de los datos. Implementar herramientas de monitoreo y alerta que permitan identificar problemas y tomar medidas correctivas es esencial para mantener la salud y el rendimiento del sistema.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En resumen, la ingesta de datos en entornos de *big data* enfrenta una serie de desafíos, desde la escalabilidad y la velocidad hasta la variedad y la confiabilidad de los datos. Abordar estos desafíos de manera efectiva es crucial para garantizar el éxito de las iniciativas de *big data* y análisis de datos.

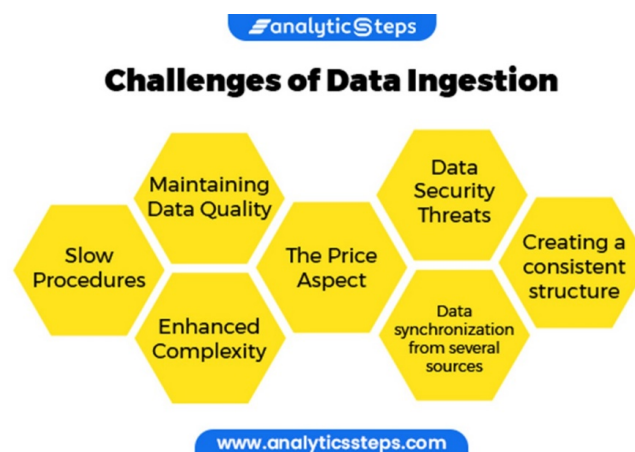


Figura 4. Desafíos de la ingesta de datos. Fuente: Vrinda Mathur, 2022.

Paradigmas de ingesta de datos

Los **paradigmas de ingesta de datos ETL** (*extract, transform, load*) y **ELT** (*extract, load, transform*) son enfoques utilizados en la preparación y procesamiento de datos antes de ser almacenados en un sistema de destino, como un almacén de datos o un lago de datos. A continuación, se desarrolla cada uno.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

ETL, que significa *extract, transform, load* (extraer, transformar, cargar), es un proceso fundamental en el campo del *big data* y la gestión de datos en general. A continuación, tienes una descripción detallada de cada una de las fases del proceso ETL:

- ▶ **Extract (extraer).** En la etapa de extracción, los datos se recopilan de múltiples fuentes, que pueden incluir bases de datos, archivos, sistemas en línea, API, entre otros. Los datos extraídos pueden ser crudos y no estructurados, y pueden estar en diferentes formatos y ubicaciones.
- ▶ **Transform (transformar).** En la etapa de transformación, los datos extraídos se procesan y se transforman en un formato adecuado para su almacenamiento y análisis. Esto puede implicar la limpieza de datos para eliminar valores atípicos y datos incompletos, la normalización de datos para asegurar la coherencia y la consistencia, y la agregación de datos para obtener resúmenes y estadísticas útiles. También puede implicar la conversión de datos a un esquema común y la combinación de datos de diferentes fuentes.
- ▶ **Load (cargar).** En la etapa de carga, los datos transformados se cargan en un sistema de destino, como un almacén de datos o un lago de datos, donde estarán disponibles para su análisis posterior. Los datos cargados suelen estar estructurados y listos para su uso en informes, análisis y aplicaciones de inteligencia empresarial.

El **proceso ELT** (*extract, load, transform*) en *big data* es una variante del proceso ETL en el que la transformación de datos se realiza después de la carga en el sistema de destino en lugar de antes. A continuación, se presenta una descripción de cada una de las fases del proceso ELT en el contexto del *big data*:

- ▶ **Extract (extraer).** En el paradigma ELT, los datos se extraen de las fuentes de origen de la misma manera que en ETL, sin realizar transformaciones significativas en este punto.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Load (cargar).** Después de extraer los datos, se cargan directamente en el sistema de destino sin transformaciones significativas. Los datos se almacenan en su forma original, a menudo en un lago de datos o un almacén de datos sin procesar.
- ▶ **Transform (transformar).** En la etapa de transformación, los datos se transforman y se procesan en el sistema de destino según sea necesario. Esto puede implicar la ejecución de transformaciones complejas, consultas SQL o procesos de análisis en los datos cargados en el sistema de destino.

Diferencias entre ETL y ELT

- ▶ **ETL** se centra en transformar los datos antes de cargarlos en el sistema de destino, lo que puede ser beneficioso para limpiar y normalizar los datos antes de su almacenamiento.
- ▶ **ELT** carga los datos en el sistema de destino sin transformaciones significativas, lo que puede ser útil cuando se trabaja con grandes volúmenes de datos y se requiere una mayor escalabilidad.

En resumen, ETL y ELT son dos paradigmas de ingesta de datos comúnmente utilizados en el procesamiento de datos antes de su almacenamiento en un sistema de destino. La **elección entre ETL y ELT** dependerá de los requisitos específicos del proyecto, incluida la naturaleza de los datos, el volumen de datos y los requisitos de transformación.

La Figura 5 ilustra las diferencias entre los dos paradigmas de acuerdo con el orden de acciones en el proceso de ingesta de datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

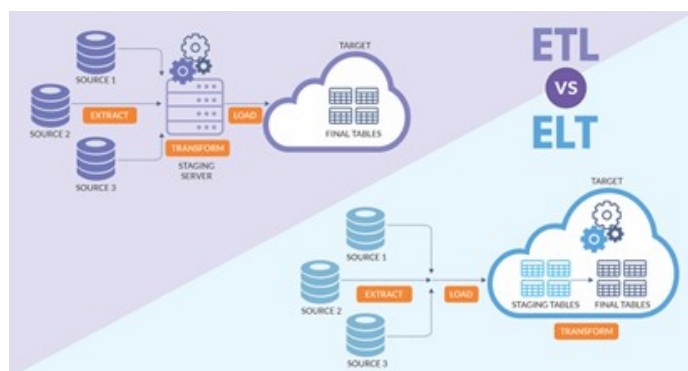


Figura 5. Diferentes entre ETL y ELT. Fuente: Halder, 2023.

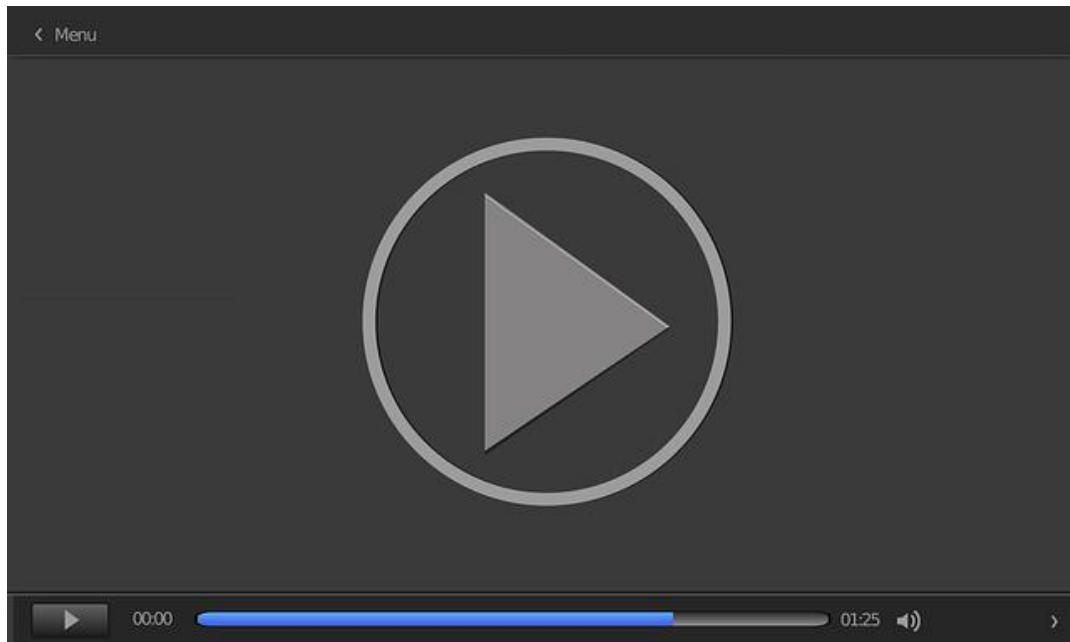
AWS Glue

Introducción a AWS Glue

En el mundo de la analítica de datos, la **preparación de datos** es una parte fundamental del proceso. AWS Glue es una herramienta desarrollada por AWS que facilita la preparación y el procesamiento de datos para su análisis en la nube. Con Glue, las empresas pueden automatizar tareas de extracción, transformación y carga (ETL) de datos, lo que les permite centrarse en el análisis y la generación de *insights* en lugar de en la gestión de infraestructura compleja.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

El siguiente vídeo, *AWS Glue Overview | Amazon Web Services* (Amazon Web Services, 2022), muestra un *overview* del servicio AWS Glue.



AWS Glue Overview | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/u14iVEc-C6E>

Descripción de AWS Glue

AWS Glue es un servicio de ETL completamente administrado que ofrece una amplia gama de características y funcionalidades para ayudar a las organizaciones a preparar y procesar sus datos de manera eficiente y escalable. Está diseñado para trabajar con **grandes volúmenes de datos y flujos de trabajo complejos**, lo que lo hace ideal para aplicaciones en la nube que requieren análisis avanzados y procesamiento de datos en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Características principales de AWS Glue

Las **características principales** de AWS Glue son las siguientes:

- ▶ **Descubrimiento automático de esquemas.** Glue utiliza *crawlers* para explorar las fuentes de datos y descubrir automáticamente la estructura y el esquema de los datos. Esto simplifica el proceso de preparación de datos al eliminar la necesidad de definir manualmente los esquemas de datos.
- ▶ **Transformaciones ETL.** Glue proporciona una funcionalidad robusta para realizar transformaciones ETL en los datos, lo que permite limpiar, normalizar y enriquecer los datos antes de cargarlos en un almacén de datos o un lago de datos. Las transformaciones se pueden definir utilizando lenguajes de *script*, como Python o Scala, o utilizando una interfaz gráfica de usuario intuitiva.
- ▶ **Programación de trabajos ETL.** Los trabajos de Glue permiten programar y ejecutar flujos de trabajo de ETL de manera automatizada y periódica. Esto simplifica la gestión de flujos de trabajo complejos y permite manejar grandes volúmenes de datos de manera eficiente.
- ▶ **Integración con otros servicios de AWS.** Glue se integra estrechamente con otros servicios de AWS, como S3, Redshift y Athena, lo que permite cargar datos en almacenes de datos y realizar análisis avanzados. Esta integración facilita el acceso a una amplia gama de herramientas y servicios de AWS para el análisis de datos.
- ▶ **Completamente administrado.** Glue es un servicio completamente administrado, lo que significa que AWS se encarga de la infraestructura subyacente, incluida la escalabilidad, la disponibilidad y la seguridad. Esto les permite a los usuarios centrarse en el análisis de datos en lugar de en la gestión de infraestructura.

En resumen, AWS Glue es una herramienta poderosa y versátil que simplifica y automatiza el **proceso de preparación de datos** para análisis en la nube. Con su capacidad para descubrir automáticamente esquemas de datos, realizar

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

transformaciones ETL y programar flujos de trabajo de manera automatizada, Glue permite que las organizaciones aceleren el tiempo de obtención de información y obtener *insights* valiosos de sus datos.

Componentes de AWS Glue

Para el **descubrimiento e inferencia de esquemas**, se utiliza el **Glue Data Catalog**, que es el catálogo de datos de AWS Glue (AWS Glue Data Catalog). Este es un servicio fundamental que actúa como un repositorio centralizado y gestionado de metadatos sobre sus datos en AWS. A continuación, tienes una descripción detallada de los servicios y capacidades del catálogo de datos de AWS Glue:

- ▶ **Almacenamiento de metadatos.** El catálogo de datos de AWS Glue almacena metadatos sobre sus conjuntos de datos, incluidos esquemas, tablas, particiones y dependencias. Estos metadatos proporcionan información estructural y descriptiva sobre los datos, lo que facilita su descubrimiento, comprensión y uso.
- ▶ **Descubrimiento automático de esquemas.** El catálogo de datos de Glue utiliza *crawlers* para explorar las fuentes de datos y descubrir automáticamente la estructura y el esquema de los datos. Esto elimina la necesidad de definir manualmente los esquemas de datos y simplifica el proceso de preparación de datos.

Los ***crawlers*** son herramientas que exploran las fuentes de datos para descubrir automáticamente la estructura y el esquema de los datos. Utilizan técnicas de inferencia para identificar metadatos relevantes, como tipos de datos, claves primarias y campos de partición. Los *crawlers* simplifican el proceso de preparación de datos al eliminar la necesidad de definir manualmente los esquemas de datos.

En la Figura 6, se muestra cómo los rastreadores de AWS Glue interactúan con almacenes de datos y otros elementos para rellenar el catálogo de datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

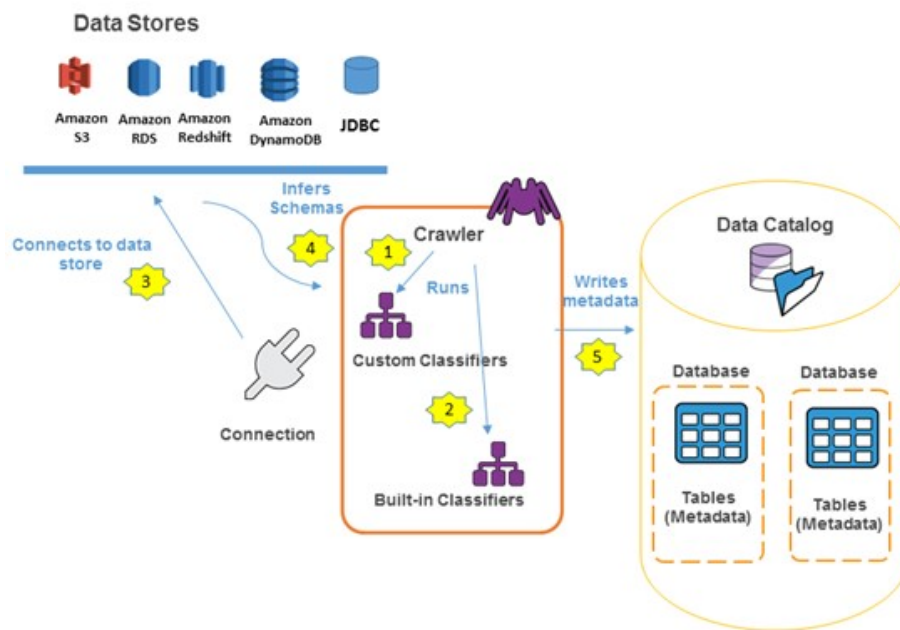


Figura 6. Catálogo. Fuente: Amazon Web Services, s. f-a.

Este es el flujo de flujo de trabajo general de rellenado de AWS Glue Data Catalog por parte de un rastreador (Tabla 1).

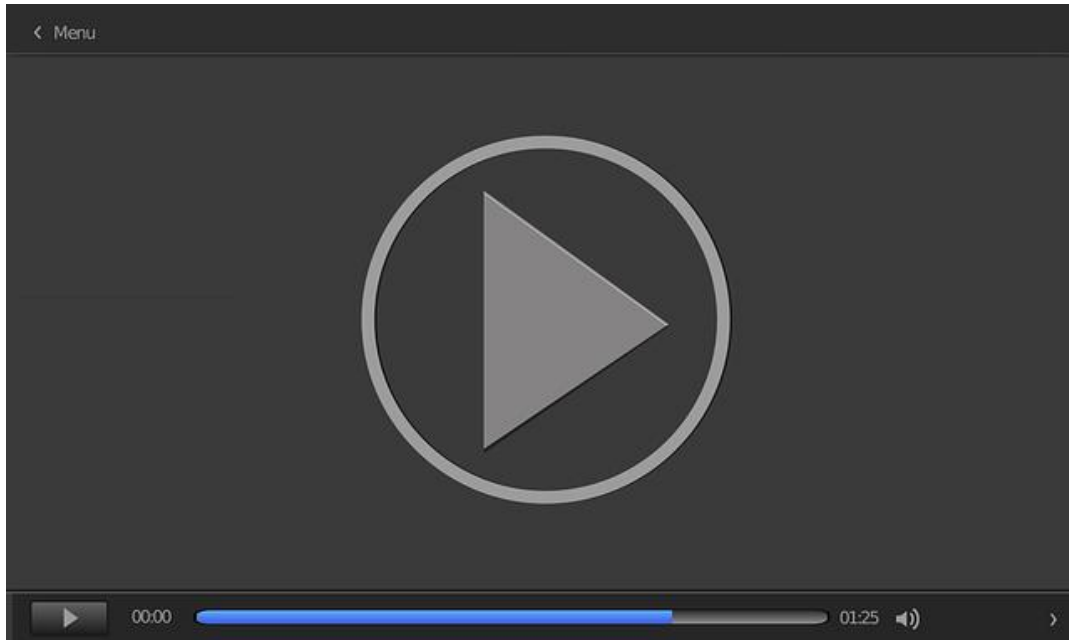
Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Flujo de trabajo	
1	Un rastreador ejecuta cualquier clasificador personalizado que elija para inferir el formato y el esquema de sus datos. Debe proporcionar el código para clasificadores personalizados, que se ejecutan en el orden especificado.
2	El primer clasificador personalizado en reconocer correctamente la estructura de sus datos se usa para crear un esquema. Los clasificadores personalizados que aparecen más abajo en la lista se omiten.
3	Si no coincide ningún clasificador con el esquema de sus datos, los clasificadores integrados intentarán reconocer el esquema de sus datos. Un ejemplo de un clasificador integrado es uno que reconoce JSON.
4	El rastreador se conecta al almacén de datos. Algunos almacenes de datos requieren propiedades de conexión para el acceso del rastreador.
5	El esquema inferido se crea para sus datos.
6	El rastreador escribe los metadatos en el catálogo de datos. Una definición de tabla contiene metadatos acerca de los datos de su almacén de datos. La tabla se escribe en una base de datos, que es un contenedor de tablas en el catálogo de datos. Entre los atributos de una tabla se incluye la clasificación, que es una etiqueta creada por el clasificador que determinó el esquema de tabla.

Tabla 1. Flujo de trabajo. Fuente: elaboración propia.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

El siguiente vídeo, *Descubre y cataloga tus datos con AWS Glue – Español* (AWS LATAM, 2021a), es un tutorial que explica cómo catalogar los datos con AWS Glue.



Descubre y cataloga tus datos con AWS Glue – Español.

Accede al vídeo:

<https://www.youtube.com/embed/pbKnfjiVsx4>

- ▶ **Integración con otros servicios de AWS.** El catálogo de datos de Glue se integra estrechamente con otros servicios de AWS, como S3, Redshift y Athena. Esto permite cargar datos en almacenes de datos y realizar análisis avanzados utilizando herramientas y servicios familiares de AWS.
- ▶ **Compatibilidad con particiones y vistas.** El catálogo de datos de Glue es compatible con particiones, lo que facilita la organización y el acceso a grandes volúmenes de datos. También admite la creación de vistas que pueden simplificar y mejorar el acceso a los datos al proporcionar una capa de abstracción sobre los datos subyacentes.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Gestión de metadatos.** El catálogo de datos de Glue proporciona herramientas para gestionar metadatos, incluida la edición, el borrado y la visualización de metadatos. También permite etiquetar y categorizar los datos para facilitar su organización y búsqueda.
- ▶ **Acceso controlado.** El catálogo de datos de Glue ofrece capacidades de acceso controlado que permiten controlar quién puede acceder y modificar los metadatos. Esto ayuda a garantizar la seguridad y la privacidad de los datos y cumple con los requisitos de cumplimiento normativo.
- ▶ **API y SDK.** Glue proporciona una API y SDK que permiten integrar el catálogo de datos con aplicaciones y herramientas personalizadas. Esto facilita la automatización de tareas y la integración con el ecosistema de herramientas existente.

Para el **motor de transformaciones ETL**, AWS Glue proporciona una funcionalidad robusta para realizar transformaciones ETL en los datos. Esto incluye limpiar, normalizar y enriquecer los datos antes de cargarlos en un almacén de datos o un lago de datos. Las transformaciones se pueden definir utilizando lenguajes de *script*, como Python o Scala, o utilizando una interfaz gráfica de usuario intuitiva. Glue permite que los usuarios definan transformaciones complejas y personalizadas para adaptarse a sus necesidades específicas.

Los trabajos de Glue permiten programar y ejecutar **flujos de trabajo de ETL** de manera automatizada y periódica. Esto simplifica la gestión de flujos de trabajo complejos y permite manejar grandes volúmenes de datos de manera eficiente. Los trabajos de Glue son completamente administrados y escalables, lo que garantiza un rendimiento confiable y consistente incluso con grandes cargas de trabajo.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Entre los componentes que ofrece AWS Glue para los flujos de transformaciones de ETL se puede destacar **AWS Glue Studio**, que es una herramienta visual desarrollada por AWS que simplifica y acelera el proceso de preparación de datos y creación de flujos de trabajo ETL. A continuación, tienes una descripción detallada de AWS Glue Studio y sus principales funcionalidades:

- ▶ **Interfaz gráfica de usuario intuitiva.** AWS Glue Studio proporciona una interfaz gráfica de usuario intuitiva que les permite a los usuarios diseñar y construir flujos de trabajo ETL de manera visual. Utilizando una interfaz de tipo «arrastrar y soltar», los usuarios pueden crear y personalizar flujos de trabajo sin necesidad de escribir código.
- ▶ **Creación de flujos de trabajo ETL.** Glue Studio permite que los usuarios creen flujos de trabajo ETL para extraer, transformar y cargar datos desde diversas fuentes a destinos como almacenes de datos, *data lakes* o bases de datos. Los flujos de trabajo pueden incluir múltiples pasos de transformación y manipulación de datos.
- ▶ **Amplia gama de conectores.** AWS Glue Studio proporciona una amplia gama de conectores predefinidos que facilitan la conexión a diversas fuentes de datos, como Amazon S3, Amazon RDS, Amazon Redshift y más. Esto simplifica el proceso de integración y extracción de datos desde múltiples fuentes.
- ▶ **Herramientas de transformación visual.** Glue Studio ofrece herramientas visuales para realizar transformaciones de datos, como filtrado, unión, agregación, y limpieza de datos. Los usuarios pueden arrastrar y soltar operadores de transformación para definir el flujo de datos y aplicar transformaciones a los datos de manera visual.
- ▶ **Monitoreo y depuración.** El servicio incluye herramientas integradas para monitorear y depurar flujos de trabajo en tiempo real. Los usuarios pueden ver el progreso de los flujos de trabajo, detectar y solucionar problemas, y optimizar el rendimiento de manera eficiente.

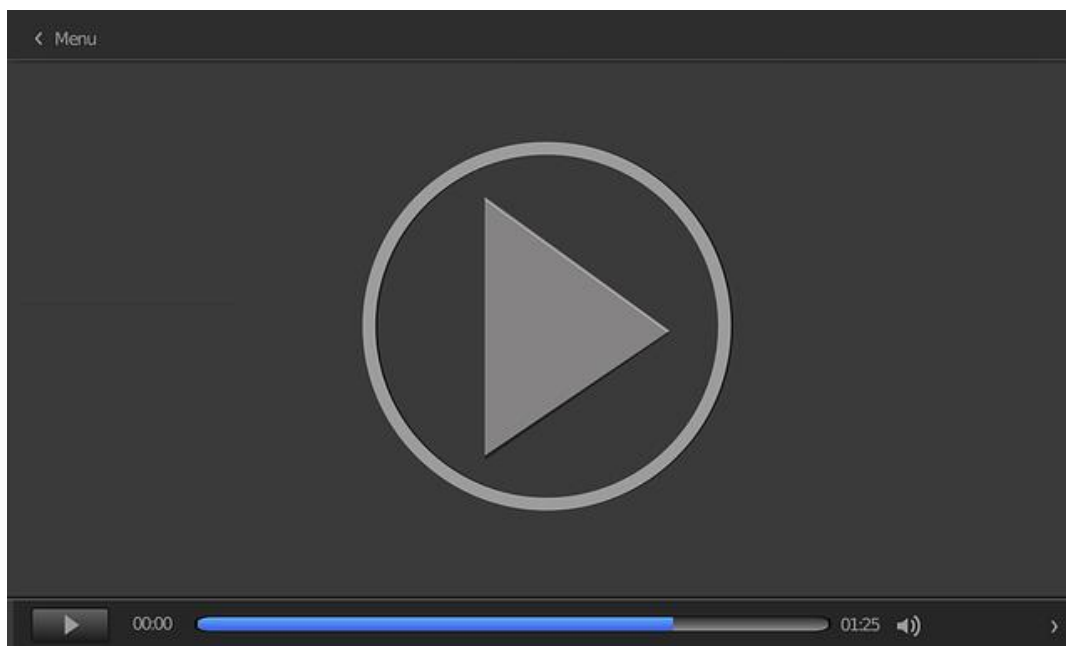
Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Colaboración y versionado.** Glue Studio permite que los equipos colaboren en el diseño y desarrollo de flujos de trabajo mediante la compartición de proyectos y la gestión de versiones. Los usuarios pueden trabajar en equipo para crear y mantener flujos de trabajo de manera colaborativa y controlar los cambios a lo largo del tiempo.
- ▶ **Integración con servicios de AWS.** AWS Glue Studio se integra estrechamente con otros servicios de AWS, como AWS Glue Data Catalog, AWS Glue Jobs, Amazon S3, Amazon Redshift y más. Esto permite que los usuarios aprovechen el ecosistema completo de AWS para el procesamiento y el análisis de datos.

En resumen, **AWS Glue Studio** es una herramienta poderosa y versátil que permite que los usuarios diseñen, construyan y administren flujos de trabajo ETL de manera visual y colaborativa. Con su interfaz gráfica intuitiva, amplia gama de conectores, herramientas de transformación visual, y capacidades de monitoreo y depuración, Glue Studio simplifica y agiliza el proceso de preparación de datos en la nube.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

El siguiente vídeo, *Prepara tus datos y administra tus ETLs con AWS Glue – Español* (AWS LATAM, 2021b), es un tutorial que explica cómo preparar y administrar los datos con AWS Glue.



Prepara tus datos y administra tus ETLs con AWS Glue – Español.

Accede al vídeo:

https://www.youtube.com/embed/mw6nu7-_4PI

Los **pasos básicos para crear un trabajo** con AWS Glue Studio son los siguientes:

- ▶ **Acceder a AWS Glue Studio.** Inicia sesión en la consola de AWS y navega hasta el servicio de AWS Glue. Una vez allí, selecciona «Glue Studio» en el menú de navegación.
- ▶ **Crear un nuevo proyecto.** En la interfaz de Glue Studio, selecciona «Crear un nuevo proyecto» para iniciar la creación de un nuevo proyecto.
- ▶ **Crear un flujo de trabajo.** Dentro del proyecto, selecciona «Crear un flujo de trabajo» para comenzar a diseñar tu flujo de trabajo ETL.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Seleccionar conectores de origen y destino.** Selecciona los conectores de origen y destino que desees utilizar para tu flujo de trabajo. Puedes elegir entre una amplia gama de conectores predefinidos para acceder a datos desde diferentes fuentes y cargarlos en diferentes destinos.
- ▶ **Configurar los pasos del flujo de trabajo.** Diseña el flujo de trabajo arrastrando y soltando operadores de transformación y manipulación de datos en el lienzo de Glue Studio. Configura los pasos del flujo de trabajo, como la extracción, la transformación y la carga de datos, según tus requisitos específicos.
- ▶ **Configurar parámetros y propiedades.** Configura los parámetros y las propiedades de cada paso del flujo de trabajo, como los nombres de las tablas, los tipos de datos, las opciones de partición, las claves de partición, etc. Ajusta las configuraciones según sea necesario para adaptarse a tu caso de uso.
- ▶ **Definir las transformaciones de datos.** Define las transformaciones de datos necesarias para limpiar, normalizar, enriquecer y manipular los datos según tus requisitos. Utiliza las herramientas visuales de Glue Studio para diseñar las transformaciones de manera eficiente y efectiva.
- ▶ **Guardar y ejecutar el flujo de trabajo.** Una vez que hayas configurado tu flujo de trabajo, guarda tus cambios y ejecuta el flujo de trabajo para procesar los datos. Glue Studio te permite monitorear y gestionar el progreso de la ejecución del flujo de trabajo en tiempo real.
- ▶ **Monitorear y depurar.** Monitorea el progreso de la ejecución del flujo de trabajo y realiza cualquier depuración necesaria para solucionar problemas o mejorar el rendimiento. Glue Studio proporciona herramientas integradas para monitorear y depurar flujos de trabajo en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- **Guardar y desplegar el proyecto.** Una vez que hayas completado la creación y la ejecución del flujo de trabajo, guarda tus cambios y despliega el proyecto para hacerlo disponible para su uso en producción. Glue Studio te permite gestionar versiones y controlar los cambios a lo largo del tiempo.

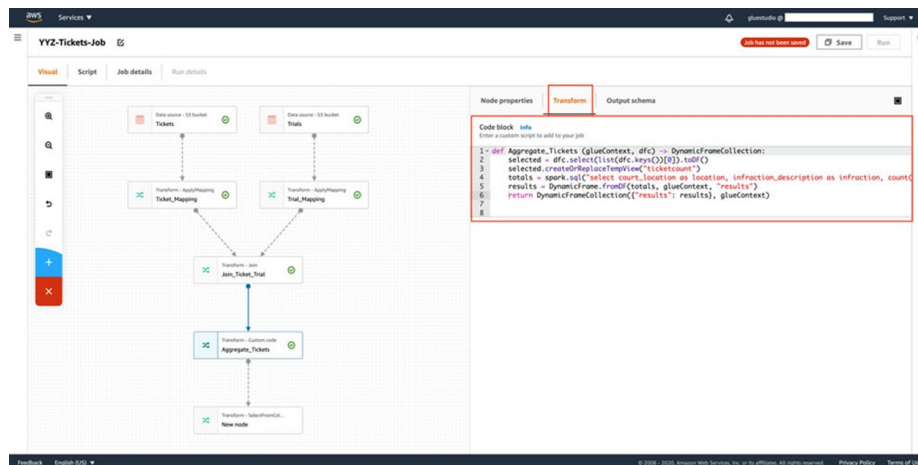
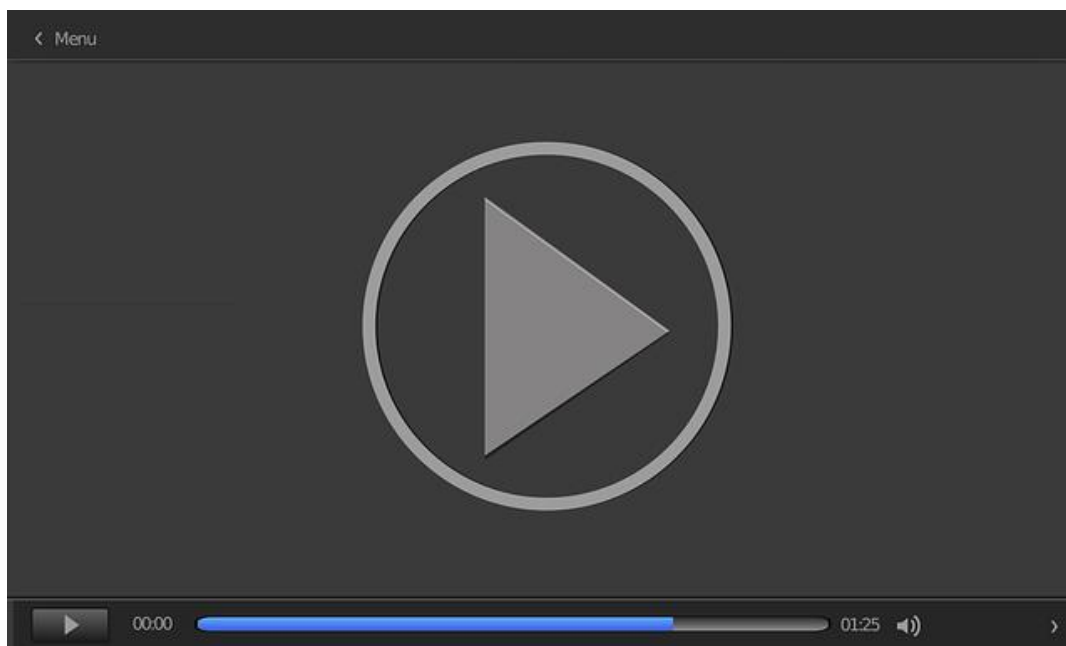


Figura 7. Creación de un trabajo con AWS Glue Studio. Fuente: elaboración propia.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

El siguiente vídeo, *AWS Glue Studio - Visual data pipeline demo* | Amazon Web Services (Amazon Web Services, 2023a), muestra como realizar un *pipeline* de ETL con AWS Studio.



AWS Glue Studio - Visual data pipeline demo | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/ckxwnd4BQmk>

AWS Glue Studio permite crear trabajos de forma interactiva en una interfaz de cuaderno basada en **cuadernos o notebooks de Jupyter**. A través de los cuadernos en AWS Glue Studio, es posible editar *scripts* de trabajos y el código de integración de datos y ver el resultado sin que sea necesario ejecutar un trabajo completo. También es posible agregar un marcado y guardar cuaderno como archivos `.ipynb` y *scripts* de trabajo. Puede iniciar un cuaderno sin instalar ningún *software* en forma local ni administrar servidores. Una vez que esté satisfecho con el código, AWS Glue Studio puede convertir el cuaderno en un trabajo de Glue con solo hacer clic en un botón.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Algunos de los **beneficios** de utilizar los cuadernos son los siguientes:

- ▶ No hay clúster que aprovisionar o administrar.
- ▶ No hay que pagar por clústeres inactivos.
- ▶ No se requiere una configuración inicial.
- ▶ No se requiere instalación de *notebooks* de Jupyter.
- ▶ Mismo tiempo de ejecución y plataforma que ETL de AWS Glue.

Al iniciar un cuaderno a través de AWS Glue Studio, todos los **pasos de configuración** ya han sido completados para que, apenas después de unos segundos, pueda explorar los datos y comenzar a desarrollar el *script* de trabajo. AWS Glue Studio configura un cuaderno de Jupyter con el *kernel* de Jupyter de AWS Glue. No es necesario configurar VPC (nube privada virtual), conexiones de red ni puntos de conexión de desarrollo para utilizar este cuaderno.

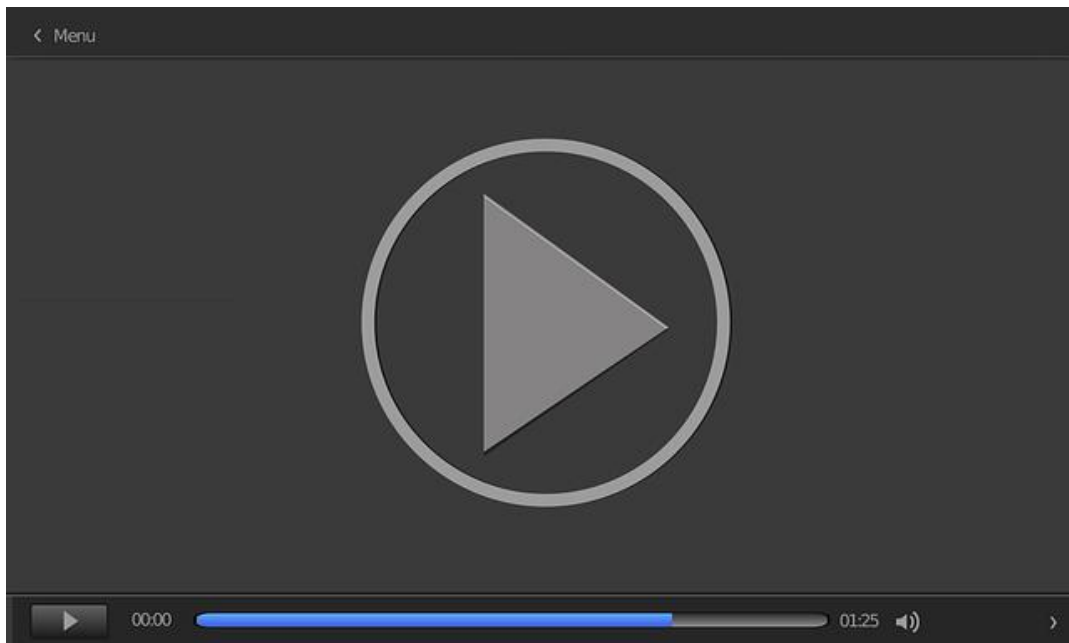
Para crear **trabajos** mediante la interfaz de cuaderno, se deben seguir los siguientes pasos:

- ▶ Configure los permisos de IAM (administración de identidad y acceso) necesarios.
- ▶ Inicie una sesión de cuaderno para crear un trabajo.
- ▶ Escriba código en las celdas en el cuaderno.
- ▶ Ejecute y pruebe el código para ver el resultado.
- ▶ Guarde el trabajo.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Una vez guardado, el cuaderno es un **trabajo completo** de AWS Glue. Puede administrar todos los aspectos del trabajo, tales como la programación de ejecuciones de trabajos, la configuración de parámetros del trabajo y la visualización del historial de ejecuciones de trabajos justo al lado del cuaderno.

En el vídeo *AWS Tutorials - Interactively Develop Glue Job using Jupyter Notebook* (AWS Tutorials, 2022), se profundiza sobre los pasos de creación y ejecución de un *notebook* con AWS Glue.



AWS Tutorials - Interactively Develop Glue Job using Jupyter Notebook.

Accede al vídeo:

<https://www.youtube.com/embed/ckxwnd4BQmk>

AWS Glue proporciona **motores de integración de datos** que son fundamentales para el procesamiento y la transformación de datos en la plataforma de AWS:

- ▶ El motor de Apache Spark.
- ▶ AWS Glue para Ray.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ AWS Glue para Python Shell.

En el caso de **AWS Glue con motor de Apache Spark**, AWS Glue brinda una infraestructura optimizada para el rendimiento y sin servidor destinada a la ejecución de Apache Spark para llevar a cabo trabajos de integración y extracción, transformación y carga (ETL) de datos. AWS Glue para Apache Spark admite el procesamiento de lotes y transmisiones, y acelera la ingesta, el procesamiento y la integración de datos. Puede crear y actualizar su lago de datos y el almacenamiento de datos, y extraer información de los datos con más rapidez.

Las **características clave** de este motor son las siguientes:

- ▶ **Procesamiento distribuido.** Apache Spark es un motor de procesamiento de datos distribuido que permite procesar grandes volúmenes de datos de manera eficiente y escalable.
- ▶ **Lenguajes de programación.** Soporta varios lenguajes de programación, incluidos Python, Scala, Java y SQL, lo que les permite a los usuarios implementar lógica de transformación compleja utilizando el lenguaje de su elección.
- ▶ **Optimización de consultas.** Utiliza un optimizador de consultas avanzado que mejora el rendimiento de las operaciones de ETL al planificar y ejecutar consultas de manera eficiente.
- ▶ **Capacidad de paralelización.** Permite la ejecución de operaciones en paralelo en clústeres de computación distribuida, lo que acelera el procesamiento de datos y reduce los tiempos de ejecución.
- ▶ **Integración con servicios de AWS.** Se integra estrechamente con otros servicios de AWS, como S3, Redshift, RDS, y más, lo que facilita el acceso a datos y la realización de análisis avanzados.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Por otro lado, **AWS Glue para Ray** es una opción de motor de integración de datos en AWS Glue, ya está disponible para el público en general. AWS Glue para Ray ayuda a los ingenieros de datos y a los desarrolladores de ETL a escalar sus trabajos de Python. AWS Glue es un servicio de integración de datos escalable y sin servidor que se utiliza para descubrir, preparar, trasladar e integrar datos de varias fuentes. AWS Glue para Ray combina esa capacidad sin servidor para la integración de datos con Ray (ray.io), un nuevo y conocido marco informático de código abierto que lo ayuda a escalar las cargas de trabajo de Python.

Al igual que en los motores Apache Spark y Python de AWS Glue, solo paga por los **recursos** que utiliza al ejecutar el código y no necesita configurar ni ajustar los recursos. AWS Glue para Ray facilita el **procesamiento** distribuido de su código Python en clústeres de varios nodos. Puede crear y ejecutar trabajos de Ray en cualquier lugar donde pueda ejecutar trabajos de ETL de AWS Glue. Esto incluye trabajos existentes de AWS Glue, interfaces de línea de comandos (CLI) y API. Puede seleccionar el motor AWS Glue para Ray de forma local o a través de cuadernos en AWS Glue Studio y el cuaderno de Amazon SageMaker Studio. Cuando el trabajo de Ray esté listo, puede ejecutarlo bajo demanda o según un cronograma.

Para trabajar con AWS Glue para Ray, se utilizan los mismos trabajos de AWS Glue y las mismas sesiones interactivas que se usan con AWS Glue para Spark. Los **trabajos** de AWS Glue están diseñados para ejecutar el mismo *script* de forma periódica, mientras que las **sesiones** interactivas están diseñadas para permitir ejecutar fragmentos de código de forma secuencial con los mismos recursos aprovisionados.

En la página «Trabajos» de la consola de AWS Glue Studio, se puede seleccionar una nueva opción al crear un trabajo en el editor de scripts AWS Glue Studio Ray tal como podemos ver en la Figura 8.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

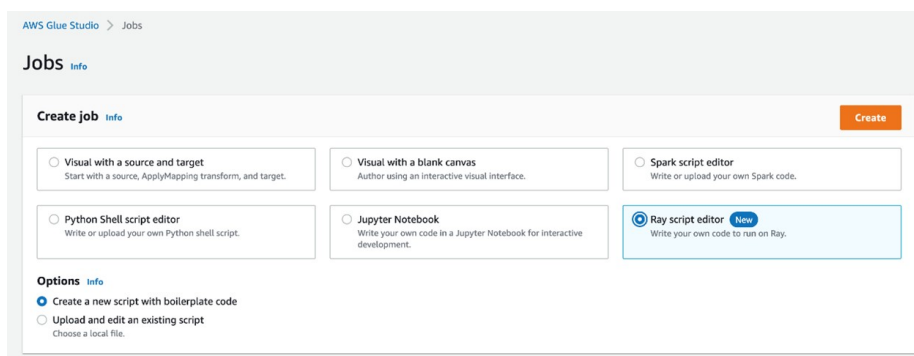


Figura 8. Crear un trabajo en AWS Glue Studio. Fuente: elaboración propia.

Por último, con **AWS Glue para Python Shell**, se puede utilizar un trabajo de Python Shell para ejecutar *scripts* de Python en AWS Glue. Mediante estos trabajos, es posible escribir trabajos de análisis y de integración de datos complejos en Python. Los trabajos de AWS Glue para Python Shell ahora ofrecen bibliotecas de análisis comunes listas para usar, incluidas Pandas, NumPy y Amazon SageMaker Data Wrangler. Puede usar la funcionalidad integrada para conectarse a una gran variedad de bases de datos, almacenamientos de datos y servicios de AWS.

Un **trabajo de shell** de Python puede usarse para ejecutar *scripts* de Python como un *shell* en AWS Glue. Con un trabajo de *shell* de Python puede ejecutar *scripts* compatibles con Python 2.7, Python 3.6 o Python 3.9.

Existen dos **alternativas** para utilizar AWS Glue Python para *shell*: con AWS Glue Studio o mediante el CLI de AWS. A continuación, se explica el funcionamiento de ambas.

Al definir su **trabajo de shell de Python en AWS Glue Studio**, debe proporcionar algunas de las siguientes propiedades:

- **Rol de IAM.**

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Especificación del rol de AWS IAM**, que se usa para dar una autorización a los recursos que se utilizan para ejecutar el flujo de trabajo y obtener acceso a los almacenes de datos. Para obtener más información acerca de los permisos para ejecutar trabajos en AWS Glue, puedes consultar la administración de identidades y accesos para AWS Glue.
- ▶ **Tipo**. Elija Python *shell* (*shell* de Python) para ejecutar un *script* de Python con el comando de trabajo llamado `pythonshell`.
- ▶ **Versión de Python**. Elija la versión de Python. El valor por defecto es Python 3.9. Las versiones válidas son Python 3.6 y Python 3.9.
- ▶ **Cargar bibliotecas de análisis comunes** (recomendado). Elija esta opción para incluir bibliotecas comunes para Python 3.9 en el *shell* de Python. Si sus bibliotecas son personalizadas o entran en conflicto con las preinstaladas, puede optar por no instalar bibliotecas comunes. Sin embargo, puede instalar bibliotecas adicionales además de las bibliotecas comunes. Si selecciona esta opción, la opción «library-set» se establece en «analytics». Al anular la selección de esta opción, la opción «library-set» se establece en «none».
- ▶ **Nombre de archivo de *script* y ruta de *script***. El código del *script* define la lógica de procedimiento del trabajo. Proporcione el nombre del *script* y la ubicación en Amazon S3. Compruebe que no haya un archivo con el mismo nombre que el directorio de *script* en la ruta. Para obtener más información acerca de cómo usar *scripts*, puedes consultar la guía de programación de AWS Glue.
- ▶ **Script**. El código del *script* define la lógica de procedimiento del trabajo. Puede codificar el *script* en Python 3.6 o Python 3.9. Puedes editar *scripts* en AWS Glue Studio.
- ▶ **Unidades de procesamiento de datos**. Es el número máximo de unidades de procesamiento de datos (DPU) de AWS Glue que se pueden asignar cuando se ejecute este trabajo. Una DPU es una medida relativa de la potencia de

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

procesamiento que consta de 4 vCPU de capacidad de cómputo y 16 GB de memoria —para más información, puedes consultar los precios de AWS Glue—.

Puede establecer el valor en 0,0625 o 1. El valor predeterminado es 0,0625. En cualquier caso, el disco local para la instancia será de 20 GB.

Respecto al **uso de CLI en AWS Glue para Python Shell**, también se puede crear un trabajo de *shell* de Python con la AWS CLI, como en el siguiente ejemplo:

```
aws glue create-job --name python-job-cli --role Glue_DefaultRole
--command '{"Name" : "pythonshell", "PythonVersion": "3.9",
"ScriptLocation" : "s3://DOC-EXAMPLE-BUCKET/scriptname.py"}'
--max-capacity 0.0625
```

Orquestación en AWS Glue

En AWS Glue, puedes iniciar trabajos (*jobs*) y *crawlers* mediante desencadenadores. Los **desencadenadores** son eventos que activan la ejecución de un trabajo o un *crawler* en respuesta a ciertas condiciones predefinidas. A continuación, tienes una descripción del funcionamiento para iniciar trabajos y *crawlers* en AWS Glue mediante desencadenadores:

► Desencadenadores de AWS Glue:

- **Desencadenadores de trabajo** (*job triggers*). Se utilizan para iniciar la ejecución de un trabajo de AWS Glue en respuesta a un evento específico. Puedes configurar desencadenadores para que se activen en horarios programados, en función de cambios en los datos, o en respuesta a eventos externos.
- **Desencadenadores de *crawler*** (*crawler triggers*). Se utilizan para iniciar la ejecución de un *crawler* de AWS Glue en respuesta a un evento específico. Puedes configurar desencadenadores para que se activen en horarios programados, en función de cambios en los metadatos de los datos, o en respuesta a eventos externos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

► Configuración de desencadenadores:

- **Horarios programados.** Puedes configurar desencadenadores para que se activen en horarios programados, como cada día a una hora específica o cada hora. Esto te permite automatizar la ejecución de trabajos y *crawlers* en intervalos regulares.
- **Eventos de cambio de datos.** Los desencadenadores también se pueden configurar para que se activen en respuesta a cambios en los datos. Por ejemplo, puedes configurar un desencadenador para que inicie un trabajo cada vez que se agreguen nuevos datos a un *bucket* de Amazon S3.
- **Eventos externos.** Además, puedes configurar desencadenadores para que se activen en respuesta a eventos externos, como notificaciones de Amazon CloudWatch, cambios en un repositorio de código fuente o mensajes en una cola de Amazon SQS.

► Proceso de ejecución.

Una vez que se activa un desencadenador, AWS Glue inicia la ejecución del trabajo o *crawler* asociado. Durante la ejecución, AWS Glue provisiona automáticamente la infraestructura necesaria, como instancias de procesamiento y almacenamiento, para realizar las tareas especificadas en el trabajo o *crawler*.

► Monitoreo y gestión.

AWS Glue proporciona herramientas integradas para monitorear y gestionar la ejecución de trabajos y *crawlers*. Puedes ver el estado de ejecución en tiempo real, monitorear el progreso, detectar y solucionar problemas, y optimizar el rendimiento según sea necesario.

► Gestión de versiones y auditoría.

AWS Glue también proporciona capacidades para gestionar versiones de desencadenadores y realizar auditorías de ejecuciones anteriores. Puedes ver un historial de ejecuciones, revisar los registros de actividad y realizar un seguimiento de los cambios en la configuración de los desencadenadores a lo largo del tiempo.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En resumen, los **desencadenadores** en AWS Glue te permiten automatizar la ejecución de trabajos y *crawlers* en respuesta a eventos específicos, lo que simplifica la administración y la ejecución de tareas de procesamiento y preparación de datos en la nube.

Uso de flujos de trabajo en AWS Glue

En AWS Glue se pueden utilizar **flujos de trabajo** para crear y visualizar actividades de ETL complejas que implican varios rastreadores, trabajos y desencadenadores. Cada flujo de trabajo administra la ejecución y el monitoreo de todos sus trabajos y rastreadores. Un flujo de trabajo registra el progreso de ejecución y el estado, ya que ejecuta cada componente. Esto le proporciona información general de la tarea de mayor envergadura y los detalles de cada paso. La consola de AWS Glue ofrece una representación visual de un flujo de trabajo en forma de gráfico.

Propiedades de ejecución del flujo de trabajo

Para compartir y administrar el estado en toda la ejecución de flujo de trabajo, se pueden definir **propiedades de ejecución** de flujo de trabajo predeterminadas. Estas propiedades, que son pares nombre-valor, están disponibles para todos los trabajos en el flujo de trabajo. Puedes utilizar la AWS Glue API para recuperar las propiedades de ejecución de flujo de trabajo y es necesario modificarlas para los trabajos que vengan después en el flujo de trabajo.

Gráfico del flujo de trabajo

La Figura 9 muestra el gráfico de un flujo de trabajo básico en la consola de AWS Glue. Su flujo de trabajo podría tener decenas de componentes.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

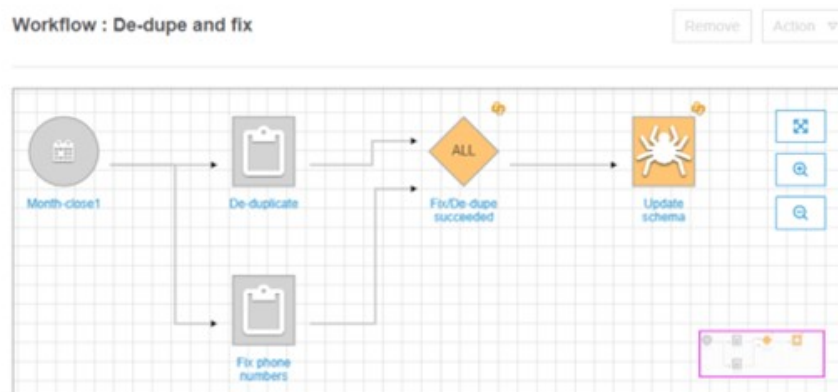


Figura 9. Flujo de trabajo. Fuente: Amazon Web Service, s. f.-b.

Este **flujo de trabajo** se inicia mediante un desencadenador de programación, Month-close1, que inicia dos trabajos, De-duplicate y Fix phone numbers. Tras la realización completa de ambos trabajos, un desencadenador de eventos, Fix/De-dupe succeeded, inicia un rastreador, Update schema.

Configurar e implementar un flujo de trabajo en Glue

Se puede usar la consola de AWS Glue para crear y construir un flujo de trabajo y un nodo a la vez y de forma manual. Un flujo de trabajo contiene trabajos, rastreadores y desencadenadores. Antes de crear un **flujo de trabajo** de forma manual, hay que crear los trabajos y los rastreadores que este vaya a incluir. Es mejor especificar los rastreadores de ejecución bajo demanda para los flujos de trabajo. Se pueden crear nuevos desencadenadores mientras se desarrolla su flujo de trabajo o puede clonar desencadenadores existentes en el flujo de trabajo. Al clonar un **desencadenador**, se agregan al flujo de trabajo todos los objetos del catálogo asociados a este (los trabajos o los rastreadores que lo activan y los trabajos o los rastreadores que lo inician).

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Se puede diseñar un propio flujo de trabajo añadiendo desencadenadores al gráfico de flujo de trabajo y definiendo los eventos vistos y las acciones para cada desencadenador. Puedes comenzar con un **desencadenador de arranque**, que puede ser un desencadenador bajo demanda o programado, y completar el gráfico añadiendo desencadenadores de eventos (condicional). Los pasos se desarrollarán a continuación.

Paso 1. Crear el flujo de trabajo

- ▶ Inicia sesión en la AWS Management Console y abre la consola de AWS Glue en <https://console.aws.amazon.com/glue/>. En el panel de navegación, en ETL, elige «Workflows» ('flujos de trabajo').
- ▶ Selecciona «Add workflow» ('añadir flujo de trabajo') y completa el formulario «Add a new ETL workflow» ('añadir un nuevo flujo de trabajo de ETL'). Las propiedades de ejecución predeterminadas opcionales que añadas estarán disponibles como argumentos para todos los trabajos en el flujo de trabajo. Para obtener más información, consulta obtención y configuración de propiedades de ejecución de flujo de trabajo en AWS Glue.
- ▶ Selecciona «Add workflow». El nuevo flujo de trabajo aparecerá en la lista en la página «Workflows».

Paso 2. Añadir un desencadenador de arranque

- ▶ En la página «Workflows», selecciona el nuevo flujo de trabajo. A continuación, en la parte inferior de la página, asegúrate de seleccionar la pestaña «Graph» ('gráfico').
- ▶ Selecciona «Add trigger» ('añadir desencadenador') y en el cuadro de diálogo «Add trigger», realiza uno de los procedimientos detallados en la Tabla 2.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Procedimientos disponibles		
A	Elige «Clone existing» ('clonación existente') y seleccione un desencadenador a clonar. A continuación, elige «Add» ('añadir').	El desencadenador aparece en el gráfico, junto con los trabajos y los rastreadores que ve e inicia.
		Si has seleccionado por error el desencadenador erróneo, selecciona el desencadenador en el gráfico y, a continuación, elige «Remove» ('eliminar').
B	Elige «Add new» ('añadir nuevo') y completa el formulario «Add trigger».	Para «Trigger type» ('tipo de desencadenador'), selecciona «Schedule» ('programación'), «On demand» ('bajo demanda') o «EventBridge event» ('evento de EventBridge'). ▶ Para el tipo de desencadenador «Schedule», elige una de las opciones en «Frequency» ('frecuencia'). Selecciona «Custom» ('personalizado') e ingresa una expresión cron. ▶ Para el tipo de desencadenador «EventBridge», ingresa «Number of events» ('número de eventos') — tamaño del lote —, y, opcionalmente, ingresa «Time delay» ('tiempo de retraso') —ventana por lotes—. Si omite «Time delay», la ventana por lotes tiene un valor predeterminado de quince minutos.
C	Elige «Add». El desencadenador aparece en el gráfico junto con un nodo de marcador de posición con la etiqueta «Add node» ('añadir nodo'). En el ejemplo de a continuación, el desencadenador de inicio es un desencadenador de programación denominado Month-close1. En este momento, el desencadenador no está guardado aún.	---

Tabla 2. Procedimientos. Fuente: elaboración propia.

Un gráfico con dos nodos rectangulares: un desencadenador y un nodo de marcador de posición. Una flecha señala desde el nodo del desencadenador hasta el nodo de marcador de posición.

Si has añadido un nuevo desencadenador, realiza una de las **acciones** siguientes:

- ▶ Elige el nodo del marcador de posición «Add node».

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ Asegúrate de que el desencadenador de arranque esté seleccionado y en el menú «Action» ('acción') de arriba del gráfico, elige «Add jobs/crawlers to trigger» ('añadir trabajos/rastreadores al desencadenador').
- ▶ En el cuadro de diálogo «Add job(s) and crawler(s) to trigger» ('añadir trabajos y rastreadores al desencadenador'), selecciona uno o más trabajos, o desencadenadores y, a continuación, elige «Add».

El **desencadenador** se guarda y los trabajos y los rastreadores seleccionados aparecen en el gráfico con los conectores del desencadenador. Si has añadido por error los trabajos y los rastreadores erróneos, puedes seleccionar el desencadenador o un conector y elegir «Remove».

Paso 3. Agregar más desencadenadores

Continúa construyendo su flujo de trabajo al agregar más **desencadenadores** de tipo «Event» ('evento'). Para ampliar o reducir la imagen, o para agrandar el lienzo del gráfico, utiliza los íconos a la derecha del gráfico. Para agregar los desencadenadores, realice los pasos que se mencionan a continuación. En primer lugar, realice una de las acciones siguientes:

- ▶ Para clonar un desencadenador existente, asegúrese de que no se selecciona ningún nodo en el gráfico y, en el menú «Action» elige «Add trigger».
- ▶ Para añadir un nuevo desencadenador que vea un trabajo o rastreador determinado en el gráfico, selecciona el nodo de trabajo o rastreador y, a continuación, elige el nodo de marcador de posición «Add trigger».

Puedes añadir más trabajos o rastreadores para ver este desencadenador en un paso posterior.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En el cuadro de diálogo «Add trigger», realiza una de las siguientes acciones:

- ▶ Elige «Add new» y completa el formulario «Add trigger». A continuación, elige «Add».
- El desencadenador aparecerá en el gráfico. Se completará el desencadenador en un paso posterior.
- ▶ Elige «Clone existing» ('clonación existente') y selecciona un desencadenador a clonar. A continuación, elige «Add».
- El desencadenador aparecerá en el gráfico, junto con los trabajos y los rastreadores que ve e inicia.
- ▶ Si has elegido por error el desencadenador erróneo, selecciona el desencadenador en el gráfico y, a continuación, elige «Remove».

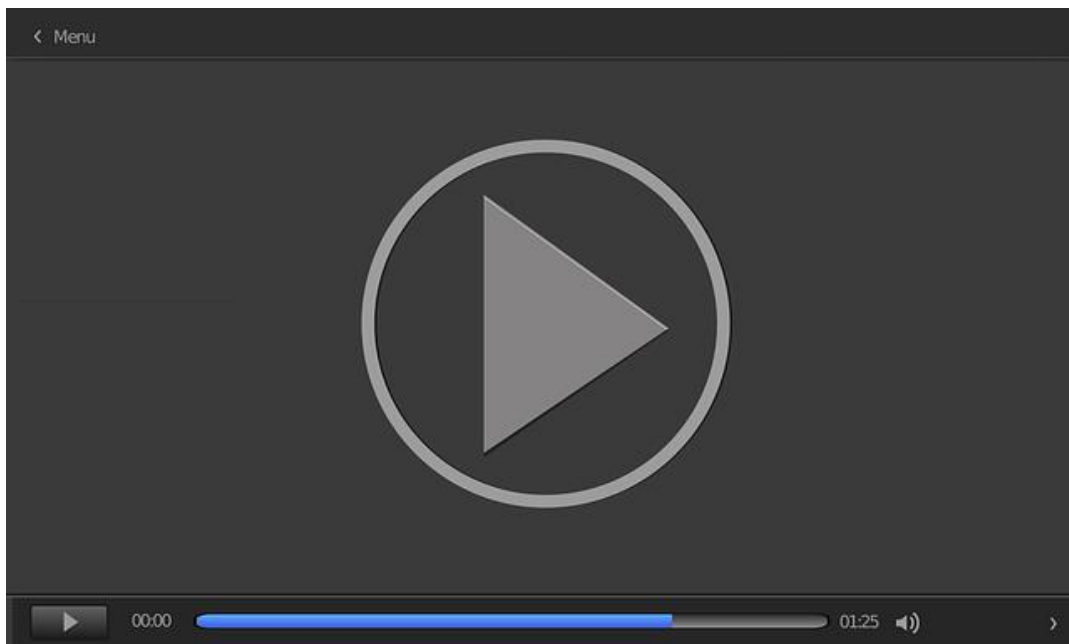
Si ha añadido un nuevo desencadenador, complete estos pasos:

- ▶ **Seleccione el nuevo desencadenador.** Se selecciona el desencadenador «De-dupe/fix succeeded», y los nodos de marcador de posición aparecen para eventos para ver (número uno) y acciones (número dos). Resulta en un gráfico con muchos nodos, dos de los cuales son nodos de marcador de posición, que se denominan números uno y dos.
- ▶ Este paso es **opcional** si el desencadenador ya ve un evento y desea añadir más trabajos o rastreadores que ver. Elige el nodo de marcador de posición de eventos que ver y, en el cuadro de diálogo «Add job(s) and crawler(s) to watch», selecciona uno o más trabajos o rastreadores. Elige un evento que ver («SUCCEEDED» —'correcto'—, «FAILED» —'erróneo'—, etc.) y haz clic en «Add».
- Asegúrate de que se seleccione el desencadenador y elige el nodo de marcador de posición de acciones.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ En el cuadro de diálogo «Add job(s) and crawler(s) to watch», selecciona uno o más trabajos o desencadenadores y, a continuación, elige «Add».
- Los trabajos y los rastreadores seleccionados aparecerán en el gráfico con los conectores del desencadenador.

A continuación, en el vídeo *Automated ETL Workflow Orchestration with AWS Glue, Athena, Lambda, EventBridge, and Step Functions* (Wall Street Mindset, 2024), se muestra un ejemplo de orquestación con AWS Glue, más otros servicios de AWS.



Automated ETL Workflow Orchestration with AWS Glue, Athena, Lambda, EventBridge, and Step Functions.

Accede al vídeo:

<https://www.youtube.com/embed/Olt0XekklhU>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Integración con otros servicios de AWS

Glue se integra estrechamente con otros servicios de AWS, como S3, Redshift y Athena, lo que facilita la carga de datos en almacenes de datos y la realización de análisis avanzados. Esta integración permite que los usuarios aprovechen una amplia gama de herramientas y servicios de AWS para el **análisis de datos**, lo que les permite obtener *insights* valiosos de sus datos de manera eficiente y escalable.

En resumen, estos son los principales **servicios de AWS Glue** que proporcionan una plataforma completa y escalable para la preparación y el procesamiento de datos en la nube. Con sus capacidades de descubrimiento automático de esquemas, transformaciones ETL flexibles, programación de trabajos automatizada y sólida integración con otros servicios de AWS, Glue les permite a las organizaciones acelerar el tiempo de obtención de información y obtener *insights* valiosos de sus datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

2.3. Ingesta de datos en streaming

La **ingesta de datos** en *streaming* es el proceso de capturar, procesar y almacenar datos en tiempo real a medida que se generan. A diferencia de la ingesta de datos tradicional, que puede implicar la recopilación y el procesamiento de grandes volúmenes de datos estáticos, la ingesta de datos en *streaming* se centra en el flujo continuo de datos que proviene de diversas fuentes, como sensores, dispositivos IoT, aplicaciones web, redes sociales y sistemas de monitoreo, como podemos ver en la Figura 10.



Figura 10. Fuentes. Fuente: Qlik, s. f.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Componentes de la ingesta de datos en *streaming*

Los principales componentes de la ingesta de datos en *streaming* son los siguientes:

- ▶ **Fuente de datos.** La fuente de datos es el origen de los datos en *streaming*. Puede ser cualquier dispositivo o sistema que genere datos en tiempo real, como sensores, dispositivos IoT, aplicaciones móviles, redes sociales, sistemas de monitoreo de servidores, entre otros.
- ▶ **Broker de mensajes.** El *broker* de mensajes es un componente clave en la ingesta de datos en *streaming*. Actúa como intermediario entre las fuentes de datos y los consumidores de datos, recibiendo los datos entrantes y distribuyéndolos a los procesadores o los sistemas de almacenamiento correspondientes.
- ▶ **Procesamiento de datos.** Los datos en *streaming* suelen ser procesados en tiempo real para realizar diversas acciones, como filtrado, transformación, enriquecimiento, agregación o análisis. Esto puede involucrar el uso de herramientas y plataformas de procesamiento de datos en *streaming*, como Apache Kafka, Apache Flink, Apache Storm, AWS Kinesis, o Google Cloud Dataflow.
- ▶ **Almacenamiento de datos.** Los datos procesados en *streaming* pueden ser almacenados en sistemas de almacenamiento de datos en tiempo real o en almacenes de datos tradicionales. Esto permite que los datos estén disponibles para consultas y análisis posteriores. Algunas opciones comunes de almacenamiento incluyen Apache Cassandra, Amazon S3, Google Cloud Storage, y bases de datos en memoria, como Redis o Memcached.
- ▶ **Consumidores de datos.** Los consumidores de datos son los sistemas o las aplicaciones que utilizan los datos procesados en *streaming* para realizar acciones específicas, como generar informes, activar alertas, tomar decisiones en tiempo real o alimentar modelos de aprendizaje automático. Estos consumidores pueden ser aplicaciones web, servicios de análisis, sistemas de monitoreo o cualquier otra entidad que necesite acceso a los datos en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Desafíos de la ingesta de datos en streaming

- ▶ **Latencia.** Reducir la latencia en la ingesta de datos es fundamental para garantizar que los datos estén disponibles para procesamiento y análisis en tiempo real.
- ▶ **Escalabilidad.** Los sistemas de ingesta de datos en *streaming* deben ser capaces de escalar para manejar grandes volúmenes de datos y picos de tráfico.
- ▶ **Tolerancia a fallos.** Es importante diseñar sistemas de ingesta de datos en *streaming* que sean robustos y tolerantes a fallos, para garantizar la disponibilidad y la integridad de los datos.
- ▶ **Integridad de los datos.** Mantener la integridad de los datos en un entorno de *streaming* puede ser un desafío debido a la naturaleza dinámica y continua de los datos.
- ▶ **Seguridad.** Garantizar la seguridad de los datos en un entorno de *streaming* es fundamental para proteger la privacidad y la confidencialidad de la información.

La Figura 11 muestra un conjunto de bloques de desafíos para tener en cuenta en un proceso en *streaming*.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

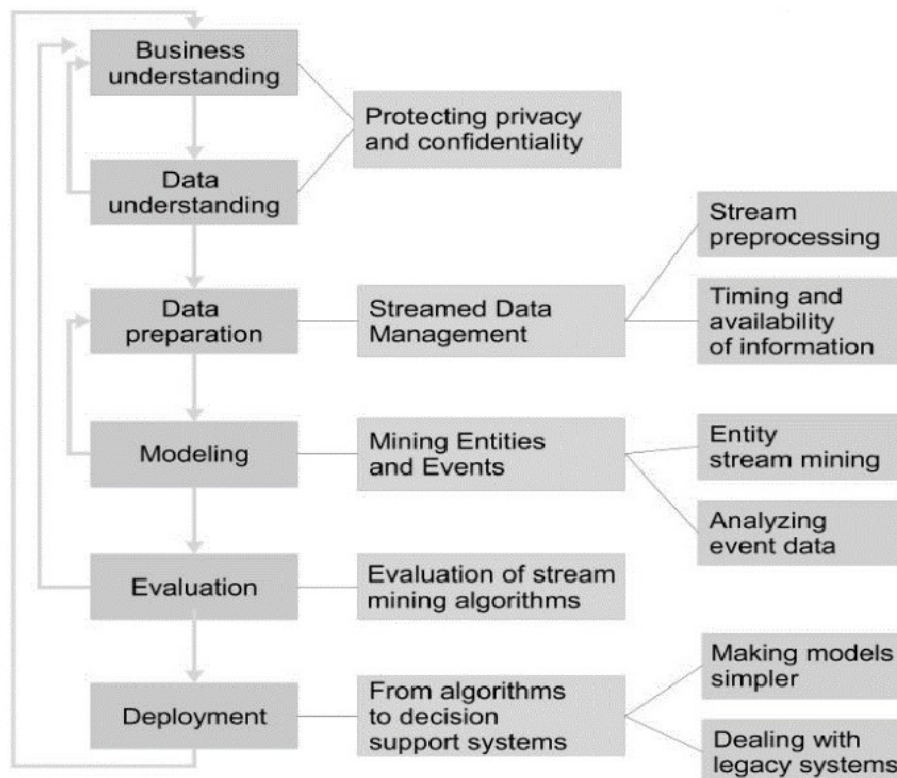


Figura 11. Conjunto de bloques de desafíos. Fuente: Saad et al., 2021.

En resumen, la **ingesta de datos en streaming** es un componente fundamental en los sistemas modernos de análisis de datos en tiempo real. Permite a las organizaciones capturar, procesar y actuar sobre los datos en tiempo real, lo que les permite tomar decisiones más informadas y rápidas en un mundo cada vez más conectado y dinámico.

¿Qué son los datos de transmisión?

Los **datos de streaming** son datos emitidos a un alto volumen de manera gradual y continua con el objetivo de lograr un procesamiento de baja latencia. Las organizaciones poseen cientos de orígenes de datos que suelen emitir simultáneamente mensajes, registros o datos, con un tamaño que va desde unos pocos bytes a varios megabytes (MB).

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Los datos de *streaming* incluyen **datos de sensor, eventos y ubicación** que las empresas usan para lograr un análisis en tiempo real y la visibilidad de muchos aspectos de su negocio. Por ejemplo, las empresas pueden rastrear cambios en la opinión pública sobre sus marcas y productos al analizar de manera continua las secuencias de clic y las publicaciones de clientes en flujos de redes sociales y responder rápidamente según sea necesario.

¿Cuáles son las características de los datos de *streaming*?

Un flujo de datos posee las siguientes características específicas que lo definen:

- ▶ **Importante a nivel cronológico.** Los elementos individuales en una secuencia de datos contienen marcas de tiempo. La secuencia de datos en sí misma puede ser urgente, con una importancia disminuida después de un intervalo de tiempo específico. Por ejemplo, su aplicación realiza recomendaciones de restaurantes de acuerdo con la ubicación actual del usuario. Tiene que responder a los datos de geolocalización del usuario en tiempo real o los datos perderán importancia.
- ▶ **De flujo continuo.** Una secuencia de datos no tiene principio ni final. Recopila datos de manera continua y constante mientras sea necesario. Por ejemplo, los registros de actividad del servidor se acumulan mientras el servidor esté en ejecución.
- ▶ **Única.** Repetir la transmisión de una secuencia de datos es todo un desafío debido a la urgencia. Por lo tanto, el procesamiento de datos preciso y en tiempo real es fundamental. Lamentablemente, las provisiones para la retransmisión son limitadas en la mayoría de los orígenes de datos de *streaming*.
- ▶ **No homogénea.** Algunos orígenes pueden transmitir datos en varios formatos estructurados como JSON, Avro y valores separados por comas (CSV) con tipos de datos que incluyen cadenas, números, fechas y tipos binarios. Sus sistemas de procesamiento de secuencias deben tener las capacidades para manejar tales variaciones en los datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Imperfecta.** Los errores temporales en el origen pueden resultar en elementos dañados o faltantes en los datos transmitidos. Puede ser difícil garantizar la coherencia de datos debido a la naturaleza continua de la transmisión. Los sistemas de análisis y procesamiento de secuencias suelen incluir lógica para la validación de datos a fin de mitigar o minimizar esos errores.

¿Por qué son importantes los datos de *streaming*?

Los sistemas tradicionales de procesamiento de datos capturan datos en un almacenamiento de datos central y los procesan en grupos o lotes. Estos sistemas fueron creados para capturar y estructurar datos antes del análisis. Sin embargo, en los últimos años, la **naturaleza de los datos** empresariales y los **sistemas de procesamiento** de datos subyacentes han cambiado de manera significativa.

- ▶ **Volumen de datos infinito.** Los volúmenes de datos generados de orígenes de secuencias pueden ser muy grandes, lo cual se traduce en un desafío para el análisis en tiempo real: la regulación de la integridad (validación), la estructura (evolución) o la velocidad (rendimiento y latencia) de los datos de *streaming*.
- ▶ **Sistemas de procesamiento de datos avanzados.** Al mismo tiempo, la infraestructura en la nube ha introducido la flexibilidad en la escala y el uso de recursos informáticos. Usa exactamente lo que necesita y paga solo por lo que utiliza. Tiene las opciones de agregación o filtros en tiempo real tanto antes como después de almacenar datos de *streaming*. La arquitectura de datos de *streaming* utiliza tecnologías en la nube para consumir, enriquecer, analizar y almacenar de forma permanente datos de *streaming* según sea necesario.

¿Cuáles son los casos de uso para el *streaming* de datos?

El sistema de procesamiento de secuencias resulta beneficioso en la mayoría de las situaciones en las que se generan **datos nuevos y dinámicos** de forma continua. Aplica a la mayoría de los sectores y casos de uso de macrodatos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Por lo general, las empresas comienzan con aplicaciones sencillas, como la recopilación de registros del sistema y el procesamiento rudimentario, como la implementación de cálculos mínimos y máximos. Más adelante, estas **aplicaciones** evolucionan y se pasa al procesamiento más sofisticado casi en tiempo real.

Otros **ejemplos de datos** de *streaming* serían:

- ▶ **Análisis de datos.** Las aplicaciones procesan secuencias de datos para producir informes y realizar acciones como respuesta, por ejemplo, emitir alertas cuando las medidas clave superen ciertos umbrales. Las aplicaciones de procesamiento de secuencias más sofisticadas extraen información más profunda mediante la aplicación de algoritmos de ML a los datos de actividad de los clientes y la empresa.
- ▶ **Aplicaciones de IoT.** Los dispositivos de IoT son otro caso de uso para los datos de *streaming*. Los sensores en vehículos, equipo industrial y maquinaria agrícola envían datos a una aplicación de *streaming*. La aplicación supervisa el rendimiento, detecta posibles defectos de forma anticipada y envía el pedido de un recambio automáticamente, lo que evita el tiempo de inactividad del equipo.
- ▶ **Análisis financiero.** Las instituciones financieras usan datos de secuencias para controlar los cambios en la bolsa en tiempo real, procesar el valor en riesgo y modificar las carteras automáticamente en función de los cambios en los precios de las acciones. Otro caso de uso financiero es la detección de fraudes en transacciones con tarjetas de crédito mediante la inferencia en tiempo real frente a datos de transacción de *streaming*.
- ▶ **Recomendaciones en tiempo real.** Las aplicaciones de bienes raíces rastrean datos de geolocalización de los dispositivos móviles de los consumidores y realizan recomendaciones en tiempo real de propiedades para visitar. De manera similar, las aplicaciones de consumidores, ventas, alimentos y publicidades pueden integrar recomendaciones en tiempo real para brindar más valor a los clientes.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Garantías de servicio.** Puede implementar el procesamiento de secuencias de datos para rastrear y mantener niveles de servicios en las aplicaciones y el equipo. Por ejemplo, una compañía de energía solar debe mantener constante el suministro de electricidad a sus clientes, ya que si no lo hace se la sanciona. Implementa una aplicación de datos de *streaming* que supervise todos los paneles en el campo y programe el servicio en tiempo real. Por lo tanto, puede minimizar los períodos de bajo rendimiento de cada panel y los pagos de penalidades asociados.
- ▶ **Multimedia y videojuegos.** Los editores de medios transmiten miles de millones de registros de secuencias de clic de sus propiedades en línea, agregan y enriquecen los datos con información demográfica del usuario y optimizan la colocación de contenido. Esto ayuda a los editores a ofrecer al público una experiencia mejor y más relevante. De manera similar, las empresas de videojuegos en línea utilizan el procesamiento de secuencias de eventos para analizar las interacciones entre el jugador y el juego, y ofrecer experiencias dinámicas para involucrar a los jugadores.
- ▶ **Control de riesgos.** Las transmisiones en directo y las plataformas de redes sociales capturan datos del comportamiento del usuario en tiempo real para controlar riesgos en la actividad financiera de los usuarios, como las recargas, los reintegros y las recompensas. Visualizan paneles en tiempo real para ajustar las estrategias de riesgos de manera flexible.

En la Figura 12, podemos ver un *overview* de casos de uso de *streaming*.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

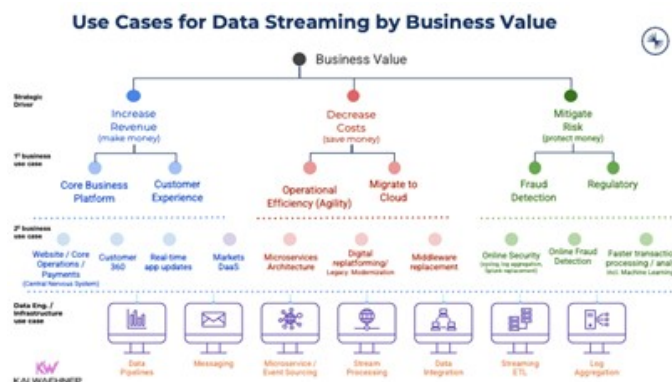


Figura 12. Overview de casos de uso de *streaming*. Fuente: Waehner, 2022.

¿Cuál es la diferencia entre los datos de lotes y los datos de *streaming*?

El **procesamiento por lotes** es el método que utilizan las computadoras para completar trabajos de datos repetitivos y de gran volumen de forma periódica. Puede ser utilizado para calcular consultas arbitrarias en diferentes conjuntos de datos. Por lo general, deriva los resultados informáticos de todos los datos que abarca y permite un análisis profundo de conjuntos de macrodatos. Los sistemas basados en MapReduce, como Amazon EMR, son ejemplos de plataformas compatibles con los trabajos por lotes.

Por el contrario, el **procesamiento de secuencias** requiere capturar una secuencia de datos y actualizar de forma gradual las métricas, los informes y las estadísticas de resumen en respuesta a cada registro de datos que llega. Es más adecuado para las funciones de respuesta y análisis en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

AWS Kinesis

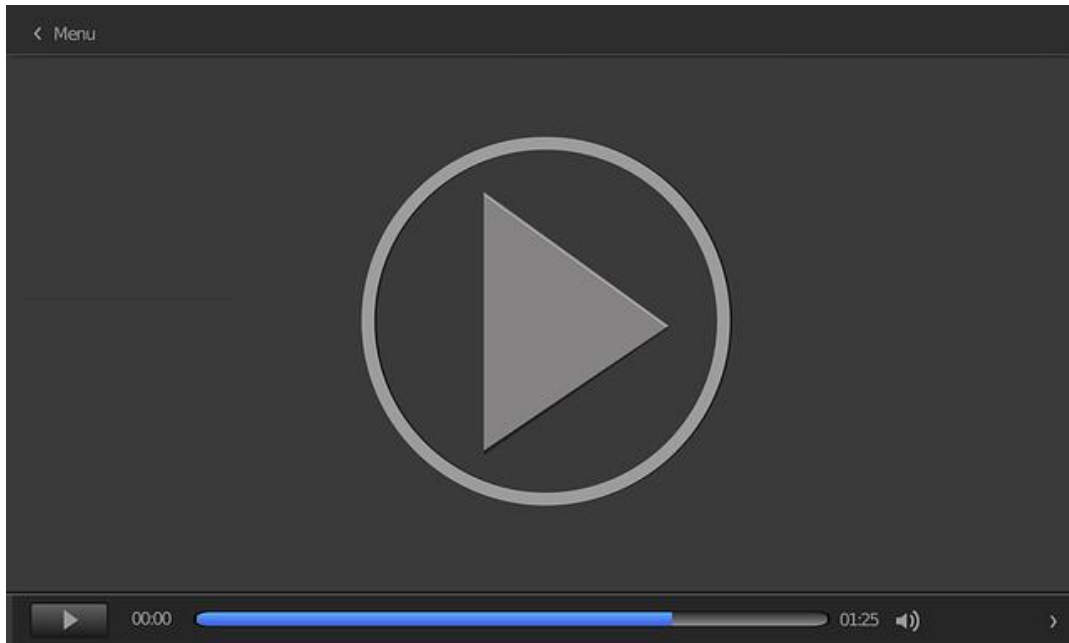
AWS Kinesis es un servicio de *streaming* de datos completamente administrado que permite que las empresas procesen y analicen grandes volúmenes de datos en tiempo real de manera eficiente y escalable. Diseñado para aplicaciones que requieren el procesamiento de datos en tiempo real, Kinesis facilita la ingesta, el almacenamiento, el procesamiento y la entrega de datos en *streaming* de forma rápida y confiable.

A continuación, tienes una descripción más detallada de los **componentes y características** clave de AWS Kinesis:

- ▶ **Kinesis Data Streams.** Kinesis Data Streams es el componente principal de AWS Kinesis. Permite a los usuarios ingresar datos en tiempo real a través de productores (como aplicaciones, dispositivos IoT, servidores de registros, etc.) y luego procesar esos datos utilizando consumidores (aplicaciones, análisis en tiempo real, sistemas de almacenamiento, etc.). Los datos se dividen en fragmentos (*shards*) para permitir la escalabilidad horizontal y la distribución de la carga.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, se muestra el vídeo *Amazon Kinesis Data Streams Fundamentals* (Amazon Web Services, 2020), que es una introducción sobre AWS Kinesis Data Streams.



Amazon Kinesis Data Streams Fundamentals.

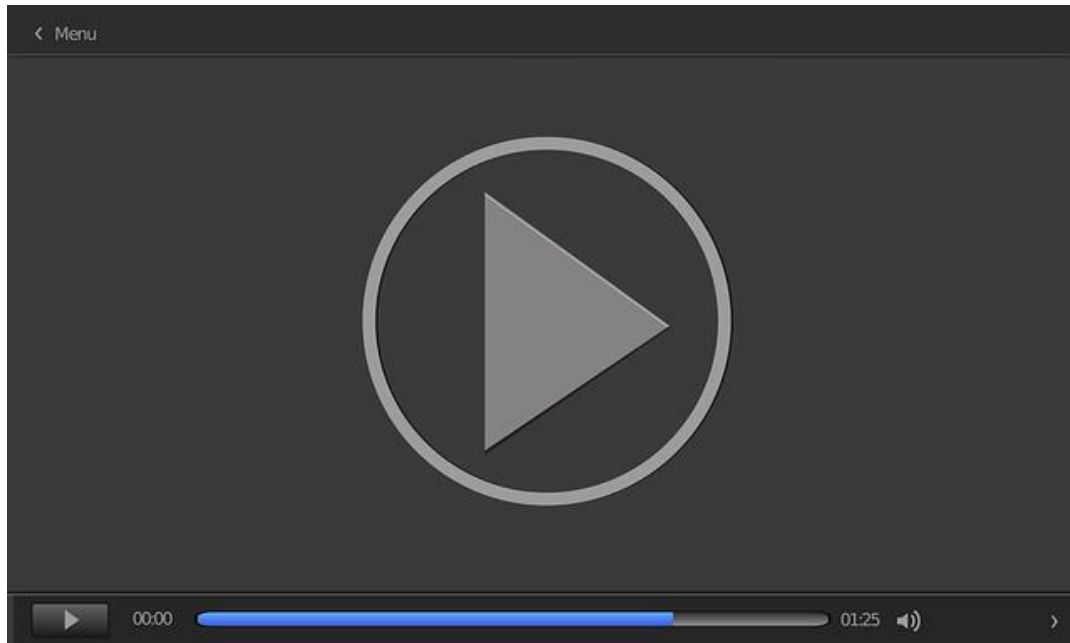
Accede al vídeo:

<https://www.youtube.com/embed/hLLgkTUmwOU>

- **Kinesis Data Firehose.** Kinesis Data Firehose es un servicio que permite la carga directa y continua de datos en Amazon S3, Amazon Redshift, Amazon Elasticsearch Service y Splunk para su posterior análisis. Elimina la complejidad de administrar infraestructura y reduce el esfuerzo requerido para procesar y almacenar datos en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, se muestra el vídeo *Introduction to Kinesis Data Firehose | Amazon Web Services* (Amazon Web Services, 2023b), que es una introducción sobre AWS Kinesis Firehose.



Introduction to Kinesis Data Firehose | Amazon Web Services.

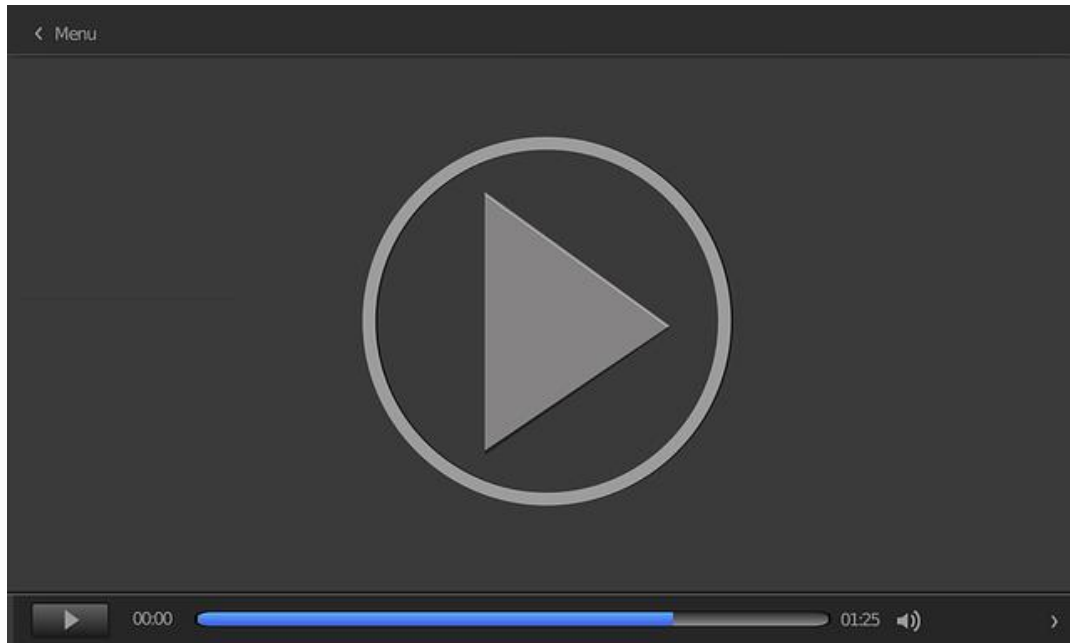
Accede al vídeo:

<https://www.youtube.com/embed/qRoyF9dEqgw>

- **AWS Flink Administrador** (antiguo Kinesis Data Analytics). Kinesis Data Analytics es un servicio que les permite a los usuarios realizar análisis en tiempo real de datos en *streaming* utilizando SQL estándar, Java o Scala a través del poderoso motor de procesamiento de Flink.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, se muestra el vídeo *Introduction to Amazon Managed Service for Apache Flink | Amazon Web Services* (Amazon Web Services, 2023c), es una introducción sobre Amazon Flink Adminnistrador.



Introduction to Amazon Managed Service for Apache Flink | Amazon Web Services.

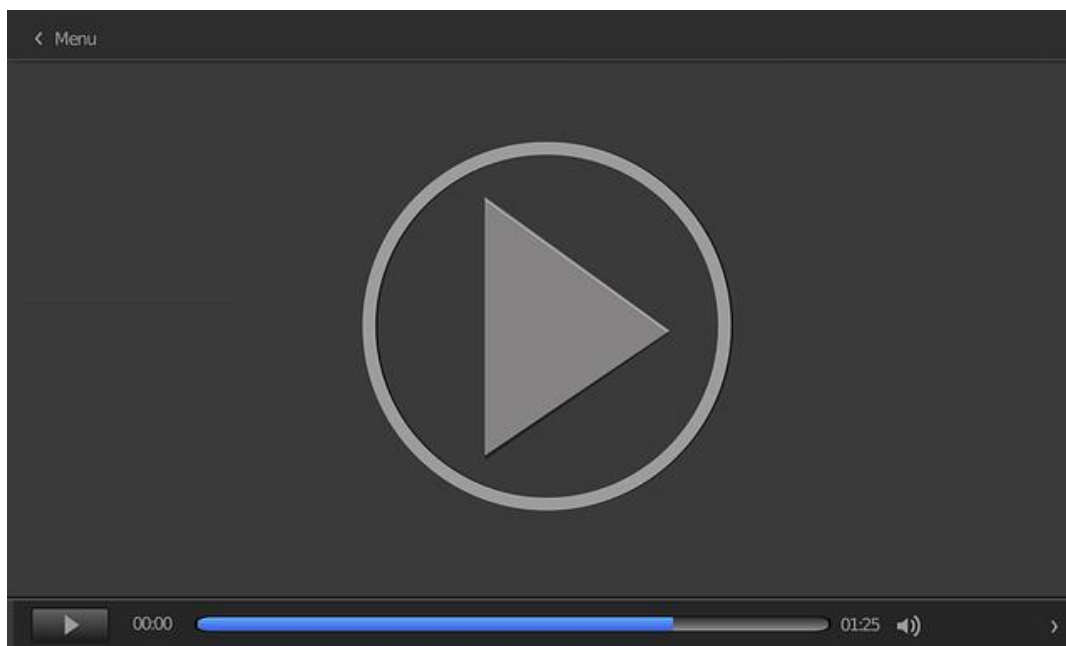
Accede al vídeo:

<https://www.youtube.com/embed/vl1GiMSHuxM>

- **Kinesis Vídeo Streams.** Kinesis Vídeo Streams es un servicio para la captura, procesamiento y almacenamiento de secuencias de vídeo en tiempo real. Permite a los usuarios transmitir secuencias de vídeo a escala desde dispositivos como cámaras IP, drones, cámaras de vigilancia y dispositivos móviles, y, luego, procesar y analizar esas secuencias utilizando otros servicios de AWS.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, se muestra el vídeo *AWS re:Invent 2017 - Introducing Amazon Kinesis Video Streams* (Amazon Web Services, 2023d), que es una introducción sobre AWS Kinesis Vídeo Streams.



AWS re:Invent 2017 - Introducing Amazon Kinesis Vídeo Streams.

Accede al vídeo:

<https://www.youtube.com/embed/STEMa3t5NOQ>

Las **características** clave de AWS Kinesis:

- ▶ **Escalabilidad y durabilidad.** Kinesis Data Streams es altamente escalable y duradero, lo que permite manejar grandes volúmenes de datos en tiempo real y proporcionar una alta disponibilidad y tolerancia a fallos.
- ▶ **Bajo latencia.** Kinesis Data Streams ofrece baja latencia para la ingesta y la entrega de datos, lo que les permite a los usuarios procesar datos en tiempo real y tomar decisiones instantáneas.
- ▶ **Integración con servicios de AWS.** Kinesis se integra estrechamente con otros servicios de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Service y AWS Lambda, lo que facilita el procesamiento y análisis de datos en tiempo real utilizando la infraestructura de AWS.

- ▶ **Facilidad de uso.** Kinesis proporciona una interfaz de usuario intuitiva y API para facilitar la configuración, administración y supervisión de flujos de datos en tiempo real.
- ▶ **Seguridad y cumplimiento normativo.** Kinesis ofrece opciones de seguridad avanzadas, como cifrado de datos en tránsito y en reposo, control de acceso basado en roles (IAM) y cumplimiento de normativas como HIPAA y GDPR.

En resumen, **AWS Kinesis** es una solución completa y escalable para el procesamiento y análisis de datos en tiempo real. Con sus componentes y características clave, les permite a las empresas capturar, procesar y analizar datos en *streaming* de manera eficiente y confiable, lo que les permite tomar decisiones más informadas y rápidas en un mundo cada vez más conectado y dinámico.

Arquitectura de AWS Kinesis

La **arquitectura de AWS Kinesis** está diseñada para facilitar la ingesta, el procesamiento y el análisis de datos en tiempo real a escala. La arquitectura de Kinesis se compone de varios componentes que trabajan juntos para permitir el flujo continuo de datos en *streaming* y su posterior procesamiento. A continuación, tienes una descripción de la arquitectura típica de AWS Kinesis:

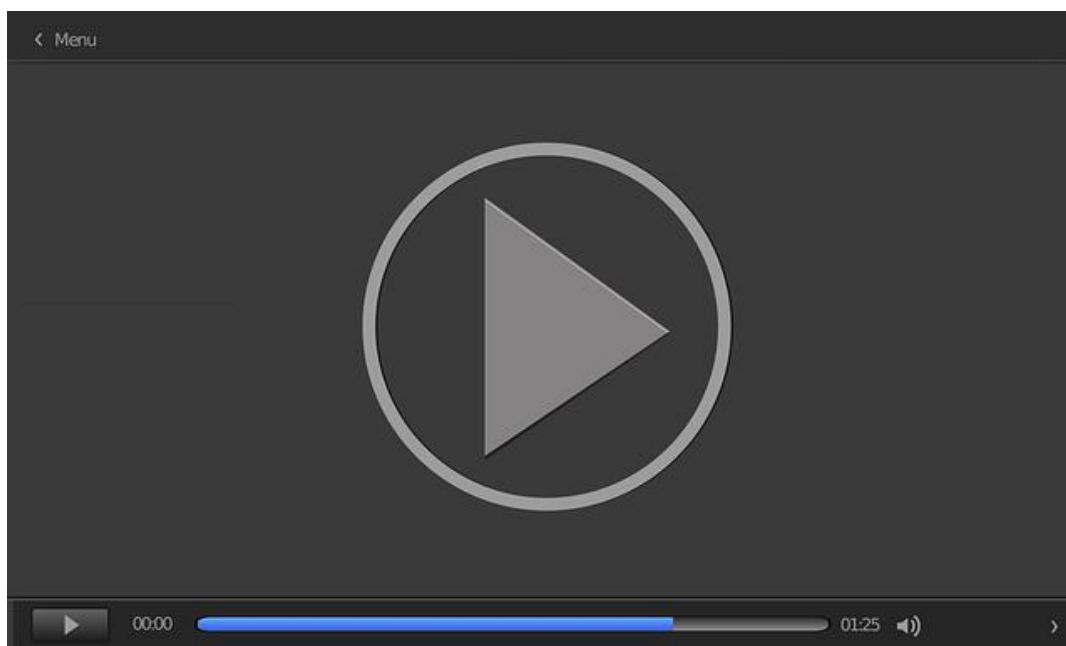
- ▶ **Kinesis Data Streams.** Es el componente central de la arquitectura de Kinesis. Permite a los usuarios capturar y procesar datos en tiempo real mediante la ingestión de datos en fragmentos (*shards*) distribuidos. Cada fragmento es una unidad de capacidad de procesamiento que puede manejar una cantidad específica de datos. Los datos ingresados en un *stream* se almacenan temporalmente en los fragmentos y se pueden procesar utilizando consumidores de datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- **Productores y consumidores de datos.** Los productores son las fuentes de datos que ingresan datos en un *stream* de Kinesis. Pueden ser aplicaciones, dispositivos IoT, servidores de registros, sensores, etc. Los productores envían datos al *stream* de Kinesis utilizando la API de Kinesis.

Los consumidores son las **aplicaciones** o los **sistemas** que procesan y analizan los datos en *streaming*. Pueden ser aplicaciones de análisis en tiempo real, sistemas de almacenamiento, servicios de procesamiento de datos, etc. Los consumidores obtienen acceso a los datos del *stream* de Kinesis utilizando la API de Kinesis o integrándose directamente con otros servicios de AWS.

A continuación, en el vídeo *Data Consumer Options for Amazon Kinesis Data Streams* / *Amazon Web Services* (Amazon Web Services, 2023e), se habla sobre las posibilidades de consumidores de datos dentro de Kinesis Data Streams.



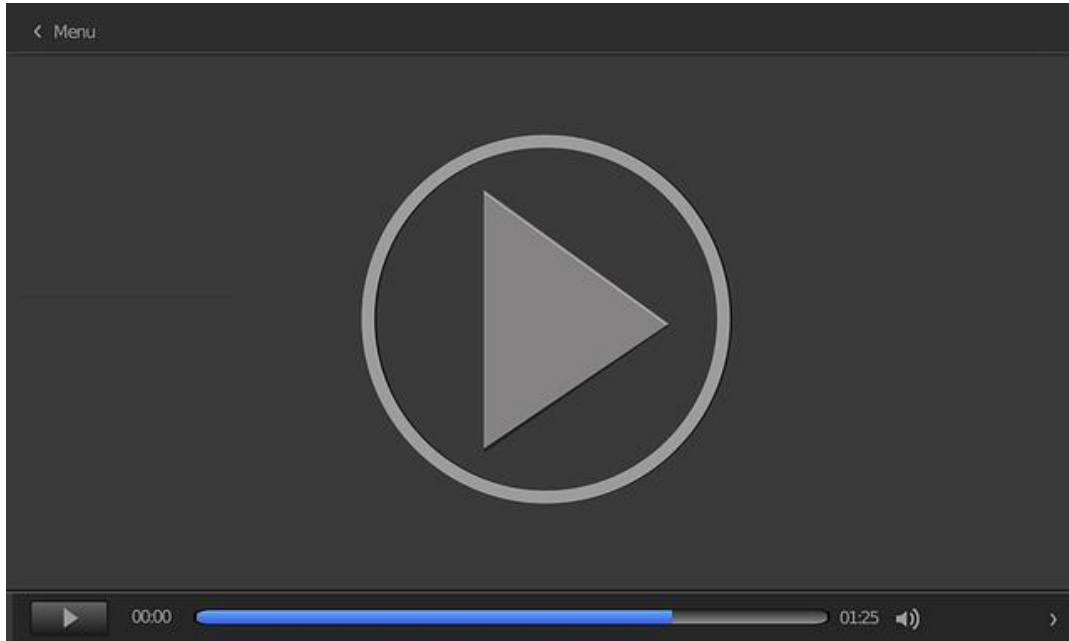
Data Consumer Options for Amazon Kinesis Data Streams | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/STEMa3t5NOQ>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, en el vídeo *Data Producer Options for Amazon Kinesis Data Streams* / *Amazon Web Services* (Amazon Web Services, 2023f), se habla sobre las posibilidades de productores de datos dentro de Kinesis Data Streams.



Data Producer Options for Amazon Kinesis Data Streams | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/tmhCRGV0XOM>

- **Kinesis Data Firehose.** Es un servicio opcional que simplifica la carga directa y continua de datos en tiempo real desde Kinesis Data Streams a otros servicios de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch Service y Splunk. Elimina la necesidad de administrar infraestructura adicional y proporciona una forma fácil de procesar y almacenar datos en tiempo real para análisis posteriores.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Kinesis Data Analytics.** Es otro servicio opcional que permite realizar análisis en tiempo real de datos en *streaming* utilizando SQL estándar. Facilita la ejecución de consultas SQL sobre datos en movimiento para extraer información valiosa y tomar decisiones instantáneas. Kinesis Data Analytics puede integrarse directamente con Kinesis Data Streams para procesar datos en tiempo real y generar resultados analíticos en tiempo real.
- ▶ **Integración con servicios de AWS.** Kinesis se integra estrechamente con otros servicios de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch Service, AWS Lambda y más. Esta integración permite que los usuarios utilicen la infraestructura de AWS para procesar y analizar datos en *streaming* de manera eficiente y escalable.
- ▶ **Seguridad y gestión de acceso.** Kinesis ofrece opciones avanzadas de seguridad y control de acceso, como cifrado de datos en tránsito y en reposo, control de acceso basado en roles (IAM), y políticas de acceso granular. Esto permite que los usuarios protejan los datos en *streaming* y garanticen el cumplimiento normativo.

En resumen, la **arquitectura de AWS Kinesis** está diseñada para facilitar la ingestión, el procesamiento y el análisis de datos en tiempo real a escala. Con sus componentes y características clave, Kinesis permite que las empresas capturen, procesen y analicen datos en *streaming* de manera eficiente y confiable, lo que les permite tomar decisiones más informadas y rápidas en un mundo cada vez más conectado y dinámico.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

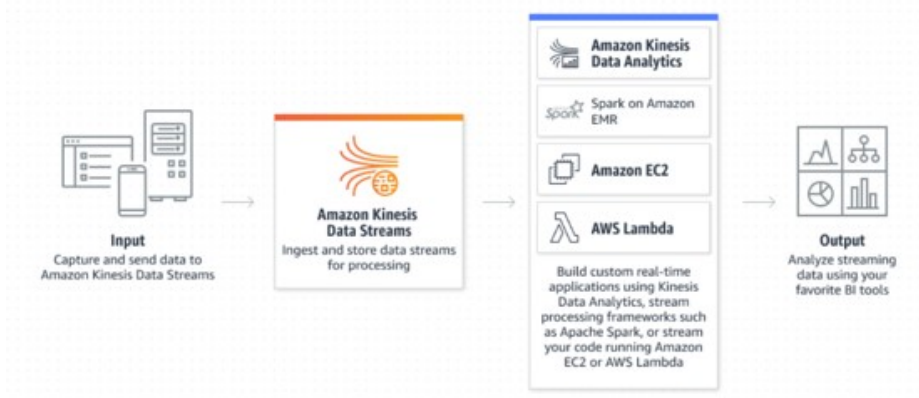


Figura 13. Amazon Kinesis Data Streams. Fuente: Amazon Web Services, s. f.-c.

Kinesis Data Streams

Kinesis Data Streams es un servicio de *streaming* de datos completamente administrado de AWS diseñado para facilitar la ingesta, el procesamiento y el análisis de grandes volúmenes de datos en tiempo real. Como parte de la plataforma de AWS, Kinesis Data Streams les permite a las empresas capturar datos en *streaming* desde diversas fuentes, como aplicaciones, dispositivos IoT, servidores de registros y más, y, luego, procesar y analizar esos datos de manera eficiente y escalable.

Con Kinesis Data Streams, las empresas pueden construir **aplicaciones de análisis en tiempo real**, que les permitan tomar decisiones basadas en datos en tiempo real. El servicio está diseñado para ser altamente escalable, duradero y confiable, lo que lo hace adecuado para una amplia gama de casos de uso, como análisis de clic en sitios web, procesamiento de registros de servidor, monitoreo de flujos de sensores, análisis de registros de aplicaciones y mucho más.

Al aprovechar Kinesis Data Streams, las empresas pueden procesar y analizar grandes **volúmenes de datos** en tiempo real utilizando herramientas y servicios familiares de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch Service y más. Además, el servicio ofrece opciones avanzadas de seguridad y control de acceso, como cifrado de datos en tránsito y en reposo, control de acceso

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

basado en roles (IAM) y políticas de acceso granular, lo que garantiza la protección de los datos en *streaming* y el cumplimiento normativo.

En resumen, **Kinesis Data Streams** es una solución poderosa y versátil para la ingestión, procesamiento y análisis de datos en tiempo real en la nube de AWS. Con su capacidad para capturar y procesar grandes volúmenes de datos en tiempo real, Kinesis Data Streams les permite a las empresas obtener información valiosa y tomar decisiones informadas de manera más rápida y eficiente en un mundo cada vez más conectado y dinámico.

Características de Kinesis Data Streams

Amazon Kinesis Data Streams ofrece una serie de características que lo convierten en una opción poderosa y versátil para la ingestión, procesamiento y análisis de datos en tiempo real. A continuación, tienes algunas de las características clave de AWS Kinesis Data Streams:

- ▶ **Escalabilidad horizontal.** Kinesis Data Streams puede manejar grandes volúmenes de datos y cargas de trabajo escalando horizontalmente a través de la adición o eliminación de *shards* según sea necesario.
- ▶ **Baja latencia.** Ofrece baja latencia para la ingestión y la recuperación de datos, lo que permite el procesamiento y el análisis de datos en tiempo real con tiempos de respuesta rápidos.
- ▶ **Durabilidad.** Los datos ingresados en un *stream* de Kinesis se almacenan de forma duradera en el servicio, lo que garantiza que no se pierdan incluso en caso de fallos o interrupciones.
- ▶ **Procesamiento paralelo.** Los datos ingresados en un *stream* se dividen en fragmentos (*shards*), lo que permite el procesamiento paralelo de datos y una mayor capacidad de procesamiento.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Integración con servicios de AWS.** Kinesis Data Streams se integra estrechamente con otros servicios de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch Service y más, lo que facilita el procesamiento y el análisis de datos en tiempo real utilizando la infraestructura de AWS.
- ▶ **Control de acceso y seguridad.** Ofrece opciones avanzadas de seguridad y control de acceso, como cifrado de datos en tránsito y en reposo, control de acceso basado en roles (IAM) y políticas de acceso granular, lo que garantiza la protección de los datos en *streaming* y el cumplimiento normativo.
- ▶ **Monitoreo y gestión.** Kinesis Data Streams proporciona herramientas y métricas para monitorear y gestionar el rendimiento y la salud de los *streams*, lo que facilita la detección y la resolución de problemas de manera proactiva.
- ▶ **Coste eficiente.** Con una estructura de precios basada en el número de *shards* y el volumen de datos procesados, Kinesis Data Streams ofrece una opción costo-eficiente para la ingestión y el procesamiento de datos en tiempo real.

Estas **características** hacen de Amazon Kinesis Data Streams una opción popular para una amplia gama de aplicaciones, desde análisis de clic en tiempo real hasta procesamiento de registros de servidor y monitoreo de flujos de sensores. Permite a las empresas capturar, procesar y analizar datos en tiempo real de manera eficiente y escalable, lo que les permite tomar decisiones más informadas y rápidas en un mundo cada vez más conectado y dinámico.

Componentes principales de Kinesis Data Streams

Los principales **componentes** de Amazon Kinesis Data Streams son:

- ▶ **Stream.** Un *stream* es el componente central de Kinesis Data Streams. Representa un flujo continuo de datos en tiempo real y actúa como el punto de entrada para los datos que ingresan al sistema. Los datos se dividen en fragmentos (*shards*) dentro del *stream* para permitir la escalabilidad horizontal y la distribución de la carga.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

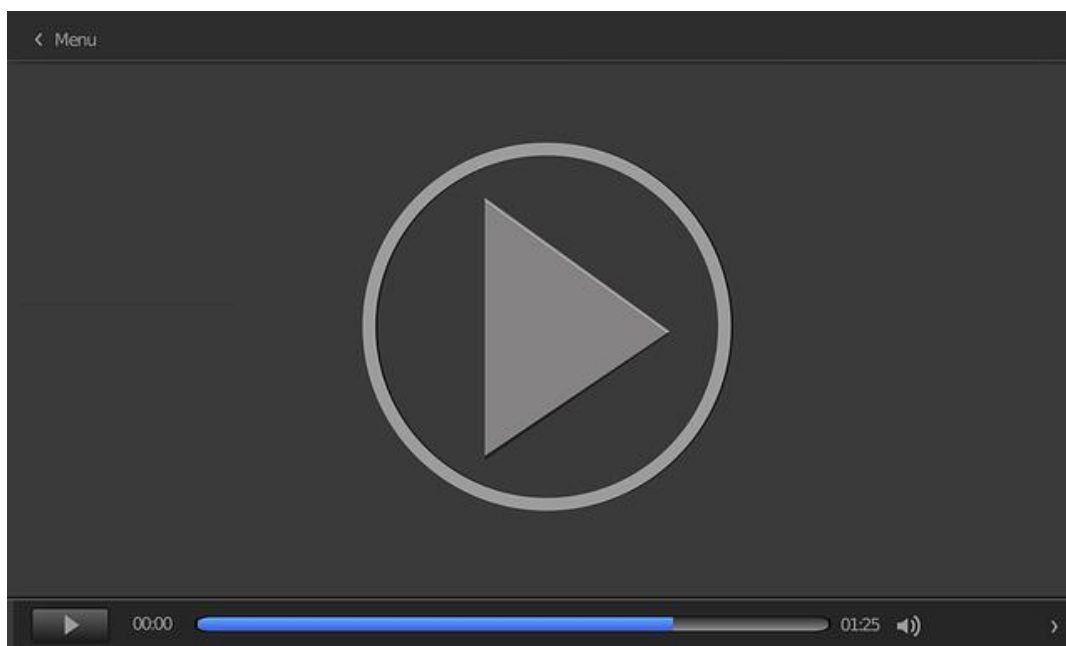
- ▶ **Shard.** Un *shard* es una unidad de capacidad de procesamiento en un *stream* de Kinesis. Cada *shard* tiene una capacidad de ingestión y recuperación de datos específica y puede manejar un volumen determinado de datos por segundo. Los datos ingresados en un *stream* se distribuyen entre los *shards* disponibles.
- ▶ **Productores.** Los productores son las fuentes de datos que ingresan datos en un *stream* de Kinesis. Pueden ser aplicaciones, dispositivos IoT, servidores de registros, sensores, etc. Los productores envían datos al *stream* de Kinesis utilizando la API de Kinesis.
- ▶ **Consumidores.** Los consumidores son las aplicaciones o los sistemas que procesan y analizan los datos en *streaming*. Pueden ser aplicaciones de análisis en tiempo real, sistemas de almacenamiento, servicios de procesamiento de datos, etc. Los consumidores obtienen acceso a los datos del *stream* de Kinesis utilizando la API de Kinesis o integrándose directamente con otros servicios de AWS.
- ▶ **Particiones.** Dentro de cada *shard*, los datos se dividen en particiones. Estas particiones permiten el procesamiento paralelo de datos y ayudan a garantizar que los datos se procesen de manera eficiente y escalable.
- ▶ **AWS Management Console.** La consola de administración de AWS proporciona una interfaz gráfica para configurar, monitorear y administrar *streams* de Kinesis. Permite a los usuarios crear, modificar y eliminar *streams*, así como supervisar el rendimiento y el estado de los *streams* existentes.
- ▶ **AWS SDK.** AWS proporciona SDK (*software development kits*) para varios lenguajes de programación que facilitan la integración de aplicaciones con Kinesis Data Streams. Estos SDK permiten que los desarrolladores interactúen con los *streams* de Kinesis y envíen y reciban datos de manera programática.

Estos **componentes** trabajan juntos para permitir la ingestión, el procesamiento y el análisis de datos en tiempo real en AWS utilizando Kinesis Data Streams. Los productores envían datos al *stream*, que se dividen en *shards* para su procesamiento

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

paralelo. Los consumidores acceden a los datos del *stream* para realizar análisis en tiempo real, almacenamiento o procesamiento adicional.

En el vídeo *Getting Started with Kinesis Data Streams | Amazon Web Services* (Amazon Web Services, 2023g) aprenderás a utilizar esta herramienta.



Getting Started with Kinesis Data Streams | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/111DcJvmd4w>

AWS Data Firehose (anterior Kinesis Data Firehose)

Amazon Kinesis Data Firehose es un servicio completamente administrado de AWS que facilita la carga directa y continua de datos en tiempo real desde múltiples fuentes a servicios de almacenamiento y análisis en la nube de AWS. Diseñado para simplificar el proceso de ingestión de datos en *streaming* y reducir la complejidad operativa, Kinesis Data Firehose automatiza gran parte del trabajo necesario para cargar, transformar y almacenar datos en tiempo real, permitiendo a las empresas enfocarse en el análisis y la generación de información valiosa a partir de sus datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

A continuación, tendrás una descripción profunda de sus características y componentes principales.

Características principales

- ▶ **Carga automática de datos.** Kinesis Data Firehose permite cargar automáticamente datos en tiempo real desde múltiples fuentes, como *streams* de Kinesis, servicios de mensajería, como Amazon SQS, flujos de *log*, como Amazon CloudWatch Logs, y eventos de servicios, como AWS Lambda.
- ▶ **Transformación de datos.** El servicio ofrece opciones para transformar los datos en tiempo real antes de cargarlos en el destino final. Esto incluye transformaciones simples, como el cambio de formato, así como transformaciones más complejas utilizando AWS Lambda para filtrar, enriquecer o procesar los datos antes de cargarlos.
- ▶ **Integración con servicios de AWS.** Kinesis Data Firehose se integra estrechamente con otros servicios de AWS, como Amazon S3, Amazon Redshift, Amazon Elasticsearch Service y Splunk. Esto permite cargar datos en tiempo real en servicios de almacenamiento y análisis en la nube de AWS de manera eficiente y escalable.
- ▶ **Gestión automatizada de recursos.** Kinesis Data Firehose gestiona automáticamente los recursos subyacentes necesarios para la carga y el procesamiento de datos en tiempo real, lo que simplifica la administración y reduce la sobrecarga operativa.
- ▶ **Escalabilidad y durabilidad.** El servicio es altamente escalable y duradero, lo que permite manejar grandes volúmenes de datos y cargas de trabajo sin problemas. Los datos se almacenan de forma duradera en el destino final, garantizando la integridad y la disponibilidad de los datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Monitorización y métricas.** Kinesis Data Firehose proporciona métricas y registros detallados para monitorear el rendimiento y la salud de los flujos de datos en tiempo real, lo que facilita la detección y resolución de problemas.

Componentes principales

- ▶ **Delivery stream.** Un *delivery stream* es el componente central de Kinesis Data Firehose. Representa el flujo continuo de datos en tiempo real que se carga en un destino específico, como Amazon S3 o Amazon Redshift.
- ▶ **Productores y fuentes de datos.** Los productores son las fuentes de datos que envían datos al *delivery stream* de Kinesis Data Firehose. Pueden ser *streams* de Kinesis, servicios de mensajería, flujos de log, eventos de servicios, etc.
- ▶ **Destinos.** Los destinos son los servicios de almacenamiento y análisis en la nube de AWS donde se cargan los datos en tiempo real. Estos pueden incluir Amazon S3 para almacenamiento a largo plazo, Amazon Redshift para análisis de datos, Amazon Elasticsearch Service para búsqueda y análisis de registros, y Splunk para análisis de registros y seguridad.

Casos de uso

- ▶ **Análisis de datos en tiempo real.** Kinesis Data Firehose permite cargar datos en tiempo real en servicios de análisis como Amazon Redshift y Amazon Elasticsearch Service para realizar análisis y generación de informes en tiempo real.
- ▶ **Almacenamiento de datos en tiempo real.** El servicio facilita la carga de datos en tiempo real en Amazon S3 para almacenamiento a largo plazo y análisis posterior.
- ▶ **Procesamiento de logs.** Kinesis Data Firehose puede utilizarse para cargar flujos de log en tiempo real desde servicios como CloudWatch Logs y enviarlos a destinos como Amazon S3 o Splunk para análisis y monitoreo.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En resumen, **Amazon Kinesis Data Firehose** es una solución poderosa y completamente administrada para la carga y procesamiento de datos en tiempo real en la nube de AWS. Con su capacidad para automatizar gran parte del trabajo necesario para cargar y transformar datos en tiempo real, Kinesis Data Firehose permite a las empresas capturar, almacenar y analizar datos de manera eficiente y escalable, lo que les permite obtener información valiosa y tomar decisiones informadas en tiempo real.

Funcionamiento

Amazon Data Firehose proporciona la forma más sencilla de adquirir, transformar y entregar secuencias de datos en cuestión de segundos a lagos de datos, almacenamientos de datos y servicios de análisis. Para utilizar Amazon Data Firehose, debe configurar una secuencia con un origen, un destino y las transformaciones necesarias. Amazon Data Firehose procesa continuamente la secuencia, la escala automáticamente en función de la cantidad de datos disponibles y la envía en cuestión de segundos.

- ▶ **Origen.** Seleccione el origen de su secuencia de datos, como un tema en Amazon Managed Streaming para Kafka (MSK), una secuencia en Kinesis Data Streams o escriba datos mediante la API Firehose Direct PUT. Amazon Data Firehose está integrado en más de veinte servicios de AWS, por lo que puede configurar una secuencia de orígenes, como registros de Amazon CloudWatch, registros de ACL web de AWS WAF, AWS Network Firewall Logs, Amazon SNS o AWS IoT.
- ▶ **Transformación de datos (opcional).** Especifique si desea convertir la secuencia de datos en formatos como Parquet u ORC, descomprimir los datos, realizar transformaciones de datos personalizadas con la función de AWS Lambda o dividir dinámicamente los registros de entrada en función de los atributos para entregarlos en diferentes ubicaciones.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- **Destino.** Seleccione un destino para su secuencia, como Amazon S3, Amazon OpenSearch Service, Amazon Redshift, Splunk, Snowflake o un punto de conexión HTTP personalizado.

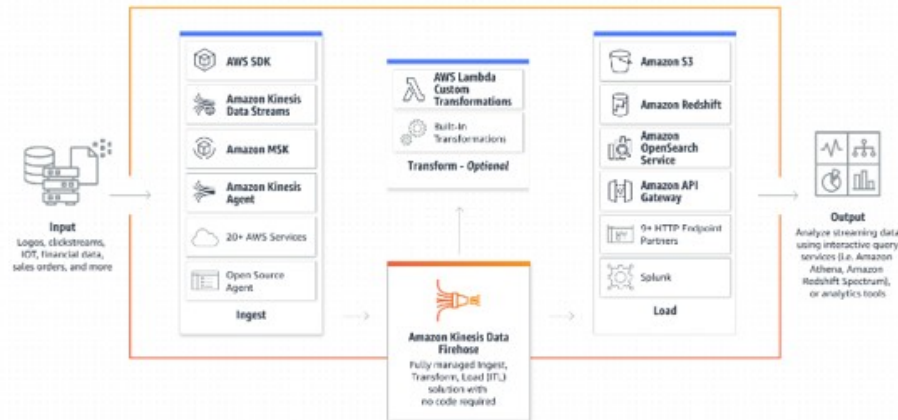
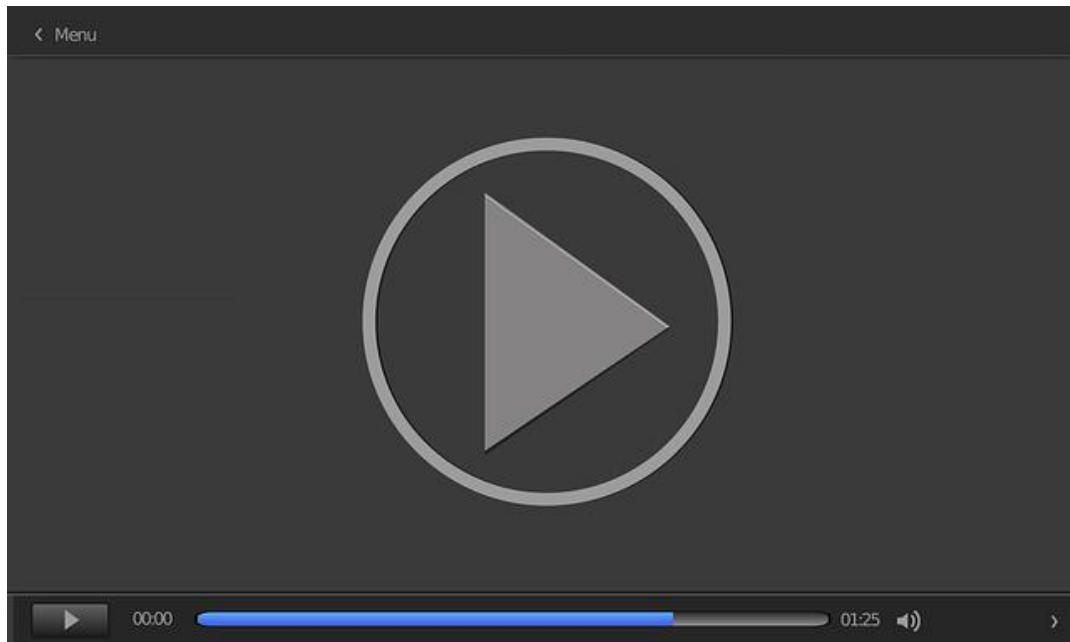


Figura 14. Amazon Data Firehose. Fuente: Amazon Web Service, s. f.-d.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En el vídeo *Introduction to Amazon Kinesis Firehose* (Amazon Web Services, 2016), se profundiza sobre el funcionamiento de Amazon Firehose.



Introduction to Amazon Kinesis Firehose.

Accede al vídeo:

<https://www.youtube.com/embed/8L3ILSPPxpY>

AWS Flink Administrado

Amazon Managed Service para Apache Flink es un servicio completamente administrado con el que puede procesar y analizar datos de transmisión mediante Java, Python, SQL o Scala. El servicio permite crear y ejecutar con rapidez código Java, SQL o Scala en orígenes de transmisión para realizar análisis de series temporales, alimentar paneles y crear métricas en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Managed Service para Apache Flink proporciona la **infraestructura** subyacente para sus aplicaciones de Apache Flink. Gestiona las capacidades principales, como el aprovisionamiento de recursos informáticos, la resiliencia de la conmutación por error de AZ, la computación paralela, el escalado automático y las copias de seguridad de las aplicaciones (implementadas como puntos de control e instantáneas).

Características de Apache Flink Administrado

- ▶ **Código abierto.** Amazon Managed Service para Apache Flink incluye bibliotecas de código abierto tales como Apache Flink, Apache Beam, Apache Zeppelin, SDK de AWS e integraciones de servicios de AWS. Apache Flink es un marco y motor que sirve para crear aplicaciones de *streaming* precisas y de alta disponibilidad. Apache Beam es un modelo unificado para definir aplicaciones de *streaming* y de procesamiento de datos por lotes que se ejecutan en varios motores de ejecución. El SDK de AWS elimina la complejidad de la codificación para muchos servicios de AWS al proporcionar API en su idioma preferido e incluye bibliotecas de AWS, ejemplos de código y documentación.
- ▶ **Apache Flink.** Apache Flink es un marco de procesamiento de datos de código abierto y distribuido diseñado para realizar análisis avanzados y procesamiento de datos en tiempo real y por lotes a gran escala. Es conocido por su capacidad para procesar datos de manera eficiente, con baja latencia y alta precisión, lo que lo hace adecuado para una amplia gama de casos de uso en entornos de *big data*. A continuación, tienes una descripción más detallada de sus características y componentes principales.

Características principales

- ▶ **Procesamiento de datos en tiempo real.** Flink es especialmente conocido por su capacidad para procesar datos en tiempo real con baja latencia. Permite a las empresas realizar análisis de datos en tiempo real y tomar decisiones basadas en eventos a medida que ocurren.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Procesamiento de datos por lotes.** Además del procesamiento de datos en tiempo real, Flink también admite el procesamiento de datos por lotes, lo que permite a las empresas realizar análisis retrospectivos y generar informes a partir de grandes volúmenes de datos históricos.
- ▶ **Modelo de programación rico.** Flink ofrece un modelo de programación rico y expresivo que permite a los desarrolladores escribir fácilmente aplicaciones complejas de procesamiento de datos. Soporta API en Java, Scala y Python.
- ▶ **Escalabilidad y tolerancia a fallos.** Flink está diseñado para ser altamente escalable y tolerante a fallos, lo que permite procesar grandes volúmenes de datos y manejar fallos de manera robusta. Puede ejecutarse en clústeres distribuidos de gran tamaño y escalar horizontalmente según sea necesario.
- ▶ **Soporte para *streams* y gráficos de datos.** Flink admite tanto el procesamiento de datos en *streaming* como el procesamiento de datos por lotes, lo que permite a las empresas analizar datos en tiempo real y procesar grandes volúmenes de datos históricos de manera eficiente.
- ▶ **Integración con ecosistema de *big data*.** Flink se integra con otros proyectos y tecnologías de *big data*, como Apache Hadoop, Apache Kafka, Apache HBase, Apache Cassandra y más, lo que facilita su incorporación en entornos existentes de *big data*.

Componentes principales

- ▶ **Flink Core.** El núcleo de Apache Flink proporciona las API y los motores de ejecución necesarios para procesar datos en tiempo real y por lotes. Incluye una amplia variedad de operadores y transformaciones para manipular y analizar datos de manera eficiente.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Flink Streaming.** Es una API de alto nivel que permite el procesamiento de datos en tiempo real utilizando flujos de datos continuos. Permite el procesamiento de datos en *streaming* con baja latencia y alta precisión.
- ▶ **Flink Batch.** Flink Batch es una API de alto nivel que permite el procesamiento de datos por lotes utilizando conjuntos de datos finitos. Permite el procesamiento de grandes volúmenes de datos históricos de manera eficiente y escalable.
- ▶ **Flink SQL.** Es una API que permite realizar consultas SQL sobre datos en *streaming* y por lotes. Facilita el análisis y la generación de informes utilizando lenguaje SQL estándar.
- ▶ **Flink ML.** Flink ML es una biblioteca de aprendizaje automático que permite realizar análisis predictivo y modelado de datos utilizando algoritmos de aprendizaje automático.

En resumen, **Apache Flink** es un marco de procesamiento de datos de código abierto y distribuido que ofrece capacidades avanzadas para realizar análisis en tiempo real y por lotes a gran escala. Con su rico modelo de programación, escalabilidad y tolerancia a fallos, Flink es ampliamente utilizado en entornos de *big data* para una variedad de casos de uso, desde análisis de clics en tiempo real hasta procesamiento de registros de servidor y análisis predictivo.

Apache Flink es un marco de procesamiento de datos de código abierto y distribuido que ha ganado una gran popularidad en el mundo de la **analítica de datos** en los últimos años. Se destaca por su capacidad para procesar datos tanto en tiempo real como en lotes, su alto rendimiento y su eficacia en el manejo de grandes volúmenes de datos.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Arquitectura de componentes

- ▶ **Cliente Flink (*Flink client*).** Es la interfaz de usuario y línea de comandos que permite a los usuarios enviar programas y comandos al clúster de Flink. A través del cliente, los desarrolladores pueden enviar y monitorear sus aplicaciones de Flink.
- ▶ **Clúster de Flink (*Flink cluster*).** El clúster de Flink es el entorno donde se ejecutan las aplicaciones de Flink. Está compuesto por un conjunto de nodos (o máquinas) que ejecutan el *software* de Flink y coordinan el procesamiento de datos.
- ▶ **JobManager .** El **JobManager** es el nodo maestro en el clúster de Flink y es responsable de coordinar y administrar las ejecuciones de los trabajos de Flink. Se encarga de recibir y programar los trabajos, coordinar la ejecución de las tareas y supervisar el progreso de los trabajos.
- ▶ **TaskManager .** Los **TaskManagers** son los nodos de trabajo en el clúster de Flink y son responsables de ejecutar las tareas individuales de los trabajos de Flink. Cada **TaskManager** ejecuta una o más instancias de tarea y se comunica con el **JobManager** para coordinar la ejecución de las tareas.
- ▶ **Tareas (*tasks*).** Las tareas son las unidades de procesamiento en Flink y representan las operaciones individuales que se aplican a los datos. Las tareas pueden ser operaciones de lectura, transformación o escritura, y se ejecutan de manera paralela en los **TaskManagers** .
- ▶ **Grafo de tareas (*task graph*).** El grafo de tareas es la representación interna de un programa de Flink y describe las dependencias entre las tareas. Está compuesto por nodos que representan las operaciones y los arcos que representan las comunicaciones entre las tareas.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **State backend.** El *state backend* es el mecanismo que Flink utiliza para almacenar y administrar el estado de las aplicaciones de Flink. Puede ser configurado para utilizar diferentes sistemas de almacenamiento, como memoria, sistemas de archivos distribuidos (por ejemplo, HDFS) o bases de datos externas.
- ▶ **Checkpointing y savepoints.** Flink utiliza el *checkpointing* para garantizar la tolerancia a fallos y la recuperación de las aplicaciones en caso de fallo. Los *savepoints* son puntos de control manuales que permiten a los usuarios guardar el estado de una aplicación y reiniciarla desde ese punto en el futuro.

Flujo de ejecución en Flink

El proceso de ejecución de una aplicación de Flink sigue los siguientes pasos:

- ▶ **Despliegue de la aplicación.** El usuario despliega una aplicación de Flink utilizando el cliente de Flink, especificando el programa que desea ejecutar y las configuraciones de ejecución.
- ▶ **Inicio del JobManager y los TaskManagers.** Cuando se inicia la aplicación, se inicia un JobManager y uno o más TaskManagers en el clúster de Flink para ejecutar la aplicación.
- ▶ **Planificación y ejecución de tareas.** El JobManager recibe el programa de la aplicación y lo traduce en un grafo de tareas. Luego, planifica y coordina la ejecución de las tareas en los TaskManagers, asignando las tareas a los TaskManagers disponibles.
- ▶ **Ejecución de tareas.** Los TaskManagers ejecutan las tareas asignadas, procesando los datos de entrada, aplicando las operaciones definidas por el programa y enviando los resultados a las tareas subsiguientes.

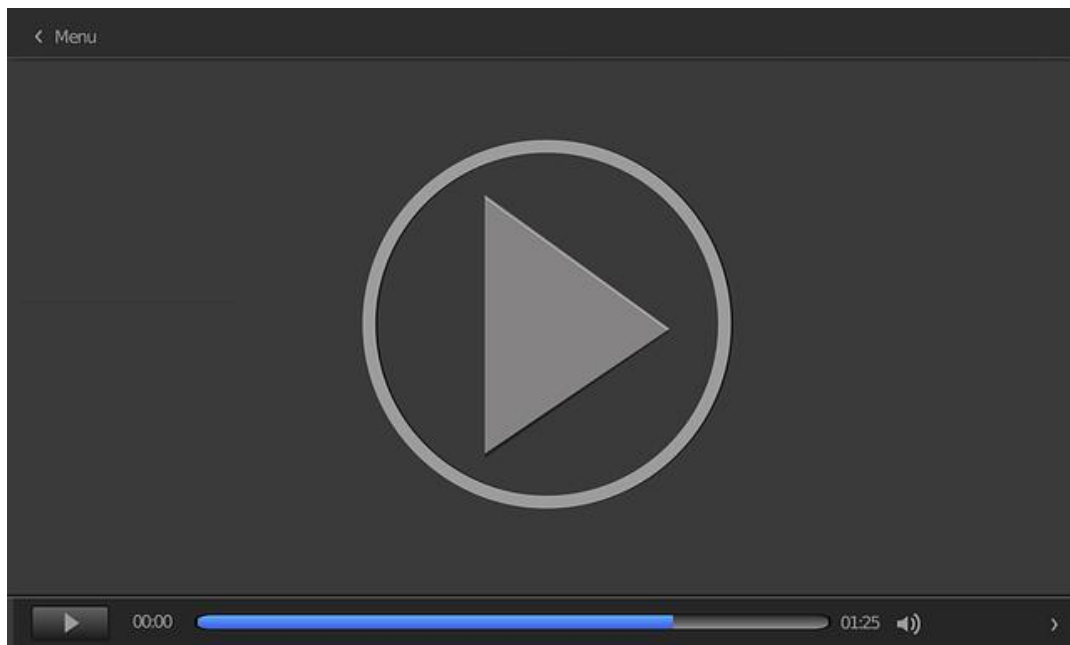
Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Checkpointing y tolerancia a fallos.** Durante la ejecución, Flink realiza *checkpointing* periódicamente para guardar el estado de la aplicación. En caso de fallo, Flink puede recuperar la aplicación desde el último punto de control para garantizar la tolerancia a fallos.
- ▶ **Finalización de la aplicación.** Una vez que se completan todas las tareas de la aplicación, Flink finaliza la ejecución y proporciona los resultados al usuario.

En resumen, **Apache Flink** ofrece una arquitectura robusta y escalable para el procesamiento de datos distribuidos en tiempo real y por lotes. Con su conjunto de componentes y su enfoque en la eficiencia y la tolerancia a fallos, Flink es ampliamente utilizado en una variedad de aplicaciones de *big data* y análisis de datos en tiempo real.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En el vídeo *One-Click Streaming with Amazon Managed Service For Apache Flink Blueprints* | Amazon Web Services (Amazon Web Services, 2023h), se explica como utiliza los *blueprints* de AWS Flink Administrado, que son la forma más fácil de ponerse en marcha y comenzar con una canalización de *streaming* completa de un extremo a otro. Con solo hacer clic en un botón, se puede poner en marcha una fuente de *streaming*, una aplicación de procesamiento de *streaming* y comenzar a procesar estos datos. Los proyectos contienen *scripts* especializados de Amazon CloudFormation para lanzar la infraestructura modular en su cuenta.



One-Click Streaming with Amazon Managed Service For Apache Flink Blueprints | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/mWTnArl8xCI>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Casos de uso

Apache Flink se utiliza en una amplia variedad de casos de uso en diferentes industrias. Algunos ejemplos comunes incluyen:

- ▶ **Análisis en tiempo real de eventos de transmisión**, como análisis de clics en sitios web, detección de fraudes, monitoreo de aplicaciones y sistemas, y más.
- ▶ **Procesamiento de lotes de grandes volúmenes de datos históricos**, como análisis de registros de servidor, procesamiento de registros de máquinas, análisis de datos de transacciones y más.
- ▶ **Procesamiento de datos en tiempo real para aplicaciones de IoT**, como monitoreo y análisis de sensores, control de procesos industriales, análisis de telemetría y más.

En resumen, **Apache Flink** es un marco de procesamiento de datos potente y versátil que ofrece capacidades avanzadas para analizar datos en tiempo real y por lotes a gran escala. Con su rico modelo de programación, escalabilidad y tolerancia a fallos, Flink es ampliamente utilizado en una variedad de casos de uso en diferentes industrias para obtener información valiosa a partir de grandes volúmenes de datos.

Las API de Flink

Amazon Managed Service para Apache Flink es compatible con las **API flexibles de Flink** en Java, Scala, Python y SQL especializadas para diferentes casos de uso, incluido el procesamiento de eventos con estado, ETL de *streaming* y análisis en tiempo real. Con operadores prediseñados y capacidades de análisis, puede crear una aplicación de *streaming* de Apache Flink en cuestión de horas en lugar de meses y las bibliotecas son ampliables, por lo que puede realizar el procesamiento en tiempo real para varios casos de uso.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Apache Beam

Apache Beam proporciona un modelo de programación unificado para el procesamiento de datos distribuidos. Permite a los desarrolladores escribir código una vez utilizando un conjunto de API coherentes y expresivas, y, luego, ejecutar ese código en múltiples motores de ejecución de procesamiento de datos, como Apache Flink, Apache Spark, Google Cloud Dataflow y más. Esto hace que sea fácil desarrollar aplicaciones de procesamiento de datos portátiles que puedan ejecutarse en una variedad de entornos de procesamiento de datos sin necesidad de modificar el código.

Características clave de Apache Beam

- ▶ **Modelo de programación unificado.** Apache Beam ofrece un modelo de programación unificado que les permite a los desarrolladores escribir código una vez y ejecutarlo en múltiples motores de ejecución de procesamiento de datos. Esto simplifica el desarrollo de aplicaciones de procesamiento de datos y facilita la portabilidad entre diferentes entornos de ejecución.
- ▶ **API expresivas y cohesivas.** Beam proporciona un conjunto de API coherentes y expresivas para la manipulación de datos, que incluyen API para la lectura, la transformación y la escritura de datos. Estas API están diseñadas para ser intuitivas y fáciles de usar, lo que facilita el desarrollo de aplicaciones de procesamiento de datos complejas.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Portabilidad y flexibilidad.** Una de las principales ventajas de Apache Beam es su capacidad para ejecutar código en múltiples motores de ejecución de procesamiento de datos. Esto permite a los desarrolladores escribir aplicaciones de procesamiento de datos una vez y ejecutarlas en una variedad de entornos de procesamiento de datos sin necesidad de modificar el código.
- ▶ **Optimización automática.** Beam realiza la optimización automática de los programas de procesamiento de datos, lo que permite mejorar el rendimiento y la eficiencia de las aplicaciones. Utiliza técnicas como la fusión de operaciones, la partición de datos y la planificación de consultas para optimizar la ejecución de los programas.
- ▶ **Integración con ecosistema de *big data*.** Apache Beam se integra con varios motores de ejecución de procesamiento de datos y herramientas de *big data*, como Apache Flink, Apache Spark, Google Cloud Dataflow y más. Esto facilita su incorporación en entornos existentes de *big data* y permite que los desarrolladores aprovechen las herramientas y las plataformas que ya están familiarizados.

Casos de uso de Apache Beam

Apache Beam se utiliza en una variedad de casos de uso en diferentes industrias. Algunos ejemplos comunes incluyen:

- ▶ Procesamiento de datos en tiempo real y por lotes.
- ▶ Análisis de datos de registros de servidor y telemetría.
- ▶ Procesamiento de eventos de transmisión, como análisis de clic en sitios web.
- ▶ Análisis de datos de aplicaciones de IoT.
- ▶ Procesamiento de datos en tiempo real para aplicaciones de análisis y monitoreo de sistemas.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

En resumen, **Apache Beam** es un modelo de programación unificado y un conjunto de API de procesamiento de datos de código abierto que simplifica el desarrollo de aplicaciones de procesamiento de datos distribuidos. Con su capacidad para ejecutar código en múltiples motores de ejecución de procesamiento de datos y su conjunto de API expresivas y coherentes, Beam facilita el desarrollo de aplicaciones de procesamiento de datos portátiles y escalables que pueden ejecutarse en una variedad de entornos de procesamiento de dato.

Arquitectura y componentes de Apache Beam

La **arquitectura** de Apache Beam está diseñada para ofrecer un modelo de programación unificado y portátil que permita el procesamiento de datos distribuidos en una variedad de motores de ejecución de procesamiento de datos. A continuación, describiré los componentes principales de Apache Beam y cómo se interrelacionan en su arquitectura.

Componentes principales

El **pipeline** es el componente central de Apache Beam y representa el flujo de datos que será procesado. Un *pipeline* consta de una serie de transformaciones que se aplican a los datos para realizar operaciones como lectura, transformación y escritura.

- ▶ **PCollections** (colecciones paralelas). Las PCollections son las estructuras de datos que fluyen a través del *pipeline*. Representan conjuntos de datos distribuidos que pueden ser procesados en paralelo. Las PCollections son inmutables y pueden contener datos en tiempo real o en lotes.
- ▶ **Transformaciones**. Las transformaciones son operaciones que se aplican a las PCollections para procesar los datos. Estas transformaciones pueden ser operaciones de lectura, transformación o escritura, y se combinan para formar el grafo de ejecución del *pipeline*.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Runners** (ejecutores). Los *runners* son los motores de ejecución que ejecutan el *pipeline* y realizan las transformaciones en los datos. Apache Beam proporciona un conjunto de *runners* prediseñados para ejecutar *pipelines* en diferentes entornos, como Apache Flink, Apache Spark, Google Cloud Dataflow y más.
- ▶ **Pipeline options**. Las opciones del *pipeline* son configuraciones que controlan el comportamiento y la ejecución del *pipeline*, como la configuración del entorno de ejecución, la ubicación de los recursos y más.

Proceso de ejecución en Apache Beam

La arquitectura de Apache Beam se basa en un modelo de programación unificado y portátil que permite a los desarrolladores escribir código una vez y ejecutarlo en diferentes motores de ejecución de procesamiento de datos. El **proceso de ejecución** de un *pipeline* en Apache Beam sigue los siguientes pasos:

- ▶ **Creación del *pipeline***. El proceso comienza con la creación del *pipeline*, donde se definen las transformaciones y las fuentes de datos que formarán parte del flujo de datos.
- ▶ **Configuración del *pipeline options***. A continuación, se configuran las opciones del *pipeline*, que incluyen la configuración del entorno de ejecución, las opciones de procesamiento y más.
- ▶ **Ejecución del *pipeline***. Una vez que el *pipeline* está configurado, se ejecuta en un *runner* específico que se encarga de ejecutar las transformaciones en los datos. El *runner* se encarga de gestionar la ejecución del *pipeline*, coordinar las transformaciones y garantizar que los resultados se procesen correctamente.
- ▶ **Procesamiento distribuido**. Durante la ejecución del *pipeline*, las transformaciones se aplican de manera distribuida en los datos, lo que permite procesar grandes volúmenes de datos de manera eficiente y escalable.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Escritura de resultados.** Finalmente, los resultados del *pipeline* se escriben en un destino específico, como un sistema de almacenamiento o un servicio de análisis, donde pueden ser utilizados para generar informes, tomar decisiones o alimentar otras aplicaciones.

En resumen, la **arquitectura de Apache Beam** está diseñada para ofrecer un modelo de programación unificado y portátil que permite el procesamiento de datos distribuidos en una variedad de motores de ejecución de procesamiento de datos. Con su conjunto de componentes y su arquitectura modular, Apache Beam facilita el desarrollo de aplicaciones de procesamiento de datos escalables y portátiles que pueden ejecutarse en diferentes entornos de procesamiento de datos sin necesidad de modificar el código.

Integraciones de servicios de AWS

Es posible configurar e integrar un destino u origen de datos con un código mínimo. Utiliza las bibliotecas de Amazon Managed Service para Apache Flink para integrarlas con los siguientes servicios de AWS:

- ▶ Amazon S3.
- ▶ Amazon MSK.
- ▶ Amazon OpenSearch Service.
- ▶ Amazon DynamoDB.
- ▶ Amazon Kinesis Data Streams.
- ▶ Amazon Kinesis Data Firehose.
- ▶ Amazon CloudWatch.
- ▶ AWS Glue Schema Registry.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Capacidades avanzadas de integración

Además de las **integraciones** de AWS, las bibliotecas de Amazon Managed Service para Apache Flink incluyen más de cuarenta conectores de Apache Flink y la capacidad de crear integraciones personalizadas. Con algunas líneas de código más, puede modificar el comportamiento de cada integración con la funcionalidad avanzada. También puede crear integraciones personalizadas mediante un conjunto de tipos primitivos de Apache Flink que le permiten leer y escribir desde archivos, directorios, conectores u otros orígenes a las que puede acceder a través de Internet.

- ▶ **Procesamiento único.** Con Amazon Managed Service para Apache Flink, puedes crear aplicaciones en las que los registros procesados afectan los resultados exactamente una vez, lo que se conoce como procesamiento único. Incluso en el caso de una interrupción de la aplicación, como el mantenimiento del servicio interno o la actualización de la aplicación iniciada por el usuario, el servicio garantiza que todos los datos se procesen y que no haya datos duplicados.
- ▶ **Procesamiento con estado.** El servicio almacena el procesamiento o estado previo y en curso en el almacenamiento de la aplicación en ejecución. Compara los resultados pasados y actuales durante cualquier período de tiempo y logra una recuperación rápida durante las interrupciones de la aplicación. El estado siempre está cifrado y se guarda de manera progresiva en el almacenamiento de la aplicación en ejecución.
- ▶ **Copias de seguridad de aplicaciones duraderas.** Se puede crear y eliminar copias de seguridad de aplicaciones duraderas a través de una simple llamada a la API. Restaura inmediatamente sus aplicaciones desde la última copia de seguridad después de una interrupción o su aplicación a una versión anterior.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

- ▶ **Integración de ML.** Amazon Managed Service para Apache Flink admite algoritmos de aprendizaje automático (ML). Puedes crear aplicaciones en tiempo real para la clasificación, la agrupación en clústeres, la evaluación, las recomendaciones de ingeniería de características, las regresiones y las estadísticas.
- ▶ **Compatibilidad con el registro de esquemas de AWS Glue.** Amazon Managed Service para Apache Flink es compatible con AWS Glue Schema Registry. Schema Registry permite mejorar la calidad de los datos y protegerse frente a cambios inesperados mediante comprobaciones de compatibilidad que rigen la evolución de los esquemas en Amazon Managed Service para Apache Flink para cargas de trabajo conectadas a Apache Kafka, Amazon MSK o Amazon Kinesis Data Streams, ya sea como conector de origen o de destino.

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

2.4. Referencias bibliográficas

Aitor Medrano. (s. f.). *Ingesta de datos*. GitHub. <https://aitor-medrano.github.io/iabd/de/etl.html>

Amazon Web Services. (2016, septiembre 8). *Introduction to Amazon Kinesis Firehose* [Vídeo]. YouTube. <https://youtu.be/8L3ILSPxpY>

Amazon Web Services. (2020, julio 2). *Amazon Kinesis Data Streams Fundamentals* [Vídeo]. YouTube. <https://youtu.be/hLLgkTUmwOU>

Amazon Web Services. (2022, agosto 23). *AWS Glue Overview | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/u14iVEc-C6E>

Amazon Web Services. (2023a, julio 18). *AWS Glue Studio - Visual data pipeline demo | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/ckxwnd4BQmk>

Amazon Web Services. (2023b, febrero 17). *Introduction to Kinesis Data Firehose | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/qRoyF9dEqgw>

Amazon Web Services. (2023c, agosto 30). *Introduction to Amazon Managed Service for Apache Flink | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/vl1GiMSHuxM>

Amazon Web Services. (2023d, noviembre 29). *AWS re:Invent 2017 - Introducing Amazon Kinesis Video Streams* [Vídeo]. YouTube. <https://youtu.be/STEMa3t5NOQ>

Amazon Web Services. (2023e). *Data Consumer Options for Amazon Kinesis Data Streams | Amazon Web Services* [Vídeo]. YouTube. https://youtu.be/opY-56_9AOg

Amazon Web Services. (2023f, febrero 14). *Data Producer Options for Amazon Kinesis Data Streams | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/tmhCRGV0XOM>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Amazon Web Services. (2023g, enero 25). *Getting Started with Kinesis Data Streams* | Amazon Web Services [Vídeo]. YouTube. <https://youtu.be/1l1DcJvmd4w>

Amazon Web Services. (2023h, octubre 16). One-Click Streaming with Amazon Managed Service For Apache Flink Blueprints | Amazon Web Services [Vídeo]. YouTube. <https://youtu.be/mWTnArl8xCi>

Amazon Web Services. (2024). *Developer Guide. Amazon Kinesis Data Streams*. <https://docs.aws.amazon.com/pdfs/streams/latest/dev/kinesis-dg.pdf>

Amazon Web Services. (s. f.-a). *Catalog and search*. <https://docs.aws.amazon.com/whitepapers/latest/building-data-lakes/data-cataloging.html>

Amazon Web Service. (s. f.-b). *Información general de los flujos de trabajo en AWS Glue*. https://docs.aws.amazon.com/es_es/glue/latest/dg/workflows_overview.html

Amazon Web Service. (s. f.-c). *Amazon Kinesis*. Fuente: <https://aws.amazon.com/es/kinesis/>

Amazon Web Service. (s. f.-d). *Amazon Data Firehose*. Fuente: <https://aws.amazon.com/es/firehose/>

Amazon Web Services. (s. f.-e). *AWS Documentation*. https://docs.aws.amazon.com/es_es/

Amazon Web Services. (s. f.-f). *Componentes de AWS Glue*. https://docs.aws.amazon.com/es_es/glue/latest/dg/components-overview.html#data-catalog-intro

Amazon Web Services. (s. f.-g). *Creación de trabajos de ETL visuales con AWS Glue Studio*. https://docs.aws.amazon.com/es_es/glue/latest/dg/author-job-glue.html

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Amazon Web Services. (s. f.-h). *Detección y catalogación de datos en AWS Glue*.

https://docs.aws.amazon.com/es_es/glue/latest/dg/catalog-and-crawler.html

Amazon Web Services. (s. f.-i). *Documentación de Amazon Kinesis*.

https://docs.aws.amazon.com/es_es/kinesis/

Amazon Web Services. (s. f.-j). *¿Qué es Amazon Kinesis Data Streams?*

https://docs.aws.amazon.com/es_es/streams/latest/dev/introduction.html

Amazon Web Services. (s. f.-k). *¿Qué es Amazon Data Firehose?*

https://docs.aws.amazon.com/es_es/firehose/latest/dev/what-is-this-service.html

Amazon Web Services. (s. f.-l). *¿Qué es Amazon Managed Service para Apache Flink?* https://docs.aws.amazon.com/es_es/managed-flink/latest/java/what-is.html

AWS LATAM. (2021a, diciembre 6). *Descubre y cataloga tus datos con AWS Glue – Español* [Vídeo]. YouTube. <https://youtu.be/pbKnfjVsx4>

AWS LATAM. (2021b, diciembre 6). *Prepara tus datos y administra tus ETLs con AWS Glue - Español* [Vídeo]. YouTube. <https://youtu.be/mw6nu7-4PI>

AWS Tutorials. (2022, marzo 7). *AWS Tutorials - Interactively Develop Glue Job using Jupyter Notebook* [Vídeo]. YouTube. https://youtu.be/n4PVC5O_tJo

Blokdyk, G. (2021). *AWS Glue Second Edition*. 5STARCooks.

Development Team. (2018). *Amazon Kinesis Firehose Developer Guide*. Samurai Media Limited.

Halder, N. (2023, mayo 27). *ETL vs. ELT: A Comprehensive Comparison and Guide to Modern Data Integration Strategies*. Medium. <https://medium.com/analysts-corner/etl-vs-elt-a-comprehensive-comparison-and-guide-to-modern-data-integration-strategies-f2968bc64651>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

IMPACT Unofficial. (2022, noviembre 4). *AI Foundation Models to Augment Scientific Data and the Research Lifecycle*. Medium. <https://impactunofficial.medium.com/ai-foundation-models-to-augment-scientific-data-and-the-research-lifecycle-d5c16054df89>

Kumar V, S. (2023). *Stream Processing with Apache Flink & Pyflink: A Comprehensive Guide* [autoedición].

Makota, T., Maguire, B., Gagne, D. y Chakrabarti, R. (2023). *Scalable Data Streaming with Amazon Kinesis: Design and secure highly available, cost-effective data streaming applications with Amazon Kinesis*. Packt Publishing.

Murat Sivri. (2022, septiembre 7). *Batch processing - stream processing*. Medium. <https://medium.com/i%CC%87stanbuldatascienceacademy/batch-processing-stream-processing-74a88a7c0dc7>

Saad, A., Salem, R. y Abd eldkader, H. A. (2021). Frequent Pattern Mining over Streaming Data: From models to research challenges. *IJCI International Journal of Computers and Information*, 8(2), 156-161. https://www.researchgate.net/publication/356996515_Frequent_Pattern_Mining_over_Streaming_Data_From_models_to_research_challenges

The Art of Service - AWS Glue Publishing (Author). (2021). *AWS Glue A Complete Guide*.

Qlik. (s. f.). *Streaming Data*. <https://www.qlik.com/us/streaming-data>

Vrinda Mathur. (2022, junio 21). *What is Data Ingestion? Challenges and Types*. AnalyticSteps. <https://www.analyticsteps.com/blogs/what-data-ingestion-challenges-and-types>

Tema 2. Ingesta de datos para big data e inteligencia artificial en cloud

Waehner, K. (2022, junio 27). *Data Warehouse vs. Data Lake vs. Data Streaming – Friends, Enemies, Frenemies?* Kai Waehner. <https://www.kai-waehner.de/blog/2022/06/27/data-warehouse-vs-data-lake-vs-data-streaming-friends-enemies-frenemies/>

Wall Street Mindset. (2024, enero 13). *Automated ETL Workflow Orchestration with AWS Glue, Athena, Lambda, EventBridge, and Step Functions* [Vídeo]. YouTube. <https://youtu.be/Olt0XekklhU>

Tema 3. Data science cloud storage

3.1. Introducción y objetivos

Siguiendo con las etapas del ciclo de vida del dato en el ámbito de *big data* y *data science*, y la etapa de ingesta del dato, entramos a revisar los principales conceptos, paradigmas y tecnologías en el ámbito del almacenamiento para la ciencia del dato.

Los **objetivos** para este tema son:

- ▶ Entender y diferenciar los conceptos de *data lake*, *data warehouse* y *data lakehouse*.
- ▶ Entender los nuevos paradigmas de arquitecturas de almacenamiento de datos.
- ▶ Comprender el paradigma *data mesh* y aprender sus elementos clave.
- ▶ Comprender el paradigma *data fabric* y aprender sus elementos clave.
- ▶ Conocer y aprender las principales características y funcionalidades del servicio de almacenamiento masivo AWS S3.
- ▶ Conocer y aprender las principales características y funcionalidades del servicio de almacenamiento HDFS.
- ▶ Conocer y aprender las principales características y funcionalidades del servicio de *data warehouse* en la nube de AWS Redshift.

Tema 3. Data science cloud storage

3.2. Sistemas de almacenamiento: big data

Sistemas de almacenamiento

Imagina un sistema de almacenamiento de datos como un gigantesco almacén donde guardas todas tus pertenencias. Ahora, en lugar de ropa, muebles o recuerdos personales, este almacén guarda datos, por ejemplo, desde simples números hasta imágenes, vídeos, documentos y mucho más. Por tanto, ¿qué **características** debe tener un sistema de almacenamiento?

- ▶ **Propósito.** Un sistema de almacenamiento de datos existe para mantener registros de información valiosa para una organización o individuo. Esta información puede ser cualquier cosa, desde datos transaccionales en una empresa hasta fotos familiares en una nube personal.
- ▶ **Estructura.** Al igual que en un almacén físico, un sistema de almacenamiento de datos tiene una estructura organizada. Esto puede ser en forma de bases de datos, sistemas de archivos o incluso una combinación de ambos. Estas estructuras están diseñadas para almacenar y organizar datos de manera eficiente, permitiendo un fácil acceso y manipulación cuando sea necesario.
- ▶ **Tipos de datos.** Los sistemas de almacenamiento de datos pueden manejar una amplia variedad de tipos de datos, desde simples cadenas de texto hasta datos complejos, como imágenes, vídeos, registros de transacciones financieras, datos biométricos, etc. Dependiendo de la naturaleza de los datos que se están almacenando, el sistema puede estar optimizado para manejar diferentes tipos de carga de trabajo y consultas.
- ▶ **Escalabilidad.** Uno de los desafíos clave en el diseño de un sistema de almacenamiento de datos es la escalabilidad. A medida que la cantidad de datos aumenta con el tiempo, el sistema debe ser capaz de crecer y manejar esta carga adicional sin comprometer el rendimiento. Esto puede implicar la adición de más

Tema 3. Data science cloud storage

servidores, el uso de tecnologías de almacenamiento distribuido o la migración a soluciones en la nube.

- ▶ **Seguridad.** Dado que los datos almacenados son a menudo valiosos y sensibles, la seguridad es una consideración crítica en cualquier sistema de almacenamiento de datos. Esto implica medidas como el cifrado de datos, el control de acceso basado en roles, la monitorización de la actividad del usuario y la implementación de políticas de retención de datos.
- ▶ **Recuperación y copia de seguridad.** Los sistemas de almacenamiento de datos también deben estar preparados para situaciones de pérdida de datos, ya sea por error humano, fallos de *hardware* o ciberataques. Por lo tanto, suelen incluir capacidades de copia de seguridad y recuperación para garantizar la integridad y disponibilidad de los datos en todo momento.

Por tanto, el **sistema de almacenamiento de datos** es un repositorio central donde se guarda información para que se pueda consultar, analizar o visualizar para tomar decisiones mejor informadas. Los datos fluyen hacia un almacenamiento de datos desde sistemas transaccionales, IoT, *logs*, bases de datos relacionales y otros orígenes. Los analistas empresariales, los ingenieros de datos, los científicos de datos y los responsables de la toma de decisiones obtienen acceso a los esos repositorios de almacenamiento a través de diferentes herramientas.

Los **datos** y el **análisis** se han vuelto fundamentales para que las empresas mantengan la competitividad. Las empresas utilizan informes, *dashboards* y herramientas de análisis para extraer información de los datos, monitorear el desempeño de la empresa y respaldar la toma de decisiones.

En resumen, un sistema de almacenamiento de datos es la **infraestructura subyacente** que permite a las organizaciones y personas almacenar, organizar, acceder y proteger grandes volúmenes de información de manera eficiente y segura, por lo que desempeña un papel fundamental en la gestión de datos en la era digital.

Tema 3. Data science cloud storage

Concepto de almacenamiento en *big data*

El **almacenamiento** en *big data* es un componente fundamental en la gestión y el análisis de grandes volúmenes de datos. Este concepto se refiere a la capacidad de almacenar, gestionar y acceder a enormes cantidades de datos de manera eficiente y escalable. En el contexto de *big data*, el almacenamiento debe ser capaz de manejar datos estructurados, no estructurados y semiestructurados, provenientes de diversas fuentes, como redes sociales, sensores, registros de transacciones, archivos de texto, imágenes, vídeos, entre otros.

A continuación, se explican algunos aspectos clave del almacenamiento en *big data*:

- ▶ **Escalabilidad.** El almacenamiento en *big data* debe ser altamente escalable para poder manejar la creciente cantidad de datos de manera efectiva. Esto implica la capacidad de agregar más capacidad de almacenamiento y procesamiento según sea necesario, sin comprometer el rendimiento.
- ▶ **Tolerancia a fallos.** Dado que los sistemas de *big data* suelen trabajar con enormes cantidades de datos distribuidos en múltiples nodos, es crucial que el almacenamiento sea tolerante a fallos. Esto significa que el sistema debe poder mantener la disponibilidad y la integridad de los datos incluso en caso de que uno o varios nodos fallen.
- ▶ **Distribución.** Los datos en *big data* se distribuyen en múltiples nodos o servidores para permitir un procesamiento paralelo y una mayor velocidad de acceso. El almacenamiento debe ser capaz de distribuir y gestionar estos datos de manera eficiente, lo que optimiza la carga de trabajo en toda la infraestructura.
- ▶ **Velocidad.** El acceso rápido a los datos es fundamental en el análisis de *big data*. Por lo tanto, el almacenamiento debe ser capaz de proporcionar altas velocidades de lectura y escritura, incluso cuando se trabaja con conjuntos de datos masivos.

Tema 3. Data science cloud storage

- **Flexibilidad.** Dado que los datos en *big data* pueden tener diferentes estructuras y formatos, el almacenamiento debe ser flexible y compatible con una variedad de tipos de datos. Esto puede implicar el uso de bases de datos NoSQL, sistemas de archivos distribuidos, como HDFS, almacenamiento en la nube, entre otros.

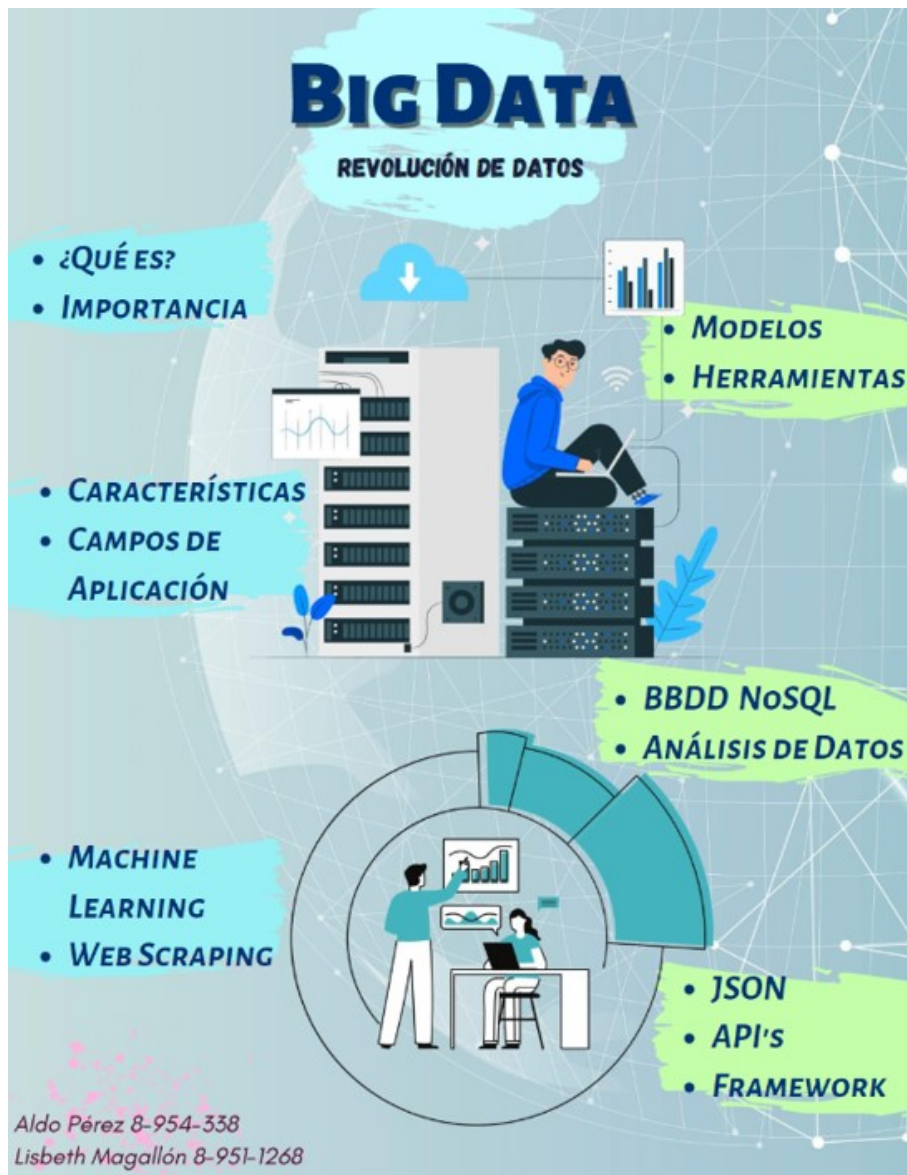


Figura 1. *Big data*. Fuente: LisM24, 2021.

Tema 3. Data science cloud storage

En resumen, el almacenamiento en *big data* es esencial para la gestión eficiente de grandes volúmenes de datos, garantizando escalabilidad, tolerancia a fallos, distribución, velocidad y flexibilidad para satisfacer las demandas del análisis de datos a gran escala.

Paradigma de almacenamiento distribuido

El **paradigma de los sistemas de almacenamiento distribuido** en *big data* es una pieza fundamental en la gestión eficiente de grandes volúmenes de datos. Este enfoque surge como respuesta a los desafíos que enfrentan las organizaciones al intentar almacenar, procesar y analizar cantidades masivas de datos que superan la capacidad de los sistemas de almacenamiento tradicionales. Seguidamente se explica con detalle los conceptos, las características, el origen y la utilidad de este paradigma.

- ▶ **Concepto.** El sistema de almacenamiento distribuido en *big data* se refiere a una arquitectura en la que los datos se distribuyen y almacenan en múltiples nodos o servidores, en lugar de concentrarse en una única ubicación centralizada. Estos nodos pueden estar ubicados en diferentes lugares geográficos y se comunican entre sí a través de una red. Los datos se replican y fragmentan en estos nodos para permitir un procesamiento paralelo y una mayor tolerancia a fallos.
- ▶ **Características.** Los sistemas de almacenamiento distribuido pueden escalar horizontalmente agregando más nodos al sistema, lo que permite manejar grandes volúmenes de datos sin sacrificar el rendimiento.
- **Tolerancia a fallos.** Al distribuir los datos en múltiples nodos, el sistema puede tolerar la falla de uno o varios nodos sin perder datos o interrumpir el servicio. La replicación de datos y la redundancia juegan un papel clave en este aspecto.
- **Procesamiento paralelo.** La distribución de datos permite que múltiples nodos

Tema 3. Data science cloud storage

procesen los datos de forma simultánea, lo que acelera el procesamiento y el análisis de grandes conjuntos de datos.

- **Alto rendimiento.** Gracias al procesamiento paralelo y la escalabilidad horizontal, los sistemas de almacenamiento distribuido pueden ofrecer un alto rendimiento en términos de velocidad de acceso y procesamiento de datos.
- **Flexibilidad.** Estos sistemas son flexibles y pueden manejar una amplia variedad de tipos de datos, desde datos estructurados hasta datos no estructurados y semiestructurados.
- ▶ **Origen.** El surgimiento del paradigma de almacenamiento distribuido en *big data* está estrechamente relacionado con el crecimiento exponencial de datos en la era digital. Con la proliferación de dispositivos conectados, redes sociales, sensores y otras fuentes de datos, las organizaciones se enfrentaron al desafío de gestionar y analizar enormes cantidades de información de manera eficiente. Los sistemas de almacenamiento distribuido surgieron como una solución escalable y flexible para abordar estos desafíos. Además, el concepto de almacenamiento distribuido en *big data* no tiene un creador único, sino que surge como resultado de la evolución de varias tecnologías y enfoques a lo largo del tiempo. Sin embargo, hay ciertos hitos y contribuciones importantes que han dado forma a este paradigma. Aquí hay algunos puntos clave:
 - **Google File System (GFS).** Uno de los hitos fundamentales en el desarrollo de sistemas de almacenamiento distribuido fue el GFS, presentado por Google en un artículo de investigación en 2003. GFS fue diseñado para satisfacer las necesidades de almacenamiento de Google, lo que permitió el almacenamiento de grandes volúmenes de datos en clústeres de servidores.
 - **MapReduce.** Otro avance crucial fue la introducción de MapReduce, también por Google en 2004. MapReduce es un modelo de programación y procesamiento distribuido diseñado para procesar grandes conjuntos de datos en paralelo en un clúster de servidores. Este modelo se convirtió en la base de muchas tecnologías de

Tema 3. Data science cloud storage

big data posteriores.

- **Hadoop.** Basándose en los principios establecidos por GFS y MapReduce, Doug Cutting y Mike Cafarella crearon Hadoop, en 2005, como un proyecto de código abierto inspirado en los trabajos de Google. Hadoop consiste en un sistema de archivos distribuido llamado HDFS y un marco de procesamiento distribuido basado en MapReduce. Hadoop fue fundamental en popularizar el concepto de almacenamiento distribuido en el ámbito de *big data*.
- **Apache Spark.** Aunque Hadoop fue un avance significativo, tenía algunas limitaciones en términos de rendimiento y flexibilidad. Apache Spark surgió como una alternativa más rápida y versátil al modelo MapReduce. Desarrollado en la Universidad de California, Berkeley, y lanzado como proyecto de código abierto en 2010, Apache Spark permite un procesamiento de datos distribuido en memoria, lo que lo hace más adecuado para aplicaciones de análisis en tiempo real y aprendizaje automático.
- **Otras tecnologías.** Además de Hadoop y Spark, han surgido muchas otras tecnologías de almacenamiento y procesamiento distribuido en el ecosistema de *big data*. Estas incluyen sistemas de bases de datos NoSQL, como Cassandra y MongoDB, sistemas de procesamiento de transmisiones, como Apache Kafka, sistemas de almacenamiento en la nube, como Amazon S3 y Google Cloud Storage, entre otros.

En resumen, el concepto de **almacenamiento distribuido en *big data*** surge como resultado de la combinación de varios avances tecnológicos y contribuciones de investigación, con hitos clave como el Google File System, MapReduce, Hadoop y Apache Spark. Estas tecnologías han sentado las bases para la gestión eficiente de grandes volúmenes de datos en entornos distribuido.

Tema 3. Data science cloud storage

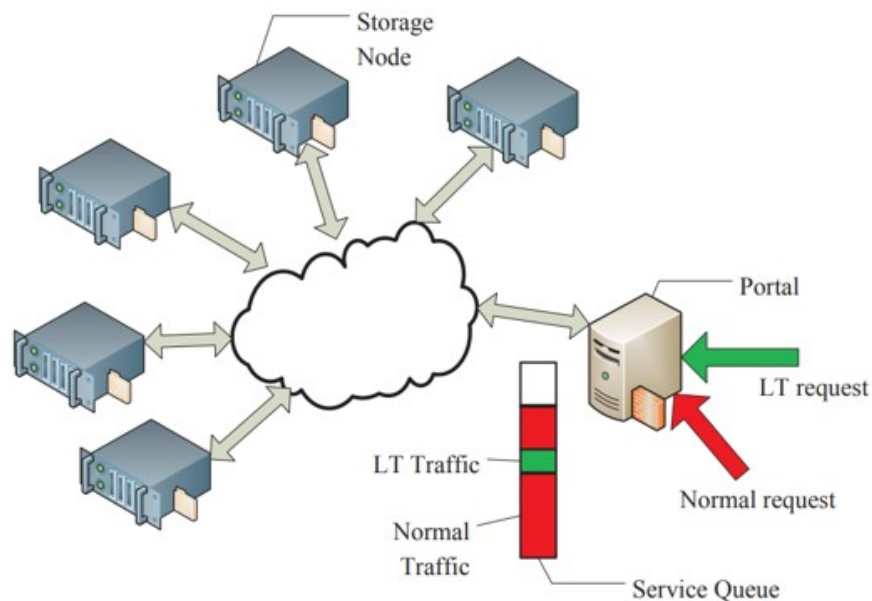


Figura 2. Arquitectura de un sistema de almacenamiento distribuido de datos. Fuente: Lu et al., 2012.

- **Utilidad.** El paradigma de almacenamiento distribuido en *big data* es esencial para una variedad de aplicaciones y casos de uso, incluyendo análisis de datos en tiempo real:
 - Procesamiento de datos masivos para informes y análisis.
 - Almacenamiento y análisis de registros de aplicaciones y sistemas.
 - Aplicaciones de inteligencia artificial y aprendizaje automático que requieren acceso a grandes conjuntos de datos.
 - Análisis de datos en la nube y entornos de computación distribuida.

En resumen, el **paradigma** de almacenamiento distribuido en *big data* proporciona una infraestructura robusta y escalable para almacenar y procesar grandes volúmenes de datos, lo que les permite a las organizaciones extraer información valiosa y tomar decisiones basadas en datos en un entorno cada vez más digitalizado y complejo.

Tema 3. Data science cloud storage

Tipos de almacenamiento de datos

Como hemos visto, el almacenamiento de datos es fundamental en la gestión de la información en entornos empresariales y de computación en la nube. Cada tipo de almacenamiento tiene sus propias características y se adapta mejor a diferentes tipos de datos y aplicaciones. Comprender las diferencias entre ellos es crucial para tomar decisiones informadas sobre cómo almacenar y gestionar datos de manera eficiente y efectiva. Existen tres **tipos principales de almacenamiento de datos**: almacenamiento de objetos, almacenamiento de archivos y almacenamiento de bloques.

El **almacenamiento de objetos** es un tipo de almacenamiento de datos que organiza la información en forma de objetos individuales, donde cada objeto contiene datos, metadatos y un identificador único. Los objetos se almacenan en un repositorio centralizado y se accede a ellos a través de una interfaz de programación de aplicaciones (API) estándar. Las **características** clave del almacenamiento de objetos incluyen:

- ▶ **Escalabilidad.** Puede manejar grandes volúmenes de datos y escalar horizontalmente según sea necesario.
- ▶ **Flexibilidad.** Admite una amplia variedad de tipos de datos, desde documentos e imágenes hasta vídeos y archivos de audio.
- ▶ **Durabilidad.** Proporciona una alta durabilidad de datos mediante la replicación y la distribución de objetos en múltiples ubicaciones.
- ▶ **Acceso mediante API.** Los objetos se acceden a través de una API estándar, lo que facilita su integración con aplicaciones y sistemas existentes.

Tema 3. Data science cloud storage

Por otro lado, el **almacenamiento de archivos** organiza la información en forma de archivos individuales, donde cada archivo tiene una ubicación y una estructura definida en el sistema de archivos. Los archivos se almacenan en un sistema de archivos jerárquico y se accede a ellos a través de rutas de acceso. Las **características** clave del almacenamiento de archivos incluyen:

- ▶ **Organización jerárquica.** Los archivos se organizan en una estructura de directorios y subdirectorios, lo que facilita la organización y la gestión de datos.
- ▶ **Acceso basado en rutas de acceso.** Los archivos se acceden a través de rutas de acceso, como URL o rutas de archivos locales, que especifican su ubicación en el sistema de archivos.
- ▶ **Compatibilidad con sistemas de archivos estándar.** Admite sistemas de archivos estándar, como NTFS en Windows y ext4 en Linux, lo que facilita la integración con los sistemas operativos y las aplicaciones existentes.
- ▶ **Eficiencia en la transferencia de archivos.** Permite la transferencia eficiente de archivos grandes mediante técnicas como la compresión y el almacenamiento en caché.

Por último, el **almacenamiento de bloques** organiza la información en forma de bloques de datos individuales, donde cada bloque tiene un identificador único y se almacena en un dispositivo de almacenamiento físico, como un disco duro o una unidad de estado sólido (SSD). Los bloques se acceden a través de un sistema de gestión de almacenamiento y se utilizan para construir archivos y sistemas de archivos. Las **características** clave del almacenamiento de bloques incluyen:

- ▶ **Eficiencia en el almacenamiento de datos.** Permite un uso eficiente del espacio de almacenamiento mediante la asignación de bloques de datos en función de las necesidades de cada archivo.

Tema 3. Data science cloud storage

- ▶ **Rendimiento optimizado.** Proporciona un rendimiento óptimo para operaciones de lectura y escritura, especialmente en entornos de almacenamiento de alta velocidad, como servidores de bases de datos y sistemas de almacenamiento de archivos.
- ▶ **Seguridad y protección de datos.** Proporciona mecanismos de seguridad y protección de datos, como la encriptación y la replicación de datos, para garantizar la integridad y la disponibilidad de la información almacenada.

En resumen, el almacenamiento de objetos, archivos y bloques son tres tipos principales de **almacenamiento de datos**, cada uno con sus propias características y casos de uso. Comprender las diferencias entre ellos es crucial para tomar decisiones informadas sobre cómo almacenar y gestionar datos de manera eficiente y efectiva en entornos empresariales y de computación en la nube.

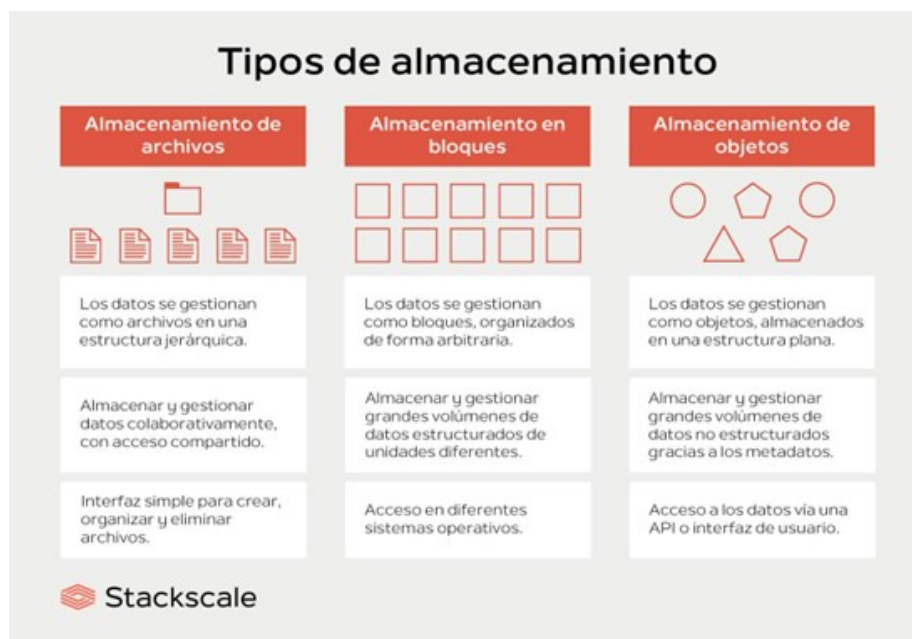


Figura 3. Tipos de almacenamiento. Fuente: Stackscale, 2023.

Tema 3. Data science cloud storage

Almacenamiento por niveles

El paradigma de **almacenamiento por niveles**, también conocido como jerarquía de almacenamiento, es una estrategia utilizada en la gestión de datos para optimizar el rendimiento y la eficiencia en el acceso y almacenamiento de información. Este enfoque implica organizar los datos en diferentes niveles o capas de almacenamiento, donde cada nivel tiene características y costos diferentes. A continuación, tienes una explicación detallada.

El almacenamiento por niveles se basa en la premisa de que no todos los datos son igualmente importantes ni requerirán el mismo nivel de acceso o rendimiento en todo momento. Por lo tanto, los datos se organizan en varias **capas de almacenamiento**, desde niveles de almacenamiento rápidos y costosos hasta niveles más lentos, pero más económicos. Muy relacionado con el sistema de almacenamiento por niveles, es el concepto de temperatura del dato, que se explica a continuación.

La **temperatura del dato** es un concepto que se utiliza en el ámbito del almacenamiento de datos para referirse a la frecuencia y la recencia con la que se accede a un determinado conjunto de datos. De manera similar al término «calor» en física, que indica la actividad o la importancia de una entidad, la temperatura del dato se refiere a la actividad o el nivel de uso de un conjunto de datos en particular. La relación entre la temperatura del dato y el sistema de almacenamiento por niveles es fundamental para optimizar el rendimiento y los costos en la gestión de datos. Las principales temperaturas del dato que se distinguen son la siguiente:

- ▶ **Datos calientes (*hot data*)**. Se refiere a los datos que se acceden con frecuencia o recientemente. Estos datos suelen ser críticos para las operaciones en curso y necesitan un acceso rápido y eficiente.
- ▶ **Datos tibios (*warm data*)**. Son datos que se acceden ocasionalmente pero que aún tienen cierta relevancia y pueden necesitar un acceso rápido en determinadas circunstancias.

Tema 3. Data science cloud storage

- **Datos fríos (cold data).** Son datos que rara vez se acceden, generalmente son datos históricos o de archivo que se mantienen por motivos de cumplimiento normativo o para referencia futura.

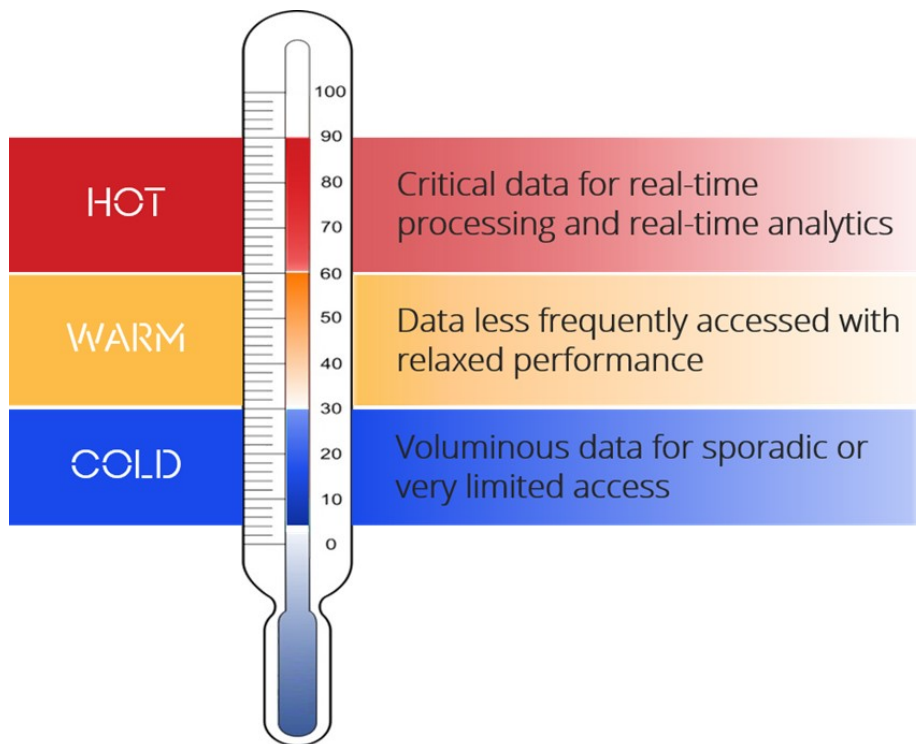


Figura 4. Temperaturas del dato. Fuente: SAPTOOLS, s. f.

En cuanto a la **relación con el sistema de almacenamiento por niveles**, el concepto de temperatura del dato se integra perfectamente con la estrategia de almacenamiento por niveles para optimizar el rendimiento y los costos en la gestión de datos. Seguidamente, profundizamos cómo se relacionan:

- **Asignación de datos a niveles de almacenamiento adecuados:**
 - Los **datos calientes** se colocan en niveles de almacenamiento de alta velocidad y baja latencia, como la memoria RAM y los discos de estado sólido (SSD), para garantizar un acceso rápido y eficiente.

Tema 3. Data science cloud storage

- Los **datos tibios** pueden colocarse en niveles de almacenamiento de capacidad media, como discos duros tradicionales (HDD).
- Los **datos fríos** se pueden almacenar en niveles de almacenamiento de gran capacidad, pero de menor velocidad, como almacenamiento en cinta o almacenamiento en la nube de bajo costo.
- ▶ **Movimiento dinámico de datos entre niveles.** Los sistemas de almacenamiento por niveles pueden monitorear continuamente la actividad de acceso a los datos y mover dinámicamente los datos entre los diferentes niveles de almacenamiento según su temperatura. Por ejemplo, si un conjunto de datos tibios comienza a experimentar un aumento en su actividad de acceso, el sistema puede moverlo automáticamente a un nivel de almacenamiento más rápido para garantizar un rendimiento óptimo.
- ▶ **Reducción de costes.** Al asignar los datos a niveles de almacenamiento adecuados según su temperatura, el almacenamiento por niveles puede ayudar a reducir los costos generales de almacenamiento al utilizar dispositivos de almacenamiento más económicos para almacenar datos menos activos.

En resumen, la **temperatura del dato** y el **sistema de almacenamiento por niveles** están estrechamente relacionados en la gestión eficiente de datos, lo que permite una asignación óptima de recursos de almacenamiento según la actividad y la importancia de los datos en un entorno de almacenamiento de datos cada vez más complejo y diverso.

Respecto a un ejemplo de **infraestructura** típica de sistema de almacenamiento por niveles, podemos describir la siguiente:

- ▶ **Nivel 1 (almacenamiento de alta velocidad).** Este nivel incluye dispositivos de almacenamiento de alta velocidad y baja latencia, como la memoria RAM y los SSD. Se utilizan para almacenar datos que necesitan un acceso rápido y frecuente, como los datos en caché y los conjuntos de datos activos.

Tema 3. Data science cloud storage

- ▶ **Nivel 2 (almacenamiento de capacidad media).** Este nivel consiste en dispositivos de almacenamiento con una capacidad mayor, pero un rendimiento menor en comparación con el nivel 1. Esto puede incluir discos duros tradicionales (HDD) u otros medios de almacenamiento de capacidad moderada. Se utilizan para almacenar datos que se acceden con menos frecuencia pero que aún necesitan estar disponibles de manera rápida cuando sea necesario.
- ▶ **Nivel 3 (almacenamiento de gran capacidad).** Este nivel implica dispositivos de almacenamiento de alta capacidad, pero con una velocidad de acceso más baja en comparación con los niveles anteriores. Esto puede incluir almacenamiento en cinta, almacenamiento en la nube de acceso esporádico o incluso almacenamiento en disco de baja velocidad. Se utilizan para almacenar datos de archivo o de respaldo que se acceden con poca frecuencia pero que aún necesitan estar disponibles para recuperación o referencia.

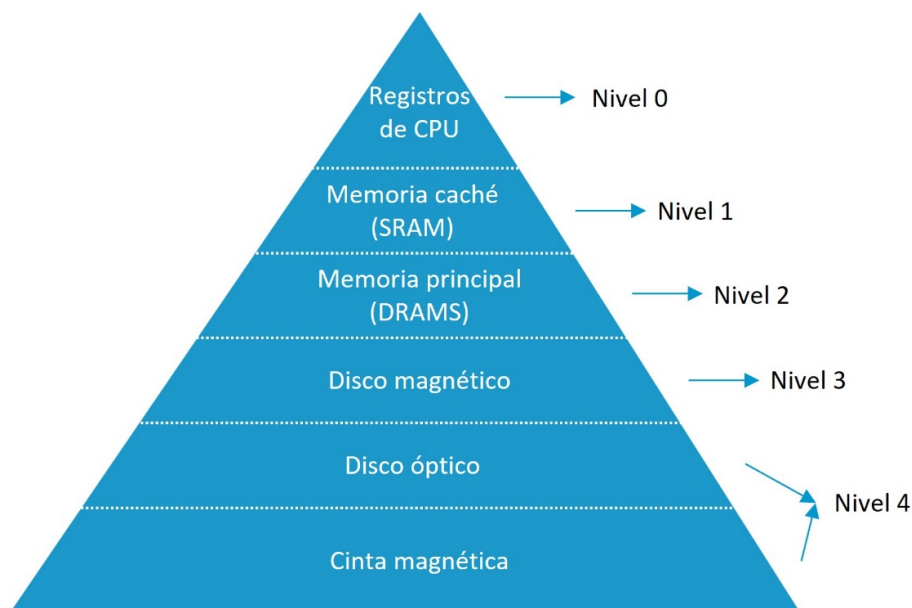


Figura 5. Jerarquía de almacenamiento. Fuente: elaboración propia.

Tema 3. Data science cloud storage

El almacenamiento por niveles se utiliza en una variedad de entornos y aplicaciones, incluyendo:

- ▶ **Sistemas de gestión de bases de datos.** Los sistemas de gestión de bases de datos pueden utilizar el almacenamiento por niveles para optimizar el rendimiento de las consultas y las transacciones, lo que coloca los datos más activos en los niveles de almacenamiento más rápidos y costosos.
- ▶ **Almacenamiento de datos empresariales.** Las empresas pueden implementar almacenamiento por niveles para administrar eficientemente grandes volúmenes de datos, lo que asegura que los datos críticos estén disponibles de manera rápida y que los datos menos activos se almacenen de manera rentable.
- ▶ **Aplicaciones de almacenamiento en la nube.** Los proveedores de servicios en la nube pueden utilizar el almacenamiento por niveles para optimizar la asignación de recursos y los costos, lo que coloca a los datos de los clientes en diferentes niveles de almacenamiento según sus necesidades y patrones de acceso.

Los **beneficios** son:

- ▶ **Optimización del rendimiento.** Al colocar los datos más activos en niveles de almacenamiento más rápidos, el almacenamiento por niveles puede mejorar significativamente el rendimiento de las aplicaciones y consultas.
- ▶ **Reducción de costos.** Al utilizar dispositivos de almacenamiento más económicos para almacenar datos menos activos, el almacenamiento por niveles puede ayudar a reducir los costos generales de almacenamiento.
- ▶ **Mejora de la eficiencia.** Al distribuir los datos en diferentes niveles de almacenamiento según su importancia y frecuencia de acceso, el almacenamiento por niveles puede mejorar la eficiencia en la gestión de datos y recursos de almacenamiento.

Tema 3. Data science cloud storage

En resumen, el paradigma de almacenamiento por niveles es una estrategia efectiva para optimizar el rendimiento, reducir costos y mejorar la eficiencia en la gestión de datos en una variedad de entornos y aplicaciones.

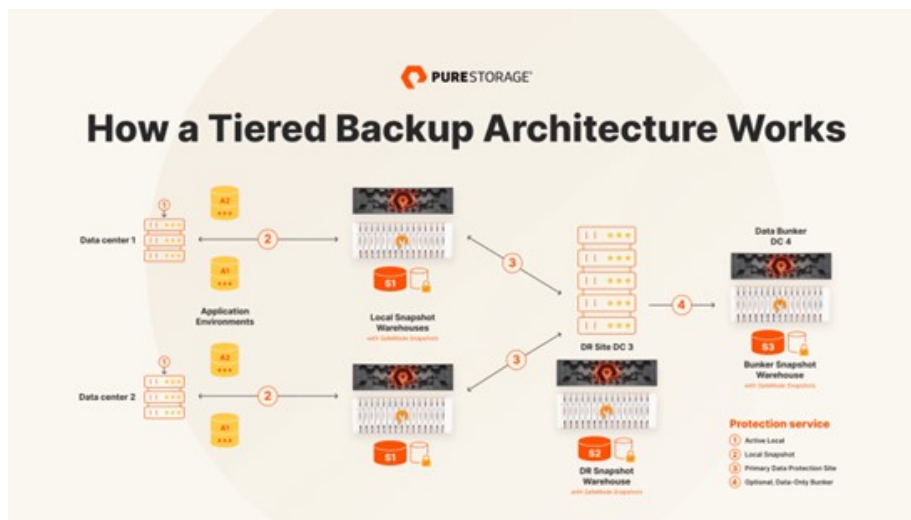


Figura 6. Clases de datos. Fuente: Pure Storage, s. f.

Arquitecturas de repositorios de datos: data lake, data warehouse, y data lakehouse

Los repositorios de datos son sistemas diseñados para almacenar, gestionar y proporcionar acceso a conjuntos de datos. Hay varios enfoques de repositorios de datos que varían en su estructura, características y casos de uso. En los siguientes apartados, vamos a profundizar en los principales enfoques existentes.

Data lake

Un **data lake** es un repositorio de datos centralizado que almacena grandes volúmenes de datos en su forma original, sin procesar, en una variedad de formatos y estructuras. A diferencia de los sistemas de almacenamiento tradicionales, como las bases de datos relacionales, que requieren que los datos estén estructurados

Tema 3. Data science cloud storage

antes de almacenarlos, un *data lake* permite almacenar datos de cualquier tipo, incluidos datos estructurados, semiestructurados y no estructurados. Estos datos pueden incluir archivos de texto, imágenes, vídeos, registros de eventos, datos de sensores, datos de redes sociales y mucho más.

Entre sus principales **características**, se pueden citar:

- ▶ **El uso de múltiples herramientas y productos.** Un *data lake* cuenta con una serie de herramientas y productos que potencian su almacenaje y gestión de datos. En efecto, los más importantes forman parte de las siguientes características a mencionar, como, por ejemplo, el acceso y la interacción de los usuarios con los datos en remoto.
- ▶ **Capacidad de almacenamiento original.** Los datos se almacenan en su forma original, sin procesar ni estructurar previamente, lo que proporciona una mayor flexibilidad y diversidad de datos.
- ▶ **Almacenamiento seguro y catalogación de los datos.** Los *data lakes* le permiten almacenar datos relacionales, como los que surgen de bases de datos operativas y datos de aplicaciones de línea de negocio, y datos no relacionales, como los provenientes de aplicaciones móviles, dispositivos de IoT y redes sociales. También le brindan la capacidad de comprender qué datos hay en el *lake* a través del rastreo, la catalogación y la indexación de datos. Finalmente, los datos deben estar seguros para garantizar que sus activos de datos estén protegidos.
- ▶ **La gestión automatizada de metadatos.** Este factor se establece como una de las características de *data lakes* más destacables, puesto que este sistema de gestión automatizada permite que las actualizaciones de los metadatos se realicen de manera continua y programada. Además, esto te ahorrará tiempo y trabajo durante la gestión de metadatos.
- ▶ **La interacción de los usuarios con los datos.** Gracias a que *data lake* es un repositorio con acceso flexible y remoto, los usuarios interesados en la información

Tema 3. Data science cloud storage

podrán acceder a esta desde diferentes partes y comprender la gestión de datos que van realizando. De esta manera, se otorga a un grupo de trabajo o empresa la posibilidad de implementarlo como una estrategia empresarial que logre una mejora en la toma de decisiones y las rutas de acción.

- ▶ **Los flujos de trabajo de ingestión configurables.** Dentro de las características de *data lakes*, esta es una de las que ofrece mayor flexibilidad, puesto que brinda una gran variedad de posibilidades para modificar cómo se desarrolla la ingesta. De esta manera, podrás establecer ciertos parámetros o dinámicas al flujo de trabajo de ingestión.
- ▶ **El *data lake* como un repositorio vivo.** Por otra parte, los *data lakes* se posicionan dentro del mundo *big data* gracias a su carácter de repositorio vivo, es decir, la posibilidad de gestionar y transformar los datos mientras estos se encuentran almacenados, a diferencia de otros sistemas, como *data warehouse*, que tarda mucho tiempo en llevar a cabo las modificaciones.

Entre sus principales **usos**, podemos destacar:

- ▶ **Análisis de *big data*.** Los *data lakes* son fundamentales para el análisis de grandes volúmenes de datos en entornos de *big data*, lo que permite que las organizaciones extraigan información valiosa y descubren patrones ocultos en los datos.
- ▶ **Exploración de datos.** Facilitan la exploración y el descubrimiento de datos, lo que les permite a los usuarios buscar, acceder y analizar datos de manera flexible y sin restricciones.
- ▶ **Integración de datos.** Permiten la integración de datos de diversas fuentes, incluyendo sistemas empresariales, aplicaciones web, dispositivos IoT, redes sociales y más.
- ▶ **Desarrollo de modelos de ML.** Son utilizados para entrenar y desarrollar modelos de ML y análisis predictivo, lo que proporciona un gran conjunto de datos para el aprendizaje automatizado.

Tema 3. Data science cloud storage

- ▶ **Análisis de *streaming*.** Pueden utilizarse para el análisis en tiempo real de datos de *streaming*, lo que permite la detección de patrones y la toma de decisiones en tiempo real.

Los ***data lakes*** son fundamentales en el ecosistema de datos moderno, ya que permiten a las organizaciones gestionar y analizar grandes volúmenes de datos de manera eficiente y flexible. Proporcionan una infraestructura escalable y económica para el almacenamiento de datos, así como capacidades avanzadas de análisis y exploración de datos, lo que les permite aprovechar al máximo su información y tomar decisiones basadas en datos de manera informada y estratégica. Seguidamente, examinemos las ventajas y las desventajas de un *data lake*.

Por el lado de las **ventajas** tenemos:

- ▶ **Almacenamiento sin procesar.** Permite almacenar datos en su forma original, sin procesar ni estructurar previamente, lo que proporciona una mayor flexibilidad y diversidad de datos.
- ▶ **Escalabilidad.** Los *data lakes* pueden manejar grandes volúmenes de datos, desde terabytes hasta petabytes o más, escalando horizontalmente según sea necesario.
- ▶ **Diversidad de datos.** Pueden almacenar una amplia variedad de tipos de datos, incluyendo datos estructurados, semiestructurados y no estructurados, así como datos en tiempo real y datos históricos.
- ▶ **Bajo costo.** Utilizan un enfoque de almacenamiento económico, ya que no requieren una estructura predefinida y pueden utilizar almacenamiento en la nube u otras soluciones de almacenamiento de bajo costo.
- ▶ **Facilidad de acceso.** Proporcionan capacidades de búsqueda y acceso flexible a los datos, lo que permite a los usuarios explorar y analizar datos de manera ágil y sin restricciones.

Tema 3. Data science cloud storage

- ▶ **Análisis avanzado.** Facilitan el análisis avanzado de datos a través de herramientas y plataformas de análisis de datos, lo que permite a las organizaciones extraer información valiosa y descubrir patrones ocultos en los datos.

Como **desventajas** tenemos:

- ▶ **Gestión de la calidad de los datos.** Debido a la diversidad y volumen de datos almacenados en un *data lake*, la gestión de la calidad de los datos puede ser un desafío, ya que es necesario garantizar la integridad, precisión y consistencia de los datos.
- ▶ **Seguridad y privacidad.** Los *data lakes* pueden contener datos sensibles o confidenciales, lo que plantea preocupaciones sobre la seguridad y la privacidad de los datos. Es importante implementar medidas de seguridad robustas, como cifrado de datos, control de acceso y monitorización de actividades, para proteger los datos almacenados en el *data lake*.
- ▶ **Complejidad de la gestión.** La gestión de un *data lake* puede ser compleja y requerir habilidades técnicas especializadas, e incluir la integración de datos, la optimización del rendimiento y la administración de la infraestructura de almacenamiento.
- ▶ **Riesgo de convertirse en un *data swamp*.** Si no se gestionan adecuadamente, los *data lakes* pueden correr el riesgo de convertirse en un *data swamp*, donde los datos se acumulan sin un propósito claro o sin ser utilizados de manera efectiva. Es importante establecer políticas y procedimientos para la gestión y gobernanza de datos con el fin de evitar este escenario.

Tema 3. Data science cloud storage

En resumen, aunque los **data lakes** ofrecen muchas ventajas en términos de flexibilidad, escalabilidad y análisis avanzado de datos, también presentan desafíos en términos de gestión de la calidad de los datos, seguridad y privacidad, complejidad de la gestión y riesgo de convertirse en un *data swamp*. Es importante abordar estos desafíos de manera proactiva para maximizar el valor de un *data lake* y garantizar su éxito en la organización.

Data warehouse

Un **data warehouse** es una base de datos diseñada específicamente para análisis y *reporting*, donde los datos son organizados y estructurados de manera que facilitan la consulta y el análisis. Utiliza un modelo dimensional o en estrella, que consiste en hechos (medidas cuantificables) que están rodeados por dimensiones (atributos descriptivos). Esto permite a los usuarios realizar consultas analíticas complejas y generar informes detallados sobre el rendimiento y las tendencias de negocio.

El **concepto** de *data warehouse* surgió en la década de 1980 y fue impulsado por las necesidades de las organizaciones para analizar grandes volúmenes de datos dispersos en diferentes sistemas operativos y bases de datos. La idea era crear un repositorio centralizado de datos que permitiera a las empresas consolidar y analizar información de múltiples fuentes para obtener una visión holística de sus operaciones y rendimiento.

William H. Inmon y **Ralph Kimball** son dos figuras clave en el desarrollo del concepto de *data warehouse* y en su evolución a lo largo de los años. Ambos han tenido un impacto significativo en la forma en que se diseñan, implementan y utilizan los *data warehouses* en la industria.

William H. Inmon, a menudo considerado como el padre del *data warehousing*, ha sido una figura influyente en el campo de la gestión de datos durante décadas. En la década de 1980, Inmon fue pionero en el desarrollo del concepto de *data warehouse*

Tema 3. Data science cloud storage

como una solución centralizada para almacenar y gestionar grandes volúmenes de datos empresariales.

Es conocido por su **enfoque de arquitectura centrado en los datos**, que enfatiza la importancia de la integración y la consistencia de los datos en un único repositorio centralizado. Inmon acuñó el término *data warehouse* y definió su arquitectura en su libro *Building the data warehouse*, publicado en 1992. En este libro, describió que un enfoque de *data warehouse* que prioriza la calidad y la integridad de los datos para garantizar la confiabilidad de la información analítica.

Ralph Kimball es otro influyente experto en el campo del *data warehousing*, quién es conocido por su enfoque de arquitectura dimensional. A diferencia de Inmon, Kimball aboga por un enfoque pragmático y orientado a usuarios para el diseño e implementación de *data warehouses*. Kimball es conocido por popularizar el **modelo dimensional**, que utiliza un diseño basado en estrellas o copos de nieve para representar los datos.

En contraste con el enfoque de Inmon, que se centra en la integración de datos y la normalización, el **enfoque de Kimball** prioriza la simplicidad y la facilidad de uso, lo que permite a los usuarios acceder y analizar datos de manera rápida y eficiente. Kimball ha escrito varios libros sobre *data warehousing*, incluyendo *The data warehouse Toolkit*, que proporciona orientación práctica sobre cómo diseñar y construir *data warehouses* utilizando su enfoque dimensional.

Las **contribuciones** de Inmon y Kimball al campo del *data warehousing* han sido significativas y complementarias. Mientras que Inmon estableció los fundamentos del *data warehouse* como una solución centralizada para la gestión de datos empresariales, Kimball introdujo enfoques prácticos y orientados a usuarios para su diseño e implementación. Ambos enfoques tienen sus ventajas y desventajas, y la elección entre ellos depende de los requisitos específicos y las prioridades de una organización. En la práctica, muchos proyectos de *data warehousing* combinan

Tema 3. Data science cloud storage

elementos de ambos enfoques para aprovechar sus fortalezas y mitigar sus debilidades.

Entre sus principales **características**, podemos destacar:

- ▶ **Orientado a temas.** Los datos se organizan en torno a temas de negocio específicos, lo que facilita la comprensión y el análisis de estos.
- ▶ **Integración de datos.** Los datos son extraídos, transformados y cargados desde diferentes fuentes de datos operativos para crear un repositorio centralizado y coherente de información.
- ▶ **Almacena datos estructurados y preprocesados** optimizados para consultas analíticas.
- ▶ **Utiliza un esquema predefinido y una estructura organizada** para garantizar la consistencia y la integridad de los datos.
- ▶ **Modelo dimensional.** Utiliza un modelo dimensional o en estrella para representar los datos, con hechos y dimensiones que permiten realizar consultas analíticas complejas.
- ▶ **Historización de datos.** Los *data warehouses* suelen mantener un historial de cambios en los datos a lo largo del tiempo, lo que permite realizar análisis comparativos y de tendencias.
- ▶ **Generación de informes.** Proporciona herramientas y funcionalidades para generar informes detallados y análisis de datos para apoyar la toma de decisiones empresariales.
- ▶ **Proporciona un rendimiento optimizado** para consultas complejas y análisis *ad hoc*.

En resumen, un **data warehouse** es un componente clave en la infraestructura de datos de una organización, proporcionando una plataforma centralizada y

Tema 3. Data science cloud storage

estructurada para el análisis y la generación de informes sobre el rendimiento y las operaciones de negocio. Surgió en la década de 1980 como respuesta a la necesidad de las organizaciones de gestionar y analizar grandes volúmenes de datos dispersos en diferentes sistemas y fuentes.

Los *data warehouses* son fundamentales para una variedad de aplicaciones empresariales y análisis de datos. Aquí tienes una descripción de los principales **usos** de un *data warehouse*:

- ▶ **Análisis de negocios.** Permite la generación de informes. Un *data warehouse* proporciona una plataforma centralizada para generar informes y paneles de control que muestran métricas clave de negocio, como ventas, ingresos, inventario y rendimiento financiero.
- ▶ **Análisis de tendencias y patrones históricos.** Permite analizar datos históricos y tendencias a lo largo del tiempo para identificar patrones, oportunidades y áreas de mejora en las operaciones comerciales.
- ▶ **Análisis de clientes.** Facilita el análisis del comportamiento del cliente, la segmentación de clientes y la evaluación del valor del cliente para mejorar la retención y la satisfacción del cliente.
- ▶ **Planificación empresarial.** Proporciona datos y análisis que respaldan la planificación estratégica y la toma de decisiones a largo plazo, lo que incluye la expansión de mercado, las fusiones y adquisiciones, y el desarrollo de productos.
- ▶ **Gestión del rendimiento.** Ayuda a monitorear y evaluar el rendimiento empresarial en relación con los objetivos y metas estratégicas, e identifica las áreas de mejora y las oportunidades para optimizar la eficiencia operativa.
- ▶ **Segmentación de mercado.** Permite segmentar el mercado en función de diferentes criterios demográficos, geográficos y de comportamiento, para orientar las estrategias de *marketing* y ventas de manera más efectiva.

Tema 3. Data science cloud storage

- ▶ **Análisis de canales.** Facilita el análisis del rendimiento de diferentes canales de *marketing* y ventas, como publicidad en línea, redes sociales, correo electrónico y ventas directas, para optimizar el retorno de la inversión (ROI).
- ▶ **Planificación de la demanda.** Ayuda a predecir la demanda de productos y servicios utilizando datos históricos y análisis predictivo, lo que permite una planificación más precisa de la producción y el inventario.
- ▶ **Optimización de inventarios.** Permite monitorear y optimizar los niveles de inventario en toda la cadena de suministro para reducir los costos de almacenamiento y mejorar la disponibilidad de productos.
- ▶ **Auditoría y cumplimiento.** Proporciona un registro centralizado y detallado de datos para fines de auditoría y cumplimiento normativo, ayudando a garantizar la precisión, la integridad y la seguridad de los datos.
- ▶ **Informes regulatorios.** Facilita la generación de informes y documentación necesaria para cumplir con regulaciones específicas de la industria, como Sarbanes-Oxley (SOX), GDPR y HIPAA.

Seguidamente, examinemos las ventajas y las desventajas de un *data warehouse*.

Por el lado de las **ventajas** tenemos:

- ▶ **Centralización de datos.** Un *data warehouse* centraliza datos de múltiples fuentes en un único repositorio, lo que facilita el acceso y la gestión de la información empresarial.
- ▶ **Datos estructurados para análisis.** Los datos en un *data warehouse* están estructurados y optimizados para análisis, lo que permite a los usuarios realizar consultas complejas y generar informes detallados sobre el rendimiento y las tendencias del negocio.

Tema 3. Data science cloud storage

- ▶ **Mejora la toma de decisiones.** Proporciona a los usuarios acceso a información actualizada y coherente, lo que facilita la toma de decisiones basada en datos y ayuda a identificar oportunidades de mejora y optimización.
- ▶ **Soporte para análisis avanzado.** Un *data warehouse* proporciona una plataforma robusta para análisis avanzado de datos, que incluye el análisis predictivo, la minería de datos y el modelado estadístico.
- ▶ **Histórico de datos.** Permite mantener un historial de cambios en los datos a lo largo del tiempo, lo que facilita el análisis comparativo y la identificación de tendencias a lo largo del tiempo.

Como **desventajas** tenemos:

- ▶ **Costo inicial y complejidad.** La implementación y mantenimiento de un *data warehouse* puede ser costosa y compleja, ya que requiere infraestructura de *hardware* y *software* especializada, así como personal técnico capacitado.
- ▶ **Tiempo de implementación.** El proceso de diseño, desarrollo e implementación de un *data warehouse* puede llevar mucho tiempo, lo que puede retrasar la disponibilidad de información para la toma de decisiones.
- ▶ **Dependencia de la calidad de los datos.** La calidad de los datos en un *data warehouse* es fundamental para la precisión y la confiabilidad de los análisis. Si los datos de entrada no son precisos o completos, los resultados del análisis pueden ser incorrectos o sesgados.
- ▶ **Rigidez en los esquemas de datos.** Los esquemas de datos en un *data warehouse* suelen ser rígidos y difíciles de cambiar una vez que están establecidos, lo que puede limitar la capacidad de adaptarse a cambios en los requisitos empresariales o en los datos de entrada.

Tema 3. Data science cloud storage

- ▶ **Volumen de datos limitado.** Aunque un *data warehouse* puede manejar grandes volúmenes de datos, puede haber limitaciones en términos de escalabilidad y rendimiento a medida que los volúmenes de datos continúan creciendo.

En resumen, un ***data warehouse*** ofrece numerosas ventajas en términos de centralización de datos, soporte para análisis avanzado y mejora de la toma de decisiones, pero también presenta desafíos en términos de costos, complejidad, calidad de los datos y rigidez en los esquemas de datos. Es importante evaluar cuidadosamente estas ventajas y desventajas al considerar la implementación de un *data warehouse* en una organización.

Data lakehouse

El concepto de ***data lakehouse*** surge como una combinación de las ventajas del *data lake* y del *data warehouse* que intenta abordar las limitaciones individuales de cada uno. Veamos más en detalle, un *data lakehouse* es un enfoque arquitectónico que combina las características de un *data lake* y un *data warehouse* en una única plataforma unificada. Esta combinación busca aprovechar la capacidad de almacenamiento ilimitada y la diversidad de datos del *data lake*, junto con la estructura y el rendimiento optimizado para el análisis del *data warehouse*.

Sus principales **características** son:

- ▶ **Almacenamiento flexible.** Al igual que un *data lake*, un *data lakehouse* permite el almacenamiento de datos en su forma original, sin procesar ni estructurar previamente, lo que proporciona una gran flexibilidad y diversidad de datos.
- ▶ **Estructura para el análisis.** A diferencia de un *data lake* puro, un *data lakehouse* proporciona una estructura optimizada para el análisis de datos, similar a un *data warehouse*, lo que facilita consultas analíticas complejas y generación de informes.

Tema 3. Data science cloud storage

- ▶ **Integración de datos.** Permite la integración de datos de múltiples fuentes en un único repositorio centralizado, lo que facilita el acceso y la gestión de la información empresarial.
- ▶ **Escalabilidad y rendimiento.** Un *data lakehouse* combina la escalabilidad del *data lake* con el rendimiento optimizado del *data warehouse*, lo que permite manejar grandes volúmenes de datos y realizar análisis avanzados de manera eficiente.
- ▶ **Análisis avanzado.** Proporciona capacidades avanzadas de análisis de datos, incluyendo análisis predictivo, ML y análisis de *big data*, que pueden aprovechar la diversidad y la profundidad de los datos almacenados en el *data lakehouse*.

Sus **ventajas** son:

- ▶ **Integración de capacidades.** Un *data lakehouse* combina las ventajas del *data lake* y del *data warehouse* en una única plataforma unificada, proporcionando flexibilidad, escalabilidad y capacidades avanzadas de análisis de datos.
- ▶ **Flexibilidad en el almacenamiento de datos.** Permite almacenar una amplia variedad de tipos de datos, desde datos estructurados hasta no estructurados, y utilizarlos para una variedad de aplicaciones y análisis.
- ▶ **Escalabilidad.** Puede manejar grandes volúmenes de datos y escalar horizontalmente según sea necesario para satisfacer las demandas cambiantes de almacenamiento y análisis.
- ▶ **Rendimiento optimizado para análisis de datos.** Proporciona un rendimiento optimizado para análisis de datos, lo que permite realizar consultas analíticas complejas de manera eficiente.
- ▶ **Integración de datos de múltiples fuentes.** Permite la integración de datos de múltiples fuentes en un único repositorio centralizado, lo que facilita el acceso y la gestión de la información empresarial.

Tema 3. Data science cloud storage

- ▶ **Capacidades avanzadas de análisis de datos.** Proporciona capacidades avanzadas de análisis de datos, incluyendo análisis predictivo, ML y análisis de *big data*, que pueden aprovechar la diversidad y la profundidad de los datos almacenados en el *data lakehouse*.
- ▶ **Facilidad de acceso y exploración de datos.** Facilita el acceso y la exploración de datos para usuarios de negocio y analistas, ya que permite realizar consultas *ad hoc* y descubrir *insights* de manera ágil.
- ▶ **Costos potencialmente reducidos.** Aunque la implementación y el mantenimiento de un *data lakehouse* pueden ser costosos, puede resultar en una reducción de costos a largo plazo al consolidar y optimizar la infraestructura de datos de la organización.

En resumen, un *data lakehouse* ofrece una serie de **ventajas** significativas en términos de flexibilidad, escalabilidad, rendimiento y capacidades avanzadas de análisis de datos, lo que lo convierte en una opción atractiva para las organizaciones que buscan aprovechar al máximo su información empresarial.

Aunque un *data lakehouse* ofrece una serie de ventajas en términos de flexibilidad, escalabilidad y capacidades avanzadas de análisis de datos, también presenta algunas desventajas y desafíos. Aquí hay algunas **desventajas** potenciales de un *data lakehouse*:

- ▶ **Complejidad de implementación y mantenimiento.** Integrar las capacidades de un *data lake* y un *data warehouse* en una única plataforma unificada puede ser complejo y requerir una planificación cuidadosa, así como habilidades técnicas especializadas para la implementación y el mantenimiento.

Tema 3. Data science cloud storage

- ▶ **Costes.** La implementación y el mantenimiento de un *data lakehouse* pueden ser costosos, ya que puede requerir infraestructura de *hardware* y *software* especializada, así como recursos humanos calificados para gestionar y mantener la plataforma.
- ▶ **Gestión de la calidad de los datos.** Mantener la calidad y la integridad de los datos en un entorno tan diverso puede ser un desafío, especialmente cuando se combinan datos estructurados y no estructurados de múltiples fuentes.
- ▶ **Seguridad y privacidad de los datos.** Almacenar una gran cantidad de datos en un único repositorio puede plantear preocupaciones sobre la seguridad y la privacidad de los datos, lo que requiere medidas de seguridad robustas para proteger los datos contra accesos no autorizados y filtraciones de datos.
- ▶ **Gestión del ciclo de vida de los datos.** Gestionar el ciclo de vida completo de los datos, incluyendo la ingestión, transformación, almacenamiento y eliminación de datos, puede ser complejo y requerir una planificación cuidadosa para garantizar la eficiencia y la conformidad con los requisitos regulatorios.
- ▶ **Riesgo de convertirse en un *data swamp*.** Si no se gestionan adecuadamente, un *data lakehouse* corre el riesgo de convertirse en un *data swamp*, donde los datos se acumulan sin un propósito claro o sin ser utilizados de manera efectiva. Es importante establecer políticas y procedimientos para la gestión y la gobernanza de datos con el fin de evitar este escenario.

En resumen, aunque un *data lakehouse* ofrece muchas ventajas en términos de flexibilidad, escalabilidad y capacidades avanzadas de análisis de datos, también presenta desafíos en términos de complejidad de implementación y mantenimiento, costos, gestión de la calidad de los datos, seguridad y privacidad de los datos, gestión del ciclo de vida de los datos y riesgo de convertirse en un *data swamp*. Es importante abordar estos desafíos de manera proactiva para maximizar el valor de un *data lakehouse* y garantizar su éxito en la organización.

Tema 3. Data science cloud storage

Sus principales **desafíos** son los siguientes:

- ▶ **Gestión de la calidad de los datos.** Mantener la calidad y la integridad de los datos en un entorno tan diverso puede ser un desafío, especialmente cuando se combinan datos estructurados y no estructurados.
- ▶ **Seguridad y privacidad.** Almacenar una gran cantidad de datos en un único repositorio puede plantear preocupaciones sobre la seguridad y la privacidad de los datos, lo que requiere medidas de seguridad robustas.
- ▶ **Gestión del ciclo de vida de los datos.** Gestionar el ciclo de vida de los datos, incluyendo la ingestión, transformación, almacenamiento y eliminación de datos, puede ser complejo y requerir una planificación cuidadosa.

En resumen, un **data lakehouse** combina las ventajas del *data lake* y del *data warehouse* en una única plataforma unificada, proporcionando flexibilidad, escalabilidad, rendimiento y capacidades avanzadas de análisis de datos. Aunque presenta desafíos en términos de gestión de la calidad de los datos, seguridad y privacidad, y gestión del ciclo de vida de los datos, ofrece un enfoque poderoso para el almacenamiento y el análisis de datos en entornos empresariales modernos.

Enfoques de gestión de arquitectura de datos

Data mesh

Si tenemos que definir el concepto de diseño **data mesh** en forma simple, deberíamos decir que es la construcción de una infraestructura de autoservicio que permite a las diferentes unidades de negocio dentro de una empresa de utilizar recursos y herramientas bajo demanda, para acceder a los datos correctos, procesarlos, prepararlos y analizarlos.

Data mesh nace con el claro **propósito** de la descentralización del dato. Dota de la capacidad a toda organización de deconstruir los silos de información aislada tan existentes en las empresas y eliminar la complejidad que conlleva la centralización.

Tema 3. Data science cloud storage

De esta manera, permite impulsar nuevos planteamientos estratégicos, lograr una gestión eficiente del negocio y caminar hacia una cultura *data-driven*. **Zhamak Dehghani** es la principal fundadora e impulsora de este paradigma.

¿Como definiríamos *data mesh*? Como una **técnica organizativa** de la descentralización del dominio de datos, la transformación y la entrega de los datos. Surge como una solución a arquitecturas centralizadas en las que su crecimiento se ve limitado por sus dependencias y su complejidad.

A medida que crece el número de fuentes y consumidores de datos, también lo hace el número de **canalizaciones de datos** necesarios para conectarlos. Esto hizo imprescindible contar con equipos especialistas en la carga de datos y habilidades para desarrollar tecnologías y administrar toda esa información. Esto fue generando paulatinamente un distanciamiento de quienes necesitan utilizar esos datos y aquellos que los gestionan.

Por eso, durante los últimos años, la **arquitectura** de *data mesh* surgió para resolver el desafío de las barreras de datos y así lograr una democratización de la información. A continuación, profundizaremos sobre sus características esenciales.

La **evolución** de las arquitecturas centralizadas tradicionales hacia una arquitectura descentralizada *data mesh* se puede ver en la Figura 7.

Tema 3. Data science cloud storage

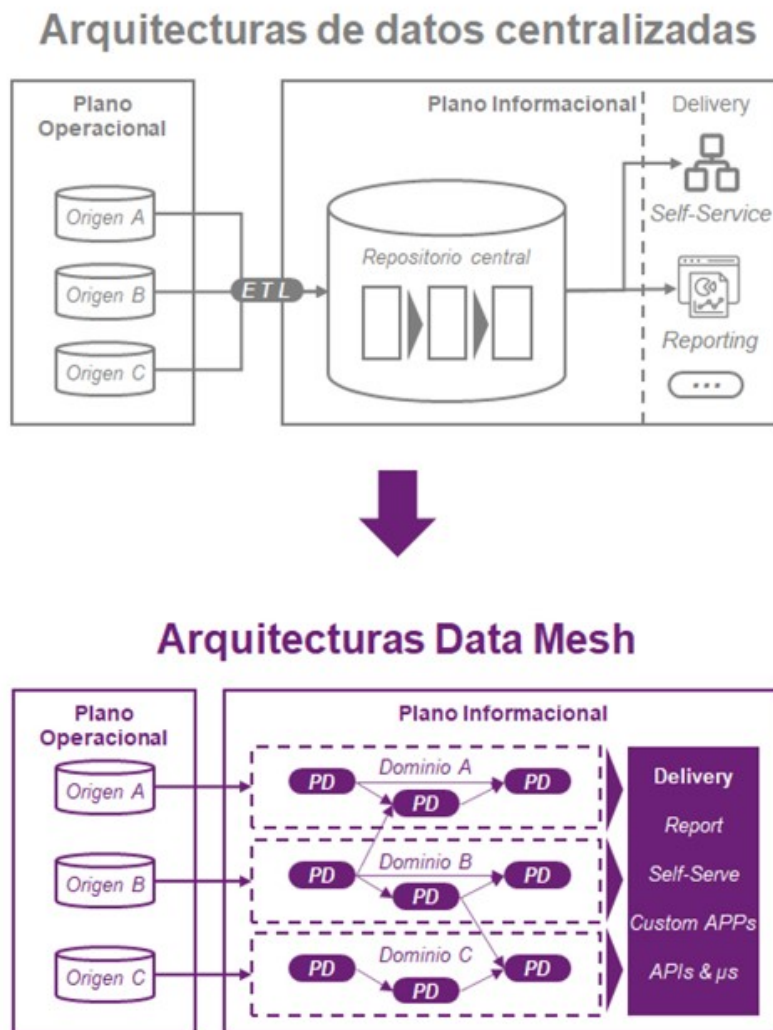


Figura 7. Evolución de las arquitecturas. Fuente: Rigoberto M., 2022.

Los **principios** del *data mesh* son los fundamentos que guían la implementación de este enfoque en la gestión de datos dentro de una organización. Teniendo en cuenta que la propiedad fundamental de una arquitectura *data mesh* es la descentralización de la información, sus principales principios de diseño serían los siguientes:

- **Descentralización.** Promueve la descentralización de la gestión de datos, donde cada dominio de negocio es responsable de sus propios datos y procesos de datos.

Tema 3. Data science cloud storage

- ▶ **Autonomía de los equipos.** Fomenta la formación de equipos multidisciplinares y autónomos en cada dominio de negocio, que son responsables de definir, gestionar y operar sus propios datos.
- ▶ **Orientación a servicios.** Se centra en la exposición de datos a través de servicios estandarizados y bien definidos, que facilitan la integración y el intercambio de datos entre diferentes dominios de negocio.
- ▶ **Gobernanza distribuida.** Establece políticas y prácticas de gobernanza de datos en cada dominio de negocio, que son aplicadas y gestionadas localmente por los equipos responsables de esos datos.
- ▶ **Arquitectura distribuida.** Propone una arquitectura distribuida y descentralizada para manejar la complejidad de los sistemas de datos, donde los datos se distribuyen y almacenan en cada dominio de negocio en lugar de centralizarse en un único repositorio de datos.
- ▶ **Evita el modelo de ingestión-centralización-transformación-distribución.** Se aleja del enfoque tradicional de ingestión centralizada, transformación y distribución de datos, y propone un modelo donde los datos se gestionan y transforman en el origen, antes de ser expuestos a través de servicios.
- ▶ **Cultura de datos como producto.** Fomenta una cultura centrada en los datos, donde los datos se tratan como productos que se desarrollan, gestionan y mejoran continuamente para satisfacer las necesidades cambiantes del negocio.
- ▶ **Transparencia y visibilidad.** Promueve la transparencia y la visibilidad de los datos en toda la organización, lo que permite que los usuarios entiendan y accedan fácilmente a los datos disponibles en la empresa.

Tema 3. Data science cloud storage

- **Colaboración y comunidad.** Fomenta la colaboración y la construcción de una comunidad en torno a la gestión de datos, donde los equipos comparten conocimientos, herramientas y mejores prácticas para mejorar la eficacia y la calidad de los datos en toda la organización.

Estos son algunos de los principios comunes asociados con el *data mesh*, que proporcionan una **guía para la implementación** de este enfoque en la gestión de datos dentro de una organización.

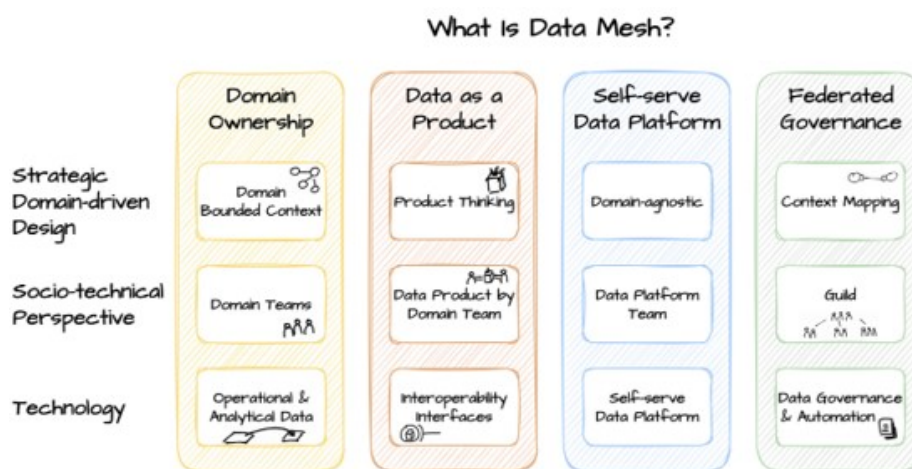


Figura 8. *Data mesh*. Fuente: Kumar, 2023.

Una **arquitectura** de *data mesh* se compone de varios elementos fundamentales que trabajan en conjunto para facilitar la gestión descentralizada y distribuida de los datos en una organización. Aquí están los elementos principales:

- **Dominios de datos.** Los dominios de datos representan áreas de negocio o funciones dentro de la organización, cada uno con su propio conjunto de datos y procesos de datos. Estos dominios son propietarios de sus datos y responsables de su gestión y operación.
- **Equipos autónomos.** Cada dominio de datos está asociado con un equipo multidisciplinario y autónomo que es responsable de definir, gestionar y operar los

Tema 3. Data science cloud storage

datos en ese dominio. Estos equipos tienen autoridad para tomar decisiones sobre la gestión de datos en su dominio, incluyendo la definición de modelos de datos, la implementación de políticas de gobernanza y la exposición de datos a través de servicios.

- ▶ **Una plataforma de autoservicio** puede tener múltiples planos, cada uno de los cuales sirve a un perfil diferente de usuarios. En el siguiente ejemplo, se enumeran tres planos de plataforma de datos diferentes:
 - **Plano de aprovisionamiento de infraestructura de datos.** Admite el aprovisionamiento de la infraestructura subyacente, necesaria para ejecutar los componentes de un producto de datos y la malla de productos. Esto incluye el aprovisionamiento de un almacenamiento de archivos distribuido, cuentas de almacenamiento, sistema de administración de control de acceso, la orquestación para ejecutar el código interno de los productos de datos, el aprovisionamiento de un motor de consulta distribuido en un gráfico de productos de datos, etc. Solo los desarrolladores de productos de datos avanzados usan esta interfaz directamente. Este es un plano de gestión del ciclo de vida de la infraestructura de datos de nivel bastante bajo.
 - **Plano de experiencia del desarrollador de productos de datos.** Es la interfaz principal que utiliza un desarrollador de productos de datos típico. Esta interfaz abstrae muchas de las complejidades de lo que implica respaldar el flujo de trabajo de un desarrollador de productos de datos. Proporciona un mayor nivel de abstracción que el plano de aprovisionamiento. Utiliza interfaces declarativas simples para administrar el ciclo de vida de un producto de datos. Implementa automáticamente las preocupaciones transversales que se definen como un conjunto de estándares y convenciones globales, que se aplican a todos los productos de datos y sus interfaces.
 - **Plano de supervisión de malla de datos.** Hay un conjunto de capacidades que se proporcionan mejor a nivel de malla, un gráfico de productos de datos conectados, globalmente. Si bien la implementación de cada una de estas interfaces puede

Tema 3. Data science cloud storage

dependen de capacidades de productos de datos individuales, es más conveniente proporcionar estas capacidades al nivel de la malla. Por ejemplo, la capacidad de descubrir productos de datos para un caso de uso particular se proporciona mejor mediante la búsqueda o la exploración de la red de productos de datos; o la correlación de múltiples productos de datos para crear una perspectiva de mayor orden, se proporciona mejor a través de la ejecución de una consulta semántica de datos que puede operar en múltiples productos de datos en la malla.

En la Figura 9 podemos ver una ilustración de estos tres planos.

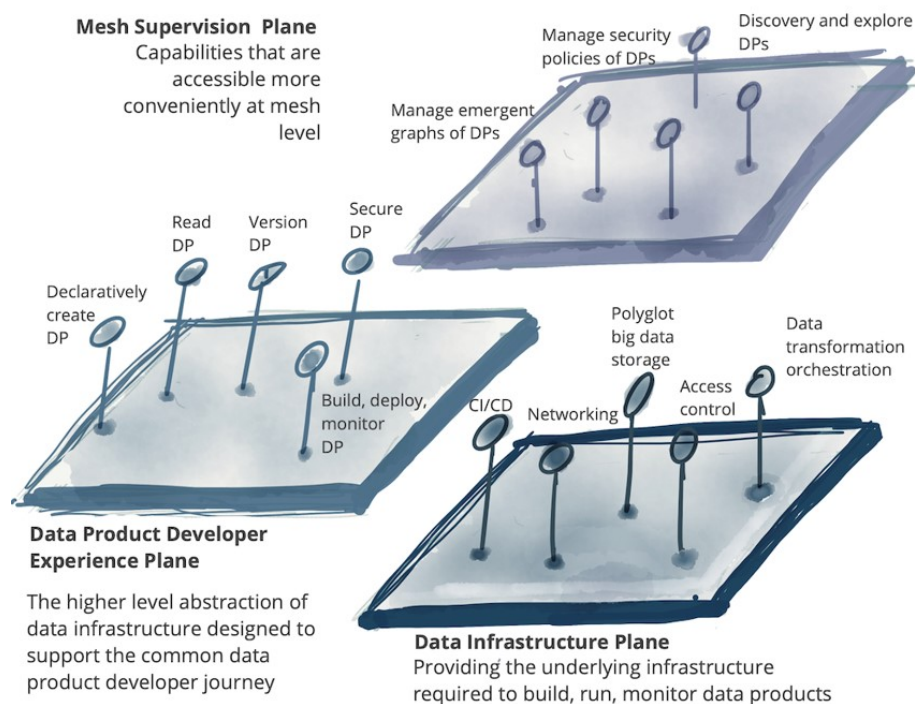


Figura 9. Planos de una plataforma de autoservicio. Fuente: Dehghani, 2020.

- **Catálogo de datos.** Proporciona un catálogo centralizado de datos que permite a los usuarios descubrir, entender y acceder a los datos disponibles en la organización. Este catálogo incluye metadatos sobre los datos, como su origen, estructura, calidad y uso.

Tema 3. Data science cloud storage

Algunas de las **ventajas** que detectamos son las siguientes:

- ▶ **Descentralización y autonomía.** La arquitectura de *data mesh* permite a cada dominio de negocio ser responsable de sus propios datos y procesos de datos, lo que fomenta la autonomía y la toma de decisiones localizada.
- ▶ **Flexibilidad y agilidad.** Al distribuir la gestión de datos en equipos autónomos, la arquitectura de *data mesh* permite una mayor flexibilidad y agilidad para adaptarse a las necesidades y a los cambios en el negocio.
- ▶ **Mejora de la calidad de los datos.** Al fomentar la propiedad y la responsabilidad de los datos en cada dominio de negocio, la arquitectura de *data mesh* puede mejorar la calidad y la integridad de los datos en toda la organización.
- ▶ **Escalabilidad.** La arquitectura de *data mesh* puede escalar horizontalmente según sea necesario para manejar grandes volúmenes de datos y demandas de procesamiento, ya que cada dominio de negocio puede escalar de forma independiente.
- ▶ **Fomenta la innovación y la experimentación.** Al permitir que los equipos autónomos experimenten e iteren con sus propios datos y procesos de datos, la arquitectura de *data mesh* fomenta la innovación y la mejora continua en la gestión de datos.

Sin embargo, también supone **retos** a resolver:

- ▶ **Complejidad organizativa y cultural.** La implementación de una arquitectura de *data mesh* puede requerir cambios significativos en la estructura organizativa y en la cultura empresarial, lo que puede ser difícil de lograr en organizaciones grandes y establecidas.

Tema 3. Data science cloud storage

- ▶ **Gestión de la complejidad técnica.** La distribución y la descentralización de los datos y los procesos de datos pueden introducir una mayor complejidad técnica en el manejo de datos, lo que puede requerir nuevas herramientas y habilidades para gestionar eficazmente esta complejidad.
- ▶ **Coordinación y alineación.** Aunque la descentralización puede fomentar la agilidad y la autonomía en los equipos de datos, también puede plantear desafíos en términos de coordinación y alineación entre diferentes dominios de negocio, especialmente en organizaciones grandes y complejas.
- ▶ **Riesgo de duplicación de esfuerzos.** La descentralización de la gestión de datos puede llevar a la duplicación de esfuerzos y recursos en diferentes dominios de negocio, lo que puede resultar en redundancias y falta de eficiencia en la gestión de datos.
- ▶ **Gestión de la seguridad y privacidad.** La distribución de los datos en diferentes dominios de negocio puede plantear desafíos en términos de seguridad y privacidad de los datos, lo que requiere medidas de seguridad robustas para proteger los datos contra accesos no autorizados y filtraciones de datos.

En resumen, una **arquitectura de data mesh** ofrece una serie de ventajas en términos de descentralización, autonomía, flexibilidad y escalabilidad, pero también presenta desafíos en términos de complejidad organizativa y técnica, coordinación y alineación, riesgo de duplicación de esfuerzos y gestión de la seguridad y privacidad. Es importante evaluar cuidadosamente estas ventajas y desventajas al considerar la implementación de una arquitectura *data mesh*.

Tema 3. Data science cloud storage

El **paradigma data mesh** es reciente y difícil de implementar debido a su naturaleza descentralizada, pero se convierte en un requisito indispensable para resolver problemas de escalabilidad a los que muchas compañías se enfrentan hoy en día. Las arquitecturas centralizadas sirven mejor a compañías pequeñas o medianas que no son *data-driven* o, dicho de otra manera, que no basan sus decisiones estratégicas en el análisis de datos.

Entonces, ¿cómo saber si una compañía necesita migrar a un *data mesh*? La respuesta a esta pregunta depende de las **resistencias** que tenga actualmente una organización. Si encuentra problemas de escalabilidad, dependencias complejas entre equipos, problemas con la calidad del dato, cuellos de botella en flujos de datos o problemas de gobernanza y seguridad, probablemente necesite plantear una posible migración al paradigma *data mesh*.

Hay muchos factores, como los mencionados anteriormente, que se deben analizar antes de dar este paso. Dicho esto, cuando se pretenda responder a esta pregunta se deberán tener en cuenta principalmente tres **factores**:

- ▶ El número de **fuentes de datos** de la compañía.
- ▶ El tamaño del **equipo** dedicado a los datos.
- ▶ El número de **dominios de negocio** que tiene la compañía.



Figura 10. Factores de necesidad en el *data mesh*. Fuente: Grande y Labella, 2022.

Tema 3. Data science cloud storage

Pero ¿cuándo tiene sentido aplicar este enfoque de arquitectura de datos? Existen muchos ejemplos y escenarios tanto desde el punto de vista para eficientizar la operativa de la información como desde el punto de vista de explotación y analítica. Algunos ejemplos de **casos de uso** serían:

- ▶ **Análisis de datos distribuidos.** Una empresa con múltiples unidades de negocio puede utilizar una arquitectura de *data mesh* para integrar datos dispersos en diferentes sistemas y ubicaciones, lo que les permite a los analistas acceder y analizar datos de manera centralizada para obtener una visión unificada del rendimiento empresarial.
- ▶ **Integración de datos en la nube.** Una empresa que migra sus sistemas a la nube puede utilizar una arquitectura de *data mesh* para integrar datos dispersos en diferentes plataformas en la nube, lo que les permite a los usuarios acceder y analizar datos de manera transparente desde cualquier ubicación.
- ▶ **Gestión de datos de IoT.** Una empresa que recopila datos de sensores y dispositivos de IoT puede utilizar una arquitectura de *data mesh* para integrar y procesar datos de manera distribuida, lo que permite el análisis en tiempo real y la detección de patrones y anomalías en los datos.
- ▶ **Análisis de datos en tiempo real.** Una empresa que necesita analizar grandes volúmenes de datos en tiempo real puede utilizar una arquitectura de *data mesh* para distribuir el procesamiento de datos en diferentes nodos, lo que permite análisis en tiempo real y la toma de decisiones basada en datos en tiempo real.
- ▶ **Gestión de datos de clientes.** Una empresa que recopila datos de clientes de diferentes fuentes, como transacciones, interacciones en redes sociales y comentarios en línea, puede utilizar una arquitectura de *data mesh* para integrar y analizar estos datos de manera centralizada, lo que permite una comprensión más profunda del comportamiento del cliente y la personalización de la experiencia del cliente.

Tema 3. Data science cloud storage

- **Análisis de riesgos financieros.** Una institución financiera puede utilizar una arquitectura de *data mesh* para integrar y analizar datos dispersos en diferentes sistemas y ubicaciones, lo que permite la identificación y la mitigación de riesgos financieros de manera más eficiente y efectiva.

Estos son solo algunos ejemplos de casos de uso de una arquitectura de *data mesh*, que demuestran su versatilidad y aplicabilidad en una amplia variedad de entornos empresariales y verticales de la industria.

Data fabric

Data fabric es un enfoque arquitectónico que aborda los desafíos de la gestión de datos en entornos empresariales cada vez más complejos y distribuidos. Se trata de una estrategia integral que busca proporcionar una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, lo que facilita su acceso, gestión y análisis de manera eficiente y efectiva.

En un mundo donde los datos están distribuidos en una amplia variedad de sistemas y ubicaciones, como bases de datos locales y en la nube, sistemas de archivos, dispositivos móviles y dispositivos de IoT, entre otros, gestionar y operar estos datos de manera eficiente puede ser un desafío significativo para las organizaciones. *Data fabric* aborda este desafío proporcionando una **capa unificada** de virtualización, integración, gestión de metadatos, seguridad y cumplimiento sobre los datos dispersos, lo que les permite a los usuarios acceder y analizar los datos de manera fluida y transparente, independientemente de su ubicación o formato.

Tema 3. Data science cloud storage

La clave de *data fabric* es su capacidad para crear una **vista unificada** de los datos dispersos en diferentes sistemas y ubicaciones, lo que permite a los usuarios acceder a ellos de manera transparente sin necesidad de conocer su ubicación física o la complejidad subyacente de la infraestructura de datos. Esto simplifica la gestión y operación de los datos en la organización, mejora la flexibilidad y la agilidad para adaptarse a las necesidades y cambios en el negocio, y acelera el desarrollo y la entrega de soluciones basadas en datos.

En resumen, *data fabric* es un **enfoque arquitectónico integral** diseñado para abordar los desafíos de la gestión de datos en entornos empresariales complejos y distribuidos, proporcionando una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, lo que facilita su acceso, gestión y análisis de manera eficiente y efectiva.

Sus **características** principales son las siguientes:

- ▶ **Virtualización de datos.** *Data fabric* utiliza técnicas de virtualización para proporcionar una vista unificada de los datos, lo que les permite a los usuarios acceder a ellos sin necesidad de conocer su ubicación física o la complejidad subyacente de la infraestructura de datos.
- ▶ **Integración de datos.** Permite la integración de datos de múltiples fuentes, incluyendo sistemas locales y en la nube, bases de datos relacionales y no relacionales, sistemas de archivos y servicios web, entre otros.
- ▶ **Gestión de metadatos.** *Data fabric* incluye una capa de metadatos que proporciona información sobre los datos disponibles en la organización, incluyendo su origen, estructura, calidad y uso. Esto facilita la búsqueda, descubrimiento y comprensión de los datos por parte de los usuarios.

Tema 3. Data science cloud storage

- ▶ **Seguridad y cumplimiento.** Proporciona mecanismos de seguridad robustos para proteger los datos contra accesos no autorizados y cumplir con los requisitos regulatorios y de cumplimiento, independientemente de su ubicación o formato.
- ▶ **Escalabilidad y rendimiento.** *Data fabric* está diseñado para escalar horizontalmente según sea necesario para manejar grandes volúmenes de datos y demandas de procesamiento, proporcionando un rendimiento óptimo para el acceso y análisis de datos.

Los **componentes** de una arquitectura *data fabric* son:

- ▶ **Capa de virtualización de datos.** Proporciona una vista unificada de los datos dispersos en diferentes sistemas y ubicaciones, lo que les permite a los usuarios acceder a ellos de manera transparente.
- ▶ **Capa de integración de datos.** Facilita la integración de datos de múltiples fuentes, transformando y combinando los datos según sea necesario para satisfacer las necesidades de los usuarios.
- ▶ **Capa de gestión de metadatos.** Almacena y gestiona los metadatos sobre los datos disponibles en la organización, lo que proporciona información sobre su origen, estructura, calidad y uso.
- ▶ **Capa de seguridad y cumplimiento.** Proporciona mecanismos de seguridad y cumplimiento para proteger los datos contra accesos no autorizados y garantizar el cumplimiento de los requisitos regulatorios y de cumplimiento.

Tema 3. Data science cloud storage

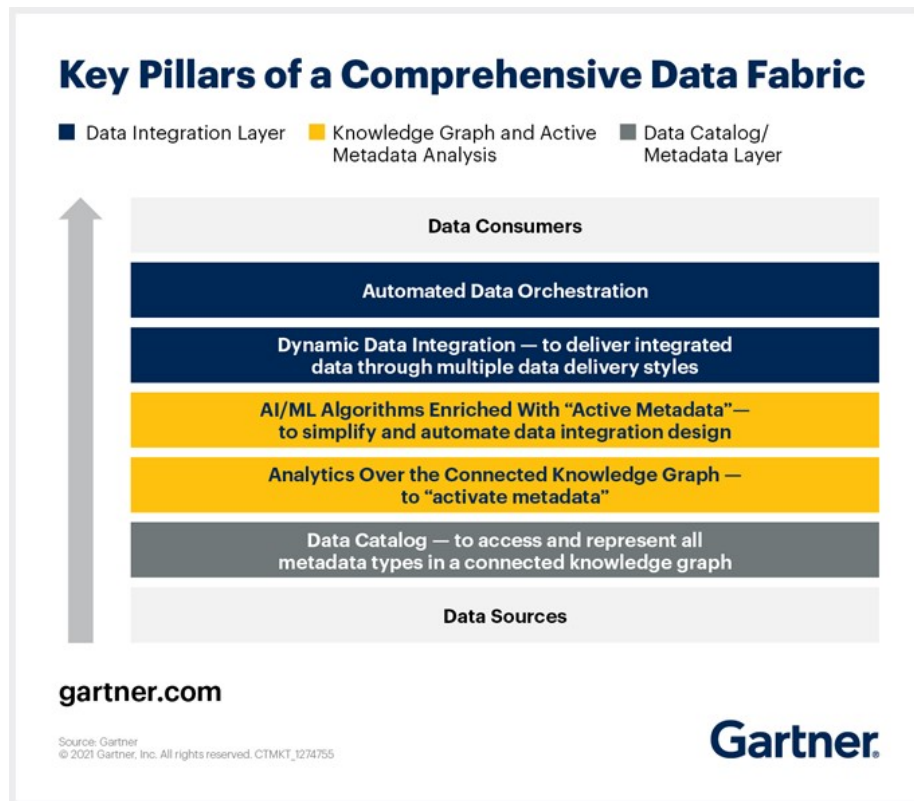


Figura 11. Pilares del *data fabric*. Fuente: [Gartner](https://www.gartner.com).

Sus principales **ventajas** son:

- ▶ **Simplificación de la gestión de datos.** Proporciona una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, lo que simplifica la gestión y operación de los datos en la organización.
- ▶ **Flexibilidad y agilidad.** Permite a los usuarios acceder y analizar datos de manera fluida y eficiente, independientemente de su ubicación o formato, lo que facilita la toma de decisiones basada en datos y la innovación en la organización.
- ▶ **Mejora del tiempo de comercialización.** Al facilitar la integración y el acceso a los datos, *data fabric* puede acelerar el desarrollo y la entrega de soluciones basadas en datos, lo que reduce el tiempo de comercialización y mejora la competitividad de la empresa.

Tema 3. Data science cloud storage

Entre los principales **desafíos de implementación** de una arquitectura *data fabric*, podemos destacar:

- ▶ **Complejidad técnica.** Implementar y gestionar una arquitectura de *data fabric* puede ser complejo, ya que requiere integrar múltiples tecnologías y sistemas en una única plataforma unificada.
- ▶ **Gestión del cambio organizativo.** Adoptar un enfoque de *data fabric* puede requerir cambios significativos en la cultura y la estructura organizativa, así como en las prácticas de gestión de datos, lo que puede ser difícil de lograr en organizaciones grandes y establecidas.

En resumen, ***data fabric*** es una arquitectura que proporciona una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, lo que facilita el acceso, la gestión y el análisis de datos en la organización. Si bien ofrece numerosas ventajas en términos de simplificación de la gestión de datos, flexibilidad y mejora del tiempo de comercialización, también presenta desafíos en términos de complejidad técnica y gestión del cambio organizativo.

Una vez que las organizaciones resuelven estos problemas, pueden comenzar a explorar nuevos **casos de uso** de estructuras de datos como los siguientes.

- ▶ **Mejorar la aplicación de IA sobre los datos.** Un enfoque de arquitectura *data fabric* puede proporcionar a los *data science* el acceso integral a los datos que les permita aplicar técnicas avanzadas de IA para mejorar la toma de decisiones. Por ejemplo, el mantenimiento predictivo requiere un acceso optimizado a los datos en tiempo real que proporciona una estructura de datos.
- ▶ **Mejorar la seguridad.** Una estructura de datos también puede mejorar las aplicaciones de seguridad al vincular datos y aplicaciones de todos los sistemas físicos y de TI. Por ejemplo, un equipo podría mejorar la seguridad al vincular la información de los lectores de llaves que se utilizan para abrir puertas, lo que podría correlacionarse con los datos de eventos de los sistemas informáticos a los que se

Tema 3. Data science cloud storage

accede desde dentro de la instalación. Esto permitiría realizar un análisis más sofisticado del comportamiento típico y anómalo para activar alertas de seguridad en tiempo real cuando sea necesario.

- ▶ **Crear una visión holística del cliente.** Las organizaciones también pueden usar una arquitectura *data fabric* para entrelazar datos de las actividades de un cliente junto con los diversos roles que interactúan con ellos para obtener una visión más holística. Esto podría incorporar datos en tiempo real de diversas actividades de ventas, realización de ingresos potenciales, tiempo de incorporación de clientes o métricas de satisfacción del cliente.
- ▶ **Mejorar la comprensión empresarial.** Las empresas también pueden usar la estructura de datos para crear una visión más holística del negocio en todas las actividades y departamentos.
- ▶ **Facilitar la productivización de algoritmos.** *Data fabric* se pueden usar para entrenar, configurar e implementar algoritmos de predicción simples y desencadenar acciones que se ejecutan en varios puntos finales de aplicaciones empresariales. Estos tipos de casos de uso abarcan todo, desde la trazabilidad de la seguridad hasta el cumplimiento de auditorías y los eventos que generan ingresos, como la acción de abandono del carrito, la optimización de anuncios, la retención de clientes, el *marketing* e incluso la venta orquestada.
- ▶ **Implementación de *market places* de datos.** Las empresas que implementan una arquitectura de estructura de datos también pueden configurar un mercado de datos más accesible que facilite a los desarrolladores ciudadanos entretener fuentes de datos dispares en nuevos modelos. Un mercado de datos permite a los ingenieros de datos configurar una infraestructura que se puede usar en múltiples casos de uso en lugar de crear una nueva infraestructura para cada caso de uso individualmente.

Tema 3. Data science cloud storage

Diferencias data mesh vs. data fabric

Tanto *data mesh* como *data fabric* son enfoques arquitectónicos destinados a abordar los desafíos de la gestión de datos en entornos empresariales cada vez más complejos y distribuidos. Aunque comparten algunos conceptos y objetivos comunes, también presentan diferencias significativas en términos de enfoque, implementación y alcance. Aquí están las principales **diferencias** entre *data mesh* y *data fabric*.

Enfoque

- ▶ **Data mesh.** *Data mesh* se centra en descentralizar la gestión de datos al promover la autonomía de los equipos en cada dominio de negocio para definir, gestionar y operar sus propios datos. Proporciona una arquitectura distribuida y descentralizada para manejar la complejidad de los sistemas de datos, donde los datos se distribuyen y almacenan en cada dominio de negocio en lugar de centralizarse en un único repositorio de datos.
- ▶ **Data fabric.** *Data fabric* se centra en proporcionar una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones. Utiliza técnicas de virtualización de datos, integración de datos, gestión de metadatos, seguridad y cumplimiento para crear una vista unificada de los datos, lo que facilita su acceso, gestión y análisis de manera eficiente y efectiva.

Alcance

- ▶ **Data mesh.** *Data mesh* se centra en la gestión descentralizada de datos en cada dominio de negocio, lo que promueve la autonomía y la responsabilidad de los equipos en la gestión de sus propios datos. Proporciona un marco para integrar y operar datos de manera distribuida, lo que permite a los equipos acceder y analizar datos de manera fluida y transparente.

Tema 3. Data science cloud storage

- ▶ **Data fabric.** *Data fabric* se centra en proporcionar una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, independientemente de quién sea responsable de su gestión. Proporciona una infraestructura tecnológica para integrar, gestionar y analizar datos de manera centralizada, lo que facilita el acceso y el análisis de datos en toda la organización.

Implementación

- ▶ **Data mesh.** La implementación de *data mesh* requiere cambios significativos en la estructura organizativa y en la cultura empresarial, así como en las prácticas de gestión de datos. Se centra en la descentralización de la gestión de datos en cada dominio de negocio al promover la autonomía y la responsabilidad de los equipos en la gestión de sus propios datos.
- ▶ **Data fabric.** La implementación de *data fabric* requiere integrar múltiples tecnologías y sistemas en una única plataforma unificada. Se centra en proporcionar una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones, lo que facilita su acceso, gestión y análisis de manera eficiente y efectiva.

En resumen, *data mesh* y *data fabric* son enfoques arquitectónicos destinados a abordar los desafíos de la gestión de datos en entornos empresariales cada vez más complejos y distribuidos. Aunque comparten algunos conceptos y objetivos comunes, también presentan **diferencias significativas** en términos de enfoque, implementación y alcance. *Data mesh* se centra en descentralizar la gestión de datos en cada dominio de negocio, mientras que *data fabric* se centra en proporcionar una capa unificada y coherente sobre los datos dispersos en diferentes sistemas y ubicaciones.

Tema 3. Data science cloud storage

3.3. Sistemas de almacenamiento: AWS

Sistema de almacenamiento AWS S3

Amazon S3 es un servicio ofrecido por AWS que proporciona almacenamiento de objetos a través de una interfaz de servicio web. Es uno de los servicios de almacenamiento en la nube más populares y ampliamente utilizados ofrecidos por AWS. Es un servicio de almacenamiento de objetos altamente escalable, seguro y duradero, diseñado para almacenar y recuperar grandes volúmenes de datos de manera eficiente y confiable. S3 es una solución versátil que se adapta a una amplia gama de casos de uso, desde almacenamiento de datos empresariales hasta alojamiento de sitios web estáticos y entrega de contenido multimedia a escala mundial, que ha creado el usuario a dicho archivo y adjunta un identificador personalizado.

Respecto a AWS S3, sus creadores lo definen como un **servicio de almacenamiento multipropósito basado en la nube**, que permite guardar y recuperar datos, cualquiera que sea su volumen, en el que se paga de acuerdo con la capacidad de almacenamiento utilizada y al volumen de transferencias realizadas. En ese sentido, puede ser utilizado para almacenar sitios web estáticos, realizar cualquier *backup* o copia de seguridad de archivos, alojar imágenes, entre otros.

Características clave de Amazon S3

- ▶ **Escalabilidad.** Amazon S3 permite almacenar una cantidad ilimitada de datos y escalar horizontalmente según sea necesario para satisfacer las demandas de almacenamiento de cualquier tamaño.
- ▶ **Durabilidad y disponibilidad.** Ofrece una alta durabilidad de datos mediante la replicación y distribución de objetos en múltiples ubicaciones dentro de una región de AWS, lo que garantiza la disponibilidad de datos incluso en caso de fallas de *hardware* o interrupciones del servicio.

Tema 3. Data science cloud storage

- ▶ **Seguridad.** Amazon S3 ofrece una amplia gama de características de seguridad, incluyendo control de acceso basado en políticas, cifrado de datos en reposo y en tránsito, y monitoreo de acceso a los datos para proteger la confidencialidad, integridad y disponibilidad de la información almacenada.
- ▶ **Facilidad de uso.** S3 proporciona una interfaz de usuario intuitiva y una API robusta que facilita la creación, gestión y automatización de operaciones de almacenamiento de datos.
- ▶ **Flexibilidad.** Soporta una variedad de tipos de datos, incluyendo archivos, imágenes, vídeos, documentos y datos estructurados, lo que lo hace adecuado para una amplia gama de aplicaciones y casos de uso.
- ▶ **Integración.** S3 se integra estrechamente con otros servicios de AWS, como Amazon EC2, AWS Lambda y Amazon CloudFront, lo que permite construir aplicaciones escalables y altamente disponibles en la nube de AWS.

En resumen, **Amazon S3** es un servicio de almacenamiento en la nube altamente escalable, seguro y duradero, diseñado para almacenar y recuperar grandes volúmenes de datos de manera eficiente y confiable. Con su amplia gama de características y su integración con otros servicios de AWS, S3 es una solución versátil que se adapta a una amplia variedad de casos de uso en entornos empresariales y de desarrollo de aplicaciones.

Un esquema de AWS S3 y sus características lo podemos ver en la Figura 12.

Tema 3. Data science cloud storage



Figura 12. Amazon S3. Fuente: Amazon Web Services, s. f.-a.

Funcionamiento y componentes de AWS S3

Amazon S3 es un servicio de almacenamiento en la nube altamente escalable, seguro y duradero ofrecido por AWS. Su **funcionamiento** se basa en una arquitectura distribuida que permite almacenar y recuperar grandes volúmenes de datos de manera eficiente y confiable. A través de una interfaz simple y robusta, Amazon S3 ofrece una solución versátil para almacenar una amplia variedad de datos, desde archivos estáticos hasta datos dinámicos generados por aplicaciones en tiempo real.

El funcionamiento de Amazon S3 se fundamenta en varios **componentes** clave:

- **Buckets.** Los *buckets* son contenedores de nivel superior para almacenar datos en Amazon S3. Cada *bucket* tiene un nombre único a nivel global y puede contener un número ilimitado de objetos. Los *buckets* se utilizan para organizar y gestionar los datos almacenados en S3, y se pueden configurar con políticas de almacenamiento para controlar el acceso y la protección de los datos.
- **Objetos.** Los objetos son los elementos individuales almacenados en un *bucket* de Amazon S3. Cada objeto consiste en datos (generalmente un archivo) y metadatos asociados, como su nombre, tamaño, tipo de contenido y fecha de modificación. Los objetos se identifican mediante una clave única dentro de un *bucket* y se accede a ellos a través de una URL única generada por S3.

Tema 3. Data science cloud storage

- ▶ **Claves.** Las claves son los identificadores únicos de los objetos dentro de un *bucket*. Una clave es una cadena de caracteres que especifica la ubicación y el nombre de un objeto dentro de un *bucket*. Por ejemplo, la clave «documentos/factura.pdf» especifica un objeto llamado «factura.pdf» que está almacenado en una carpeta llamada «documentos» dentro de un *bucket*.
- ▶ **Consistencia de lectura y escritura.** Amazon S3 ofrece una fuerte consistencia de escritura y una consistencia eventual de lectura. Esto significa que las operaciones de escritura (por ejemplo, subir un nuevo objeto) son consistentes en todas las zonas de disponibilidad inmediatamente, mientras que las operaciones de lectura (por ejemplo, obtener un objeto existente) pueden tomar algún tiempo para propagarse a todas las zonas de disponibilidad.
- ▶ **Interfaz de acceso.** Amazon S3 proporciona una interfaz de acceso basada en HTTP/HTTPS que permite a los usuarios y aplicaciones acceder a los datos almacenados en S3 desde cualquier lugar del mundo. Esta interfaz de acceso incluye una API RESTful, así como herramientas de línea de comandos y SDK para varios lenguajes de programación.
- ▶ **Regiones y zonas de disponibilidad.** Amazon S3 está disponible en varias regiones geográficas en todo el mundo. Cada región de AWS consta de varias zonas de disponibilidad, que son ubicaciones físicas separadas y aisladas dentro de la región. Los datos almacenados en S3 se replican automáticamente en múltiples zonas de disponibilidad dentro de una región para garantizar la durabilidad y la disponibilidad de los datos.
- ▶ **Políticas de almacenamiento.** Amazon S3 permite configurar políticas de almacenamiento para controlar el acceso, la protección y el ciclo de vida de los datos almacenados en S3. Estas políticas pueden aplicarse a nivel de *bucket* u objeto y permiten gestionar de manera flexible el acceso y la protección de los datos almacenados en S3.

Tema 3. Data science cloud storage

En resumen, **Amazon S3** es un servicio de almacenamiento en la nube altamente escalable, seguro y duradero, que permite almacenar y recuperar grandes volúmenes de datos de manera eficiente y confiable. Sus principales componentes incluyen *buckets*, objetos, regiones y zonas de disponibilidad, interfaz de acceso, políticas de almacenamiento y versiones. Estos componentes trabajan juntos para proporcionar una plataforma robusta y flexible para el almacenamiento y la gestión de datos en la nube.

Clases de almacenamiento en S3

Amazon S3 ofrece una variedad de **clases de almacenamiento** entre las cuales puede elegir en función de los requisitos de rendimiento, acceso a los datos, resiliencia y costos de sus cargas de trabajo. Las clases de almacenamiento de S3 se crearon específicamente para brindar el menor costo posible de almacenamiento para los diferentes patrones de acceso. Amazon S3 ofrece diferentes clases de almacenamiento diseñadas para adaptarse a diversos casos de uso y requisitos de rendimiento y costes.

Amazon S3 proporciona el **almacenamiento más duradero** de la nube. Gracias a su arquitectura única, S3 está diseñado para superar el 99,999999999 % (once noes) de durabilidad de los datos. Además, S3 almacena los datos de forma redundante en un mínimo de tres zonas de disponibilidad de forma predeterminada, lo que proporciona resiliencia integrada contra desastres generalizados. Los clientes pueden almacenar los datos en una única zona de disponibilidad para minimizar los costos de almacenamiento o la latencia, en varias zonas de disponibilidad para evitar la pérdida permanente de todo un centro de datos o en varias regiones de AWS para cumplir con los requisitos de resiliencia geográfica.

Tema 3. Data science cloud storage

Amazon S3 también ofrece capacidades que sirven para administrar sus datos durante todo su ciclo de vida. Una vez configurada una **política de ciclo de vida** de S3, sus datos se transferirán.

A continuación, repasamos las diferentes clases de almacenamiento de S3. En primero lugar, tenemos **Amazon S3 estándar** (S3 estándar). Es la clase de almacenamiento estándar de Amazon S3. Ofrece alta durabilidad, disponibilidad y rendimiento, lo que lo hace ideal para una amplia variedad de casos de uso, incluyendo almacenamiento de datos empresariales, alojamiento de sitios web, distribución de contenido multimedia y copias de seguridad. Sus **características** clave son:

- ▶ **Alta durabilidad.** S3 Standard proporciona una durabilidad excepcional para los datos almacenados. Amazon S3 está diseñado para ofrecer una durabilidad del 99,999999999 % (once nueves), lo que significa que es extremadamente poco probable que se pierdan datos.
- ▶ **Alta disponibilidad.** Los datos almacenados en S3 Standard están disponibles para su acceso casi instantáneo en todo momento. Amazon S3 está diseñado para ofrecer una disponibilidad del 99,99 %, lo que significa que los datos son accesibles con muy poco tiempo de inactividad.
- ▶ **Rendimiento consistente.** S3 Standard ofrece un rendimiento consistente y predecible para el acceso a los datos, independientemente del tamaño del objeto o el volumen de tráfico. Esto lo hace adecuado para una amplia variedad de aplicaciones, desde sitios web hasta aplicaciones empresariales.
- ▶ **Escalabilidad ilimitada.** S3 Standard está diseñado para escalar de manera transparente según las necesidades de almacenamiento de tu aplicación. Puedes almacenar cualquier cantidad de datos y S3 se encargará de la escalabilidad y la gestión de la infraestructura subyacente.

Tema 3. Data science cloud storage

- ▶ **Acceso a datos en tiempo real.** Los datos almacenados en S3 Standard están disponibles para su acceso en tiempo real a través de una interfaz simple basada en HTTP/HTTPS. Puedes acceder a los datos desde cualquier lugar del mundo utilizando una variedad de herramientas y servicios de AWS.
- ▶ **Compatibilidad con versiones y replicación.** S3 Standard admite la versión de objetos y la replicación de datos entre regiones, lo que te permite mantener un historial completo de cambios y revisiones para tus datos y garantizar su disponibilidad en caso de desastre.
- ▶ **Tarifas competitivas.** A pesar de ofrecer un rendimiento y una durabilidad excepcionales, S3 Standard tiene tarifas competitivas que lo hacen asequible para una amplia gama de aplicaciones y cargas de trabajo.

Por otro lado, tenemos **Amazon S3 Intelligent-Tiering** (S3 Intelligent-Tiering). La clase de almacenamiento Amazon S3 Intelligent-Tiering es una opción avanzada dentro de Amazon S3 que utiliza aprendizaje automático para analizar el patrón de acceso a los datos y mover automáticamente los objetos entre las clases de almacenamiento *standard* y *standard-IA (infrequently accessed)*. Esta opción está diseñada para optimizar los costos de almacenamiento al garantizar que los datos se almacenen en la clase de almacenamiento más adecuada en función de su frecuencia de acceso. Entre sus **características** clave tenemos:

- ▶ **Ahorros de costos automáticos** para datos con patrones de acceso desconocidos o que cambian constantemente.
- ▶ Los **niveles de acceso** frecuente, poco frecuente e instantáneo al archivo tienen la misma baja latencia y alto nivel de rendimiento de S3 Standard.
- El nivel de acceso **poco frecuente** ahorra hasta un 40 % en costos de almacenamiento.
- El nivel de acceso **instantáneo** al archivo ahorra hasta un 68 % en costos de

Tema 3. Data science cloud storage

almacenamiento.

- ▶ Incluye **capacidades opcionales** de archivo asíncrono para objetos a los que se accede de forma inusual.

Continuamos con **Amazon S3 Express One Zone**, que es una clase de almacenamiento de zona de disponibilidad única y alto rendimiento diseñada específicamente para ofrecer un acceso constante en milisegundos a los datos de un solo dígito para los datos a los que se accede con más frecuencia y las aplicaciones sensibles a la latencia.

Además, puede mejorar diez veces la velocidad de acceso a los datos y reducir los costes de solicitud en un 50 % en comparación con S3 Standard. Los datos se almacenan en un tipo de *bucket* diferente (un *bucket* de directorio de Amazon S3), que admite cientos de miles de solicitudes por segundo. Además, se puede utilizar con servicios, como el entrenamiento de modelos Amazon SageMaker, Amazon Athena, Amazon EMR y el catálogo de datos de AWS Glue, para acelerar sus cargas de trabajo de ML y análisis. Con S3 Express One Zone, el almacenamiento se incrementa o reduce automáticamente en función de su consumo y necesidad, y ya no necesita administrar varios sistemas de almacenamiento para cargas de trabajo de baja latencia. Estas son sus **características** clave:

- ▶ **Almacenamiento** de alto rendimiento para los datos a los que se accede con más frecuencia.
- ▶ **Latencia** constante de solicitudes de milisegundos de un solo dígito.
- ▶ Mejora diez veces las **velocidades** de acceso y reduce los **costos de solicitud** en un 50 % en comparación con S3 Standard.
- ▶ Optimizado para grandes **conjuntos de datos** con muchos objetos pequeños.

Tema 3. Data science cloud storage

- ▶ Permite utilizar las API existentes de Amazon S3 con un tipo de *bucket* diferente: **buckets de directorio**.
- ▶ Diseñado para ofrecer una disponibilidad del 99,95 % con un SLA (*service level agreement*) de disponibilidad del 99,9 %

Asimismo, tenemos **Amazon S3 Infrequent Access** (S3 IA o S3 estándar de acceso poco frecuente). La clase de almacenamiento Amazon S3 IA, también conocida como S3 Standard-IA, es una opción diseñada para datos que se acceden con menos frecuencia que los datos almacenados en la clase de almacenamiento S3 Standard, pero que aún requieren un acceso rápido cuando sea necesario. Es una opción intermedia entre S3 Standard y Amazon S3 Glacier, ofreciendo un equilibrio entre rendimiento y costo para datos que no se acceden con tanta frecuencia como los datos en S3 Standard, pero que aún necesitan estar disponibles de manera inmediata cuando se solicitan.

A continuación, tienes algunas **características** clave de la clase de almacenamiento Amazon S3 IA:

- ▶ **Acceso rápido.** Aunque los datos se almacenan en una clase de acceso menos frecuente que S3 Standard, aún están disponibles para su acceso inmediato cuando sea necesario. Esto significa que los datos pueden ser recuperados rápidamente sin necesidad de esperar tiempos de recuperación prolongados como en el caso de Amazon S3 Glacier.
- ▶ **Bajo coste.** Amazon S3 IA ofrece tarifas más bajas que S3 Standard, lo que la hace una opción económica para datos que no se acceden con tanta frecuencia. Esto permite a las organizaciones reducir costos al optimizar el almacenamiento de datos según su frecuencia de acceso.
- ▶ **Durabilidad y disponibilidad.** Los datos almacenados en la clase de almacenamiento Amazon S3 IA disfrutan de la misma durabilidad y disponibilidad que los datos almacenados en S3 Standard. Los objetos se replican

Tema 3. Data science cloud storage

automáticamente en múltiples zonas de disponibilidad dentro de una región para garantizar su integridad y disponibilidad.

- ▶ **Política de precios basada en el uso.** Amazon S3 IA se factura en función del volumen de almacenamiento utilizado y de las solicitudes de acceso a los datos. Esto significa que solo pagas por el almacenamiento y el acceso a los datos que realmente utilizas, lo que puede ayudar a reducir los costos operativos.
- ▶ **Políticas de ciclo de vida.** Amazon S3 IA es compatible con las políticas de ciclo de vida de Amazon S3, lo que te permite automatizar el movimiento de datos entre clases de almacenamiento en función de criterios predefinidos, como la antigüedad de los datos. Esto te permite optimizar el almacenamiento de datos y reducir costos automáticamente a lo largo del tiempo.

En resumen, **Amazon S3 IA** es una opción de almacenamiento diseñada para datos que se acceden con menos frecuencia que los datos en la clase de almacenamiento S3 Standard, pero que aún requieren un acceso rápido cuando sea necesario. Ofrece un equilibrio entre rendimiento y costo, con tarifas más bajas que S3 Standard y una disponibilidad inmediata cuando se solicita el acceso a los datos. Esto la convierte en una opción económica y práctica para almacenar datos que no se acceden con tanta frecuencia pero que aún necesitan estar disponibles de manera rápida y eficiente.

Continuamos con **Amazon S3 One Zone-Infrequent Access** (S3 One Zone-IA). La clase de almacenamiento S3 One Zone-IA es una opción de almacenamiento en frío ofrecida por AWS que está diseñada para datos que se acceden con menos frecuencia y que no requieren la redundancia de zona de disponibilidad. A diferencia de otras clases de almacenamiento, los datos en S3 One Zone-IA se almacenan en una sola zona de disponibilidad dentro de una región de AWS, lo que la hace más económica pero menos resistente a las fallas.

Tema 3. Data science cloud storage

Estas son algunas **características** clave de S3 One Zone-IA:

- ▶ **Coste reducido.** S3 One Zone-IA ofrece tarifas más bajas en comparación con otras opciones de almacenamiento en frío de Amazon S3, como S3 Standard-IA. Al almacenar los datos en una sola zona de disponibilidad, AWS puede reducir los costos operativos y ofrecer precios más competitivos.
- ▶ **Menor resistencia a fallas.** A diferencia de S3 Standard-IA, que almacena datos en múltiples zonas de disponibilidad dentro de una región, S3 One Zone-IA almacena datos en una sola zona de disponibilidad. Esto significa que es menos resistente a las fallas y puede estar sujeto a interrupciones si ocurre un evento que afecta a esa zona específica.
- ▶ **Disponibilidad inmediata.** A pesar de su menor resistencia a fallas, los datos almacenados en S3 One Zone-IA aún están disponibles para su acceso inmediato cuando sea necesario. Los tiempos de acceso y recuperación son similares a los de otras opciones de almacenamiento en frío de Amazon S3.
- ▶ **Política de precios basada en el uso.** S3 One Zone-IA se factura en función del volumen de almacenamiento utilizado y de las solicitudes de acceso a los datos, al igual que otras opciones de almacenamiento en frío de Amazon S3. Esto significa que solo pagas por el almacenamiento y el acceso a los datos que realmente utilizas.

S3 One Zone-IA es adecuado para ciertos **casos de uso** donde la redundancia de zona de disponibilidad no es crítica y los costos son un factor importante. Por ejemplo, puede ser útil para almacenar copias de respaldo de datos que ya se replican en otra ubicación o para datos no críticos que se pueden recuperar fácilmente en caso de una interrupción.

En resumen, **S3 One Zone-IA** es una opción de almacenamiento en frío de bajo costo diseñada para datos que se acceden con menos frecuencia y que no requieren la redundancia de zona de disponibilidad. Ofrece tarifas más bajas en comparación con otras opciones de almacenamiento en frío de Amazon S3, pero es menos

Tema 3. Data science cloud storage

resistente a las fallas al almacenar los datos en una sola zona de disponibilidad. Es adecuado para ciertos casos de uso donde la redundancia de zona de disponibilidad no es crítica y los costos son un factor importante.

Seguimos con **Amazon S3 Glacier**, cuya clase de almacenamiento es una opción de almacenamiento en frío de bajo costo ofrecida por AWS. Está diseñada para archivar datos a largo plazo que rara vez se acceden pero que deben mantenerse disponibles para cumplir con requisitos de cumplimiento, regulaciones o necesidades de negocio. Glacier ofrece una durabilidad excepcional y tarifas extremadamente bajas, lo que lo hace ideal para almacenar grandes volúmenes de datos que no se necesitan con frecuencia pero que deben mantenerse seguros y disponibles.

Aquí tienes algunas **características** clave de Amazon S3 Glacier:

- ▶ **Bajo costo.** Glacier ofrece tarifas extremadamente bajas para el almacenamiento de datos en comparación con otras opciones de almacenamiento en Amazon S3. Esto lo hace ideal para archivar grandes volúmenes de datos que no se acceden con frecuencia pero que deben mantenerse disponibles para cumplir con requisitos de cumplimiento o regulaciones.
- ▶ **Durabilidad extrema.** Al igual que otras clases de almacenamiento en Amazon S3, Glacier ofrece una durabilidad excepcional para los datos almacenados. Los datos se replican automáticamente en múltiples dispositivos de almacenamiento y ubicaciones geográficas para garantizar su integridad y disponibilidad a lo largo del tiempo.
- ▶ **Alta disponibilidad.** Aunque Glacier ofrece tiempos de acceso más largos que otras opciones de almacenamiento en Amazon S3, los datos almacenados en Glacier están disponibles para su acceso en cuestión de minutos u horas, dependiendo de la opción de recuperación seleccionada.

Tema 3. Data science cloud storage

- ▶ **Opciones de recuperación.** Glacier ofrece varias opciones de recuperación para adaptarse a diferentes necesidades y requisitos de tiempo. Estas opciones incluyen recuperación estándar, *expedited* y *bulk*, que varían en términos de tiempo de recuperación y tarifas asociadas.
- ▶ **Facilidad de gestión.** Glacier se integra estrechamente con otros servicios de AWS, lo que facilita la gestión y el acceso a los datos almacenados en Glacier. Puedes utilizar la consola de administración de AWS, la API de Glacier o herramientas de terceros para gestionar tus datos de forma eficiente.
- ▶ **Cumplimiento y seguridad.** Glacier ofrece características avanzadas de cumplimiento y seguridad para proteger los datos almacenados. Esto incluye cifrado de datos en reposo y en tránsito, control de acceso basado en políticas y registros de auditoría para rastrear el acceso y las modificaciones a los datos.

En resumen, **Amazon S3 Glacier** es una opción de almacenamiento en frío de bajo costo y alta durabilidad diseñada para archivar datos a largo plazo que rara vez se acceden pero que deben mantenerse disponibles para cumplir con requisitos de cumplimiento o regulaciones. Ofrece tarifas extremadamente bajas, durabilidad excepcional y varias opciones de recuperación para adaptarse a diferentes necesidades y requisitos de tiempo.

Amazon S3 Glacier Instant Retrieval es una opción de recuperación de datos dentro de Amazon S3 Glacier que permite recuperar datos de forma rápida y en tiempo real. A diferencia de las opciones de recuperación estándar, que pueden llevar horas o incluso días, *expedited retrieval* proporciona acceso inmediato a los datos almacenados en Glacier.

Tema 3. Data science cloud storage

A continuación, se muestran algunas **características** clave de Amazon S3 Glacier Instant Retrieval:

- ▶ **Acceso rápido a los datos.** Instant Retrieval ofrece acceso a los datos almacenados en Glacier en cuestión de minutos, lo que permite recuperar información de manera rápida y eficiente cuando sea necesario.
- ▶ **Adecuado para cargas de trabajo críticas.** Esta opción de recuperación es ideal para cargas de trabajo que requieren acceso inmediato a los datos almacenados en Glacier, como recuperaciones de emergencia o situaciones en las que el tiempo es esencial.
- ▶ **Coste adicional.** Si bien *expedited retrieval* ofrece acceso rápido a los datos, hay un costo adicional asociado con esta opción. El precio puede ser más alto que las opciones estándar de recuperación, por lo que es importante evaluar el equilibrio entre velocidad y costo según los requisitos de tu carga de trabajo.
- ▶ **Facilidad de uso.** *Expedited retrieval* se puede activar fácilmente al solicitar la recuperación de datos desde Glacier. Simplemente selecciona la opción de recuperación acelerada y los datos estarán disponibles en minutos.
- ▶ **Seguridad.** Al igual que con todas las opciones de almacenamiento en Amazon S3, *expedited retrieval* ofrece seguridad integrada para proteger los datos durante la transferencia y el acceso.

En resumen, **Amazon S3 Glacier Instant Retrieval** es una opción que ofrece acceso rápido a los datos almacenados en Glacier en minutos. Es adecuado para cargas de trabajo críticas que requieren recuperación inmediata de datos, aunque hay un costo adicional asociado. Esta característica proporciona un equilibrio entre velocidad y costo para satisfacer las necesidades de diferentes aplicaciones y cargas de trabajo.

Tema 3. Data science cloud storage

Amazon S3 Glacier Flexible Retrieval ofrece almacenamiento de bajo costo, hasta un 10 % menor que S3 Glacier Instant Retrieval, para los datos de archivo a los que se accede una o dos veces al año y se recuperan de manera asíncrona. Para los datos de archivo que no requieren acceso inmediato, pero necesitan la flexibilidad de recuperar grandes conjuntos de datos sin costo alguno, como los casos de uso de copias de seguridad o recuperación de desastres, S3 Glacier Flexible Retrieval (antes S3 Glacier), es la clase de almacenamiento ideal.

Además, ofrece las opciones de recuperación más flexibles que equilibran el costo con los tiempos de acceso que varían de minutos a horas, con recuperaciones masivas gratuitas. Esto es una solución ideal para las necesidades de copia de seguridad, recuperación de desastres, almacenamiento de datos fuera del sitio y para cuando algunos datos deban recuperarse ocasionalmente en minutos y no desee preocuparse por los costos. Entre sus **características** clave, tenemos:

- ▶ Crear **copias de seguridad y archivado de datos** a los que se accede con poca frecuencia y a bajo costo.
- ▶ Diseñado para ofrecer una **disponibilidad del 99,99 %** con un SLA de disponibilidad del 99,9 %.
- ▶ **Admite SSL** para los datos en tránsito y cifrado de datos en reposo.
- ▶ Es ideal para casos de uso de **copia de seguridad y recuperación de desastres** cuando, ocasionalmente, se deban recuperar grandes conjuntos de datos en minutos sin preocuparse por los costos.
- ▶ Los **tiempos de recuperación** se pueden configurar de minutos a horas con recuperaciones masivas gratuitas.
- ▶ **Tiene la API PUT de S3 para cargas directas a S3 Glacier Flexible Retrieval** y la administración del ciclo de vida de S3 para la migración automática de objetos

Tema 3. Data science cloud storage

Por último, tenemos **Amazon S3 Glacier Deep Archive**, cuya clase de almacenamiento es la opción de almacenamiento en frío de más bajo costo ofrecida por AWS. Está diseñada para archivar datos a largo plazo que rara vez se acceden y que pueden mantenerse en almacenamiento por períodos prolongados, a menudo durante varios años. Glacier Deep Archive ofrece una durabilidad excepcional y tarifas extremadamente bajas, lo que la hace ideal para almacenar grandes volúmenes de datos que no se necesitan con frecuencia pero que deben mantenerse seguros y disponibles para cumplir con requisitos de cumplimiento, regulaciones o necesidades de negocio. Estas son algunas de sus **características** clave:

- ▶ **Tarifas extremadamente bajas.** Glacier Deep Archive ofrece las tarifas más bajas entre todas las opciones de almacenamiento en frío de Amazon S3. Estas tarifas son significativamente inferiores a las de otras clases de almacenamiento en frío, lo que la convierte en una opción económica para archivar grandes volúmenes de datos a largo plazo.
- ▶ **Durabilidad excepcional.** Al igual que otras clases de almacenamiento en Amazon S3, Glacier Deep Archive ofrece una durabilidad excepcional para los datos almacenados. Los datos se replican automáticamente en múltiples dispositivos de almacenamiento y ubicaciones geográficas para garantizar su integridad y disponibilidad a lo largo del tiempo.
- ▶ **Disponibilidad bajo demanda.** Aunque Glacier Deep Archive ofrece tiempos de acceso más largos que otras opciones de almacenamiento en Amazon S3, los datos almacenados en Glacier Deep Archive están disponibles para su acceso en cuestión de horas. Esto significa que los datos pueden ser recuperados de manera rápida y eficiente cuando sea necesario.

Tema 3. Data science cloud storage

- **Política de precios basada en el uso.** Glacier Deep Archive se factura en función del volumen de datos almacenados y de las solicitudes de acceso a los datos. Esto significa que solo pagas por el almacenamiento y el acceso a los datos que realmente utilizas, lo que puede ayudar a reducir los costos operativos.
- **Cumplimiento y seguridad.** Glacier Deep Archive ofrece características avanzadas de cumplimiento y seguridad para proteger los datos almacenados. Esto incluye cifrado de datos en reposo y en tránsito, control de acceso basado en políticas y registros de auditoría para rastrear el acceso y las modificaciones a los datos.

En resumen, **Amazon S3 Glacier Deep Archive** es una opción de almacenamiento en frío de bajo costo diseñada para archivar datos a largo plazo que rara vez se acceden. Ofrece tarifas extremadamente bajas, durabilidad excepcional y disponibilidad bajo demanda, lo que la convierte en una opción económica y práctica para almacenar grandes volúmenes de datos que no se acceden con frecuencia, pero que deben mantenerse disponibles a lo largo del tiempo.

	S3 Standard	S3 Intelligent-Tiering*	S3 Standard-IA	S3 One Zone-IA†	S3 Glacier	S3 Glacier Deep Archive
Designed for durability	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)
Designed for availability	99.99%	99.9%	99.9%	99.5%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99.9%	99.9%
Availability Zones	≥3	≥3	≥3	1	≥3	≥3
Minimum capacity charge per object	N/A	N/A	128KB	128KB	40KB	40KB
Minimum storage duration charge	N/A	30 days	30 days	30 days	90 days	180 days
Retrieval fee	N/A	N/A	per GB retrieved	per GB retrieved	per GB retrieved	per GB retrieved
First byte latency	milliseconds	milliseconds	milliseconds	milliseconds	select minutes or hours	select hours
Storage type	Object	Object	Object	Object	Object	Object
Lifecycle transitions	Yes	Yes	Yes	Yes	Yes	Yes

Figura 13. Sistemas de almacenamiento AWS S3. Fuente: Jineshkumar, 2021.

Tema 3. Data science cloud storage

AWS S3 Batch Operations

Las **operaciones por lotes** (*batch operations*) en Amazon S3 son un conjunto de herramientas que te permiten realizar cambios masivos en tus objetos de almacenamiento de una manera eficiente y económica. Estas operaciones pueden aplicarse a múltiples objetos a la vez, lo que puede ser útil para tareas como la gestión de versiones, la aplicación de políticas de retención o la modificación de metadatos en grandes conjuntos de datos. Una descripción de los **componentes básicos** de las operaciones por lotes en Amazon S3 es la siguiente:

- ▶ **Job.** Un trabajo (*job*) en Amazon S3 Batch Operations es una solicitud para realizar una tarea específica en un conjunto de objetos de almacenamiento. Esto puede incluir tareas como cambiar los permisos de acceso, aplicar políticas de retención o eliminar objetos. Los trabajos se pueden configurar a través de la consola de Amazon S3, la API de S3 o la interfaz de línea de comandos (CLI).
- ▶ **Operation.** La operación es el tipo de acción de API, como copiar objetos, que desea que ejecute el trabajo de las operaciones por lotes. Cada trabajo realiza un único tipo de operación en todos los objetos especificados en el manifiesto.
- ▶ **Tarea.** Una tarea es la unidad de ejecución de un trabajo. Una tarea representa una llamada única a una operación de API de Amazon S3 o AWS Lambda para realizar la operación del trabajo en un único objeto. En el transcurso de la vida útil de un trabajo, las operaciones por lotes de S3 crea una tarea para cada objeto especificado en el manifiesto.
- ▶ **Manifiesto.** Un manifiesto (*manifest*) es un archivo JSON que contiene la lista de objetos a los que se aplicará la operación por lotes. Puedes crear el manifiesto manualmente o generar automáticamente la lista de objetos mediante un prefijo de clave, una expresión de filtro o una lista de identificadores de objeto.
- ▶ **Presupuesto de operación.** Antes de ejecutar un trabajo, puedes establecer un presupuesto de operación (*operation budget*) que limita la cantidad máxima de

Tema 3. Data science cloud storage

recursos que se utilizarán para completar la tarea. Esto te permite controlar los costos y evitar gastos inesperados al realizar cambios en grandes conjuntos de datos.

- **Notificaciones de estado.** Una vez que se completa un trabajo, Amazon S3 puede enviar notificaciones de estado (*status notifications*) para informarte sobre el progreso y el resultado de la operación. Puedes configurar notificaciones por correo electrónico, mensajes de texto o integraciones con otros servicios de AWS.
- **Control de acceso y seguridad.** Amazon S3 Batch Operations utiliza los mismos controles de acceso y seguridad que otros servicios de AWS. Esto te permite controlar quién puede crear, modificar o eliminar trabajos, así como acceder a los datos procesados durante las operaciones por lotes.

En resumen, las **operaciones por lotes en Amazon S3** son una herramienta poderosa para realizar cambios masivos en tus objetos de almacenamiento de una manera eficiente y económica. Los componentes básicos incluyen el trabajo en sí, el manifiesto que especifica los objetos afectados, el presupuesto de operación para controlar los costos, las notificaciones de estado para mantenerse informado sobre el progreso y el resultado del trabajo, y los controles de acceso y seguridad para proteger tus datos durante el proceso.

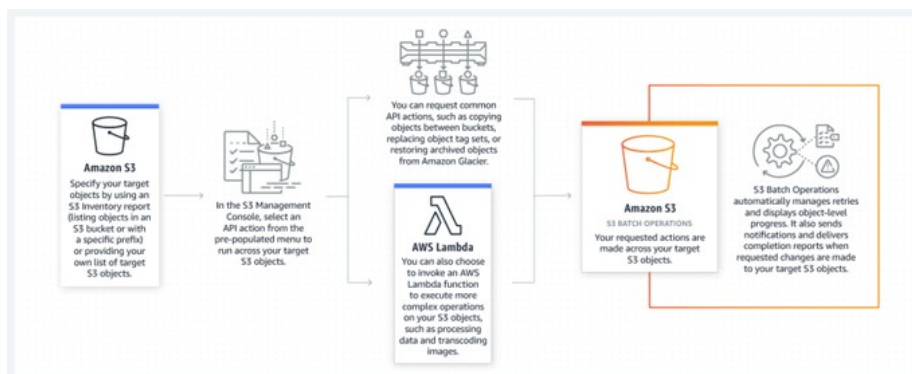


Figura 14. Operaciones por lotes de Amazon S3. Fuente: Amazon Web Services, s. f.-b.

Tema 3. Data science cloud storage

Operaciones admitidas en S3 Batch Operations

Las operaciones admitidas por Amazon S3 Batch Operations varían según la acción que desees realizar en tus objetos de almacenamiento. Aquí tienes una lista de algunas de las operaciones comunes admitidas por S3 Batch Operations:

- ▶ **Cambio de permisos (ACL).** Puedes modificar los permisos de acceso de los objetos, como agregar o eliminar usuarios o grupos con permisos de lectura, escritura o eliminación.
- ▶ **Cambio de metadatos.** Puedes actualizar los metadatos de los objetos, como el tipo de contenido, el tipo de almacenamiento o cualquier otro atributo personalizado.
- ▶ **Aplicación de políticas de retención.** Puedes aplicar o actualizar las políticas de retención de los objetos para garantizar que no sean eliminados o modificados durante un período de tiempo específico.
- ▶ **Eliminación de objetos.** Puedes eliminar objetos de almacenamiento según criterios específicos, como su antigüedad, tamaño o ubicación.
- ▶ **Copiar objetos.** Puedes realizar copias de seguridad o duplicar objetos de almacenamiento en diferentes ubicaciones o con diferentes atributos.
- ▶ **Recuperación de objetos archivados.** Puedes iniciar la recuperación de objetos archivados en Amazon S3 Glacier o S3 Glacier Deep Archive para hacerlos accesibles de nuevo en Amazon S3.

Estas son solo algunas de las **operaciones admitidas** por Amazon S3 Batch Operations. La plataforma ofrece una amplia gama de opciones para realizar cambios masivos en tus objetos de almacenamiento de una manera eficiente y controlada.

Tema 3. Data science cloud storage

Administración de ciclo de vida en AWS S3

AWS S3 ofrece la posibilidad de configurar el ciclo de vida de los objetos para administrarlos de manera que se almacenen de manera económica durante todo su ciclo de vida en **Amazon S3 Lifecycle**. La configuración de S3 Lifecycle es un conjunto de reglas que definen acciones que Amazon S3 aplica a un grupo de objetos. La configuración del ciclo de vida de Amazon S3 es un conjunto simple de reglas que se pueden usar para administrar el ciclo de vida de dichos datos en S3, de manera automatizada. A continuación, se describirá el concepto de control de versiones de AWS S3 y las configuraciones del ciclo de vida de Amazon S3 como concepto. Luego, se mostrará cómo aprovechar la configuración del ciclo de vida para caducar y eliminar objetos a fin de ahorrar costos de almacenamiento en la nube.

¿Qué es el **control de versiones** de objetos de Amazon S3? Para comprender completamente todas las diferentes aplicaciones de las configuraciones del ciclo de vida de Amazon S3, es importante comprender primero el concepto de **control de versiones** del depósito de S3. La función de control de versiones de Amazon S3 permite a los usuarios mantener varias versiones del mismo objeto en un depósito de S3 con fines de reversión o recuperación. Cuando un depósito de Amazon S3 está habilitado para el control de versiones, cada objeto del depósito recibe un identificador de versión que cambia cada vez que el objeto cambia o se sobrescribe. Cuando se sobrescribe un objeto, este nuevo objeto será la versión actual, mientras que el archivo anterior tendrá un ID de versión anterior.

Los usuarios pueden aprovechar las **versiones anteriores** para restaurar los objetos que se eliminan o sobrescriben según sea necesario. Cuando el usuario final o la aplicación elimina un objeto de S3, Amazon S3 insertará un marcador de eliminación. Este marcador de eliminación se convierte en la versión actual del objeto. Si bien el control de versiones de S3 es útil para varios propósitos de recuperación, significa

Tema 3. Data science cloud storage

que una vez habilitados, los objetos más antiguos que se eliminaron o sobrescribieron continúan consumiendo la capacidad de almacenamiento subyacente, lo que genera costos de consumo.

La configuración del **ciclo de vida de AWS S3** es una colección de reglas que definen varias acciones del ciclo de vida que se pueden aplicar automáticamente a un grupo de objetos de Amazon S3. Estas acciones pueden ser **acciones de transición** (que hacen que la versión actual de los objetos de S3 cambie entre varias clases de almacenamiento de S3) o pueden ser **acciones de caducidad** (que definen cuándo caduca un objeto de S3).

Ejemplo de configuración de ciclo de vida en S3 para eliminar objetos

La **configuración del ciclo de vida** de vencimiento de la objeción de S3 se puede crear con varias herramientas diferentes: la herramienta AWS CLI, el SDK de AWS, la consola de Amazon S3 o las llamadas a la API RESTful.

Recomendamos se consulte la **guía del usuario del ciclo de vida** de Amazon S3 para obtener información detallada paso a paso.

La Figura 15 muestra cómo se puede aprovechar la interfaz de usuario de la consola de Amazon S3 (a la que se accede a través de la pestaña «Administración» dentro del depósito de S3) para configurar una regla de ciclo de vida de S3 para caducar la versión actual de los objetos S3.

Tema 3. Data science cloud storage

The screenshot displays the AWS S3 console's 'Lifecycle rule configuration' page. The left sidebar shows the 'Amazon S3' navigation menu. The main content area is divided into several sections: 'Lifecycle rule configuration', 'Lifecycle rule actions', 'Expire current versions of objects', and 'Review transition and expiration actions'. In the 'Lifecycle rule configuration' section, the 'Lifecycle rule name' is 's3-eu-west-2-bucket-01-lifecycle-delete-all'. The 'Choose a rule scope' section has 'Apply to all objects in the bucket' selected. The 'Lifecycle rule actions' section has 'Expire current versions of objects' selected and highlighted with an orange box. The 'Expire current versions of objects' section has 'Days after object creation' set to '90' and highlighted with an orange box. The 'Review transition and expiration actions' section shows 'Current version actions' with 'Objects uploaded' at Day 0 and 'Objects expire' at Day 90. The 'Noncurrent versions actions' section shows 'No actions defined.' The 'Create rule' button is highlighted in orange.

Figura 15. Regla de ciclo de vida de S3. Fuente: elaboración propia.

A los efectos de este ejemplo, se ha configurado la **caducidad** para que se active después de noventa días de la creación del objeto para todo el *bucket* para eliminar la versión actual.

Para eliminar **versiones anteriores** de objetos S3, la misma consola de AWS se puede aprovechar fácilmente para crear la regla del ciclo de vida, como se ilustra en la Figura 16.

Tema 3. Data science cloud storage

aws Services Search for services, features, marketplace products, and a

We're continuing to improve the S3 console to make it faster and easier to use. If you have feedback on the updated experience, click here.

Lifecycle rule name
s3-eu-west-2-bucket-01-lifecycle-all-remove-old-versions
Up to 255 characters

Choose a rule scope
☐ Limit the scope of this rule using one or more filters
☒ Apply to all objects in the bucket

Apply to all objects in the bucket
If you want the rule to apply to specific objects, you must use a filter to identify those objects. Choose "Limit the scope of this rule using one or more filters". [Learn more](#)
☒ I acknowledge that this rule will apply to all objects in the bucket.

Lifecycle rule actions
Choose the actions you want this rule to perform. Per-request fees apply. [Learn more](#) or see [Amazon S3 pricing](#)

☐ Move current versions of objects between storage classes
☐ Move noncurrent versions of objects between storage classes
☐ Expire current versions of objects
☒ Permanently delete noncurrent versions of objects
☐ Delete expired object delete markers or incomplete multipart uploads
When a lifecycle rule is scoped with tags, these actions are unavailable.

Permanently delete noncurrent versions of objects
Days after objects become noncurrent
90

Review transition and expiration actions

Current version actions	Noncurrent versions actions
Day 0 No actions defined.	Day 0 • Objects become noncurrent
	↓
	Day 90 • Objects are permanently deleted

Cancel **Create rule**

Figura 16. Regla de ciclo de vida de S3. Fuente: elaboración propia.

Tema 3. Data science cloud storage

Replicación en AWS S3

La **replicación** de Amazon S3 es un elemento eficiente y gestionado encargado de replicar objetos almacenados en los *buckets*. Su funcionamiento permite mayor flexibilidad y optimización para el almacenamiento en la nube, lo que a su vez proporciona los controles pertinentes para el cumplimiento de las tareas empresariales.

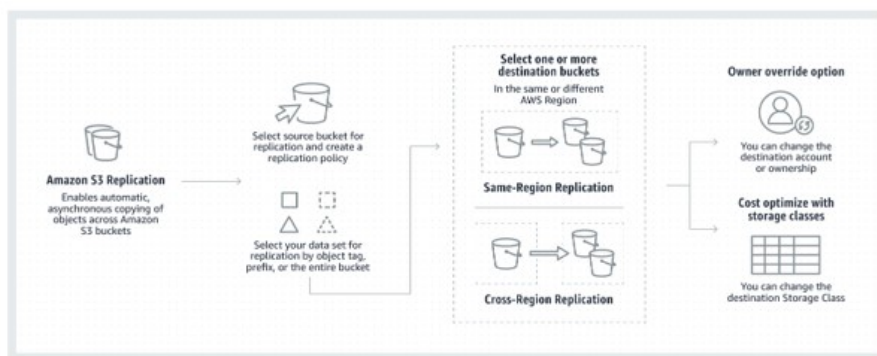


Figura 17. Cómo funciona la replicación de S3. Fuente: Amazon Web Services, s. f.-c.

En cuanto a las **características** de la replicación de objetos en Amazon S3, ofrecer la configuración de Amazon S3 para la copia automática de los objetos del sistema en distintas regiones de AWS a través de la función de réplica entre regiones (CRR) de S3 o dentro de *buckets* en una misma región de AWS por medio de la replicación en una sola región (SRR) de Amazon S3.

La replicación de Amazon S3 provee, igualmente, **notificaciones y métricas** precisas para gestionar el funcionamiento de la réplica de objetos entre *buckets*. Esto permite observar el proceso de la replicación mediante el rastreo de los bytes pendientes y supervisar las tareas pendientes, y la latencia de replicación entre los *buckets* de origen y de destino con la consola de administración de Amazon S3 o AWS CloudWatch.

Tema 3. Data science cloud storage

De la misma manera, esta característica de Amazon S3 proporciona la **configuración de notificaciones de eventos de S3** para obtener avisos sobre replicaciones erróneas y, así, emplear un análisis y reparar los errores de administración de inmediato.

Algunos **casos de uso** de la replicación de objetos son:

- ▶ **Replicar objetos conservando los metadatos.** Puede utilizar la replicación para realizar copias de los objetos en las que se conserven todos los metadatos, como la hora de creación del objeto original y los ID de versión. Esta capacidad es importante si necesita asegurarse de que la réplica sea idéntica al objeto de origen.
- ▶ **Replicar objetos en diferentes clases de almacenamiento.** Puede utilizar la replicación para colocar objetos directamente en S3 Glacier, S3 Glacier Deep Archive u otra clase de almacenamiento en los *buckets* de destino. También puede replicar los datos en la misma clase de almacenamiento y utilizar las políticas de ciclo de vida en los *buckets* de destino para mover objetos a una clase de almacenamiento con menos actividad conforme adquiere antigüedad.
- ▶ **Mantener copias de objetos con distintos propietarios.** Independientemente de quién sea el propietario del objeto de origen, puede indicar a Amazon S3 que cambie la propiedad de la réplica a la cuenta de AWS que posee el *bucket* de destino. Esto se conoce como la opción de invalidación del propietario. Puede usar esta opción para restringir el acceso a las réplicas de objetos.
- ▶ **Mantener los objetos almacenados en varias regiones de AWS.** Puede establecer varios *buckets* de destino en diferentes regiones de AWS para garantizar las diferencias geográficas en el lugar donde se guardan los datos. Esto podría ser útil para cumplir ciertos requisitos de conformidad.
- ▶ **Reproducir objetos en quince minutos.** Puede utilizar el control de tiempo de reproducción de S3 (S3 RTC) para reproducir sus datos en la misma región de AWS o en distintas regiones dentro de un período predecible. S3 RTC replica el 99,99 %

Tema 3. Data science cloud storage

de los objetos nuevos almacenados en Amazon S3 dentro de un plazo de quince minutos (con el respaldo de un acuerdo de nivel de servicio).

Amazon Web Services. (s. f.-d). *Cumplimiento de los requisitos de conformidad mediante el control de tiempo de replicación de S3 (S3 RTC)*.
https://docs.aws.amazon.com/es_es/AmazonS3/latest/userguide/replication-time-control.html

La **reproducción entre las regiones** (CRR) de S3 se utiliza para copiar objetos en *buckets* de Amazon S3 en diferentes regiones de AWS. CRR puede ayudarlo a hacer lo siguiente:

- ▶ **Cumplir los requisitos de conformidad.** Aunque Amazon S3 almacena sus datos en diversas zonas de disponibilidad alejadas geográficamente, de forma predeterminada los requisitos de conformidad pueden exigir que almacene los datos en ubicaciones aún más alejadas. La reproducción entre las regiones permite reproducir los datos entre las regiones de AWS alejadas para cumplir con estos requisitos de conformidad.
- ▶ **Minimizar la latencia.** Si sus clientes están en dos ubicaciones geográficas, puede minimizar la latencia en el acceso a los objetos mediante el mantenimiento de copias de los objetos en las regiones de AWS que estén geográficamente más cerca de sus usuarios.
- ▶ **Aumentar la eficiencia operativa.** Si tiene clústeres de cómputo en dos regiones de AWS diferentes que analizan el mismo conjunto de objetos, puede optar por mantener copias de objetos en dichas regiones.

Tema 3. Data science cloud storage

La **reproducción en la misma región** (SRR) se utiliza para copiar objetos en *buckets* de Amazon S3 en la misma región de AWS. SRR puede ayudarlo a hacer lo siguiente:

- ▶ **Agregar registros en un solo *bucket*.** Si almacena registros en varios *buckets* o en varias cuentas, puede fácilmente replicar registros en un solo *bucket* en la región. Esto permite un procesamiento más simple de los registros en una sola ubicación.
- ▶ **Configurar la replicación en vivo entre las cuentas de producción y prueba.** Si usted o sus clientes tienen cuentas de producción y de prueba que utilizan los mismos datos, puede replicar objetos entre esas cuentas múltiples, mientras mantiene metadatos de objeto.
- ▶ **Cumplir las leyes de soberanía de datos.** Es posible que tenga que almacenar varias copias de sus datos en las cuentas de AWS separadas dentro de una misma región. La replicación en la misma región puede ayudarlo a replicar automáticamente los datos críticos cuando las normativas de conformidad no permiten que los datos salgan de su país.

Apache HDFS

Apache HDFS es un sistema de archivos distribuido diseñado para almacenar grandes conjuntos de datos de manera confiable y escalable en clústeres de servidores estándar. Forma parte del ecosistema Apache Hadoop y es una de las tecnologías clave que permiten el procesamiento distribuido de datos en grandes cantidades. Una descripción de sus **características** sería la siguiente:

- ▶ **Distribuido y tolerante a fallos.** HDFS distribuye los datos a través de múltiples nodos de un clúster, lo que permite almacenar grandes volúmenes de datos de manera distribuida. Además, está diseñado para ser tolerante a fallos, lo que significa que puede manejar la pérdida de nodos individuales sin perder datos.

Tema 3. Data science cloud storage

- ▶ **Bloques de datos.** HDFS divide los datos en bloques de tamaño fijo (generalmente 128 MB o 256 MB por defecto), que luego se distribuyen y almacenan en diferentes nodos del clúster. Esta división en bloques permite el procesamiento paralelo de datos y facilita la recuperación de fallos.
- ▶ **Arquitectura maestro-esclavo.** HDFS sigue una arquitectura maestro-esclavo, donde un conjunto de nodos maestros (NameNode y Secondary NameNode) gestionan el sistema de archivos y un conjunto de nodos esclavos (DataNodes) almacenan los datos reales.
- **NameNode.** El NameNode es el nodo maestro responsable de mantener el sistema de archivos y el árbol de metadatos. Contiene información sobre la ubicación y el estado de todos los bloques de datos, así como los metadatos asociados con cada archivo.
- **DataNode.** Los DataNodes son los nodos esclavos que almacenan los bloques de datos reales. Cada DataNode es responsable de gestionar el almacenamiento de los bloques asignados a él y de comunicarse con el NameNode para informar sobre su estado y replicar los bloques según sea necesario.

Tema 3. Data science cloud storage

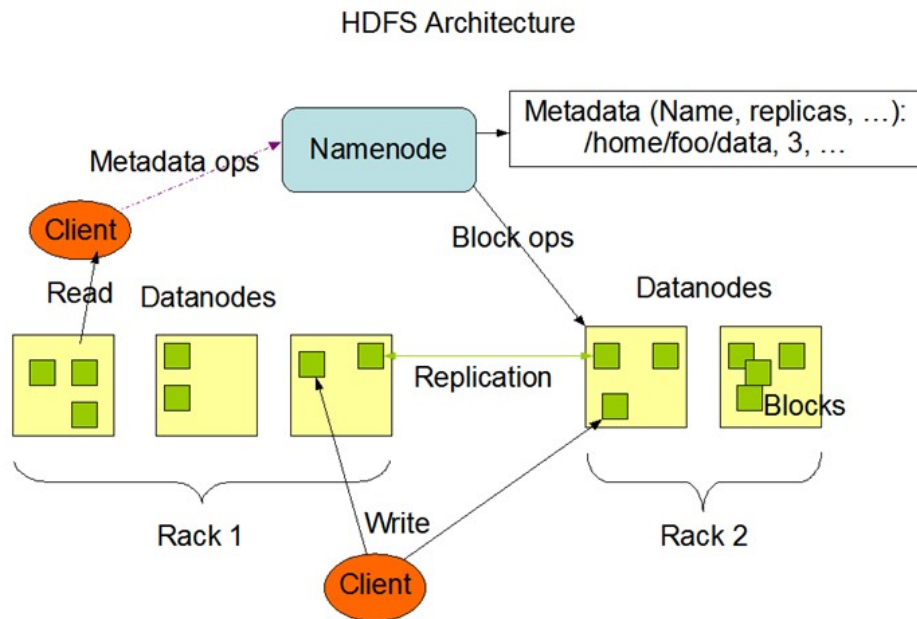


Figura 18. Arquitectura HDFS. Fuente: Mamani, 2023.

- **Replicación de datos.** HDFS utiliza la replicación de datos para garantizar la disponibilidad y la tolerancia a fallos. Por defecto, cada bloque de datos se replica en múltiples nodos DataNode (generalmente tres), lo que garantiza que los datos estén disponibles incluso en caso de fallo de un nodo.
- **API para acceso a datos.** HDFS proporciona API para acceder y manipular datos almacenados en el sistema de archivos, incluyendo una API Java, comandos de línea de Hadoop (Hadoop Shell) y API de acceso a través de Hadoop Streaming y Hadoop Pipes.

En resumen, **Apache HDFS** es un sistema de archivos distribuido diseñado para almacenar grandes conjuntos de datos de manera confiable y escalable en clústeres de servidores estándar. Ofrece distribución de datos, tolerancia a fallos, replicación de datos y API para el acceso a los datos, lo que lo convierte en una parte fundamental del ecosistema de Hadoop para el procesamiento distribuido de datos.

Tema 3. Data science cloud storage

Sistemas de archivos EMRFS

EMRFS (EMR File System) es una característica específica de Amazon Elastic MapReduce (EMR), que facilita el acceso a los datos almacenados en Amazon S3 desde clústeres de EMR. Se integra con Apache Hadoop y Apache Spark, lo que les permite a los usuarios leer y escribir datos en S3 como si estuvieran trabajando con un sistema de archivos tradicional, pero con mejoras específicas para el entorno de EMR.

Estas son hay algunas **características** importantes de EMRFS y se explica cómo se relacionan con HDFS en EMR:

- ▶ **Integración con S3.** EMRFS está diseñado específicamente para trabajar con Amazon S3 como un sistema de almacenamiento subyacente. Permite a los usuarios acceder y manipular datos en S3 de manera eficiente y confiable, aprovechando las características de S3 como escalabilidad, durabilidad y bajo costo.
- ▶ **Consistencia de metadatos.** EMRFS garantiza la consistencia de los metadatos entre el clúster de EMR y S3, lo que significa que los cambios realizados en los metadatos (como la creación, modificación o eliminación de archivos) son reflejados de manera consistente en ambos lados.
- ▶ **Optimizaciones de rendimiento.** EMRFS implementa diversas optimizaciones de rendimiento para mejorar la eficiencia de las operaciones de lectura y escritura en S3. Esto incluye el uso de cachés locales y remotos para metadatos y datos, así como técnicas de *prefetching* y *prefetching* de datos para acelerar el acceso a los datos.

Tema 3. Data science cloud storage

- ▶ **Consistencia fuerte.** A diferencia de HDFS, que utiliza una consistencia eventual para los metadatos, EMRFS ofrece consistencia fuerte para garantizar que los cambios en los metadatos sean visibles de manera inmediata en todo el clúster de EMR.
- ▶ **Soporte para características avanzadas de S3.** EMRFS es compatible con muchas características avanzadas de Amazon S3, como el cifrado de datos, control de acceso basado en políticas (IAM), versionado de objetos y Amazon S3 Cross-Region Replication (CRR).

En cuanto a **HDFS en EMR**, es importante tener en cuenta que los clústeres de EMR pueden utilizar tanto HDFS como EMRFS para el almacenamiento de datos. Sin embargo, EMRFS se considera la opción preferida para acceder a datos en S3 debido a sus ventajas en términos de escalabilidad, durabilidad y compatibilidad con las características avanzadas de S3. En resumen, EMRFS es la opción recomendada para acceder a datos en S3 desde clústeres de EMR, mientras que HDFS sigue siendo una opción válida para el almacenamiento de datos en el sistema de archivos local del clúster.



Figura 19. Flujo de trabajo ETL. Fuente: Chayel, 2014.

Tema 3. Data science cloud storage

3.4. Data warehouse en la nube

Data warehouse. Introducción

Un **data warehouse** (almacén de datos) es un sistema de almacenamiento y gestión de datos diseñado para posibilitar el análisis y la elaboración de informes sobre grandes volúmenes de datos, procedentes de múltiples fuentes. Su principal objetivo es centralizar y consolidar datos históricos para facilitar la toma de decisiones estratégicas en una organización.

Entre los principales **usos** de un *data warehouse*, tenemos los siguientes:

- ▶ **Análisis de datos.** Facilita el análisis en profundidad de los datos históricos de la empresa. Permite realizar análisis de tendencias y patrones a largo plazo.
- ▶ **Business intelligence (BI).** Provee una base para herramientas de inteligencia empresarial que ayudan a transformar datos en información útil para la toma de decisiones.
- ▶ **Soporta cuadros de mando, informes y visualización de datos.**
- ▶ **Mejora del rendimiento empresarial.** Ayuda a identificar oportunidades de negocio, mejorar procesos y aumentar la eficiencia. Permite la evaluación del rendimiento de distintas áreas de la empresa.
- ▶ **Integración de datos.** Centraliza datos provenientes de diversas fuentes y sistemas, facilitando su acceso y análisis conjunto. Mejora la coherencia y calidad de los datos.
- ▶ **Soporte a la toma de decisiones.** Proporciona una visión unificada y completa de la información empresarial. Facilita la toma de decisiones informadas basadas en datos precisos y actualizados.

Tema 3. Data science cloud storage

Sobre las **características** fundamentales de un *data warehouse*, podemos enumerar las siguientes:

- ▶ **Orientación a sujetos.** Los datos están organizados por temas o áreas de interés, como ventas, finanzas, recursos humanos, etc., en lugar de por aplicaciones.
- ▶ **Integración.** Consolida datos de diferentes fuentes, garantizando consistencia y calidad. Estandariza formatos, unidades y estructuras de datos.
- ▶ **Variabilidad en el tiempo.** Almacena datos históricos, lo que permite análisis longitudinales y comparación a lo largo del tiempo. Los datos son etiquetados con un marco temporal, lo que facilita el seguimiento de cambios y tendencias.
- ▶ **No volatilidad.** Una vez que los datos se cargan en el *data warehouse*, no se modifican ni se eliminan. Permite mantener la integridad y exactitud de los datos históricos.
- ▶ **Optimización para consulta y análisis.** Está diseñado específicamente para ejecutar consultas y análisis rápidos y eficientes. Emplea técnicas como indexación, particionamiento y cubos OLAP (*online analytical processing*) para mejorar el rendimiento de las consultas.

Para entender la **arquitectura** de un *data warehouse*, se muestra el siguiente ejemplo:

- ▶ **Fuente de datos.** Sistemas transaccionales (ERP, CRM), archivos planos, bases de datos relacionales y datos externos.
- ▶ **ETL (extracción, transformación y carga).** Procesos para extraer datos de las fuentes, transformarlos según las necesidades de análisis y cargarlos en el *data warehouse*.
- ▶ **Área de *staging*.** Espacio temporal para limpiar y transformar los datos antes de cargarlos en el *data warehouse*.

Tema 3. Data science cloud storage

- ▶ **Almacén de datos central.** Base de datos principal donde se almacenan los datos integrados y consolidados.
- ▶ **Data marts.** Subconjuntos de datos específicos orientados a necesidades particulares de negocio o departamentos.
- ▶ **Herramientas de BI y análisis.** Aplicaciones y plataformas que permiten a los usuarios finales realizar consultas, generar informes y visualizar datos.

En resumen, un **data warehouse** es una herramienta clave para la gestión y el análisis de grandes volúmenes de datos, lo que proporciona una base sólida para la inteligencia empresarial y la toma de decisiones informada.

Data warehouse en la nube

En los últimos tiempos, existe una clara simbiosis entre un *data warehouse* y el *cloud* (la nube) mediante la integración de los almacenes de datos tradicionales con las tecnologías y servicios en la nube. Esta **combinación** ofrece una serie de ventajas significativas para las organizaciones en términos de flexibilidad, escalabilidad y costo-efectividad. A continuación, se detalla esta relación y sus beneficios.

- ▶ **Escalabilidad dinámica:**
 - **Data warehouses.** Tradicionalmente, los *data warehouses* se implementaban en infraestructuras físicas limitadas por la capacidad del *hardware* disponible.
 - **Cloud.** La nube permite escalar recursos de almacenamiento y procesamiento de manera dinámica según las necesidades, sin requerir inversiones iniciales en *hardware*.

Tema 3. Data science cloud storage

► Coste-efectividad:

- **Data warehouse.** La implementación y mantenimiento de un *data warehouse* on-premise (en las instalaciones de la empresa) puede ser costosa debido a la compra de *hardware*, licencias de *software* y personal especializado.
- **Cloud.** Los servicios de *data warehouse* en la nube, como Amazon Redshift, Google BigQuery y Azure Synapse Analytics, operan bajo un modelo de pago por uso, lo que reduce significativamente los costos iniciales y permite una mejor gestión del presupuesto al pagar solo por los recursos utilizados.

► Flexibilidad y agilidad:

- **Data warehouse.** Las actualizaciones y expansiones en un entorno *on-premise* pueden ser lentas y disruptivas.
- **Cloud.** Los servicios en la nube permiten implementar nuevas capacidades, actualizar *software* y ajustar configuraciones de manera rápida y sin interrupciones, lo que facilita una respuesta ágil a las necesidades cambiantes del negocio.

► Acceso global y colaboración:

- **Data warehouse.** El acceso a los datos puede estar limitado por la ubicación física del servidor y las restricciones de red.
- **Cloud.** Los *data warehouses* en la nube están accesibles desde cualquier lugar con conexión a internet, lo que facilita la colaboración global y el acceso remoto a los datos por parte de empleados, socios y clientes.

Tema 3. Data science cloud storage

► Seguridad y recuperación ante desastres:

- **Data warehouse.** Requiere una inversión significativa en medidas de seguridad y planes de recuperación ante desastres.
- **Cloud.** Los proveedores de servicios en la nube ofrecen avanzadas características de seguridad y recuperación ante desastres como parte de sus servicios, incluyendo cifrado de datos, redundancia y copias de seguridad automáticas.

Entre las **ventajas** de implementación de un *data warehouse* en la nube, podemos destacar las siguientes:

- **Implementación rápida.** Los *data warehouses* en la nube se pueden desplegar y configurar rápidamente sin la necesidad de instalar y configurar *hardware* físico.
- **Mantenimiento reducido.** Los proveedores de la nube gestionan el mantenimiento del *hardware* y las actualizaciones del *software*, lo que permite que las organizaciones se concentren en el análisis de datos y la toma de decisiones.
- **Integración con servicios de datos.** Los servicios de *data warehouse* en la nube suelen integrarse fácilmente con otros servicios en la nube, como herramientas de análisis de datos, aprendizaje automático y *big data*, creando un ecosistema completo para la gestión y análisis de datos.
- **Elasticidad.** La capacidad de ajustar automáticamente los recursos de procesamiento y almacenamiento en respuesta a las cargas de trabajo variables, garantizando un rendimiento óptimo sin necesidad de sobreaprovisionar recursos.
- **Innovación continua.** Los proveedores de la nube introducen regularmente nuevas características y mejoras, lo que les permite a las empresas beneficiarse de las últimas innovaciones tecnológicas sin esfuerzo adicional.

Tema 3. Data science cloud storage

Por tanto, la **integración de los *data warehouses* con la nube** ofrece una poderosa combinación de escalabilidad, flexibilidad, y eficiencia en costos. Las organizaciones pueden manejar grandes volúmenes de datos con mayor agilidad, garantizar la seguridad y la recuperación de sus datos, y aprovechar un ecosistema de servicios en la nube para potenciar su inteligencia empresarial y capacidad analítica. Esta simbiosis transforma el manejo de datos en una ventaja competitiva significativa en el entorno empresarial actual.

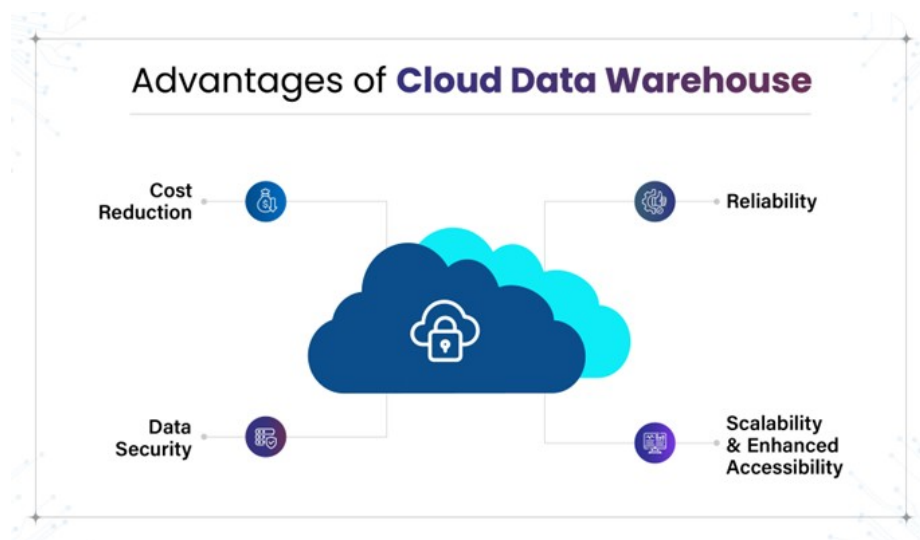


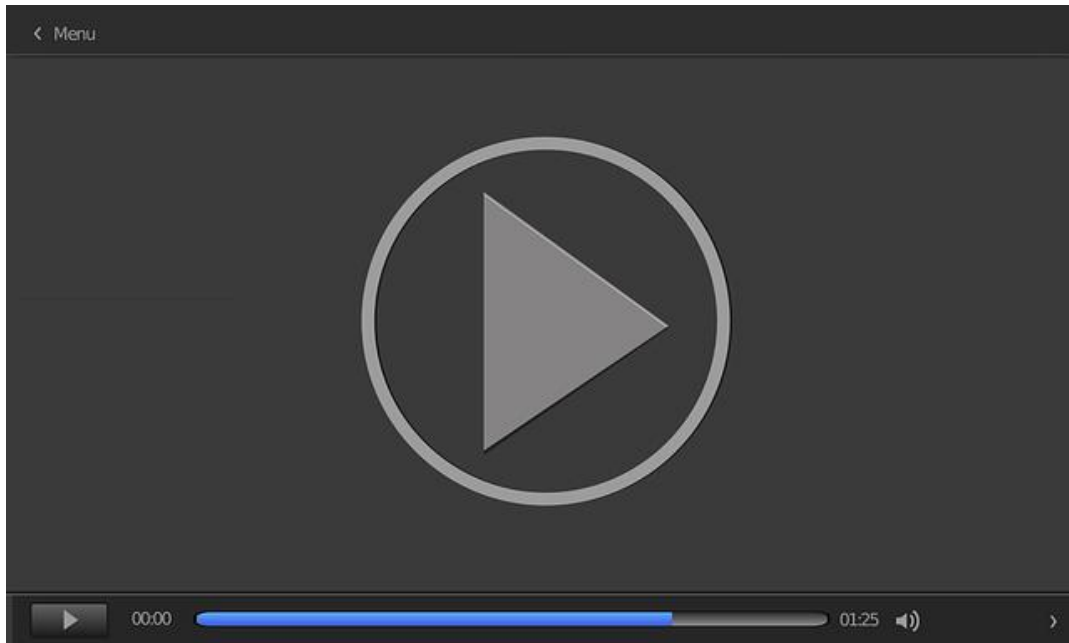
Figura 20. Ventajas del *cloud data warehouse*. Fuente: Cogent, 2024.

AWS Redshift

Amazon Redshift es un servicio de almacenamiento de datos (*data warehouse*) en la nube proporcionado por AWS. Está diseñado para manejar grandes cantidades de datos y realizar consultas analíticas rápidas y eficientes. A continuación, se describen sus características, funcionalidades, ventajas y casos de uso.

Tema 3. Data science cloud storage

En el vídeo *Introduction to Data Warehousing on AWS with Amazon Redshift | Amazon Web Services* (Amazon Web Services, 2021a), se realiza una introducción al servicio de AWS Redshift.



Introduction to Data Warehousing on AWS with Amazon Redshift | Amazon Web Services.

Accede al vídeo:

https://www.youtube.com/embed/IWwFJV_9PoE

Características de Amazon Redshift

Entre las principales características de Amazon Redshift, podemos enumerar:

- ▶ **Arquitectura distribuida.** Utiliza una arquitectura de clúster que permite distribuir las tareas de procesamiento de datos entre múltiples nodos, lo que mejora el rendimiento y la escalabilidad.
- ▶ **Compatibilidad con SQL.** Soporta SQL estándar, lo que facilita a los usuarios ejecutar consultas complejas y análisis sobre los datos almacenados.

Tema 3. Data science cloud storage

- ▶ **Integración con el ecosistema de AWS.** Se integra perfectamente con otros servicios de AWS, como S3, EMR, Kinesis y más, lo que permite una gestión y el análisis de datos más cohesiva.
- ▶ **Compresión de datos.** Redshift utiliza técnicas avanzadas de compresión de datos, lo que reduce el tamaño de almacenamiento necesario y mejora el rendimiento de las consultas.
- ▶ **Almacenamiento en columnas.** Emplea un almacenamiento en columnas, que optimiza las consultas de lectura intensiva, lo que permite un acceso rápido y eficiente a grandes conjuntos de datos.
- ▶ **Copias de seguridad automáticas.** Realiza copias de seguridad automáticas de los datos para garantizar la durabilidad y la recuperación en caso de fallos.
- ▶ **Data sharing.** Para compartir de datos entre clústeres de manera segura y eficiente.
- ▶ **Integración con Spark.**

La Figura 21 muestra las principales características y el flujo de funcionamiento de RedShift.

Tema 3. Data science cloud storage



Figura 21. Características y flujo de funcionamiento de RedShift. Fuente: Amazon Web Services, s. f.-e.

Funcionalidades de Amazon Redshift

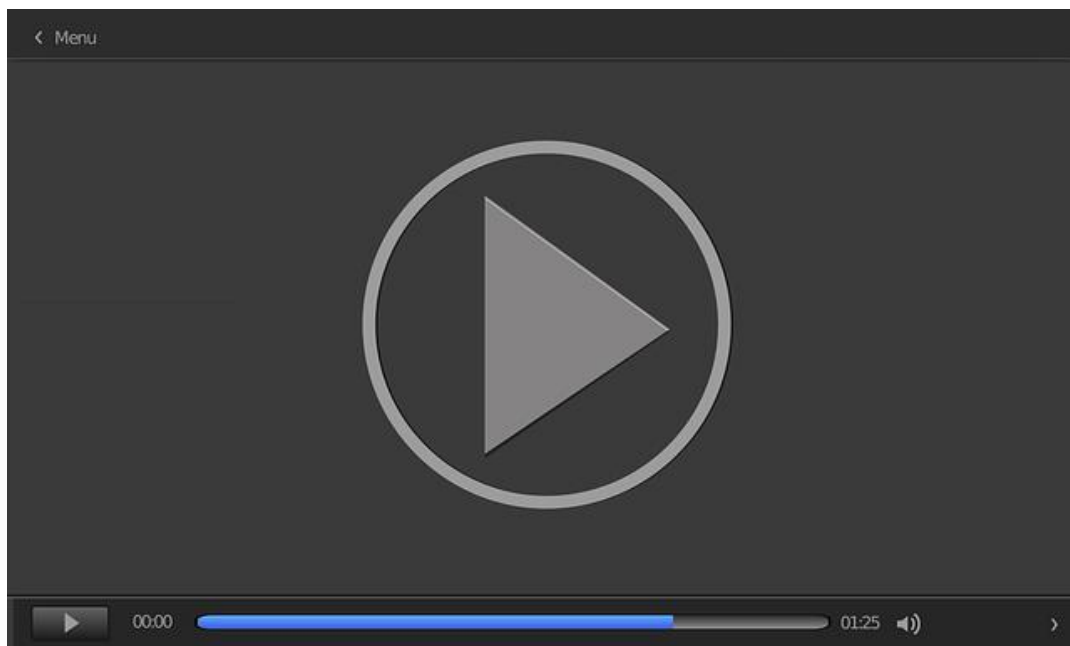
Entre las principales funcionalidades de Redshift, se pueden destacar las siguientes:

- ▶ **Redshift Spectrum.** Permite ejecutar consultas directamente sobre datos almacenados en Amazon S3 sin necesidad de cargar estos datos en Redshift, combinando el almacenamiento en columnas con la flexibilidad del almacenamiento en objetos.
- ▶ **Concurrencia y escalabilidad.** Redshift ofrece características para manejar múltiples consultas simultáneamente y ajustarse dinámicamente a las cargas de trabajo cambiantes.
- ▶ **Seguridad.** Soporta cifrado de datos en reposo y en tránsito, integración con AWS IAM para gestión de acceso y capacidades de auditoría y monitoreo a través de CloudTrail.
- ▶ **Optimización automática.** Redshift proporciona análisis y recomendaciones automáticas para optimizar las consultas y el rendimiento del clúster.

Tema 3. Data science cloud storage

- **Compatibilidad con *data lakes*.** Se integra con Amazon Lake Formation para construir *data lakes*, lo que permite que las organizaciones combinen *data warehouses* y *data lakes*.

En el vídeo *Getting Started with Amazon Redshift - AWS Online Tech Talks* (Amazon Web Services, 2020a), se realiza una introducción sobre cómo comenzar a trabajar en Amazon Redshift.



Getting Started with Amazon Redshift - AWS Online Tech Talks.

Accede al vídeo:

<https://www.youtube.com/embed/dfo4J5ZhIKI>

Tema 3. Data science cloud storage

Ventajas de Amazon Redshift

A continuación, se enumeran una serie de ventajas que aporta el uso de AWS Redshift:

- ▶ **Alto rendimiento.** La arquitectura de almacenamiento en columnas y la distribución de carga entre nodos garantizan un rendimiento elevado para consultas complejas y grandes volúmenes de datos.
- ▶ **Escalabilidad.** Redshift puede escalar fácilmente añadiendo o eliminando nodos según las necesidades de procesamiento y almacenamiento.
- ▶ **Costo-efectividad.** Ofrece un modelo de precios basado en el uso, lo que permite a las empresas pagar solo por los recursos que utilizan. Las opciones de pago por demanda y precios reservados a largo plazo permiten un mejor control de costos.
- ▶ **Fácil de usar.** La gestión y la configuración de Redshift son sencillas gracias a la integración con el ecosistema de AWS y las herramientas de gestión proporcionadas.
- ▶ **Integración y ecosistema.** La estrecha integración con otros servicios de AWS y herramientas de terceros facilita la creación de soluciones de análisis de datos completas y personalizadas.

Casos de uso de Amazon Redshift

Como casos de uso donde utilizar AWS Redshift, podemos describir los siguientes:

- ▶ **Análisis de datos empresariales.** Empresas pueden utilizar Redshift para analizar datos de ventas, *marketing*, finanzas y operaciones, lo que permite una toma de decisiones basada en datos.
- ▶ **BI y reporting.** Organizaciones utilizan Redshift para generar informes y cuadros de mando interactivos en tiempo real, integrándose con herramientas de BI como Tableau, Looker y Amazon QuickSight.

Tema 3. Data science cloud storage

- ▶ **Big data analytics.** Capaz de manejar grandes volúmenes de datos, Redshift es ideal para análisis de *big data*, lo que les permite a las empresas descubrir patrones y tendencias en sus datos.
- ▶ **Análisis de *clickstream*.** Empresas de comercio electrónico y medios digitales usan Redshift para analizar datos de *clickstream* y comprender mejor el comportamiento de los usuarios en sus plataformas.
- ▶ **Integración de *data lakes*.** Combinando Redshift con Amazon S3 y Redshift Spectrum, las organizaciones pueden analizar datos estructurados y no estructurados de manera conjunta, creando soluciones de análisis de datos más flexibles y poderosas.

En resumen, **Amazon Redshift** es una solución robusta y flexible para el almacenamiento y el análisis de grandes volúmenes de datos, y ofrece un alto rendimiento, escalabilidad y una integración fluida con otros servicios de AWS y herramientas de análisis.

Uso y funcionamiento de Redshift

A continuación, se describe el uso y funcionamiento de AWS Redshift. Comenzamos por la **configuración inicial** de AWS Redshift:

- ▶ **Crear un clúster de Redshift:**
 - Inicia sesión en la consola de administración de AWS.
 - Navega a Amazon Redshift y selecciona «Create Cluster».
 - Configura el clúster especificando el nombre, el tipo de nodos, el número de nodos y otras configuraciones de red y seguridad.

Tema 3. Data science cloud storage

► Configuración de redes y seguridad:

- Configura el VPC (*virtual private cloud*), las subredes y los grupos de seguridad para controlar el acceso al clúster.
- Asegúrate de configurar las reglas de entrada y salida para permitir el acceso desde las ubicaciones permitidas.

► Configurar usuarios y privilegios:

- Utiliza el entorno de SQL de Redshift para crear usuarios y roles, y asigna los privilegios necesarios para el acceso y la administración de los datos.

► Carga de datos:

- Los datos pueden cargarse en Redshift desde diversas fuentes, como Amazon S3, bases de datos relacionales, servicios de flujo de datos, como Kinesis, y a través de herramientas ETL.
- Utiliza el comando `COPY` para cargar datos desde Amazon S3, DynamoDB o archivos locales en las tablas de Redshift.

```
sql
COPY table_name
FROM 's3://bucket_name/file_path'
CREDENTIALS 'aws_access_key_id=...;aws_secret_access_key=...'
CSV;
```

En cuanto al funcionamiento general de Redshift, los principales **componentes** se describen a continuación:

- **Arquitectura.** Redshift usa una arquitectura MPP (procesamiento masivo en paralelo) que distribuye datos y consultas a través de múltiples nodos. Los datos se almacenan en un formato columnar, que es optimizado para las consultas analíticas. Un ejemplo de arquitectura columnar y MPP de Redshift se muestra en la Figura 22.

Tema 3. Data science cloud storage

- ▶ **Consultas SQL.** Redshift soporta SQL estándar, lo que permite realizar operaciones de selección, inserción, actualización y eliminación. Las consultas se distribuyen entre los nodos para aprovechar el procesamiento paralelo.
- ▶ **Optimización.** Redshift incluye características de optimización automática, como el ajuste de consultas, almacenamiento en caché de resultados y recomendaciones de distribución de datos.
- ▶ **Query Editor de Redshift.** El Query Editor de Redshift es una herramienta integrada en la consola de AWS que les permite a los usuarios escribir y ejecutar consultas SQL directamente en el clúster de Redshift sin necesidad de instalar un *software* adicional.

Tema 3. Data science cloud storage

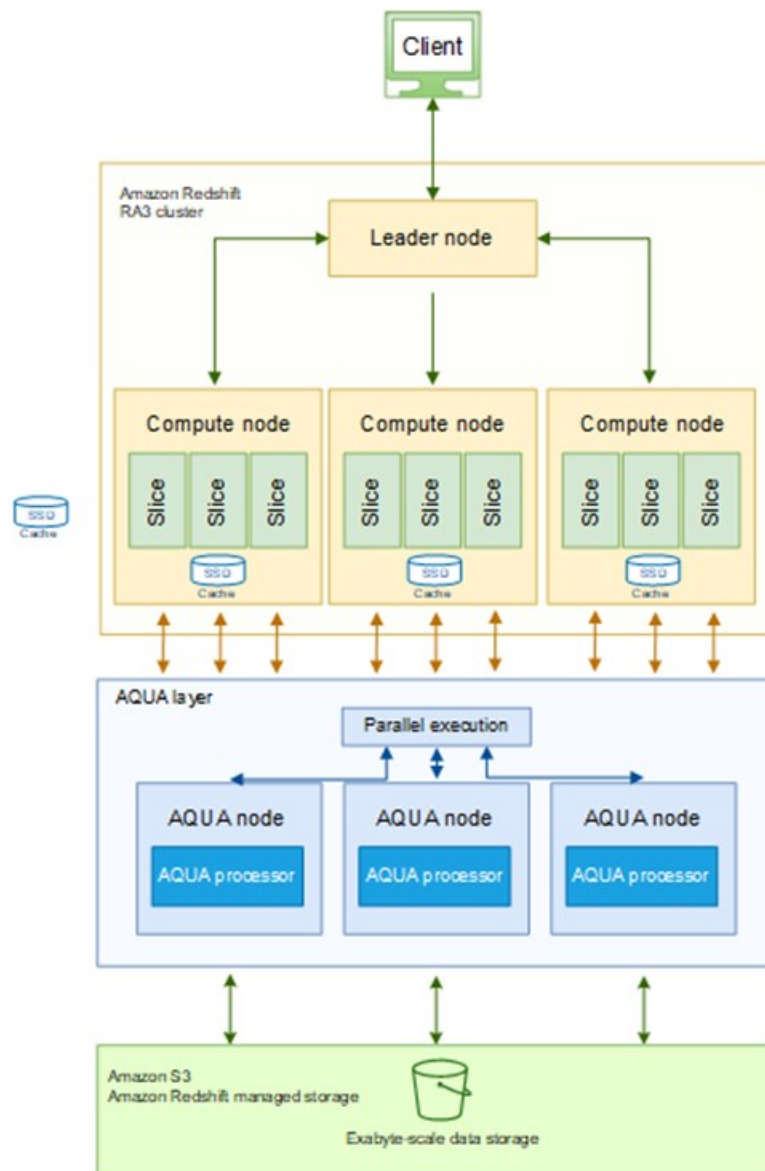


Figura 22. Arquitectura columnar y MPP de Redshift.

Tema 3. Data science cloud storage

A continuación, se muestra **cómo usar el Query Editor** y los pasos que hay que realizar:

► Accede al Query Editor:

- Inicia sesión en la consola de AWS y navega a Amazon Redshift.
- Selecciona el clúster de Redshift que deseas usar y haz clic en «Query Editor» en el panel de navegación.

► Conexión al clúster:

- Ingresa las credenciales necesarias para conectarte al clúster (nombre de usuario y contraseña).
- Selecciona la base de datos y el esquema con los que deseas trabajar.

► Escribir y ejecutar consultas:

- En el editor de consultas, escribe tus comandos SQL. Puedes realizar selecciones, inserciones, actualizaciones y eliminar datos, así como crear y modificar tablas.
- Haz clic en «Run» para ejecutar la consulta.

```
sql
SELECT * FROM users WHERE age > 30;
```

► Visualización de resultados:

- Los resultados de las consultas se mostrarán en una tabla debajo del editor.
- Puedes exportar los resultados en formatos CSV para su análisis posterior.

Tema 3. Data science cloud storage

► Administración de consultas:

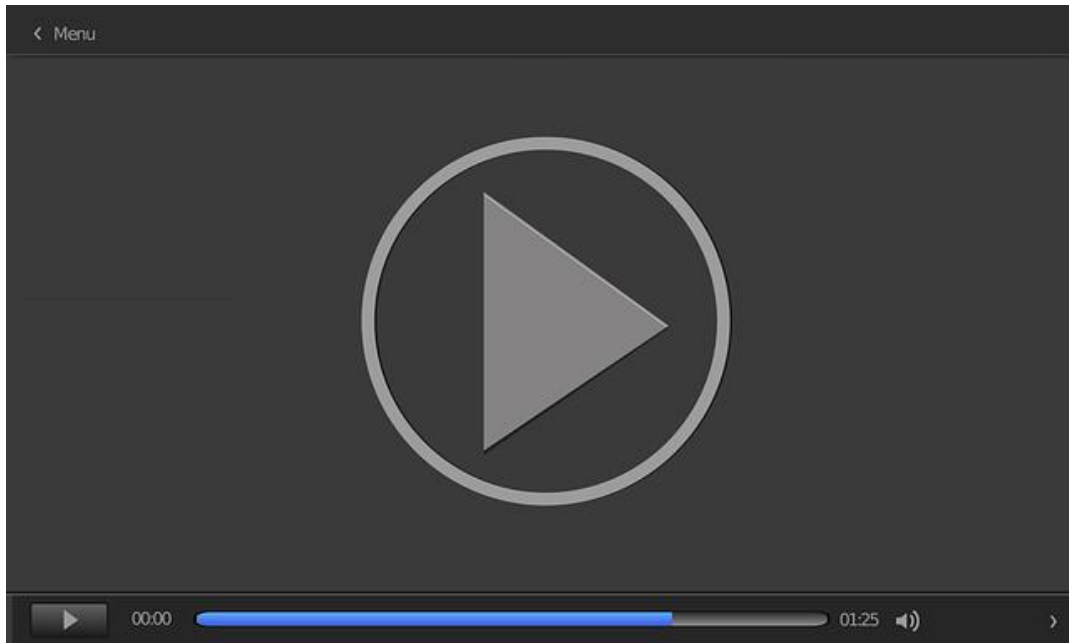
- El Query Editor permite guardar consultas frecuentes para reutilizarlas.
- Puedes ver el historial de consultas ejecutadas y reutilizar consultas anteriores.

Las **ventajas** del Query Editor son:

- **Facilidad de uso.** Proporciona una interfaz sencilla y accesible para escribir y ejecutar consultas SQL sin la necesidad de herramientas adicionales.
- **Acceso inmediato.** Permite el acceso inmediato a los datos del clúster de Redshift para realizar análisis *ad hoc*.
- **Sin instalación adicional.** Todo se maneja a través de la consola de AWS, lo que elimina la necesidad de instalar y de configurar clientes de SQL o herramientas ETL adicionales.
- **Integración completa.** El Query Editor está completamente integrado con el ecosistema de AWS, lo que facilita el acceso y la manipulación de datos en Redshift.

Tema 3. Data science cloud storage

A continuación, se muestra el vídeo *An Overview of Amazon Redshift Query Editor V2* / *Amazon Web Services* (Amazon Web Services, 2021b), que explica el AWS RedShift Query Editor.



An Overview of Amazon Redshift Query Editor V2 | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/lwZNlroJUnc>

En resumen, **Amazon Redshift** es una potente solución de *data warehouse* en la nube que ofrece escalabilidad, rendimiento y facilidad de uso. Su integración con el ecosistema de AWS y herramientas como el Query Editor facilita la gestión, el análisis y la visualización de grandes volúmenes de datos, haciendo que las empresas puedan tomar decisiones informadas basadas en datos de manera eficiente.

Redshift Serverless

Amazon Redshift Serverless es una variante del servicio de *data warehouse* en la nube Amazon Redshift que elimina la necesidad de administrar infraestructura.

Tema 3. Data science cloud storage

Proporciona una experiencia totalmente gestionada, donde AWS gestiona automáticamente el aprovisionamiento, la configuración, la escalabilidad y el mantenimiento de los recursos de almacenamiento y procesamiento de datos.

Redshift Serverless permite a los usuarios ejecutar y escalar cargas de trabajo de análisis de datos sin preocuparse por la **administración de clústeres**. Los usuarios simplemente cargan sus datos y ejecutan consultas, y Redshift Serverless ajusta automáticamente los recursos necesarios para satisfacer las demandas de carga de trabajo.

Entre las principales **funcionalidades**, tenemos las siguientes:

- ▶ **Autoescalado.** Escala automáticamente los recursos de computación y almacenamiento en función de las necesidades de las cargas de trabajo, sin intervención manual.
- ▶ **Precios basados en el uso.** Cobra solo por el tiempo de consulta y los recursos utilizados, lo que puede resultar más económico para cargas de trabajo variables o intermitentes.
- ▶ **Simplicidad de configuración.** No requiere configuración de nodos, clústeres ni administración de infraestructura. Los usuarios pueden comenzar a cargar datos y ejecutar consultas rápidamente.
- ▶ **Compatibilidad completa con Redshift.** Total compatibilidad con las funciones y las características de Amazon Redshift, lo que incluye consultas SQL, integraciones con AWS y herramientas de BI.
- ▶ **Inicio rápido.** Proporciona un punto de partida rápido para ejecutar consultas sin necesidad de aprovisionar clústeres. Ideal para pruebas, desarrollo y análisis *ad hoc*.
- ▶ **Seguridad y cumplimiento.** Ofrece características avanzadas de seguridad, como cifrado de datos en reposo y en tránsito, integración con AWS IAM y cumplimiento con regulaciones y normativas de seguridad.

Tema 3. Data science cloud storage

Respecto a las **ventajas** que ofrece Redshift Serverless, tenemos las siguientes:

- ▶ **Administración simplificada.** Elimina la necesidad de gestionar la infraestructura del *data warehouse*, lo que permite que los usuarios se centren en el análisis y la obtención de *insights*.
- ▶ **Escalabilidad automática.** Ajusta automáticamente los recursos de computación y almacenamiento para satisfacer las demandas de las cargas de trabajo, lo que asegura un rendimiento óptimo.
- ▶ **Costes reducidos.** El modelo de pago por uso puede reducir significativamente los costos operativos, especialmente para cargas de trabajo intermitentes o variables.
- ▶ **Rapidez y flexibilidad.** Permite iniciar rápidamente con análisis de datos sin la necesidad de configuraciones complejas, lo que facilita la agilidad y la flexibilidad en el trabajo con datos.
- ▶ **Integración con el ecosistema AWS.** Se integra perfectamente con otros servicios de AWS, como S3, Glue, Kinesis y Lambda, lo que facilita la creación de *pipelines* de datos y soluciones de análisis complejas.

Por último, tenemos algunos ejemplos de **casos de uso** de Redshift Serverless:

- ▶ **Análisis *ad hoc*.** Ideal para análisis de datos no planificados donde la carga de trabajo es impredecible. Los analistas pueden ejecutar consultas *ad hoc* sin preocuparse por la capacidad de la infraestructura.
- ▶ **Entornos de prueba y desarrollo.** Permite a los desarrolladores y científicos de datos probar y desarrollar nuevas consultas y modelos analíticos sin necesidad de configurar y gestionar clústeres dedicados.

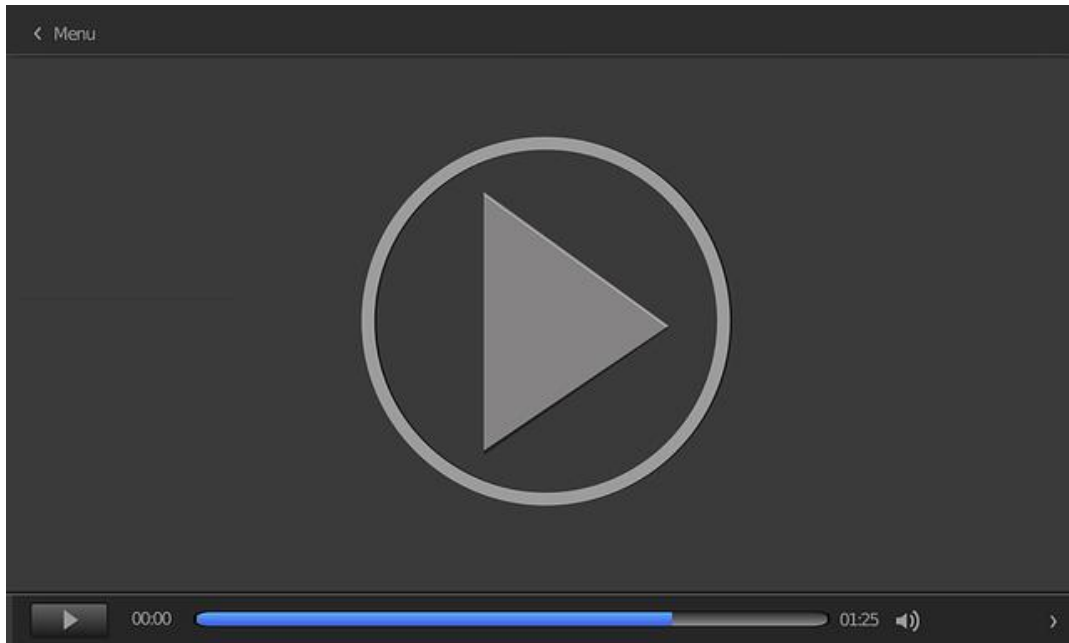
Tema 3. Data science cloud storage

- ▶ **Aplicaciones intermitentes.** Es adecuado para aplicaciones que requieren análisis de datos de manera intermitente, por lo que evita el costo de mantener clústeres en funcionamiento continuo.
- ▶ **Pequeñas y medianas empresas (pymes).** Las pymes que no tienen los recursos para gestionar infraestructura de *data warehousing* pueden beneficiarse de la simplicidad y el modelo de pago por uso de Redshift Serverless.
- ▶ **Análisis de eventos y datos de IoT.** Permite analizar grandes volúmenes de datos generados por eventos o dispositivos IoT de manera eficiente, sin necesidad de gestionar la infraestructura subyacente.

Amazon Redshift Serverless ofrece una solución de data warehousing poderosa y flexible que elimina la complejidad de la administración de infraestructura. Con su capacidad de autoescalado, precios basados en uso y fácil integración con el ecosistema de AWS, es una excelente opción para organizaciones que buscan agilizar sus procesos de análisis de datos y reducir costos operativos. Redshift Serverless proporciona las herramientas necesarias para que las empresas se centren en obtener *insights* valiosos de sus datos sin preocuparse por la gestión de recursos.

Tema 3. Data science cloud storage

A continuación, se muestra el vídeo *Amazon Redshift Serverless - End to End Use Case | Amazon Web Services* (Amazon Web Services, 2022a), que muestra el funcionamiento de AWS Redshift Serverless.



Amazon Redshift Serverless - End to End Use Case | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/gEAWSqjthXs>

Redshift Data Sharing

Amazon Redshift incluye una funcionalidad llamada **Redshift Data Sharing** que permite compartir datos de manera segura y eficiente entre distintos clústeres de Redshift. Esta característica es especialmente útil para organizaciones que necesitan colaborar entre diferentes equipos o departamentos, permitiendo un acceso rápido a los datos sin necesidad de mover o copiar grandes volúmenes de información.

Tema 3. Data science cloud storage

Como **características** del Redshift Data Sharing, se pueden enumerar las siguientes:

- ▶ **Acceso en tiempo real.** Permite compartir datos en tiempo real entre diferentes clústeres de Redshift sin necesidad de duplicar los datos. Los usuarios pueden acceder a los datos más actualizados en cualquier momento.
- ▶ **Seguridad y control de acceso.** Los permisos de acceso se pueden gestionar de manera granular. Los propietarios de los datos pueden controlar quién puede acceder a qué datos, por lo que de este modo se garantiza la seguridad y la privacidad.
- ▶ **Eficiencia en el uso de recursos.** Al compartir datos sin duplicarlos, se reduce la necesidad de almacenamiento adicional y se optimiza el uso de los recursos de almacenamiento y procesamiento.
- ▶ **Escalabilidad.** Facilita la colaboración entre múltiples equipos o departamentos, cada uno con su propio clúster de Redshift, sin preocuparse por los límites físicos de un solo clúster.
- ▶ **Simplicidad.** El proceso de compartir datos es sencillo y no requiere configuraciones complicadas ni la implementación de ETL adicionales.

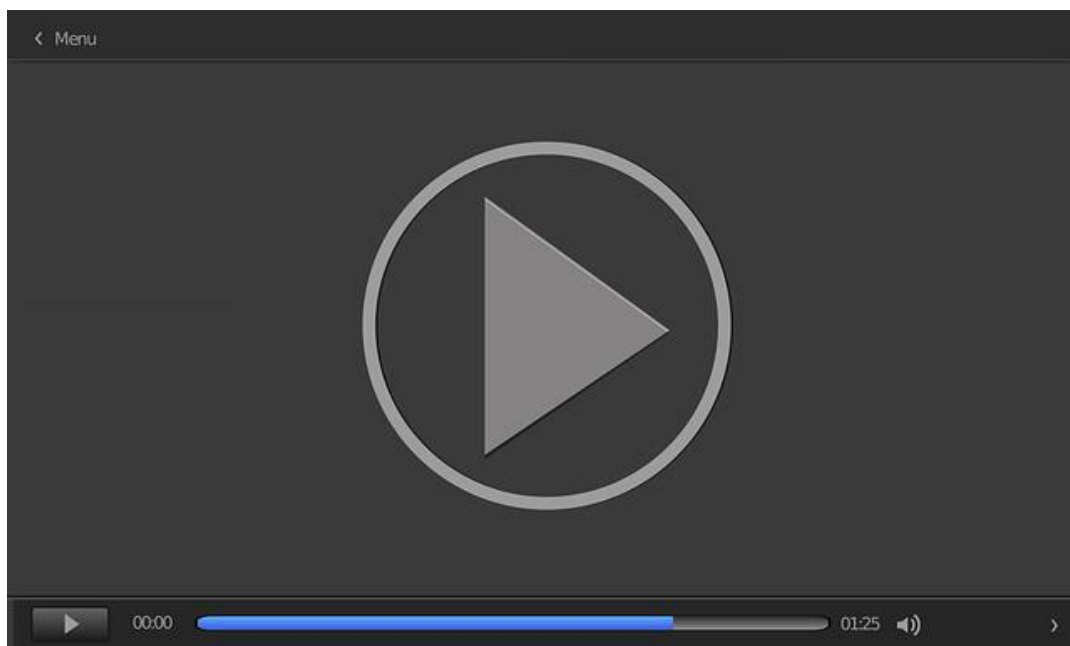
Las principales **funcionalidades** de Redshift Sharing son:

- ▶ **Compartición entre clústeres.** Permite a un clúster de Redshift (clúster productor) compartir datos con uno o más clústeres (clústeres consumidores) de manera directa.
- ▶ **Compatibilidad con SQL.** Los usuarios pueden ejecutar consultas SQL en los datos compartidos de la misma manera que lo harían con los datos locales en sus clústeres.

Tema 3. Data science cloud storage

- ▶ **Soporte para diferentes regiones.** Redshift Data Sharing soporta la compartición de datos entre clústeres en diferentes regiones de AWS, lo que facilita la colaboración global.
- ▶ **Metadatos y estadísticas.** Los clústeres consumidores tienen acceso a los metadatos y las estadísticas de los datos compartidos, lo que mejora el rendimiento de las consultas.

A continuación, se muestra el vídeo *Amazon Redshift Data Sharing Workflow* (Amazon Web Services, 2020b), que muestra un ejemplo del funcionamiento de AWS Redshift Data Sharing.



Amazon Redshift Data Sharing Workflow.

Accede al vídeo:

<https://www.youtube.com/embed/EXioFirlrNA>

Tema 3. Data science cloud storage

Las **ventajas** que ofrece Redshift Data Sharing son las siguientes:

- ▶ **Colaboración mejorada.** Facilita la colaboración entre diferentes equipos y departamentos al permitir el acceso compartido a datos críticos sin necesidad de copiar grandes volúmenes de información.
- ▶ **Aceleración del análisis.** Los datos están disponibles para análisis en tiempo real, lo que acelera los procesos de toma de decisiones basadas en datos.
- ▶ **Reducción de costos.** Al evitar la duplicación de datos, se reducen los costos de almacenamiento y se optimiza el uso de recursos.
- ▶ **Flexibilidad operacional.** Los equipos pueden trabajar de manera independiente en sus propios clústeres mientras comparten datos según sea necesario, lo que mejora la flexibilidad y la eficiencia operativa.

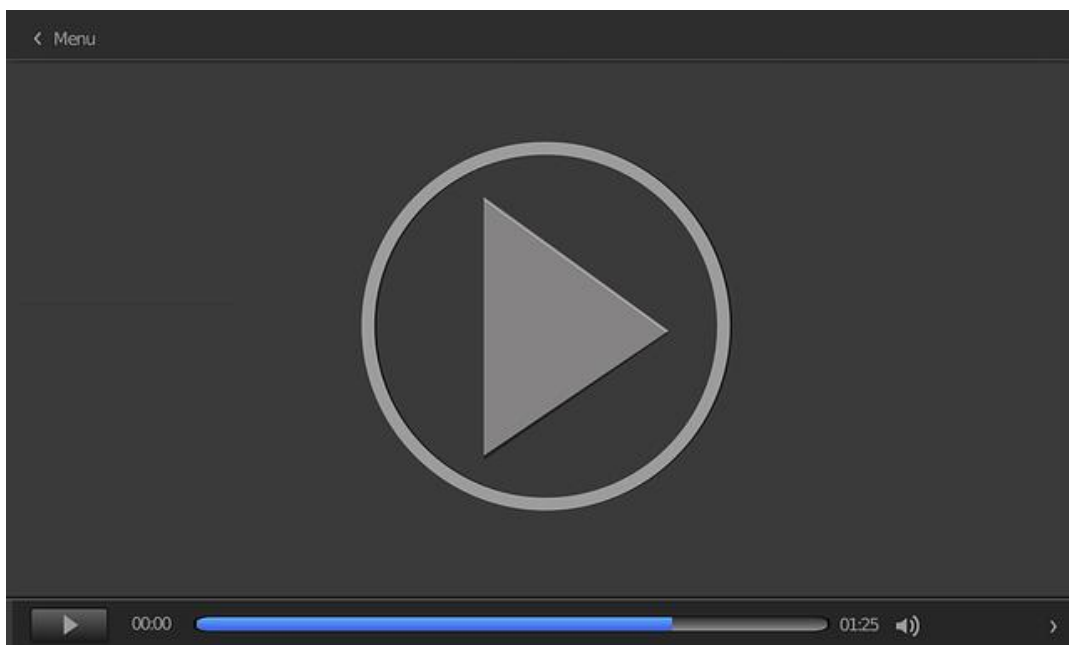
Los principales **casos de uso** de Redshift Data Sharing son:

- ▶ **Análisis conjunto entre equipos.** Equipos de *marketing*, ventas y finanzas pueden compartir datos entre sí para obtener una visión holística del rendimiento de la empresa y tomar decisiones informadas.
- ▶ **Colaboración entre departamentos.** Diferentes departamentos dentro de una empresa pueden acceder a los mismos datos para realizar análisis específicos sin necesidad de copiar los datos a cada clúster.
- ▶ **Integración con aplicaciones externas.** Empresas que trabajan con socios o clientes externos pueden compartir datos de manera segura para permitir el acceso a información relevante sin comprometer la seguridad.
- ▶ **Multirregional data analysis.** Organizaciones globales pueden compartir datos entre clústeres ubicados en diferentes regiones geográficas para realizar análisis distribuidos y coordinados.

Tema 3. Data science cloud storage

En resumen, **Redshift Data Sharing** es una poderosa funcionalidad que mejora la colaboración y eficiencia en el análisis de datos al permitir compartir datos de manera segura y en tiempo real entre distintos clústeres de Amazon Redshift. Esta característica ofrece ventajas significativas en términos de costos, rendimiento y flexibilidad operativa, haciendo que las organizaciones sean más ágiles y colaborativas.

En el vídeo *Amazon Redshift Data Sharing Use Cases* (Amazon Web Services, 2020c), se muestran casos de uso de aplicación de Redshift Data Sharing.



Amazon Redshift Data Sharing Use Cases.

Accede al vídeo:

<https://www.youtube.com/embed/sloTB8B5nn4>

Tema 3. Data science cloud storage

Redshift Machine Learning

Amazon Redshift ML es una funcionalidad de Amazon Redshift que permite a los usuarios crear, entrenar y desplegar modelos de ML directamente dentro de su *data warehouse*. Esta integración simplifica el proceso de incorporar capacidades de ML en las consultas de datos, ya que elimina la necesidad de mover datos a servicios externos para el entrenamiento y la inferencia de modelos.

El **concepto** detrás de Redshift ML es llevar el poder del ML directamente al *data warehouse* mientras aprovecha el entorno de SQL familiar para los analistas de datos y científicos de datos. Utiliza Amazon SageMaker, el servicio de ML de AWS, para el entrenamiento de modelos, donde los resultados se integran de vuelta en Redshift, lo que permite que las predicciones se realicen directamente dentro de las consultas SQL.

Entre las principales **funcionalidades** de Redshift ML, se puede destacar lo siguiente:

- ▶ **Creación de modelos desde SQL.** Permite a los usuarios crear modelos de ML utilizando comandos SQL, lo que facilita la creación de modelos sin necesidad de conocimientos profundos en ML.
- ▶ **Entrenamiento automático.** Redshift ML utiliza Amazon SageMaker para entrenar los modelos automáticamente. Los usuarios pueden especificar los datos y las características a utilizar, y SageMaker se encarga del resto.
- ▶ **Inferencia directa en SQL.** Una vez que los modelos están entrenados, las predicciones se pueden realizar directamente en las consultas SQL, lo que facilita la integración de *insights* de ML en los análisis de datos.
- ▶ **Gestión de modelos.** Ofrece herramientas para gestionar el ciclo de vida de los modelos de ML, incluyendo actualizaciones y reentrenamiento directamente desde Redshift.

Tema 3. Data science cloud storage

- **Soporte para diversos algoritmos.** Soporta una variedad de algoritmos de ML comunes, incluyendo regresión lineal, regresión logística, árboles de decisión, y redes neuronales, entre otros.

En la Figura 23, vemos un **flujo de funcionalidades** de Redshift ML.

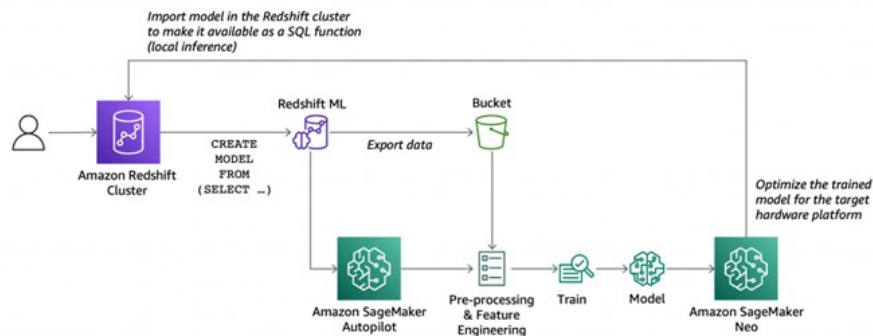


Figura 23. Flujo de funcionalidades de Redshift ML. Fuente: Poccia, 2021.

Las **ventajas** más destacables de Redshift ML son las siguientes:

- **Facilidad de uso.** Los analistas de datos pueden crear y utilizar modelos de ML sin salir de su entorno de SQL habitual, lo que reduce la barrera de entrada para la adopción de ML.
- **Integración directa.** La integración directa con Redshift elimina la necesidad de mover datos a sistemas externos para el entrenamiento y la inferencia, lo que reduce la latencia y los costos asociados a la transferencia de datos.
- **Escalabilidad.** Al utilizar Amazon SageMaker, Redshift ML se beneficia de la escalabilidad de SageMaker para manejar grandes volúmenes de datos y entrenar modelos complejos.
- **Automatización.** La automatización del proceso de entrenamiento y despliegue de modelos facilita la implementación de soluciones de ML sin la necesidad de gestionar la infraestructura subyacente.

Tema 3. Data science cloud storage

- ▶ **Coste-efectividad.** Aprovecha el modelo de precios basado en uso de SageMaker, permitiendo a las organizaciones pagar solo por los recursos de entrenamiento e inferencia que utilizan.

Los siguientes son algunos ejemplos de **casos de uso** de Redshift ML:

- ▶ **Predicción de *churn* de clientes.** Las empresas pueden utilizar Redshift ML para predecir la tasa de abandono de clientes, identificando patrones en los datos históricos que indican un mayor riesgo de *churn*.
- ▶ **Detección de fraude.** Redshift ML puede entrenar modelos para detectar transacciones fraudulentas en tiempo real al analizar los patrones y las anomalías en los datos de transacciones.
- ▶ **Segmentación de clientes.** Permite crear modelos que segmenten a los clientes en diferentes grupos basados en sus comportamientos y características, facilitando campañas de *marketing* más efectivas.
- ▶ **Pronóstico de demanda.** Utiliza datos históricos de ventas y otros factores para predecir la demanda futura de productos, lo que ayuda a optimizar la gestión de inventarios.
- ▶ **Análisis de sentimiento.** Permite analizar comentarios de clientes y reseñas de productos para identificar el sentimiento general y las opiniones sobre productos o servicios.

Se describe, a continuación, un **ejemplo de uso de Redshift ML**, con los pasos a seguir:

- ▶ **Crear un modelo de ML.** Escribir una consulta SQL para crear un modelo de predicción de *churn*.

```
sql
CREATE MODEL customer_churn_model
FROM (
```

Tema 3. Data science cloud storage

```
SELECT
    customer_id,
    churned,
    age,
    tenure,
    spend
FROM customer_data
)
TARGET churned
FUNCTION predict_churn;
```

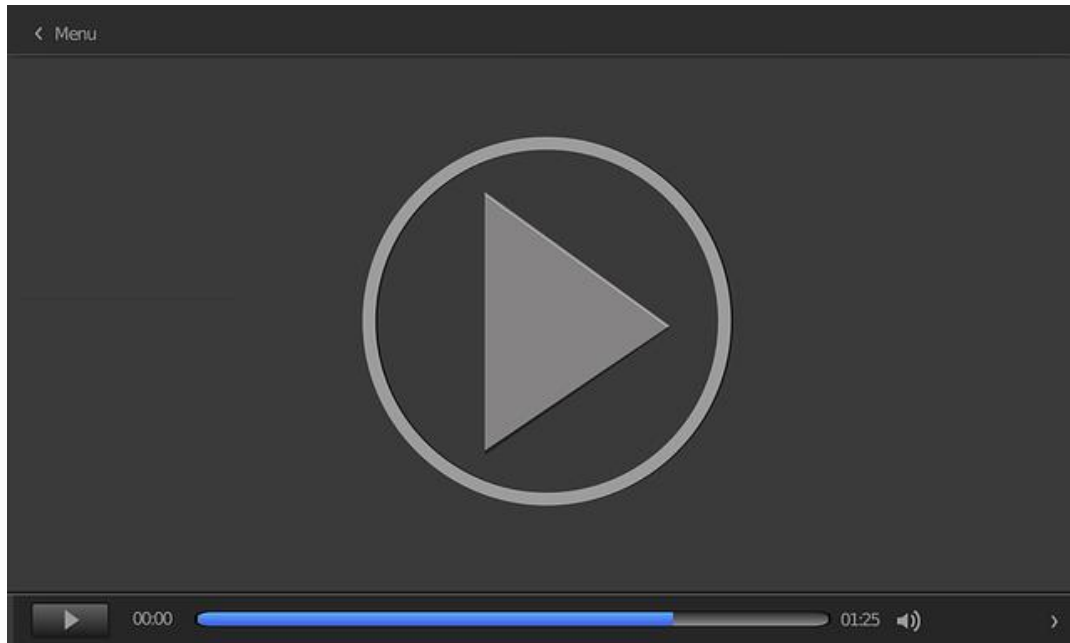
- ▶ **Entrenamiento del modelo.** Redshift ML utiliza Amazon SageMaker para entrenar el modelo automáticamente.
- ▶ **Usar el modelo para inferencia.** Una vez entrenado, realiza predicciones directamente en SQL.

```
sql
SELECT customer_id, predict_churn(age, tenure, spend)
FROM new_customer_data;
```

Amazon Redshift ML facilita la integración de ML en el análisis de datos, por lo que permite que las organizaciones aprovechen las capacidades de ML directamente en su *data warehouse*. Con su enfoque en la facilidad de uso, escalabilidad y costo-efectividad, Redshift ML empodera a los analistas de datos para implementar soluciones de ML sin necesidad de habilidades avanzadas en ML, lo que acelera el tiempo de obtención de *insights* valiosos.

Tema 3. Data science cloud storage

A continuación, se muestra el vídeo *AWS re:Invent 2020: Introducing Amazon Redshift Machine Learning* (AWS Events, 2021), que profundiza en las capacidades del servicio de AWS Redshift ML.



AWS re:Invent 2020: Introducing Amazon Redshift Machine Learning.

Accede al vídeo:

<https://www.youtube.com/embed/3BN1w8JUd4>

Integración con Spark

La **integración de AWS Redshift ML con Apache Spark** permite aprovechar las capacidades avanzadas de procesamiento y análisis de datos de Spark junto con las funcionalidades de ML de Redshift ML. Esta combinación es potente para escenarios donde se requieren análisis complejos y volúmenes masivos de datos que Spark puede manejar de manera eficiente antes de enviar datos a Redshift para el entrenamiento y las inferencias de ML.

Tema 3. Data science cloud storage

Esta integración simplifica y acelera las **aplicaciones** Apache Spark a través del acceso a los datos de Amazon Redshift de los servicios de análisis de AWS, como Amazon EMR, AWS Glue y Amazon SageMaker. Con Amazon EMR, AWS Glue y SageMaker, se pueden crear con rapidez aplicaciones de Apache Spark, que leen desde su almacenamiento de datos de Amazon Redshift y que escriben en este, sin comprometer el rendimiento ni la coherencia transaccional.

La integración entre ambos también utiliza **credenciales** basadas en AWS IAM para mejorar la seguridad. Con la integración de Amazon Redshift para Apache Spark, no hay configuración ni mantenimiento manual de versiones no certificadas de conectores de terceros. Puedes comenzar con los trabajos de Apache Spark utilizando los datos en Amazon Redshift en segundos. Esta nueva integración mejora el rendimiento de las aplicaciones Apache Spark a través del uso de los datos de Amazon Redshift.

El **objetivo** de la integración es utilizar Spark para procesar y transformar grandes volúmenes de datos y, luego, utilizar Redshift ML para entrenar modelos de ML y realizar inferencias directamente en el *data warehouse* de Redshift. Esto permite a las organizaciones utilizar una arquitectura de análisis de datos flexible y escalable.

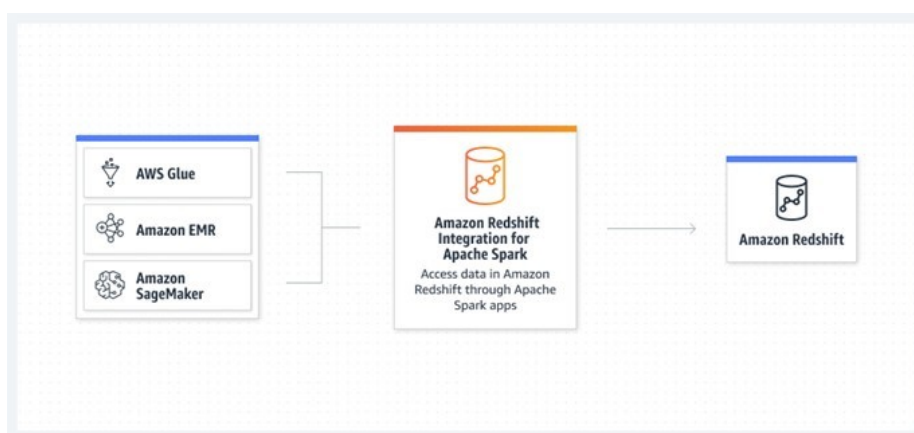


Figura 24. Diagrama de la integración. Fuente: Amazon Web Services, s. f.-f.

Tema 3. Data science cloud storage

Entre las principales **funcionalidades** de esta integración, se enumeran las siguientes:

- ▶ **Extracción y transformación de datos con Spark.** Utiliza Spark para procesar y transformar datos a gran escala. Spark es muy eficiente en el procesamiento distribuido de grandes volúmenes de datos.
- ▶ **Carga de datos a Redshift.** Después de transformar los datos, se pueden cargar a Amazon Redshift para su almacenamiento y análisis adicional. Esto se hace generalmente usando el conector Spark-Redshift.
- ▶ **Entrenamiento de modelos con Redshift ML.** Utiliza los datos cargados en Redshift para crear y entrenar modelos de ML utilizando Redshift ML.
- ▶ **Inferencia directa desde Redshift.** Realiza predicciones y análisis de ML directamente en Redshift utilizando SQL.

Los **pasos** para esta integración serían los siguientes:

- ▶ **Configuración del entorno Spark.** Configura un clúster de Spark, ya sea en AWS EMR (Elastic MapReduce) o en otra infraestructura gestionada.
- ▶ **Procesa de datos con Spark.** Usa Spark para leer, limpiar, transformar y enriquecer los datos desde diversas fuentes. Por ejemplo, leer datos desde Amazon S3:

```
python
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("RedshiftMLIntegration").getOrCreate()
df = spark.read.csv("s3://my-bucket/my-data.csv", header=True,
inferSchema=True)
```


Tema 3. Data science cloud storage

- **Carga de datos transformados a Redshift.** Utiliza el conector Spark-Redshift para cargar los datos procesados a Amazon Redshift:

```
python
df.write \
    .format("com.databricks.spark.redshift") \
    .option("url", "jdbc:redshift://redshift-cluster:5439/mydb") \
    .option("dbtable", "transformed_data") \
    .option("user", "myuser") \
    .option("password", "mypassword") \
    .option("aws_iam_role", "arn:aws:iam::account-id:role/RedshiftCopyUnload") \
    .save()
```

Entre las **ventajas** de esta integración, podemos destacar lo siguiente:

- **Eficiencia en el procesamiento de datos.** Spark puede manejar grandes volúmenes de datos y realizar transformaciones complejas de manera eficiente antes de cargar los datos en Redshift.
- **Escalabilidad.** Tanto Spark como Redshift son altamente escalables, lo que permite manejar grandes cargas de trabajo y volúmenes de datos.
- **Simplificación del flujo de trabajo de ML.** La integración permite realizar todo el flujo de trabajo de ML, desde la preparación de datos hasta la inferencia, utilizando herramientas robustas y escalables.
- **Flexibilidad.** Utilizar Spark para la preparación de datos y Redshift para ML permite aprovechar lo mejor de ambas plataformas, lo que ofrece una solución flexible para diferentes necesidades analíticas.

Los siguientes son algunos ejemplos de **casos de uso** de esta integración:

- **Análisis de logs y predicción de anomalías.** Procesar grandes volúmenes de *logs* con Spark, cargar los datos transformados a Redshift y usar Redshift ML para detectar anomalías en tiempo real.

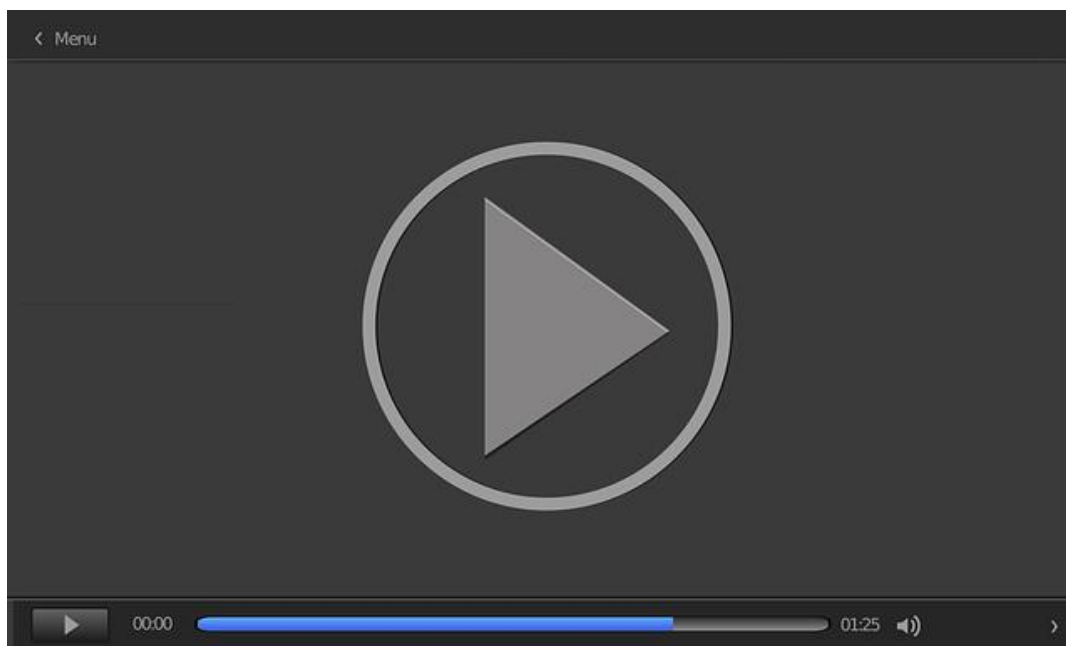
Tema 3. Data science cloud storage

- ▶ **Predicción de demanda.** Utilizar Spark para combinar y procesar datos de múltiples fuentes, cargar los datos a Redshift y entrenar modelos de predicción de demanda con Redshift ML.
- ▶ **Análisis de sentimiento en redes sociales.** Procesar datos de redes sociales con Spark para extraer características relevantes, cargar los datos en Redshift y usar modelos de ML para análisis de sentimiento.

La **integración de AWS Redshift ML con Apache Spark** proporciona una solución poderosa y flexible para manejar y analizar grandes volúmenes de datos. Al combinar la capacidad de procesamiento masivo de Spark con las capacidades de ML de Redshift ML, las organizaciones pueden implementar flujos de trabajo de análisis y ML de manera eficiente y escalable. Esta integración es ideal para escenarios donde la preparación de datos complejos y el análisis predictivo son críticos para la toma de decisiones empresariales.

Tema 3. Data science cloud storage

Seguidamente, se muestra el vídeo *Amazon Redshift integration for Apache Spark - Demo | Amazon Web Services* (Amazon Web Services, 2022b), que es un demo de la integración de AWS Redshift y Spark.



Amazon Redshift integration for Apache Spark - Demo | Amazon Web Services.

Accede al vídeo:

<https://www.youtube.com/embed/3Me8SAruNB0>

Tema 3. Data science cloud storage

3.5. Referencias bibliográficas

Amazon Web Services. (2020a, mayo 18). *Getting Started with Amazon Redshift - AWS Online Tech Talks* [Vídeo]. YouTube. <https://youtu.be/dfo4J5ZhIKI>

Amazon Web Services. (2020b, diciembre 9). *Amazon Redshift Data Sharing Workflow* [Vídeo]. YouTube. <https://youtu.be/EXioFirlrA>

Amazon Web Services. (2020c, diciembre 9). *Amazon Redshift Data Sharing Use Cases* [Vídeo]. YouTube. <https://youtu.be/sIoTB8B5nn4>

Amazon Web Services. (2021a, noviembre 30). *Introduction to Data Warehousing on AWS with Amazon Redshift | Amazon Web Services* [Vídeo]. YouTube. https://youtu.be/IWwFJV_9PoE

Amazon Web Services. (2021b, septiembre 29). *An Overview of Amazon Redshift Query Editor V2 | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/lwZNIroJUnc>

Amazon Web Services. (2022a, enero 10). *Amazon Redshift Serverless - End to End Use Case | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/gEAWSqjthXs>

Amazon Web Services. (2022b, diciembre 19). *Amazon Redshift integration for Apache Spark - Demo | Amazon Web Services* [Vídeo]. YouTube. <https://youtu.be/3Me8SAruNB0>

Amazon Web Services. (s. f.-a). *Amazon S3*. <https://aws.amazon.com/es/s3/>

Amazon Web Services. (s. f.-b). *Operaciones por lotes de Amazon S3*. <https://aws.amazon.com/es/s3/features/batch-operations/>

Amazon Web Services. (s. f.-c). *Replicación de Amazon S3*. <https://aws.amazon.com/es/s3/features/replication/>

Tema 3. Data science cloud storage

Amazon Web Services. (s. f.-d). *Cumplimiento de los requisitos de conformidad mediante el control de tiempo de replicación de S3 (S3 RTC)*.

https://docs.aws.amazon.com/es_es/AmazonS3/latest/userguide/replication-time-control.html

Amazon Web Services. (s. f.-e). *Amazon Redshift*.

<https://aws.amazon.com/es/redshift/>

Amazon Web Services. (s. f.-f). *Integración de Amazon Redshift para Apache Spark*.

<https://aws.amazon.com/es/redshift/features/integration-for-apache-spark/>

Amazon Web Services. (s. f.-g). *Amazon S3*. <https://aws.amazon.com/es/s3/>

Amazon Web Services. (s. f.-h). *Puntos de acceso de Amazon S3*.

<https://aws.amazon.com/es/s3/features/access-points/>

Amazon Web Services. (s. f.-i). *Introducción a Amazon S3*.

<https://aws.amazon.com/es/s3/getting-started/>

Amazon Web Services. (s. f.-j). *Clases de almacenamiento de Amazon S3*.

<https://aws.amazon.com/es/s3/storage-classes/>

Amazon Web Services. (s. f.-k). *Tutoriales en video de Amazon S3*.

<https://aws.amazon.com/es/s3/videos/?nc=sn&loc=6&dn=2>

Amazon Web Services. (s. f.-l). *¿Qué es Amazon S3?*

https://docs.aws.amazon.com/es_es/AmazonS3/latest/userguide/

Amazon Web Services. (s. f.-m). *Introducción a Amazon S3*.

https://docs.aws.amazon.com/es_es/AmazonS3/latest/userguide/GetStartedWithS3.html

AWS Events. (2021, febrero 5). *AWS re:Invent 2020: Introducing Amazon Redshift Machine Learning* [Vídeo]. YouTube. <https://youtu.be/3BN1w8JUtd4>

Tema 3. Data science cloud storage

Chacaltana, G. (2017, mayo 22). *El poder de las bases de datos NoSQL*.

<https://solocodigoweb.com/blog/2017/05/22/el-poder-de-las-bases-de-datos-nosql/>

Chayel, M. (2014, noviembre 26). *ETL Processing Using AWS Data Pipeline and Amazon Elastic MapReduce*. Amazon Web Services.

<https://aws.amazon.com/es/blogs/big-data/etl-processing-using-aws-data-pipeline-and-amazon-elastic-mapreduce/>

Cogent. (2024, septiembre 1). *Data Warehousing in the Cloud: Best Practices*.

<https://www.cogentinfo.com/resources/data-warehousing-in-the-cloud-best-practices>

Datos.gob.es. (s. f.). *Normas técnicas para un correcto gobierno del dato*.

<https://datos.gob.es/es/documentacion/normas-tecnicas-para-un-correcto-gobierno-del-dato>

Dehghani, Z. (2020). *Data Mesh Principles and Logical Architecture*. Martin Fowler.

<https://martinfowler.com/articles/data-mesh-principles.html>

Dehghani, Z. (2022). *Data Mesh: Delivering Data-Driven Value*. O'Reilly Media.

Deloitte. (s. f.). *Big data architect*.

Diarioti. (2015, octubre 3). *¿Qué es el almacenamiento distribuido y por qué Google está silenciosa?*

<https://diarioti.com/brand/storpool/que-es-el-almacenamiento-distribuido-y-por-que-google-esta-silenciosa>

DiegoCoronel. (2021, abril 28). *Big Data: Arquitectura e importancia* [Entrada en un foro]. Huawei.

<https://forum.huawei.com/enterprise/es/Big-Data-Arquitectura-e-importancia/thread/667224227310354432-667212895836057600>

Elternativa. (s. f.). *Data Mesh: qué es, ventajas y cómo implementar esta arquitectura de datos*.

<https://www.elternativa.com/blog-elternativa/data-mesh/>

Tema 3. Data science cloud storage

Fernández, Y. (2021, enero 8). *Big Data: qué es y para que sirve*. Xataka.

<https://www.xataka.com/basics/big-data-que-sirve>

Francis, R., Gupta, R. y Oke, M. (2023). *Amazon Redshift: The Definitive Guide: Jump-Start Analytics Using Cloud Data Warehousing*. O'Reilly Media.

Gil, E. (2016). *Big data, privacidad y protección de datos*. Agencia Española de Protección de Datos. <https://www.aepd.es/es/documento/big-data.pdf>

Grande, C. y Labella, E. (2022, abril 6). *¿Qué es un Data Mesh y cómo saber si tu empresa lo necesita?* KPMG Tendencias.

<https://www.tendencias.kpmg.es/2022/04/data-mesh-descentralizacion-organizativa-informacion/>

Graph Everywhere. (s. f.). *Bases de datos NoSQL vs SQL | Bases de datos no relacionales vs relacionales*. <https://www.grapheverywhere.com/nosql-vs-sql/>

Keepler. (2022, enero 26). *Entendiendo el concepto Data Mesh, la nueva tendencia para resolver (casi) todos los problemas de datos*.

<https://keepler.io/es/2022/01/entendiendo-concepto-data-mesh/>

MDCloud. (s. f.). *Dónde almacenar los datos de una empresa*.

IBM. (s. f.). *¿Y si una arquitectura data fabric pudiera guiar la toma de decisiones?*

<https://www.ibm.com/es-es/data-fabric>

Ilimit. (2020/ abril 28). *Bases de Datos SQL vs NOSQL: Diferencias, Ventajas y Mitos*. <https://www.ilimit.com/blog/base-de-datos-sql-nosql/>

Immon, W. H. (2005). *Building the Data warehouse Fourth Edition*. Willey.

Indra. (s. f.). *¿Cuántas V debería tener el Big Data?*

<https://www.indracompany.com/es/blogneo/cuantas-v-deberia-big-data>

Tema 3. Data science cloud storage

Jineshkumar, P. (2021, agosto 18). *All about AWS S3 Storage*. Jineshkumar Patel.

<https://blog.jineshkumar.com/all-about-aws-s3-storage>

KeepCoing. (s. f.). *¿Qué es la arquitectura Big Data y cómo funciona?*

<https://keepcoding.io/blog/que-es-arquitectura-big-data/>

Kimball, R. (2013). *The Data warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley.

Kumar, P. (2023, marzo 26). *What is data mesh, when should it be used, and what are its challenges?* LinkedIn. <https://www.linkedin.com/pulse/what-data-mesh-when-should-used-its-challenges-purushottam-kumar>

Laplante, A. (2021). *Data fabric as Modern Data Architecture*. O'Reilly Media.

LisM24. (2021, julio 23). *Big Data – Aldo Pérez_Lisbeth Magallón*. ISSUU.

https://issuu.com/lism24/docs/revista_big_data_-_aldop_rez_lisbethmagall_n

López Fandino, V. M. (2023). *Sistemas de big data*. RA-MA.

Lu, H., Foh, C. H., Wen, Y. y Cai, J. (2012). Optimizing content retrieval delay for LT-based distributed cloud storage systems. *2012 IEEE Global Communications Conference (GLOBECOM), Anaheim, CA, 1920-1925*. DOI 10.1109/GLOCOM.2012.6503396.

Mamani, J. (2023, mayo 30). *Hadoop I: Get in touch with Docker*. Medium.

<https://medium.com/@jose.rolando.m/hadoop-get-in-touch-with-docker-c55da692f06>

Marco de Desarrollo de la Junta de Andalucía. (s. f.). *Conceptos sobre la escalabilidad*.

<https://www.juntadeandalucia.es/servicios/madeja/contenido/recurso/220>

Modus. (s. f.). *Data Fabric*. <https://www.modus.es/data-fabric/>

Tema 3. Data science cloud storage

Moncho Terol. (2022). *Data Mesh: ¿qué es y cómo beneficia tu estrategia de gestión de datos?* ThinkBig. <https://blogthinkbig.com/data-mesh-que-es-y-como-beneficia-a-tu-estrategia-de-gestion-de-datos>

NetApp. (s. f.). *Almacenamiento de datos unificado*. <https://www.netapp.com/es/data-fabric/what-is-data-fabric/>

Nextu. (s. f.). *Características del big data*.

Nexus integra. (s. f.). *Big Data vs Inteligencia Artificial*. <https://nexusintegra.io/es/big-data-vs-inteligencia-artificial/>

Página de Gartner (<https://www.gartner.com/en>).

Paloma Rojo. (s. f.). *¿Por qué las empresas masivamente interactivas están impulsando la adopción de NoSQL?* DataIQ. <https://dataiq.com.ar/blog/por-que-las-empresas-masivamente-interactivas-estan-impulsando-la-adopcion-de-nosql/>

Plataforma Agua. (s. f.). *Análisis big data*.

Poccia, D. (2021, mayo 27). *Amazon Redshift ML Is Now Generally Available – Use SQL to Create Machine Learning Models and Make Predictions from Your Data*. Amazon Web Services. <https://aws.amazon.com/es/blogs/aws/amazon-redshift-ml-is-now-generally-available-use-sql-to-create-machine-learning-models-and-make-predictions-from-your-data/>

PowerData. (s. f.). *Big Data: ¿En qué consiste? Su importancia, desafíos y gobernabilidad*. <https://www.powerdata.es/big-data>

Pure Storage. (s. f.). *¿Qué es el almacenamiento de datos por niveles?* <https://www.purestorage.com/es/knowledge/what-is-tiered-data-storage.html>

Tema 3. Data science cloud storage

Reinsel, D., Gantz, J. y Ryding, J. (2018). *The Digitization of the World. From Edge to Core*. Seagate. <https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf>

Rigoberto M. (2022, julio 1). *¿Qué es Data Mesh y por qué revoluciona al mundo de los datos?* LinkedIn. <https://es.linkedin.com/pulse/qu%C3%A9-es-data-mesh-y-por-revoluciona-al-mundo-de-los-malca-la-rosa>

Rios, S. (2020, abril 26). *Los Datos como Servicio y la Modernización de Aplicaciones*. LinkedIn. <https://www.linkedin.com/pulse/los-datos-como-servicio-y-la-modernizaci%C3%B3n-de-salvador-rios-sr-/?originalSubdomain=es>

SAP. (s. f.). *¿Qué es un data fabric?* <https://www.sap.com/spain/insights/what-is-data-fabric.html>

SAPTOOLS. (s. f.). *¡Toma la temperatura a tus datos!* <https://www.saptools.es/toma-la-temperatura-a-tus-datos/>

Stackscale. (2023, junio 22). *3 Tipos de almacenamiento: de archivos, en bloques y de objetos*. <https://www.stackscale.com/es/blog/tipos-de-almacenamiento/>

Tehreem Naeem. (2024, marzo 21). *Repositorio de datos: definición, tipos y beneficios con mejores prácticas*. <https://www.astera.com/es/type/blog/data-repository/>

Telefónica Tech. (2021, mayo 9). *La evolución de las bases de datos con propósito IA en entornos Cloud*. <https://telefonicatech.com/blog/la-evolucion-de-las-bases-de-datos-con-proposito-ia-en-entornos-cloud>

T-Systems. (2018, agosto 20). *Big Data: ventajas del almacenamiento de datos por niveles*. <https://www.t-systemsblog.es/big-data-almacenamiento-datos-niveles/>

Tema 3. Data science cloud storage

T-Systems. (2020, septiembre 9). *¿Qué es Data Fabric y para qué sirve?*

<https://www.t-systemsblog.es/que-es-data-fabric-y-para-que-sirve/>

Vidal Vidal, L. H. (2019, octubre 7). *Conceptos iniciales sobre la Arquitectura del Big Data y el Gobierno de Datos*. LinkedIn.

<https://www.linkedin.com/pulse/conceptos-iniciales-sobre-la-arquitectura-del-big-data-vidal-vidal/?originalSubdomain=es>

Worlikar, S., Arumugam, T. y Patel, H. (2021). *Amazon Redshift Cookbook: Recipes for building modern data warehousing solution*. Packt Publishing.

Ye, G. (2023, marzo 15). *Data Mesh, la descentralización del dato*. Hiberus blog.

<https://www.hiberus.com/crecemos-contigo/data-mesh-la-descentralizacion-del-dato/>

Tema 4. Computación distribuida.

Computación paralela

4.1. Introducción y objetivos

En este tema aprenderás sobre el paradigma del procesamiento de la información que ha sido almacenada en dichos sistemas y cómo supone un cambio en cuanto a la manera de trabajar con respecto a lo que se venía realizando hasta ahora.

Entenderás en qué consiste el procesamiento de Hadoop con MapReduce y verás qué herramientas dentro del ecosistema de Hadoop ayudan a realizar dicho procesamiento de una manera más sencilla y cómoda siempre usando como base MapReduce. Dentro de estas aplicaciones conocerás algunas, como Hive o Impala, que permiten realizar consultas analíticas de la información almacenada en HDFS a través del procesamiento con MapReduce.

También profundizaremos en el principal de *software* de procesamiento de datos de hoy en día con Apache Spark.

Los **objetivos** que persigue este tema son los siguientes:

- ▶ Entender por qué Spark es superior a MapReduce atendiendo a su diseño.
- ▶ Identificar los módulos que componen Spark y el propósito de cada uno.
- ▶ Conocer la arquitectura y el funcionamiento interno de Spark.
- ▶ Practicar con algunas funciones típicas de procesamiento de datos con Spark.
- ▶ Conocer la API estructurada de Spark y su principal estructura de datos, el *dataframe*.
- ▶ Identificar las ventajas de usar *dataframes* en lugar de RDD (*resilient distributed datasets*).
- ▶ Conocer Spark SQL, así como sus similitudes y diferencias con la API estructurada.

Tema 4. Computación distribuida.

Computación paralela

- ▶ Practicar con algunas funciones típicas de procesamiento de *dataframes*, tanto con la API estructurada como con Spark SQL.
- ▶ Introducir al estudiante en los módulos de alto nivel de Spark.
- ▶ Presentar las capacidades de Spark MLlib mediante ejemplos de uso.
- ▶ Mostrar las ideas básicas y un ejemplo sencillo de Spark Structured Streaming.

Tema 4. Computación distribuida.

Computación paralela

4.2. Spark I

En el año 2009, en Berkeley, EE. UU., surgió una nueva propuesta para paliar estas deficiencias, que, poco a poco, ha ido imponiéndose hasta reemplazar completamente a Hadoop (más concretamente, a MapReduce).

Apache Spark es un motor unificado de cálculo en memoria y un conjunto de bibliotecas para procesamiento paralelo y distribuido de datos en clústeres de ordenadores.

Analicemos la **definición**: es un *motor* de cálculo paralelo y distribuido, esto es, un *framework* de propósito general orientado a resolver cualquier tipo de problema y con el que se puede implementar cualquier algoritmo.

- ▶ **Unificado**: el motor es único e independiente de cómo utilicemos Spark:
 - Desde la API de *dataframes* (para lenguaje R, Python, Java y Scala).
 - Desde herramientas externas que lanzan consultas SQL contra Spark (Hive, herramientas de *business intelligence* conectadas a Spark como Tableau, PowerBI, Microstrategy, QlikSense, etc.).
 - Desde una instrucción de la API de programación que recibe una consulta SQL, como `string`, tan compleja como sea necesario.
 - Cualquiera de las opciones anteriores se traduce a un grafo de tareas al que Spark aplica optimizaciones de código automáticamente; por tanto, todos (API, aplicaciones BI, SQL, etc.) se benefician de ellas.
- ▶ **En memoria**: todos los cálculos se llevan a cabo en memoria y solo se escriben resultados a disco (parciales o finales) cuando el usuario lo indica explícitamente (operación de guardado) o cuando la operación indicada por el usuario requiere

Tema 4. Computación distribuida.

Computación paralela

forzosamente realizar movimiento de datos entre nodos (shuffle , el cual se lleva a cabo desde el disco duro del emisor al disco duro del nodo receptor). Además, el movimiento de datos (y las operaciones de acceso a disco que conlleva) solo se produce cuando es irremediable, pero no de manera obligatoria, como ocurría en MapReduce. Esto consigue un rendimiento muy superior en tareas iterativas (varias pasadas sobre los mismos datos, por ejemplo, algoritmos de ML).

Componentes de Spark

Apache Spark consiste en una **API** (bibliotecas de programación) distribuida, mucho más intuitiva que MapReduce. Al igual que este, abstrae todos los detalles de comunicación de red entre las máquinas del clúster, pero, además, opera de manera similar a las consultas SQL tradicionales, como si los datos fuesen tablas (distribuidas). Esto la hace fácil de usar y de aprender.

Spark ofrece distintas API para cuatro **lenguajes**: Java, Scala (la más utilizada en la actualidad por los ingenieros de datos para aplicaciones en producción), Python (paquete de Python PySpark, muy usado por científicos de datos, con una API casi idéntica a la de Scala que, en realidad, no es más que una capa adicional que termina llamando al código de Scala) y R (paquete de R SparkR, con una API muy diferente al resto, más cercana a los comandos típicos de R).

En este tema, usaremos PySpark para todos los ejemplos.

La Figura 1 muestra los principales componentes de Spark.

Tema 4. Computación distribuida. Computación paralela

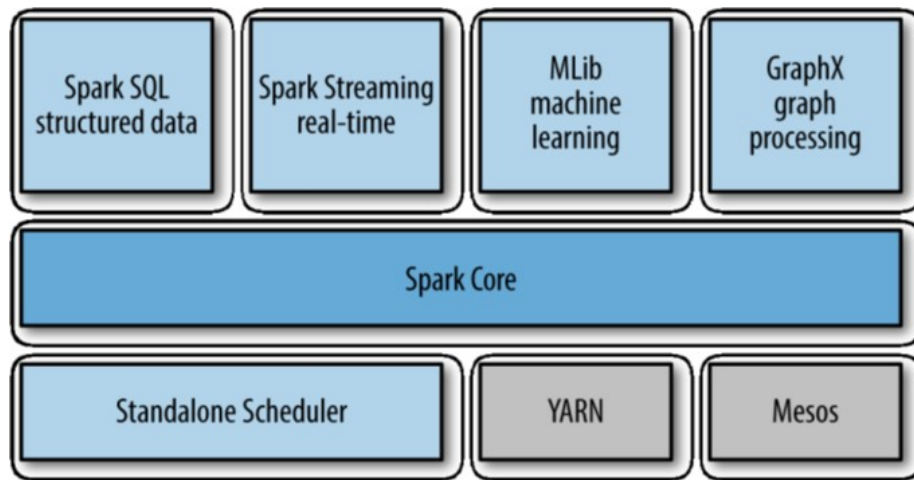


Figura 1. Módulos que componen Apache Spark. Fuente: Chambers y Zaharia 2018.

De estos, el principal es el módulo **Spark Core**. En él se encuentran las estructuras de datos fundamentales de Spark, tales como los RDD, de los que hablaremos más adelante, y las operaciones que se puede llevar a cabo con él. En última instancia, todas las operaciones que se pueden hacer con Spark, independientemente del módulo utilizado o la API donde se encuentren, se traducen automáticamente por parte de Spark a operaciones sobre RDD, que son las únicas que sabe ejecutar el motor.

Los tres módulos inferiores representan tres **gestores de recursos** de un clúster sobre los que puede ejecutarse Spark. Un gestor de recursos se encarga de asignar máquinas, CPU y memoria principal a Spark, de forma que disponga de un determinado número de nodos y de que, en cada uno, existan suficientes recursos para ejecutar la aplicación que hemos implementado con Spark.

Tema 4. Computación distribuida.

Computación paralela

El resto de los módulos cumplen las siguientes **funciones**:

- ▶ **Spark SQL y API estructurada** es una API con una serie de funciones para manejar tablas de datos distribuidas, estructuradas en columnas con nombre y tipo, denominadas *dataframes*. Un *dataframe* proporciona un nivel de abstracción adicional sobre un RDD, al cual recubre. Este módulo incluye también una función para ejecutar sentencias SQL sobre las mismas estructuras de datos distribuidas.
- ▶ **Spark Streaming** es el módulo para operar de manera distribuida sobre datos en tiempo real, según los vamos recibiendo (*stream* de datos). A partir de Spark 2.0, este módulo ha sido reemplazado por **Spark Structured Streaming**, que simplifica el procesamiento de flujos de datos en tiempo real.
- ▶ **SparkML** contiene implementaciones distribuidas de algoritmos de ML.
- ▶ **Spark GraphX** es el módulo de procesamiento de grafos, que se representan mediante RDD. Contiene algunos algoritmos de camino mínimo y similares. Actualmente, este módulo ha quedado obsoleto y ha sido reemplazado *de facto* por el paquete GraphFrames, que representa un grafo como una pareja de *dataframes*, con los nodos y los arcos respectivamente. Empezó como un proyecto separado de Spark y, finalmente, ha sido integrado en las versiones más recientes.

Arquitectura de Spark

La Figura 2 muestra la arquitectura de una aplicación que utiliza Spark al ejecutarse en un clúster. No debemos olvidar que, al escribir una aplicación en Spark, en realidad, estamos escribiendo una aplicación **secuencial** (no paralela) utilizando la biblioteca de Spark para el lenguaje en el que estemos programando (Java, Scala, Python o R). Cuando ejecutamos el código de este programa (proceso que se denomina *driver*), lo hacemos sobre una máquina concreta, que puede ser interna o externa al clúster. Por ejemplo, podríamos escribir un programa en Spark y ejecutarlo en nuestro ordenador portátil, y, desde aquí, el programa driver podría conectarse a un clúster de Spark remoto.

Tema 4. Computación distribuida. Computación paralela

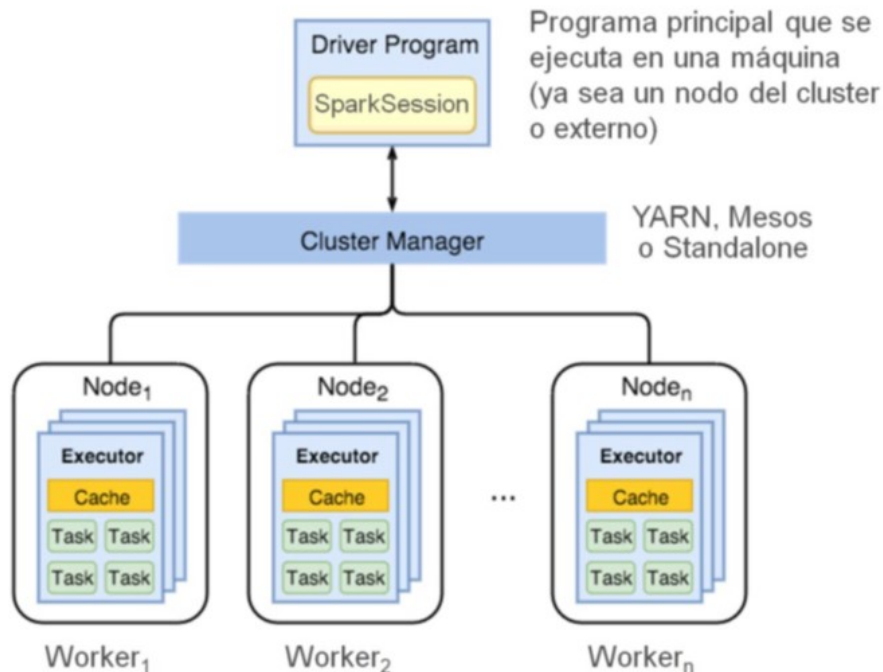


Figura 2. Arquitectura de una aplicación en Spark al ejecutarse en un clúster. Fuente: Datastrophic, 2016.

En el transcurso de la **ejecución**, normalmente necesitaremos crear un objeto denominado `sparkSession`, de la clase `SparkSession` de Spark. En el momento de crear este objeto, hay que indicar dónde (en qué dirección IP y en qué puerto) existe un clúster de Spark configurado. Ciertas aplicaciones, como, por ejemplo, Jupyter Notebook (cuando está configurado para Spark) o la línea de comandos (intérprete) de Spark, tanto en Scala (`spark-shell`) como en Python (`pyspark-shell`), ya nos dan un objeto `sparkSession` creado al arrancar la aplicación. No obstante, al iniciar el *shell*, debemos pasarle la configuración relativa al clúster para que la `sparkSession` se cree correctamente.

De lo contrario, se estará ejecutando en local, sin **conexión con un clúster**. De esta manera, la `sparkSession` establece comunicación con el gestor de clúster para poder enviar tareas a los *workers* (nodos del clúster disponibles para Spark). A partir de este momento, podemos utilizar dichos recursos para las sentencias que requieran ejecución distribuida.

Tema 4. Computación distribuida.

Computación paralela

Es importante recalcar esto último: la **ejecución** de un programa en Spark es **secuencial**, en una sola máquina, tal como sería cualquier otro programa en el lenguaje elegido, excepto cuando el flujo de programa llega a ciertas funciones específicas de Spark que desencadenan **ejecución distribuida**, que son la mayoría (pero no todas) de las que forman parte de la API de Spark. Muchas se aplican sobre el objeto `sparkSession` y otras sobre estructuras de datos distribuidas que hayamos ido creando en el programa, como, por ejemplo, RDD o *dataframes* de Spark.

En el momento de crear el objeto `sparkSession`, hemos indicado una configuración con el número de nodos, la memoria RAM y el número de núcleos físicos que necesitamos reservar en cada nodo. En un nodo, estos recursos constituyen lo que se denomina un **executor**: un proceso de la JVM (máquina virtual de Java) que se ejecuta en el nodo y que ocupa los recursos indicados (núcleos, RAM, disco duro).

El **proceso executor** es creado por el gestor de clúster cuando arranca nuestra aplicación de Spark y muere cuando la aplicación finaliza (ya sea con éxito o por alguna condición imprevista que provoca que toda la aplicación termine abruptamente). Cada executor queda preparado para ejecutar tareas de Spark, que es la unidad mínima de ejecución de trabajos. Cada tarea requiere un núcleo libre para ejecutarse, por lo que, si un *executor* tiene reservados cuatro núcleos, podrá ejecutar cuatro tareas en paralelo. Detallaremos esto más adelante. Cada uno de los **nodos del clúster** en los que se crean ejecutores se denomina *worker*.

RDD

Los RDD constituyen la abstracción fundamental de Spark.

Un RDD es una colección no ordenada (*bag*) de objetos que está distribuida en la memoria RAM de los nodos del clúster.

Tema 4. Computación distribuida.

Computación paralela

La colección está dividida en particiones, cada una de las cuales está en la memoria RAM de un nodo distinto del clúster. Al desglosar el nombre, tenemos que:

- ▶ **Resilient** ('resistente' o 'adaptable'). Es posible reconstruir un RDD que estaba en memoria, a pesar de que una de las máquinas falle, gracias al DAG de ejecución (*directed acyclic graph* o grafo de ejecución), que veremos más adelante.
- ▶ **Distributed** ('distribuido'). Los objetos de la colección están divididos en particiones que están distribuidas en la memoria principal de los nodos del clúster. La colección no está ordenada, por lo que no se puede acceder mediante una posición a objetos individuales.
- ▶ **Dataset**. La colección representa un conjunto de datos que estamos procesando de forma paralela y distribuida, para transformarlos, calcular agregaciones, etc.

La Figura 3 representa tres RDD diferentes, distribuidos en la memoria RAM de un clúster de cuatro nodos. No todos los RDD presentan el mismo número de particiones. En la Figura 3, uno de los RDD tiene solo dos, otro tiene tres y, otro, cuatro. La idea es similar al almacenamiento de los ficheros en HDFS, donde están distribuidos entre los discos duros de los nodos, con la diferencia de que, en este caso, los RDD están distribuidos en la memoria RAM de los nodos y, además, **no hay replicación de cada partición**.

Si un nodo falla, es posible reconstruir las particiones que estuvieran en ese momento en su memoria principal gracias al **DAG**, que mantiene la traza de cómo se construyeron. El DAG es otro mecanismo que proporciona robustez sin la necesidad de replicar las particiones de un RDD. Es preciso indicar también que, a pesar de que la Figura 3 muestra una sola partición de cada RDD en cada nodo, lo habitual es que, en la memoria de un mismo nodo, haya numerosas particiones de un mismo RDD (de hecho, es normal que los RDD que se van calculando tengan decenas o cientos de particiones).

Tema 4. Computación distribuida.

Computación paralela

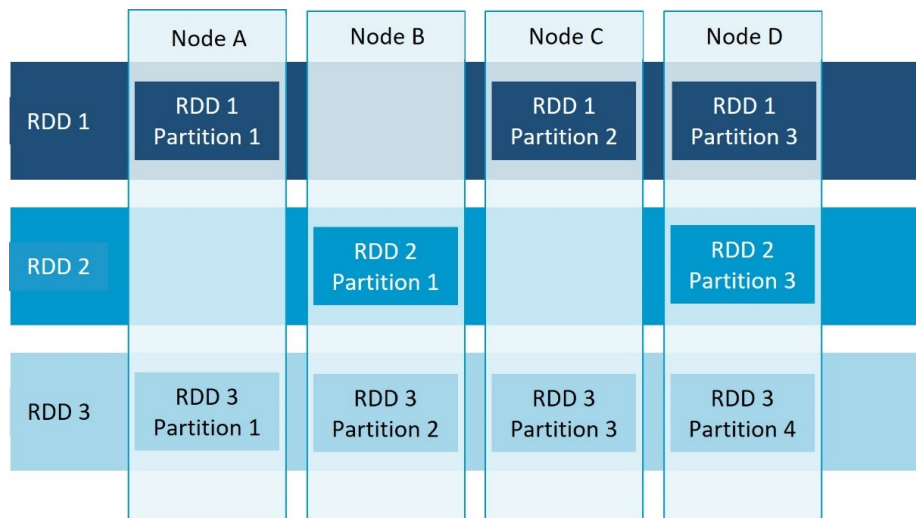


Figura 3. Representación de tres RDD en un clúster de Spark con cuatro *workers*. Fuente: elaboración propia.

En relación con los RDD, cabe destacar dos conceptos:

- **Inmutabilidad.** El contenido de un RDD no puede modificarse una vez creado. Lo que hacemos es aplicar transformaciones a los RDD para obtener otros nuevos, pero los datos del RDD existente no se alteran. La idea de la ejecución es que, cuando aplicamos una transformación, se ejecuta en paralelo sobre todas las particiones del RDD, de manera transparente al programador, para dar lugar a un nuevo RDD, cuyas particiones son el resultado de aplicar la transformación sobre cada una de las particiones del original.

Ejemplo

Dado un RDD de números reales, para multiplicar cada elemento por dos, aplicamos una transformación que actúa en cada elemento y lo multiplica por dos. Spark lleva nuestro código de la transformación (lo serializa y lo envía por la red) a cada uno de los nodos del clúster donde haya particiones de ese RDD y lo ejecuta en ellos para que actúe en cada elemento de esa partición. Todo de manera transparente al programador.

Tema 4. Computación distribuida.

Computación paralela

Una vez más, los datos son el centro, lo más importante; no se mueven salvo que sea imprescindible. Ciertos tipos de transformaciones no requieren movimiento de datos, pero otros sí, como veremos después.

- **Partición.** Es un subconjunto de los objetos de un RDD que están presentes en un mismo nodo. Es la unidad de datos mínima sobre la que se ejecuta una tarea de transformación de manera independiente al resto de particiones. Idealmente, tendría que haber, al menos, tantas particiones como núcleos físicos (procesadores) disponibles en nuestro clúster. De esta manera, nos aseguramos de que todos los núcleos estarán ocupados, incluso en el caso de que nada más que nuestra aplicación estuviese empleando ese clúster. Lo habitual es que haya muchas particiones de un mismo RDD en cada nodo, en un número muy superior al de procesadores existentes en ellos. Se recomienda que cada RDD esté dividido en un número de particiones que sea entre el doble y el triple que el número de procesadores del clúster.

Originalmente, los programadores de Spark trabajaban con RDD. Sin embargo, en Spark 1.6, se introdujeron los ***dataframes***, que definiremos más adelante. Desde Spark 2.0, los propios creadores recomiendan encarecidamente no utilizar los RDD, sino siempre *dataframes* y su API correspondiente. Aparte de la facilidad de uso, el motivo fundamental es que los *dataframes* están sujetos a importantes optimizaciones automáticas de código por parte del analizador de Spark, llamado Catalyst. Los RDD no están sujetos a estas optimizaciones y la ejecución es sensiblemente más lenta. Todos los ejemplos los presentaremos con PySpark.

Tema 4. Computación distribuida.

Computación paralela

Operaciones de transformación y de acción

Podemos realizar dos **tipos de operaciones** cuando usamos la API de Spark:

- **Transformación.** Es una operación que se ejecuta sobre un RDD y devuelve un nuevo RDD, en el que sus elementos se han modificado de algún modo. Son *lazy* (perezosas), es decir, no se ejecuta nada hasta que Spark encuentra una acción. Mientras tanto, Spark simplemente añade la transformación al grafo de ejecución (el DAG), que mantiene la trazabilidad y permite la *resiliency*. El DAG guarda toda la secuencia de transformaciones que se realizaron para obtener cada RDD concreto que se vaya creando en nuestro código.

Si la transformación no implica *shuffle* (movimiento de datos entre nodos), se denomina *narrow* y cada partición da lugar a otra en el mismo nodo. Hay que recordar que una operación *shuffle* implica una escritura previa de los datos en el disco duro local del nodo emisor y después en el disco local del nodo receptor, de manera transparente al programador.

- **Acción.** Recibe un RDD y calcula un resultado (generalmente, un tipo simple, enteros, *doubles*, etc.) y lo devuelve al *driver* (programa principal, que corre en una máquina). El tipo de dato devuelto al *driver* no es un RDD, sino un tipo nativo del lenguaje que estemos utilizando (Java/Scala/Python/R).

Importante: el resultado de la acción debe caber en la memoria de la máquina donde se está ejecutando el proceso *driver*.

Una **acción** desencadena instantáneamente el cálculo de toda la secuencia de transformaciones intermedias y la materialización de los RDD involucrados.

Una vez materializado un RDD, se aplica la **transformación** que toque, según indica el DAG, para generar el siguiente RDD y el anterior se libera (no permanece en la memoria RAM, salvo que se indique expresamente mediante el método `cache()`). Un

Tema 4. Computación distribuida.

Computación paralela

RDD cacheado permanece materializado en la RAM de los nodos y no es necesario recalcularlo después de que se haya materializado la primera vez.

Por defecto, el **punto de partida** del DAG son operaciones de lectura de datos, bien desde una fuente de datos, como, por ejemplo, HDFS o el sistema Amazon S3, bien desde alguna base de datos (distribuida o no) o similar. Si ningún RDD intermedio ha sido cacheado, cualquier operación que haga referencia a un RDD exigirá reconstruir toda la secuencia de transformaciones previas a dicho RDD, empezando por la lectura de los datos, excepto que alguno de los RDD intermedios haya sido cacheado. Esto hace que la secuencia empiece en el RDD cacheado y no haya que remontarse al origen de datos.

La **utilización del DAG** sirve para poder reconstruir cualquier RDD de una secuencia de transformaciones, tanto si la necesidad de materializar el RDD se debe a que hacemos referencia a él varias veces a lo largo de nuestro código como si obedece a que, durante el procesamiento, falla algún nodo y el contenido de su memoria RAM se pierde. Gracias al DAG es posible reconstruir cualquier partición concreta de cualquier RDD y volver a lanzar la secuencia de transformaciones en otros nodos que sí estén activos.

Existen ciertas operaciones de la API de Spark que **no son transformaciones ni acciones**, como, por ejemplo, `cache()`, sino que sirven para configurar, habilitar o deshabilitar ciertos comportamientos, o para obtener características relativas a la distribución física de un RDD (consultar el número de particiones que tiene, consultar si está cacheado, consultar el esquema —nombres y tipos de las columnas— de un *dataframe*, etc.

Transformaciones más habituales con RDD

Recordemos que, para todas las operaciones que reciben una función, Spark serializa el código de la función y la envía por red a los nodos, donde es ejecutada.

Tema 4. Computación distribuida.

Computación paralela

- ▶ `map` . Recibe como parámetro una función, que se ejecuta sobre cada uno de los elementos del RDD para transformarlos, y devuelve un nuevo RDD con los elementos transformados.
- ▶ `flatMap` . Es similar a la anterior, pero, en este caso, la función devuelve un vector de valores para cada elemento. En lugar de generar un RDD de vectores, los aplana para tener un RDD del tipo interior.
- ▶ `filter` . Recibe una función que se aplicará sobre cada elemento del RDD y que deberá devolver un valor booleano (`true` , solo si ese elemento debe ser incluido en el nuevo RDD). Devuelve otro RDD con los elementos que han devuelto `true` .
- ▶ `sample` . Devuelve una muestra aleatoria del RDD del tamaño especificado como parámetro.
- ▶ `union` . Devuelve un RDD con la unión de dos RDD pasados como parámetros.
- ▶ `intersection` . Devuelve la intersección de los dos RDD, es decir, los elementos que están presentes en ambos.
- ▶ `distinct` . Quita los elementos repetidos (retiene cada elemento una sola vez).

Transformaciones específicas para un PairRDD o RDD de pares (clave, valor)

- ▶ `groupByKey` . Cuando los elementos del RDD son tuplas (grupos de varios elementos ordenados), agrupa los elementos por la clave y considera esta como el primer elemento de la tupla.
- ▶ `reduceByKey` . Similar al anterior, pero se agregan los elementos para cada clave empleando la función especificada como parámetro. Esta debe recibir dos valores y devolver uno, y cumplir las propiedades conmutativa y asociativa.
- ▶ `sortByKey` . Ordena los elementos del RDD por clave.

Tema 4. Computación distribuida.

Computación paralela

- ▶ `join` . Combina dos RDD de tal modo que se junten los elementos que tienen la misma clave.

Acciones más habituales en RDD

Por definición, todas las acciones llevan resultados al *driver*, por lo que estos tienen que caber en la memoria del proceso *driver*.

- ▶ `reduce` . Ejecuta una agregación de los datos empleando la función especificada como parámetro. Esta agregación se calcula sobre todos los datos, independientemente de que haya o no claves.
- ▶ `collect` . Devuelve todos los elementos contenidos en el RDD como una colección del lenguaje (listas en Python y R, *arrays* en Java y Scala). Puede causar una excepción por memoria si la lista no cabe en la memoria RAM de la máquina donde está corriendo el *driver*. Se debe usar solo en casos muy controlados.
- ▶ `count` . Devuelve el número de elementos contenidos en el RDD.
- ▶ `take` . Devuelve los *n* primeros elementos contenidos en el RDD. En general, no hay garantías de ordenación en un RDD, salvo que se hayan empleado transformaciones como `sortByKey` .
- ▶ `first` . Devuelve el primer elemento del RDD. Es equivalente a `take` cuando *n* = 1.
- ▶ `takeSample` . Devuelve *n* elementos aleatorios del RDD.
- ▶ `takeOrdered` . Devuelve los *n* primeros elementos del RDD tras haber realizado una ordenación de todos los elementos contenidos en el mismo.
- ▶ `countByKey` . Cuenta el número de elementos en el RDD para cada clave diferente.
- ▶ `SaveAsTextFile` . Guarda los contenidos del RDD en un fichero de texto.

Tema 4. Computación distribuida.

Computación paralela

Ejemplo de código PySpark con RDD

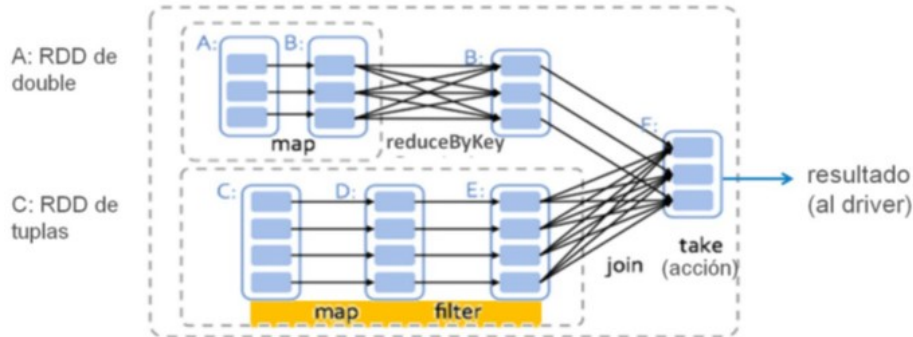


Figura 4. Ejemplo de transformaciones y de la acción `take` mediante el uso de RDD. Fuente: elaboración propia.

```
func_multiplicar = lambda x: (x, 3*x) # función que devuelve una tupla
A = sc.parallelize([5.0, 3.2, 1.1, -2.4, # distribuimos la lista como
8.9, 4.4, 3.7, 9.1], 3) # un RDD de 3 particiones
B = A.map(func_multiplicar)
B = B.reduceByKey(lambda v1, v2: v1+v2)
C = sc.parallelize([(5.0, 1.0), (1.1, -3)], 4) # lista a RDD de 4 part
D = C.map(lambda tuple: (tuple[0], 2*tuple[1]))
E = D.filter(lambda tuple: tuple[1] > 1)
F = E.join(B)
resultado = F.take(3) # ¡ahora es cuando se desencadena el cálculo!
```

Como se observa, ciertas **operaciones** como `join` y `reduceByKey` asumen un RDD de (clave, valor), lo cual no es tan frecuente y todavía se asemeja a MapReduce en la manera de pensar (clave, valor). Para simplificar, existen los *dataframes*, similares a tablas.

En el ejemplo anterior, se ha usado la variable `sc`, referida a `SparkContext`, el objeto que, en versiones anteriores, efectuaba la conexión con el **gestor de clúster**. Actualmente, el objeto `sparkSession` envuelve (e incluye) a un `SparkContext`. Lo habitual es usar la `sparkSession` para leer datos de una fuente y crear desde ellos un *dataframe*. No obstante, para poner crear un RDD (distribuido) a partir de una lista no distribuida del lenguaje, aún se necesita el objeto `sc`. En el código anterior, estamos creando, en primer lugar, un RDD llamado A, que la Figura 5 representa con tres

Tema 4. Computación distribuida.

Computación paralela

particiones, a partir de una lista de números reales. Por eso, el RDD resultante es un RDD de números reales.

Hemos definido una función `func_multiplicar`, que recibe un número y devuelve una tupla formada por el número y el resultado de multiplicarlo por tres. Dicha función la hemos aplicado a cada elemento de A mediante el método `map` de Spark. Este método es una **transformación** que aplica a cada elemento del RDD nuestra función y devuelve un nuevo RDD con el resultado. Para ello, **serializa el código** de nuestra función (en este caso, la función `func_multiplicar`) y lo envía por la red a los nodos, para que, en ellos (en aquellos nodos que contengan alguna partición del RDD A), se ejecute dicha función sobre los elementos de las particiones del RDD que estén presentes. Hemos llamado B al RDD resultante, que, en este caso, será un RDD de tuplas de dos elementos de tipo `double`, que es lo que devolvía `func_multiplicar`.

Nótese que, para llevar a cabo la **transformación** `map`, no es necesario el movimiento de datos, ya que la función se aplica elemento a elemento sobre los que haya en el nodo y no necesita de otros elementos para calcular el resultado. Se dice que `map` es una transformación *narrow* (estrecha), a diferencia de otras transformaciones que forzosamente requieren movimiento de datos para poder calcular el resultado, llamadas transformaciones *broad*.

Un **RDD de tuplas de dos elementos** se considera, a ojos de Spark, un PairRDD, es decir, un RDD de (clave, valor). Los RDD de (clave, valor) admiten operaciones adicionales, además del estándar de cualquier RDD. Una de ellas es `reduceByKey`, que agrupa elementos del RDD que tengan el mismo valor de clave y agrega entre sí sus valores, empleando la función que le pasemos como argumento. Esta función requiere movimiento de datos entre nodos, ya que existirán tuplas que compartan la misma clave, pero estén en particiones distintas, que, además, posiblemente, también estarán en nodos diferentes. El resultado de esta transformación lo hemos vuelto a asignar a la variable B.

Tema 4. Computación distribuida.

Computación paralela

Por otro lado, hemos creado otro RDD en la **variable C**, que la Figura 5 muestra con cuatro particiones, como el resultado de paralelizar una lista de tuplas de números reales. Por tanto, C ya es, desde el comienzo, un PairRDD. Le hemos aplicado, a continuación, una transformación `map`. Dado que C contiene tuplas, la función que aplicamos tiene que saber que el argumento que recibe es una tupla. En nuestro caso, a partir de cada tupla, se devuelve otra cuyo primer elemento es igual y cuyo segundo elemento es el resultado de multiplicar por DOS el segundo elemento original.

La **sintaxis** de `lambda` de Python simplemente sirve para indicar que estamos creando una función anónima, es decir, una función convencional, pero que no necesitamos invocarla desde fuera. Solo la usamos para pasarla como argumento a un método (que espera recibir una función como argumento, tal como le ocurre a `map` de Spark); por eso, no necesitamos darle nombre, aunque podríamos haberlo hecho creando una función convencional con nombre, como se suele hacer con `def` en Python.

El RDD resultante de esta transformación `map` lo almacenamos en la **variable D**, que también es un PairRDD. Sobre él aplicamos una nueva transformación `filter`, que, al igual que `map`, actúa sobre cada elemento del RDD sin necesitar ningún otro proveniente de otra partición ni de otro nodo para calcular el resultado. De hecho, lo que `filter` lleva a cabo es un filtrado, de manera que el RDD resultante solo contendrá aquellos elementos del RDD original que cumplan cierta condición. La función pasada como argumento a `filter` debe ser capaz de recibir un elemento del RDD (en este caso, una tupla de dos elementos) y siempre ha de devolver un booleano, que indica si ese elemento tiene que formar parte del resultado (`true`) o no (`false`).

A continuación, hemos invocado una **transformación** `join`, que se aplica a un PairRDD y recibe como argumento otro PairRDD. El resultado es un nuevo RDD que contiene tuplas jerárquicas tales que la clave es común entre una tupla de uno de los

Tema 4. Computación distribuida.

Computación paralela

RDD y una tupla del otro, y el valor está formado por una tupla con el valor que tenía esa clave en un RDD y el valor que tenía en el otro. El RDD resultante de la operación `join` aplicada a B y E lo almacenamos en la variable F.

Por último, hemos llamado al **método** `take` sobre el RDD F. Este método es una acción que lleva el resultado al *driver*. Esto implica que el resultado debe caber en él. En este caso, no supone un problema, ya que `take(n)` coge `n` elementos del RDD y los envía al *driver*, y solo hemos solicitado tres elementos. Además, y lo que es más importante, al ser una acción, desencadena la realización de todas las transformaciones anteriores que estaban pendientes. De hecho, la ejecución de cada una de las líneas de código anteriores a `take` simplemente provocaba que se añadiesen fases al grafo de ejecución (DAG) de Spark, pero no se materializaba ninguna. Se puede comprobar porque la ejecución en Python devuelve inmediatamente el control al intérprete de Python, sin emplear tiempo en llevarla a cabo. El resultado de `take` ya no es un RDD, sino una estructura de datos del lenguaje que se esté manejando. En este caso, es una lista de Python formada por tres elementos del RDD F, es decir, una lista de tres tuplas, que es lo que contiene F.

Jobs, stages y tasks

Por ejemplo: `df.count()`, `df.take(4)`, `df.show()`, `df.read(...)`, `df.write(...)`, etc. Cada *job* se divide en una serie de *stages* (etapas).

Un *job* de Spark es todo el procesamiento necesario para llevar a cabo una acción del usuario. Un *stage* es todo el procesamiento que puede llevarse a cabo sin mover datos entre nodos.

Cada nodo hace exactamente un procesamiento idéntico, aplicado a diferentes particiones del mismo *dataframe* que están procesando todos. Cuando en nuestro código invocamos una operación que implica movimiento de datos (`shuffle`), finaliza un *stage* y se crea otro nuevo. Por ejemplo, `df.join(...)`, `df.groupBy(...).agg(...)`.

Tema 4. Computación distribuida.

Computación paralela

Una *task* (tarea) es cada una de las transformaciones que forman una etapa. Es la unidad mínima de trabajo de Spark. Más formalmente, una **tarea** de Spark es el procesamiento aplicado por un núcleo físico (CPU) a una partición de un RDD.

La Figura 5 representa la división en tareas, etapas y trabajos para el ejemplo de código contenido en el anexo.

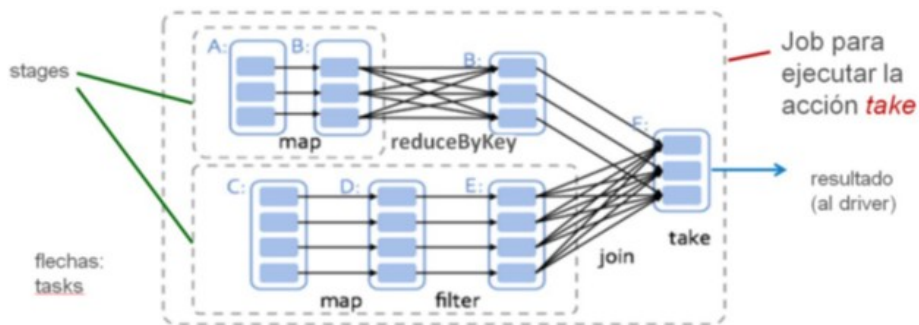


Figura 5. Representación de *jobs*, *stages* y *tasks* en Spark. Fuente: elaboración propia.

Tema 4. Computación distribuida.

Computación paralela

4.3. Spark II

Dataframes en Spark

Manejar RDD resulta tedioso cuando los tipos de datos que contienen empiezan a complicarse, ya que en todo momento necesitamos saber exactamente la estructura del dato, incluso cuando son tuplas jerárquicas (tuplas en las que algún campo es, a su vez, una tupla o lista). Sería más conveniente poder manejar los RDD como si fuesen **tablas de datos**, estructuradas en filas y columnas, lo cual aportaría mayor nivel de abstracción y más facilidad de uso. Esto es lo que nos proporcionan la API estructurada y los *dataframes* de Spark.

Un *dataframe* de Spark es una tabla de datos distribuida en la RAM de los nodos que está formada por filas y columnas con nombre y tipo (incluidos tipos complejos), similar a una tabla en una base de datos relacional

Internamente, un **dataframe** no es más que un RDD de objetos de tipo `Row` de Spark, cada uno de los cuales representa una fila de una tabla como un vector cuyos componentes (lo que serían las columnas de una tabla) tienen nombre y tipo predefinido. El esquema (*schema*) de un *dataframe* define el nombre y el tipo de dato de cada una de estas columnas. Cada *dataframe* envuelve (tiene dentro) un RDD, al que se puede acceder como el atributo `rdd`, por ejemplo, `variableDF.rdd`. Por eso, los conceptos de transformación y acción se aplican también a *dataframes*.

Hay que recordar que los *dataframe* de Spark están distribuidos en la memoria RAM de los **nodos worker**, aunque puedan parecerse a una tabla de una base de datos. Por otro lado, el nombre *dataframe* es el mismo que el definido en otras **librerías de lenguajes**, como Python (`dataframe` es del paquete `Pandas`) o R (`data.frame`). Si bien el concepto es el mismo (tabla de datos cuyas columnas tienen nombre y tipo), la

Tema 4. Computación distribuida.

Computación paralela

implementación y manejo no tienen nada que ver, más allá de que los autores eligieron el mismo nombre. Los *dataframe* de Spark son un tipo de dato definido por Spark, están distribuidos físicamente y se manejan mediante la API de Spark.

Existe, en la API de Spark, un método que permite traerse todo el contenido a una sola máquina (el *driver*) y que devuelve un *dataframe* de Pandas (no distribuido). Es el **método** `toPandas`, pero debe usarse con cuidado, ya que, de nuevo, requiere que todo el contenido distribuido en los nodos quepa en la memoria RAM del nodo donde se ejecuta el proceso *driver*. De lo contrario, provocará una excepción `OutOfMemory`.

A continuación, vamos a ver las diferentes **operaciones** que se pueden hacer con *dataframes* y la API estructurada. Empezaremos por la lectura y escritura de *dataframes* y seguiremos por las diferentes transformaciones que se pueden aplicar a los *dataframes* leídos.

API estructurada de Spark: lectura y escritura de dataframes

Como cabe esperar, lo primero que se necesita para tener un *dataframe* son los datos con los que se quiere trabajar y que, por tanto, serán cargados en un *dataframe* para su futura manipulación. Spark puede leer información de numerosas **fuentes de datos**.

Para que Spark pueda conectarse a una fuente de datos, debe existir un conector específico que indique cómo obtener datos de esa fuente y convertirlos en un *dataframe*. Entre las fuentes de datos más habituales que disponen de dicho **conector**, podemos encontrar:

- **HDFS.** Spark puede leer de HDFS diversos formatos de archivo: CSV, JSON, Parquet, ORC y texto plano. No obstante, la comunidad de desarrolladores ha proporcionado mecanismos para leer otros tipos de ficheros, como los XML, entre otros. La terminación indicada en el nombre de archivo no informa a Spark de nada, solo puede servir como pista al usuario que vaya a leer el fichero.

Tema 4. Computación distribuida.

Computación paralela

- ▶ **Amazon S3.** Almacén de objetos distribuido creado por Amazon, de donde Spark también puede leer cualquiera de los formatos de fichero nombrados anteriormente.
- ▶ **Bases de datos relacionales** mediante conexiones JDBC u ODBC. Spark es capaz de leer en paralelo a través de varios *workers* conectados simultáneamente a una base de datos relacional, cada uno de los cuales lee porciones diferentes de una misma tabla. Cada porción va a una partición del resultado. Spark puede enviar una consulta a la base de datos y leer el resultado como *dataframe*.
- ▶ Conectores para **bases de datos no relacionales**. Los conectores son específicos, desarrollados por la comunidad o por el fabricante (por ejemplo: Cassandra, MongoDB, HBase ElasticSearch, etc.).
- ▶ Cola distribuida **Kafka** para datos que se van leyendo de un *buffer*.
- ▶ También datos que llegan en **streaming a HDFS** (ficheros que se van creando nuevos en tiempo real).

Aunque hemos hablado de fuentes de datos de lectura, en el caso de querer escribir resultados del *dataframe* en el almacenamiento persistente, ya sea fichero, base de datos o servicio de mensajería, Spark proporciona los mismos mecanismos de los que hemos hablado para la lectura. A continuación, vamos a ver en detalle cómo realizar las **lecturas y escrituras** de los datos.

Lectura de *dataframes*

En general, para leer los datos desde Spark, se usa un atributo de la `SparkSession`, `spark.read`, y se especifican diferentes **opciones**, dependiendo del tipo de fichero por leer:

- ▶ **Formato.** Tal y como se mencionó anteriormente, el formato de fichero puede ser CSV, JSON, Parquet, Avro, ORC, JDBC/ODBC o texto plano, entre otros muchos.

Tema 4. Computación distribuida.

Computación paralela

- ▶ **Esquema.** Hay cuatro opciones referentes al esquema:
 - Algunos tipos de fichero almacenan el esquema junto a los datos (por ejemplo, Parquet, ORC o Avro) y, por tanto, no es necesario indicar ningún esquema adicional.
 - Para los tipos de fichero que no almacenan el esquema, es posible solicitar a Spark que trate de inferirlo con la opción `inferSchema` activada (`true`). Hay que tener en cuenta que esta opción conllevará más tiempo de lectura, dado que Spark necesita leer una serie de líneas del fichero y analizarlas para tratar de adivinar el esquema.
 - Podemos pedir a Spark que no trate de inferir el esquema. En este caso, todos los datos se leerán como si fueran texto (`string`).
 - Cabe la opción de indicar de forma explícita el esquema de los datos esperado, para evitar un incremento del tiempo de lectura y posibles incorrecciones en la inferencia del esquema por parte de Spark. Veremos estas opciones en detalle, con algún ejemplo, más adelante.
- ▶ **Modo de lectura.** Puede ser `permissive` (por defecto), que traduce como *null* aquellos registros que considere corruptos de cada fila; `dropMalformed` , que descarta las filas que contienen alguno de sus registros con un formato incorrecto, y `failFast` , que lanza un error en cuanto encuentra un registro con un formato incorrecto.
- ▶ Existen otra serie de opciones que veremos más adelante, ya que dependen del tipo específico de **fichero a leer**.

La única información obligatoria es la ruta del fichero que va a ser leído; el resto de las opciones son opcionales. Por tanto, la **estructura genérica de lectura** sería la siguiente:

```
myDF = spark.read.format(<formato>)  
.load("/path/to/hdfs/file") # spark es el objeto SparkSession  
# <formato> puede ser "parquet" | "json" | "csv" | "orc" | "avro"
```

Tema 4. Computación distribuida.

Computación paralela

En cuanto al **esquema**, recordemos que es el que describe el nombre y el tipo de cada uno de los registros (columnas) de las filas del *dataframe*. Como comentábamos, algunos tipos de fichero, como Avro, Parquet u ORC, contienen información respecto a su esquema y, por tanto, no es necesario especificarlo durante su lectura. Sin embargo, con otros formatos, como CSV o JSON, donde el esquema del fichero no está almacenado en este, podemos dejar que Spark infiera el esquema con la opción `inferSchema` configurada como `true`, indicar que no infiera nada (`inferSchema` a `false`) y lea todo como `string`, o bien especificar explícitamente cuál va a ser el esquema concreto esperado.

Cabe recordar una vez más que la opción `inferSchema` indica a Spark que trate de adivinar el **tipo de cada columna**, en cuyo caso Spark tratará de inferir lo mejor posible esta información. No obstante, puede darse el caso de que, en una columna que debería ser de tipo entero, falte alguna información y de que Spark, ante la duda, infiera que dicha columna es de tipo texto (`string`). Para evitar este tipo de incorrecciones, es mejor especificar explícitamente el esquema si se conoce de antemano.

Un **esquema** en Spark no es más que un objeto `StructType`, compuesto por un conjunto de `StructFields`. Cada `StructField` representa un registro (columna) de una fila; por tanto, se compone de un nombre (`name`), un tipo (`type`), un booleano que indica si la columna puede contener datos *null* (es decir, datos inexistentes), así como otra información opcional. Por ejemplo, podemos definir en `pyspark` el esquema de un fichero JSON donde cada fila contiene tres campos (columnas) de la siguiente forma:

```
from pyspark.sql.types import StructField, StructType, StringType, LongType
fileSchema = StructType([
    StructField("dest_country_name", StringType(), True),
    StructField("origin_country_name", StringType(), True),
    StructField("count", LongType(), False)
])
```

Tema 4. Computación distribuida. Computación paralela

Una vez que se tiene el esquema definido, solo resta leer los datos utilizando dicho esquema:

```
myDF = spark.read.format("json")  
.schema(fileSchema)  
.load("/path/to/file.json") # spark es el objeto SparkSession
```

Si no queremos que Spark infiera el esquema ni proporcionar uno nosotros, Spark leerá todas las columnas como strings :

```
myDF = spark.read.format("json")  
.options("inferSchema", "false")  
.load("/path/to/file.json") # spark es el objeto SparkSession
```

En caso de querer que Spark infiera el esquema en lugar de especificarlo, habría que usar la opción `inferSchema`, como comentábamos anteriormente:

```
myDF = spark.read.format("json")  
.options("inferSchema", "true")  
.load("/path/to/file.json") # spark es el objeto SparkSession
```

Por último, cabe mencionar que, para algunos tipos de fichero, suele estar disponible un atajo como `spark.read.<format>("/path/to/file")`, donde `format` puede ser, por ejemplo, `csv` o `avro`, aunque no todos los formatos lo tienen.

Tema 4. Computación distribuida.

Computación paralela

Vamos a ver algunas **particularidades** de ciertos formatos concretos:

- ▶ Los **ficheros CSV** son probablemente los más problemáticos, ya que la división de las filas en diferentes registros (columnas) depende de separadores que no siempre se respetan o que son confusos respecto al texto del fichero. Por ejemplo, si los registros están separados por comas, y uno de ellos es de tipo `string` y puede contener dentro comas, cabe la posibilidad de que se produzcan interpretaciones (*parser*) incorrectas. Además de las opciones que ya hemos visto, los ficheros CSV soportan más opciones. Entre las más usadas, destacan:
 - **Si el fichero incorpora cabecera**, la opción `header` indica si cabe esperar que la primera línea del fichero se corresponda con los nombres de las columnas (`true`) o no (`false`, por defecto).
 - **El carácter separador**. La opción `delimiter` permite indicar el carácter separador de registros (columnas) en una fila. Es importante tener en cuenta que Spark no soporta separadores de más de un carácter.

Veamos un ejemplo de lectura de CSV con y sin inferencia de esquema, en el que usaremos, además, el atajo `.csv(<path>)` que se comentaba anteriormente:

```
# En df1 se van a leer todas las columnas como si fuesen strings:
df1 = spark.read.option("inferSchema", "false")
.csv("/path/hdfs/file")#uso del atajo
# Para evitar que todas las columnas se lean como strings,
# vamos a indicar el esquema para que cada columna
# se lea con su tipo correspondiente y se le asigne el nombre deseado:
myschema = StructType([
  StructField("columna1", DoubleType(), nullable = False),
  StructField("columna2", DateType(), nullable = False)
])
# Pasamos el esquema para que se lean correctamente
# (el true/false como valor de option se escribe en minúscula):
df2 = spark.read.option("header", "true")\
.option("delimiter", "|")\
.schema(myschema)\
.csv("/path/hdfs/file")#uso del atajo
```

Tema 4. Computación distribuida.

Computación paralela

- ▶ Parquet , ORC , Avro . Para estos tipos de ficheros, la lectura es más directa, ya que basta con ejecutar el siguiente código (no es necesario ni inferir esquema ni especificarlo):

```
myDf = spark.read.parquet("/path/to/file.parquet") # usa atajo
```

Escritura de *dataframes*

La operación de escritura es análoga a la de lectura. En este caso, se usa el atributo `write` de los *dataframes* y se indican los siguientes aspectos:

- ▶ **Formato**, con el método `format(<formato>)` , que puede ser cualquiera de los que hemos visto para la operación de lectura.
- ▶ **Modo de escritura**. Puede ser `append` , `overwrite` , `errorIfExists` o `ignore` (si los datos o ficheros existen, no se hace nada).
- ▶ **Otras opciones específicas** de cada formato.

Así, por ejemplo, si queremos guardar un *dataframe*, `df`, en formato CSV, utilizaremos el tabulador como separador y, en modo sobreescritura, en la cabecera en el fichero, escribiremos algo como lo siguiente:

```
df.write.format("csv")  
.mode("overwrite")  
.option("sep", "\t")  
.option("header", "true")  
.save("path/to/hdfs/directory")
```

Mientras que, si queremos guardarlo en Parquet , introduciremos el siguiente código:

```
df.write.format("parquet") # equivalente en orc, avro y json  
.mode("overwrite")  
.save("path/to/hdfs/directory")
```

Tema 4. Computación distribuida.

Computación paralela

Hay que tener en cuenta que la **ruta** especificada donde se guardará la información no es un fichero, sino un directorio (que creará Spark), dentro del cual se escribirá la información del *dataframe* en diferentes partes, con la estructura `part-XXXXX-hash.csv` en el caso de CSV. También puede escribirse el resultado en otros destinos de datos muy diversos, siguiendo una sintaxis específica. Para más detalles, puedes consultar la web de Spark.

API estructurada de Spark: manipulación de *dataframes*

Una vez que sabemos qué es un *dataframe*, cómo leer datos para crear *dataframes* y cómo escribir la información que albergan en almacenamiento persistente, veamos qué tipo de **operaciones** podemos realizar sobre estas estructuras de datos distribuidas que nos proporciona Spark.

Recordemos que un *dataframe* consistía en un conjunto de filas (en el fondo, envolvían RDD de objetos de tipo `Row`), cada una de las cuales estaba formada por una serie de registros (columnas). La mayoría de las operaciones que ofrece la API estructurada son operaciones sobre columnas (clase `Column`). Spark implementa muchas operaciones entre columnas de manera distribuida (operaciones aritméticas entre columnas, manipulación de columnas de tipo `string`, comparaciones...), que debemos usar siempre que sea posible. Todas ellas son sometidas a optimizaciones por parte de **Catalyst** para ejecutar más rápidamente.

La utilización general de los métodos de la API sigue el siguiente formato: `objetodataframe.nombreDelMétodo(argumentos)`. Todas las **manipulaciones** devuelven como resultado un nuevo *dataframe*, sin modificar el original (recordamos que los RDD son inmutables y, por extensión, los *dataframes* también). Por eso, se suelen encadenar transformaciones:

```
df.método1(args1).método2(args2)
```


Tema 4. Computación distribuida.

Computación paralela

Transformaciones más frecuentes con *dataframes*

Supongamos que hemos creado una variable llamada `df` con esta línea de código:

```
df = spark.read.parquet("/ruta/hdfs/datos.parquet")
```

Antes de describir las transformaciones más frecuentes con *dataframes*, cabe mencionar que todas ellas están contenidas en la librería de `pyspark` `pyspark.sql.functions`. Por tanto, es necesario importar esta librería antes de usar cualquiera de estas funciones. Generalmente, se suele importar asignándole el alias `F`, de forma que, posteriormente, nos podremos referir a las **funciones** como `F.nombreFuncion(argumentos)`.

Una vez que sabemos cómo importar las funciones, veamos cuáles son:

- ▶ `printSchema` imprime el esquema del *dataframe*. Esta función es muy útil cuando queremos comprobar que el *dataframe* se ha leído de fichero con el tipo de datos esperado.

```
df.printSchema()
```

- ▶ `col("nombreCol")` sirve para seleccionar una columna y devuelve un objeto `Column` sobre el que podemos realizar diferentes transformaciones. Es importante tener en cuenta que no se puede mezclar código SQL (que devuelve *dataframes*) con objetos de tipo `Column`.
- ▶ `select` permite seleccionar columnas de diferentes formas:

```
import pyspark.sql.functions as F
#ambas formas de usar select devuelven el mismo resultado:
df.select("nombreColumna").show(5)
df.select(F.col("nombreColumna")).show(5)
# Crear columna con nombre diff y seleccionar esa columna,
# que es el resultado de restar el literal 18 a la columna edad.
# Mostrar 5 registros de la columna resultante seleccionada:
df.select((F.col("edad")-F.lit(18)).alias("diff")).show(5)
```

Tema 4. Computación distribuida.

Computación paralela

- ▶ `alias` le asigna un nombre a la columna sobre la que se aplica:

```
import pyspark.sql.functions as F
df.select(F.col("nombreColumna").alias("nombreNuevo")).show(5)
```

- ▶ `withColumn` devuelve un nuevo DF con todas las columnas del original más una nueva columna añadida al final, como resultado de una operación entre columnas existentes que devuelve como resultado un objeto `Column`.

```
import pyspark.sql.functions as F
# Crea una nueva columna cuyo nombre es nombreNuevaColumna
# y que almacena la suma de los valores en las columnas c1 y c2.
# Devuelve un objeto de tipo Column, del que mostramos 5 registros:
df.withColumn("nombreNuevaColumna", F.col("c1")+F.col("c2")).show(5)
```

- ▶ `drop` elimina una columna.

```
import pyspark.sql.functions as F
# Ambas formas de utilizar drop dan el mismo resultado
df.drop("nombreColumna")
df.drop(F.col("nombreOtraColumna"))
```

- ▶ `withColumnRenamed` renombra una columna:

```
import pyspark.sql.functions as F
df.withColumnRenamed("nombreExistenteColumna", "nombreNuevoColumna")
```

- ▶ `when(condición, valorReemplazo1).otherwise(valorReemplazo2)` sirve para reemplazar valores de una columna según una condición que implica a esa o a otras columnas. Si no especificamos `otherwise`, los campos donde no se cumpla la condición se rellenarán a `null`. Esta función se utiliza generalmente dentro de `withColumn`:

```
import pyspark.sql.functions as F
df.withColumn("esMayor", F.when("edad>18", "mayor").otherwise("menor"))
```

Tema 4. Computación distribuida.

Computación paralela

Transformaciones matemáticas y estadísticas con *dataframes*

Existen numerosas **funciones** matemáticas y estadísticas disponibles para su aplicación sobre *dataframes*. Dichas funciones generan columnas y *dataframes* nuevos como, por ejemplo, los siguientes:

- ▶ Columna de números aleatorios:

```
import pyspark.sql.functions as F
# Crea una nueva columna con números aleatorios de una uniforme:
df = df.withColumn("unif", F.rand())
# Crea una nueva columna con números aleatorios de una normal:
df = df.withColumn("norm", F.randn())
```

- ▶ *Dataframe* con estadísticos descriptivos (media, desviación típica, máximo, mínimo...) (método `describe`):

```
df.describe().show()
```

- ▶ Operaciones matemáticas, entre otras:
 - Seno: `F.sin()` ,
 - Coseno: `F.cos(F.col("nombreColumna"))` .
 - Raíz cuadrada: `F.sqrt(F.col("nombreColumna"))` .

Combinaciones y filtrado de *dataframes*

Otro grupo de funciones son las relacionadas con la combinación y filtrado de *dataframes*. Entre ellas, podemos encontrar:

- ▶ **Unión de *dataframes*:**

```
df3 = df1.unionAll(df2)
```

Tema 4. Computación distribuida.

Computación paralela

► Diferencia de *dataframes*:

```
df3 = df1.except(df2)
```

► Filtrado de filas de un *dataframe*. Existen dos formas de expresar la condición de filtrado:

```
import pyspark.sql.functions as F
# df3 tendrá las filas de df donde la columna c1 tenga un valor
# mayor que c2
df3 = df.where(F.col("c1")>F.col("c2"))
df3 = df.where("c1 > c2") # Es equivalente a lo anterior
```

Operaciones de RDD aplicadas a *dataframes*

En general, las operaciones que podíamos usar sobre RDD suelen estar disponibles también para los *dataframes*. No olvidemos que un *dataframe* no es más que un RDD de objetos de tipo `Row` y que podemos acceder al RDD que envuelve el *dataframe* mediante el atributo `rdd`:

```
df.rdd.take(5) # Muestra 5 objetos de tipo Row
```

► `map` y `flatMap` sobre el RDD; en este caso, iteramos sobre objetos de tipo `Row`:

```
def mifuncion(r): # r es un Row y se accede a sus campos mediante '.'
    return((r.DNI, "Bienvenido " + r.nombre + " " + r.apellido))
paresRDD = df.rdd.map(mifuncion) # map sobre un RDD devuelve un RDD
dfRenombrado = paresRDD.toDF("DNI", "mensaje") # lo convertimos a DF
```

► Otras operaciones que pueden realizarse son: transformaciones como `sample`, `sort`, `distinct`, `groupBy` ..., y acciones como `count`, `take`, `first`, etc.

A continuación, se muestra un ejemplo un poco más completo del uso de la API estructurada:

```
# el objeto spark (sparkSession) ya está creado en Jupyter. Si no,
# habría que crearlo indicando la dirección IP del máster
# de un cluster de Spark existente:
```

Tema 4. Computación distribuida.

Computación paralela

```
from pyspark.sql import functions as F
df = spark.read.parquet("/mis/datos/en/hdfs/flights.parquet")
resultDF = df.withColumn("distMetros", F.col("dist")*1000)\
.withColumn("retrasoCat",
F.when(F.col("retraso") < 15, "poco")\
.when(F.col("retraso") < 30, "medio")
.otherwise("mucho")) # sin otherwise, pondría Null!
.select(F.col("aeronave"), F.col("origen"),
F.col("disMetros"),
F.col("retrasoCat"),
(F.col("retraso") / F.col("dist")).alias("retrasoPorKm")
)\
.withColumnRenamed("dist", "distKm")
.where(F.col("aeronave") == "Boing 747") # equivale a .filter()
.where("distMetros > 20000 and origen != 'Madrid'")
# Línea anterior: where pasa una consulta SQL como string.
# En la línea siguiente, hacemos de nuevo lo mismo:
df2 = resultDF.select("retraso, compania")
.where("aeropuerto like '%Barajas' and retrasoCat = 'poco'")
df2.show() # show es una acción que provoca que se calculen todos los
# Dataframes de las transformaciones anteriores hasta show.
```

Operaciones de agrupamiento y agregación

El **método** `groupBy("nombreCol1", "nombreCol2", ...)` sobre un *dataframe* devuelve una estructura de datos llamada *RelationalGroupedDataset*, que no es un *dataframe* y sobre la que apenas se pueden aplicar operaciones. Equivale a la operación `GROUP BY` de SQL, donde se definen grupos para después calcular, para cada uno, un resultado agregado (por ejemplo, la suma, la media, un conteo, etc.) de una o varias variables numéricas.

Se suelen aplicar **operaciones** como `count()`, que efectúa un conteo del número de elementos de cada grupo, o la función `agg()`, que es la más habitual y realiza, para cada grupo, las agregaciones que le indiquemos sobre las columnas seleccionadas. El resultado solamente contendrá aquellas columnas que se incluyeron como argumentos en el `groupBy` más aquellas que sean mencionadas como argumento de alguna de las operaciones de agregación que incluimos en la función `agg`.

Tema 4. Computación distribuida.

Computación paralela

Cuando ejecutamos **funciones de agregación** sin haber llevado previamente una agrupación con `groupBy`, lo que obtenemos es un *dataframe* con una sola fila, que es el resultado de la agregación. El siguiente fragmento de código muestra cómo se utilizan `groupBy` y `agg`:

```
import pyspark.sql.functions as F
newDF = myDF.agg(max(F.col("mycol"))) # devuelve DF de una sola fila
newDF = myDF.groupBy("mycol").agg(F.max(F.col("mycol"))) #Tantas filas
# como valores distintos en mycol
newDF = myDF.withColumn("complicated", # F.sin: seno. F.lit: constante
F.lit(2)*F.sin(F.col("colA"))*F.sqrt(F.col("colB")))
newDF = myDF.groupBy("id").agg(F.count("id").alias("countId"),
F.max("date").alias("maxdate"),
F.countDistinct("prod").alias("nProd"))
# Sintaxis tipo diccionario para indicar varias agregaciones:
newDF = myDF.groupBy(F.col("someCol"))
.agg({"existingCol": "min", "otherCol": "avg"})
# Indicamos que, en cada grupo, definido por cada valor de "someCol",
# queremos el mínimo de la columna "existingCol" en dicho grupo y la
# media de los valores de la columna "otherCol". Esto devuelve
# un DF con tantas filas como valores distintos tenga someCol.
```

Spark SQL

Spark SQL ofrece una potente opción, que consiste en aplicar operaciones escritas como consultas en lenguaje SQL a *dataframes* que se hayan registrado como tablas, sin tener que utilizar la API estructurada paso a paso. Es decir, Spark SQL se integra con la API de *dataframes*; así, se puede expresar parte de una consulta en SQL y parte utilizando la API estructurada. Sea cual sea la opción elegida, cabe recordar que se compilará en el mismo plan de ejecución, dado que, como se veía en la descripción de Spark, tiene un motor de ejecución unificado.

Antes de que Spark fuera una herramienta tan usada como lo es hoy en día, la tecnología *big data* que permitía hacer consultas usando lenguaje SQL sobre grandes conjuntos de datos almacenados de forma distribuida era **Hive**. Esta herramienta fue muy popular, ya que ayudó a que Hadoop fuera usado por profesionales que no tenían conocimientos suficientes de Java u otros lenguajes de

Tema 4. Computación distribuida.

Computación paralela

programación para realizar procesamientos de los datos almacenados en HDFS utilizando MapReduce, pero que sí tenían amplios conocimientos en el uso de bases de datos SQL.

Por su parte, Spark comenzó como un **motor de procesamiento paralelo** de grandes cantidades de datos basado en RDD. Sin embargo, a partir de la versión 2.0, los autores incluyeron un *parser* de consultas SQL (que soportaba tanto ANSI-SQL como HiveQL, el lenguaje SQL de Hive), que dio lugar a una herramienta similar a Hive. Es decir, ofrecía la posibilidad de realizar consultas SQL sobre un conjunto de datos sin necesidad de tener conocimientos de Python, Java o Scala para ello, lo que hacía más accesible el procesamiento de grandes conjuntos de datos.

Cabe aclarar que Spark SQL está pensado para funcionar como un **motor de procesamiento de consultas en batch** (OLTP) y no para realizar consultas interactivas o que necesiten una baja latencia (OLAP). Exactamente igual ocurre con Hive, como se comentará en el capítulo correspondiente.

Existen dos opciones para ejecutar **consultas SQL en Spark**:

- ▶ **Interfaz de línea de comandos de Spark SQL.** Es una herramienta útil para realizar consultas sencillas en modo local desde la línea de comandos. Para acceder a esta herramienta, basta con ejecutar `./bin/spark-sql` en línea de comandos, en el directorio donde esté Spark instalado.
- ▶ **API de SparkSQL.** El método `.sql()` de la `sparkSession` recibe como argumento una consulta SQL, que puede hacer referencia a cualquier tabla registrada en Hive, y devuelve un *dataframe* con el resultado de la consulta. Para registrar un *dataframe* como una tabla de Hive, lo cual genera (solamente) los metadatos necesarios en el *metastore* de Hive, existen varios métodos; entre ellos, podemos mencionar `createOrReplaceTempView`, que la registra solo durante esta sesión. Spark ofrece más métodos para crear tablas en Hive y manejar el *metastore*, pero quedan fuera del alcance de este tema. Spark analiza automáticamente la consulta y la traduce a un

Tema 4. Computación distribuida.

Computación paralela

DAG, exactamente del mismo modo que ocurre al utilizar la API estructurada que hemos visto en las secciones anteriores. Veamos un ejemplo:

```
from pyspark.sql import functions as F
df = spark.read.parquet("/mis/datos/en/hdfs/flights.parquet")
df.createOrReplaceTempView("vuelos") # crear tabla temporal vuelos
resultDF = spark.sql("select *, 1000*dist as distMetros from vuelos")
resultsDF.show() # todas las cols originales más una nueva distMetros
```

Además, la API SQL es totalmente **interoperable** con la API estructurada, de forma que se puede crear un *dataframe*, manipularlo primero con SQL y después con la API estructurada. Es decir, se puede hacer código como el siguiente:

```
from pyspark.sql import functions as F
df = spark.read.parquet("/mis/datos/en/hdfs/flights.parquet")
df.createOrReplaceTempView("vuelos") # crear tabla temporal vuelos
resultDF = spark.sql("select *, 1000*dist as distMetros from vuelos")
.where("distMetros > 100000") # API SQL + estructurada
resultsDF.show() # Todos los vuelos con distancias >100 km
```

- **Servidor JDBC/ODBC.** Spark proporciona una interfaz JDBC/ODBC, mediante la cual aplicaciones BI (como Tableau) se pueden conectar a Spark y realizar consultas de estilo SQL.

Tablas en Spark SQL

El elemento de trabajo en Spark SQL son las **tablas**, equivalente a los *dataframes* en la API estructurada. Toda tabla pertenece a una base de datos (*database*) y, si no se especifica ninguna, lo hará a la base de datos por defecto (*default*). Las tablas siempre contienen datos y no existe el concepto de tabla temporal. En su lugar, existen vistas, que no contienen datos. Es importante tener esto en cuenta a la hora de eliminar (*drop*) vistas (no se eliminan datos) y tablas (se eliminan datos).

Otro aspecto que debemos tener en cuenta al crear tablas es si se desea que estas sean gestionadas por Spark (*managed table*) o no (*unmanaged table*). Para entender este concepto, cabe mencionar que una tabla está formada por dos tipos de

Tema 4. Computación distribuida.

Computación paralela

información: los datos que contiene y los metadatos que la describen. Con esto presente, podemos ver cómo se diferencian las **tablas gestionadas y no gestionadas** por Spark:

- ▶ **Tablas no gestionadas por Spark.** Cuando se define una tabla desde ficheros almacenados en disco, lo que se crea es una tabla no gestionada por Spark, dado que los datos ya existían previamente (no son datos nuevos creados usando Spark).
- ▶ **Tablas gestionadas por Spark.** Cuando se guarda un *dataframe* como una nueva tabla (usando, por ejemplo, *saveAsTable* sobre un *dataframe*), entonces se crea una tabla gestionada por Spark, ya que son datos nuevos que necesitan almacenarse y Spark es el encargado de ello.
- ▶ **Tablas externas.** Esta opción existe por compatibilidad con Hive y permite crear tablas no gestionadas por Spark; es decir, los metadatos estarán gestionados por Spark, pero no así los datos. Para ello, se utiliza la consulta `CREATE EXTERNAL TABLE`, tal y como se muestra en el ejemplo a continuación:

```
CREATE EXTERNAL TABLE flights (  
  DEST_COUNTRY_NAME STRING, ORIGIN_COUNTRY_NAME STRING)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION 'data/flight_info/'  
O también desde el resultado de otra tabla:  
CREATE EXTERNAL TABLE flights  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION 'data/flight_info/'  
AS SELECT * FROM flights
```

Tema 4. Computación distribuida.

Computación paralela

Cuando queremos eliminar una **tabla**, usamos la consulta `drop`. Es importante recordar que, en el caso de tablas gestionadas por Spark, se eliminarán tanto los datos como los metadatos, mientras que, si la tabla no está gestionada por Spark, se eliminarán los metadatos (no se podrá volver a hacer referencia a la tabla eliminada), pero los datos originales no (por ejemplo, si la tabla se creó a partir de un fichero, este quedará intacto).

Vistas en Spark SQL

Un elemento auxiliar en Spark SQL son las **vistas**. Una vista especifica un conjunto de transformaciones sobre una tabla existente. Es decir, no son tablas, sino que definen el conjunto de operaciones que se harán sobre los datos almacenados en cierta tabla para conseguir unos resultados. Las vistas se muestran como tablas, pero no guardan los datos en una nueva localización. Sencillamente, cuando se consultan, ejecutan las transformaciones definidas en ellas sobre la fuente de los datos. Por ejemplo, el siguiente ejemplo crea una vista:

```
CREATE VIEW flights_view AS
SELECT * FROM flights
WHERE dest_country_name = 'Spain'
```

La **vista** no contiene las filas de la tabla `flights` cuyo destino sea `Spain`, sino que únicamente almacena el plan de ejecución necesario para obtener esas filas de la tabla origen, de forma que las pueda mostrar cada vez que sea consultada. Si lo pensamos, una vista no es más que una transformación en Spark que nos devuelve un nuevo *dataframe* a partir de otro *dataframe* de origen; por tanto, no se ejecutará hasta que se haga una consulta que requiera obtener la vista. El principal beneficio es que evita escribir datos en disco repetidamente (como sería el caso de crear nuevas tablas).

Tema 4. Computación distribuida.

Computación paralela

Existen diferentes **tipos de vistas**:

- ▶ **Vistas estándar**, que están disponibles de sesión en sesión, como la que veíamos en el ejemplo previo.
- ▶ **Vistas temporales**, que solo están disponibles en la sesión actual.

```
CREATE TEMP VIEW flights_view AS
SELECT *
FROM flights
WHERE dest_country_name = 'Spain'
```

- ▶ **Vistas globales**, que son accesibles desde cualquier lugar de la aplicación Spark y no pertenecen a ninguna base de datos en concreto, pero se eliminan al final de la sesión.

```
CREATE GLOBAL TEMP VIEW flights_view AS
SELECT *
FROM flights
WHERE dest_country_name = 'Spain'
Al crear una vista, podemos indicar que reemplace otra existente:
CREATE OR REPLACE TEMP VIEW flights_view AS
SELECT *
FROM flights
WHERE dest_country_name = 'Spain'
```

Cuando ejecutamos la **sentencia** para crear una vista, vemos que no ocurre nada. Recordemos que dicha vista no se crea (no se ejecutan las transformaciones asociadas) hasta que se hacen consultas sobre ella, como, por ejemplo:

```
SELECT *
FROM flights_view; # ahora es cuando se ejecuta la vista.
```

Respecto a las consultas que se pueden hacer con las vistas, son las habituales de una tabla. Finalmente, podemos descartar una vista usando **DROP** :

```
DROP VIEW IF EXISTS flights_view;
```

Tema 4. Computación distribuida.

Computación paralela

4.4. Spark III

Spark MLlib

Spark MLlib es el módulo de Spark para **tareas** de:

- ▶ Limpieza de datos.
- ▶ Ingeniería de variables (creación de variables desde datos en crudo).
- ▶ Aprendizaje de modelos sobre *datasets* muy grandes (distribuidos).
- ▶ Ajuste de parámetros y evaluación de modelos.

No proporciona métodos para despliegue en producción de modelos entrenados. En la actualidad, es muy frecuente desplegar **modelos como microservicios**. Estos usan el modelo entrenado para realizar predicciones sobre un nuevo ejemplo, el cual reciben por medio de una llamada HTTP de una aplicación externa que les ha enviado el dato para predecir. Comentaremos más detalles en la próxima sección.

La esencia de **MLlib** es la implementación de modelos de manera distribuida utilizando Spark. Dichos modelos son capaces de aprender sobre *datasets* muy grandes almacenados de manera distribuida. No obstante, también es muy frecuente utilizar solo ciertas funcionalidades de MLlib para preprocesar variables en *datasets* masivos, limpiar, normalizar, etc., y, finalmente, para filtrar y pasar el *dataset* resultante (asumiendo que ya no es masivo) al *driver*, a fin de guardarlo en el sistema de archivos local. Posteriormente, se puede utilizar una biblioteca de *machine learning* no distribuida, como, por ejemplo, scikit-learn de Python o paquetes específicos de R, como caret o e1071. Tanto Python como R son capaces de leer ficheros no distribuidos en formato texto o CSV, entre otros, y de usarlos como entrada para un algoritmo de aprendizaje automático.

Tema 4. Computación distribuida. Computación paralela

La Figura 6 muestra una idea general del **ciclo de ajuste** de un modelo, junto a las **herramientas** que proporciona Spark para cada etapa (en color rojo). Además, Spark tiene una herramienta adicional, llamada *pipelines*, para encapsular todas estas etapas en un solo objeto indivisible y asegurarnos de que los nuevos datos recibidos, y con los que queremos realizar predicciones, pasen por exactamente las mismas etapas de preprocesamiento que el *dataset* estático con el que se entrenó el modelo. Por eso, la Figura 6 se refiere a estos pasos como un *all-in-one pipeline*.

La **etapa de preprocesamiento** incluye, además de las operaciones típicas de un proyecto de *data science*, cierto procesamiento específico de Spark. La finalidad es pasar los datos al formato adecuado de columnas que esperan recibir las implementaciones de cada algoritmo en la API de Spark ML. MLib incorpora herramientas para esto también.

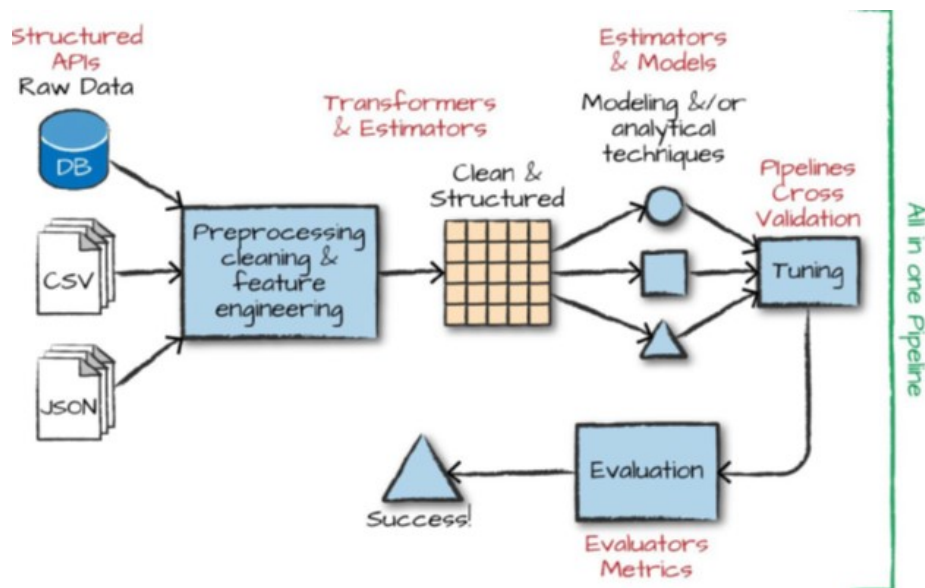


Figura 6. Etapas en el proceso de entrenamiento de un modelo a partir de datos. Fuente: Chambers y Zaharia, 2018.

Tema 4. Computación distribuida.

Computación paralela

Despliegue de modelos en producción con Spark

Spark no fue concebido para explotación *online* de modelos entrenados, es decir, para dar respuestas rápidas (predicciones) a un ejemplo nuevo que se recibe, por ejemplo, desde un sitio web. Por el contrario, la fortaleza de Spark está en entrenar **modelos en modo *batch* (*offline*)** con conjuntos de datos muy grandes.

Aun así, hay varias formas de aprovechar el **modelo entrenado** obtenido por Spark:

- ▶ Entrenar con datos *offline* almacenados en HDFS (el proceso de creación de variables y entrenamiento llevará cierto tiempo) y, justo después, usar el modelo entrenado para predecir (también en modo *batch*) otro conjunto de datos nuevos. La etapa de **predicción** es mucho más rápida que la de entrenamiento.
- Es un enfoque muy frecuente. Es posible cuando los **datos** que hay que predecir (excepto la columna objetivo) ya se conozcan en el momento de entrenar, por ejemplo, series temporales para predecir una ventana a futuro.
- Las **predicciones** precalculadas se almacenan en HDFS o en bases de datos indexadas para que sea muy rápido servir las desde un microservicio.
- ▶ También es posible hacer una **predicción en *batch* en otro momento** que no sea inmediatamente después del entrenamiento, sino cuando tengamos suficientes datos nuevos (un nuevo lote, considerable) sobre los que predecir.
- ▶ Entrenar, guardar el modelo entrenado y usarlo desde Spark para hacer **predicciones una a una**. Resulta poco recomendable lanzar un *job* para cada ejemplo que predecir implica sobrecarga. Balanceo de carga con réplicas del modelo.
- ▶ **Entrenar y exportar el modelo a un formato de intercambio**. Por ejemplo, PMML, para leerlo y explotarlo con otra herramienta no distribuida (Python en especial, aunque también R soporta archivos en formato PMML).

Tema 4. Computación distribuida.

Computación paralela

- ▶ **Entrenar en modo *online* usando *structured streaming* para recoger datos.**

Exige reentrenar desde cero, salvo que el algoritmo esté preparado para entrenamiento incremental (modelos de *online learning*, que, en la actualidad, son una minoría).

Estimadores y transformadores

Antes de describir en detalle la API del módulo Spark MLlib, hay que tener en cuenta que, en la API de Spark (Java/Scala/Python), se distinguen:

- ▶ Paquete `org.apache.spark.mllib` (en Python: `pyspark.mllib`): API antigua basada en RDD de una estructura llamada `LabeledPoint`: `LabeledPoint(etiqueta, [vector de atributos])`. Obsoleta, no debe usarse.
- ▶ Paquete `org.apache.spark.ml` (en Python: `pyspark.ml`): API actual, sobre *dataframes*. En la medida de lo posible, se debe utilizar siempre.
- ▶ Casi todo el contenido del módulo `spark.mllib` ya está migrado al módulo `spark.ml`, excepto algunas clases en métricas de evaluación y algún algoritmo de recomendación.

Para la creación de variables y el preprocesamiento, en general, se utiliza la API de Spark SQL. No obstante, Spark ML ofrece una **transformación** en la que puede escribirse código SQL arbitrario e incluir dicha transformación en un *pipeline*. Además, ciertas transformaciones relacionadas con el preprocesamiento estadístico de datos (normalización, estandarización, codificación *one-hot*, etc.) también están disponibles en Spark ML.

Por último, existen varias **transformaciones** cuyo propósito no es realmente modificar los datos, sino prepararlos (dar a las columnas del *dataframe* los tipos adecuados) para la entrada de los algoritmos de ML de Spark. En concreto, Spark requiere los formatos mencionados en la Tabla 1.

Tema 4. Computación distribuida.

Computación paralela

Problemas de clasificación (clase codificada como número real)		Problemas de regresión (el <i>target</i> ya es un número real)	
Features	Label	Features	Label
[-32.2, 4.5, 1.0, 6.7]	1.0	[-32.2, 4.5, 1.0, 6.7]	0.72
[-40.8, 2.25, 4.0, 2.3]	0.0	[-40.8, 2.25, 4.0, 2.3]	-4.56

Problemas de clustering (no existe la columna <i>label</i>)		Problemas de recomendación (con filtrado colaborativo)		
Features		User	Item	Rating
[-32.2, 4.5, 1.0, 6.7]		2	3	4.5
[-40.8, 2.25, 4.0, 2.3]		1	19	3.21

Tabla 1. Formatos de Spark. Fuente: elaboración propia.

Como puedes ver, en todos los algoritmos, los valores de las **variables** deben presentarse en una sola columna de tipo vector. Por otro lado, si se trata de un problema de aprendizaje supervisado (sea de clasificación o de regresión), la columna *target* debe ser siempre de tipo real (*double*). En el caso de los problemas de clasificación, cada clase se indica con un número real, que ha de empezar en 0.0 y con la parte decimal del número siempre a 0 (es decir, si tenemos un problema con cinco clases, se tienen que codificar como 0.0, 1.0, 2.0, 3.0 y 4.0).

En relación con los **problemas de regresión**, la columna *target* puede ser cualquier número real. Los nombres de las columnas no tienen por qué ser *features* y *label*, como en los ejemplos de arriba, sino que pueden ser cualesquiera, siempre que le indiquemos al algoritmo cómo se llama la columna (de tipo vector) que contiene las *features* en el *dataframe* que le pasamos para entrenar y cómo se llama la columna *target*, si la hay.

Si hablamos de **algoritmos de recomendación**, Spark solo permite el filtrado colaborativo. En ese caso, hay que pasarle un *dataframe* que contenga, al menos, tres columnas con los identificadores de un usuario, un ítem y el *rating* que ha dado dicho usuario a ese ítem, ya sea implícito o explícito. Los identificadores de usuarios

Tema 4. Computación distribuida.

Computación paralela

y de ítems tienen que ser códigos de tipo entero, mientras que el *rating* puede ser un número real en cualquier rango.

Según la documentación oficial de Spark, actualmente nos ofrece las siguientes posibilidades para **preprocesamiento**, divididas en varios grupos:

- ▶ **Extracción:** extraer variables a partir de datos en crudo.
- ▶ **Transformación:** reescalar, convertir o modificar variables.
- ▶ **Seleccionar:** seleccionar un buen subconjunto de variables de otro más grande.
- ▶ **Locality sensitive hashing (LSH):** combinación de transformaciones de variables con otros algoritmos.

Para más detalle, puedes consultar la documentación de Spark.

Apache Spark. (s. f.). *Extracting, transforming and selecting features*.
<https://spark.apache.org/docs/latest/ml-features.html>

Transformadores en Spark ML

Un **transformer** es un objeto que recibe como entrada un *dataframe* de Spark y uno o varios nombres de columna existentes (por ejemplo, `inputCol`), y los transforma de alguna manera. Su salida es el mismo *dataframe* con una nueva columna añadida a la derecha, con el nombre que hayamos indicado en el parámetro correspondiente (generalmente, `outputCol`, pero, a veces, puede ser `predictionCol`).

Tema 4. Computación distribuida.

Computación paralela

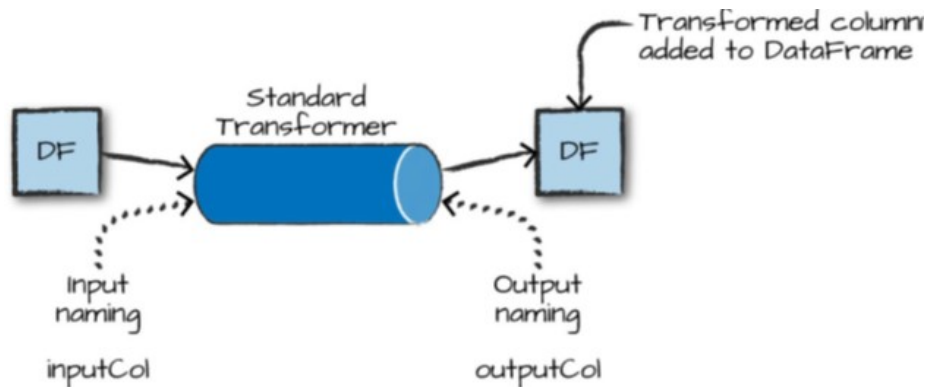


Figura 7. Funcionamiento de un transformador de Spark ML. Fuente: Chambers y Zaharia, 2018.

La **interfaz *transformer*** tiene un único método: `transform(df: dataframe)`, que recibe un *dataframe* y devuelve otro *dataframe*. Los transformadores no necesitan aprender ningún parámetro del *dataframe* de entrada. Simplemente están preparados (tienen toda la información) para transformar un *dataframe*, y esto es lo que hacen cuando llamamos a `transform()`, tal como muestra la Figura 7.

Algunos transformadores habituales

- **VectorAssembler** recibe varias columnas y las concatena en una sola de tipo vector, de longitud igual al número de columnas que se quieran ensamblar. Necesario para calcular la columna (única) de *features* en los algoritmos de aprendizaje supervisado. Por ejemplo:

```
from pyspark.ml.linalg import Vectors
from pyspark.ml.feature import VectorAssembler
dataset = spark.createDataFrame(
    [(0, 18, 1.0, Vectors.dense([0.0, 10.0, 0.51]), 1.0)],
    ["id", "hour", "mobile", "userFeatures", "clicked"])
assembler = VectorAssembler(
    inputCols=["hour", "mobile", "userFeatures"],
    outputCol="features")
output = assembler.transform(dataset)
print("Assembled hour, mobile, userFeatures to column 'features'")
output.select("features", "clicked").show(truncate = False)
```

Tema 4. Computación distribuida.

Computación paralela

id	hour	mobile	userFeatures	clicked
0	18	1.0	[0.0, 10.0, 0.5]	1.0



id	hour	mobile	userFeatures	clicked	features
0	18	1.0	[0.0, 10.0, 0.5]	1.0	[18.0, 1.0, 0.0, 10.0, 0.5]

Figura 8. Ejemplo. Fuente: elaboración propia.

Para cualquier **modelo entrenado**, el resultado de entrenar un modelo sobre un DF es un objeto *model* de la subclase específica del modelo que hayamos ajustado. También es un *transformer*, por lo que es capaz de transformar (hacer predicciones) un DF de ejemplos, siempre que tenga el mismo formato (mismos nombres de columnas y tipos de datos) que el DF que se utilizó para entrenar.

- ▶ Las **predicciones** se añaden junto a cada ejemplo en una nueva columna.
- ▶ Para facilitar que se mantenga el **mismo formato**, se suele entrenar un *pipeline* completo y utilizar su salida (*pipeline* entrenado) como transformador.

```
from pyspark.ml.regression import LinearRegression
from pyspark.ml import Transformer
training = spark.read.format("csv")\
.load("sample_linear_regression_data.csv")
lr = LinearRegression(maxIter=10, regParam=0.3, elasticNetParam=0.8)
lrModel = lr.fit(training)
lrModel.__class__ # comprobamos la clase del modelo ajustado
# Devuelve: <class 'pyspark.ml.regression.LinearRegressionModel'>
pred = lrModel.transform(training)
pred.show()
```

Estimadores en Spark ML

Un **estimator** es un objeto de Spark capaz de realizar transformaciones que primero requieren que ciertos parámetros de la transformación se ajusten (o se *aprendan*) a partir de los datos. Normalmente precisan una pasada previa (o varias) sobre la columna de datos que se desea transformar.

Tema 4. Computación distribuida.

Computación paralela

La interfaz *estimator* tiene un único **método**, `fit(df: dataframe)`, que recibe un *dataframe* y devuelve un objeto de tipo *model* (el modelo entrenado), que es, además, un *transformer*, tal como explicamos anteriormente. Es importante notar que Spark llama modelo a cualquier cosa que requiera un *fit* previo, no solo a los algoritmos de aprendizaje automático que conocemos como modelos propiamente.

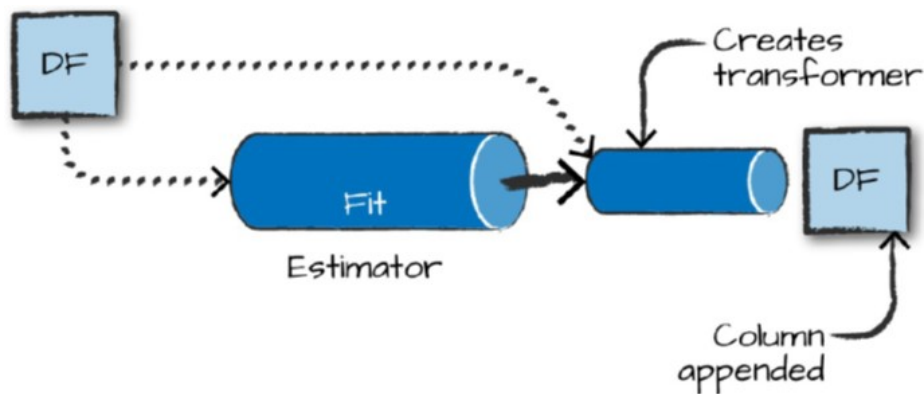


Figura 9. Estimador que genera un transformador. Fuente: Chambers y Zaharia, 2018.

Estimadores más comunes

- ▶ `StringIndexer` es un estimador para preprocesar variables categóricas. Es el más utilizado. Convierte una columna (de cualquier tipo, ya que los valores serán interpretados como categorías) en números reales (*double*), con la parte decimal a 0 y empezando en 0.0. Las categorías se representan mediante 0.0, 1.0, 2.0, etc.
- Además, añade metadatos al *dataframe* transformado devuelto por `transform()`, con los que indica que esa columna es categórica y no como cualquier otra columna numérica. Esta información es útil para los algoritmos.
- Los algoritmos que sí soportan variables categóricas (ejemplo: `DecisionTree`, `RandomForest`, `GradientBoostedTrees`) requieren que estas las pasemos indexadas.
- Los algoritmos que no soportan variables categóricas (`LinearRegression`, `LogisticRegression`) requieren el uso de `OneHotEncoder`, tal como veremos.

Tema 4. Computación distribuida.

Computación paralela

Es importante recordar que, al emplear un **modelo entrenado de *machine learning*** para predecir ejemplos nuevos, primero hay que codificar sus variables categóricas, siguiendo exactamente la misma codificación que se utilizó con los datos de entrenamiento con los que se entrenó el modelo. Por eso, cobra sentido la estructura de *pipeline*, que comentaremos más adelante.

```
from pyspark.ml.feature import StringIndexer
df = spark.createDataFrame(
    [(0,"a"), (1,"b"), (2,"c"), (3,"a"), (4,"a"), (5,"c")], ["id", "category"])
indexer = StringIndexer(inputCol =
    "category", outputCol = "categoryIndex")
indexed = indexer.fit(df).transform(df)
indexed.show()
# Guardo el transformer ajustado (indexerModel) para usarlo después:
indexerModel = indexer.fit(df)
indexed = indexerModel.transform(df)
# ... resto del código de nuestro programa. Ahora cargamos nuevos
# datos y los codificamos siguiendo exactamente la misma codificación:
indexedNuevo = indexerModel.transform(datosNuevosDF)
```

- ▶ **OneHotEncoderEstimator** recibe un conjunto de columnas y convierte cada una (de manera independiente) a un conjunto de variables *dummy* con codificación *one-hot*. Cada variable (con n categorías posibles) da lugar a n variables (condensadas en una sola columna de tipo vector), donde, para cada ejemplo, solo una de las n variables tiene valor 1 y el resto son 0. Esto indica cuál es el valor de la categoría presente en ese ejemplo.
- Spark siempre asume que los valores provienen de una indexación previa con **StringIndexer** : obligatoriamente, la columna de entrada debe estar constituida por números reales con la parte decimal a 0.

```
from pyspark.ml.feature import OneHotEncoderEstimator
df = spark.createDataFrame([
    (0.0, 1.0, 2.0),
    (1.0, 0.0, 3.0), # Spark asume que la 3ª col tiene 5 categorías
    (2.0, 1.0, 2.0), # porque el máximo valor que aparece es 4.0;
    (0.0, 2.0, 1.0), # también que vienen de una indexación previa
    (0.0, 1.0, 4.0), # con StringIndexer y, por tanto, empiezan en 0.0
    (2.0, 0.0, 4.0) ],
```

Tema 4. Computación distribuida.

Computación paralela

```
[ "categoryIndex1", "categoryIndex2", "categoryIndex3" ])
encoder = OneHotEncoderEstimator(
    inputCols = ["categoryIndex1", "categoryIndex2", "categoryIndex3"],
    outputCols = ["categoryVec1", "categoryVec2", "categoryVec3"]
)
model = encoder.fit(df)
encoded = model.transform(df)
# La siguiente línea se utiliza porque vamos a convertir vectores
# sparse a dense, ya que el show() de sparse se ve peor en pantalla
from pyspark.ml.linalg import Vectors, VectorUDT
from pyspark.sql import functions as F
toDenseUDF = F.udf(lambda r: Vectors.dense(r), VectorUDT())
encoded.withColumn("categoryVec1", toDenseUDF("categoryVec1"))\
.withColumn("categoryVec2", toDenseUDF("categoryVec2"))\
.withColumn("categoryVec3", toDenseUDF("categoryVec3"))\
.show()
```

categoryIndex1	categoryIndex2	categoryIndex3	categoryVec1	categoryVec2	categoryVec3
0.0	1.0	2.0	[1.0,0.0]	[0.0,1.0]	[0.0,0.0,1.0,0.0]
1.0	0.0	3.0	[0.0,1.0]	[1.0,0.0]	[0.0,0.0,0.0,1.0]
2.0	1.0	2.0	[0.0,0.0]	[0.0,1.0]	[0.0,0.0,1.0,0.0]
0.0	2.0	1.0	[1.0,0.0]	[0.0,0.0]	[0.0,1.0,0.0,0.0]
0.0	1.0	4.0	[1.0,0.0]	[0.0,1.0]	[0.0,0.0,0.0,0.0]
2.0	0.0	4.0	[0.0,0.0]	[1.0,0.0]	[0.0,0.0,0.0,0.0]

Tabla 2. Ejemplo. Fuente: elaboración propia.

- **Cualquier modelo de predicción (ML).** Todos los modelos heredan de *estimator* y el método `fit(df)` lanza el aprendizaje. Los *estimators* suelen tener muchos parámetros configurables antes de *fit* (en algunos *transformers*, también hay parámetros configurables, pero suelen ser menos).

En el caso de los algoritmos de ML, los parámetros que se pueden ajustar antes de entrenar el modelo (aparte de los nombres de columnas necesarios) se denominan **hiperparámetros** y afectan a la manera en la que se desarrolla dicho entrenamiento. Por ejemplo, el hiperparámetro que controla la fuerza de la regularización (λ), el número de iteraciones máximo del algoritmo de descenso en gradiente que se aplicará para aprender los parámetros del modelo o el número de árboles que se ajustarán en un algoritmo RandomForest.

Tema 4. Computación distribuida.

Computación paralela

Pipelines en Spark ML

Es frecuente, en ML, extraer características de datos en crudo (*raw*) y prepararlas antes de llamar a un algoritmo de aprendizaje. Sin embargo, puede ser difícil tener control de todos los pasos de preprocesamiento que hemos llevado a cabo al entrenar, para luego replicarlos de manera exacta en otros conjuntos de datos o en el momento de hacer predicciones para nuevos datos con el modelo entrenado. Por ejemplo, a la hora de **procesar un documento**, hemos de llevar a cabo los siguientes pasos:

- ▶ División en palabras.
- ▶ Procesamiento de palabras para obtener un vector de características numéricas.
- ▶ Preparación de esas características para el formato que requiere el algoritmo elegido en Spark.
- ▶ Finalmente, entrenamiento de un modelo.

Spark nos proporciona un mecanismo para esto, denominado *pipeline*.

*Pipeline de SparkML es la secuencia de etapas (*estimator* o *transformer*) que se ejecutan en un cierto orden para ir transformando un *dataframe*. En un *pipeline* de Spark, la salida de una etapa es entrada para alguna de las etapas posteriores (no necesariamente la inmediatamente siguiente).*

Un *pipeline* es un ***estimator***. El método `fit(df)` de un *pipeline* recorre cada **etapa**: llama a `transform()` si la etapa es un *transformer* o a `fit(df)` y, luego, a `transform(df)`, si es un *estimator*, pasando siempre el *dataframe df* tal como esté en ese punto (con las columnas originales más las que le hayan añadido las etapas previas). Es habitual (pero no obligatorio) que la última etapa del *pipeline* sea un algoritmo de ML,

Tema 4. Computación distribuida. Computación paralela

aunque podría haber varios a lo largo de un *pipeline*. Es importante recordar que un mismo objeto (sea un estimador o un transformador) no puede ser añadido como etapa a dos *pipelines* diferentes.

Veamos un ejemplo. La Figura 10 muestra un *pipeline* antes de llamar a `fit`.

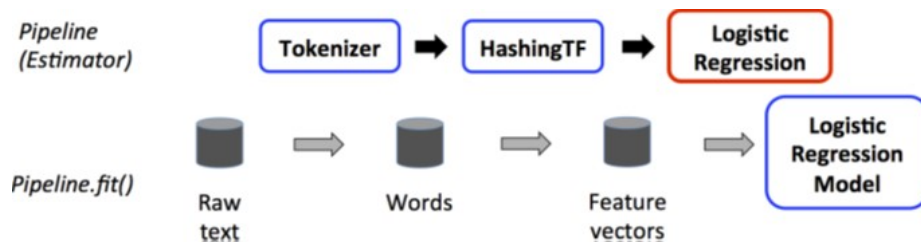


Figura 10. *Pipeline* sin ajustar (arriba) y procesamiento que ocurre al llamar a `fit` (abajo). Fuente: [Apache Spark](#).

Las etapas en azul son transformadores, mientras que las rojas son estimadores. A continuación, vemos el objeto `PipelineModel` (*pipeline* ajustado), donde las etapas que eran estimadores han pasado a ser transformadores. Si ejecutamos el método `transform()` sobre un `PipelineModel`, este irá llamando a `transform` para cada etapa y el DF de la etapa previa devuelto por `transform` pasará como argumento.

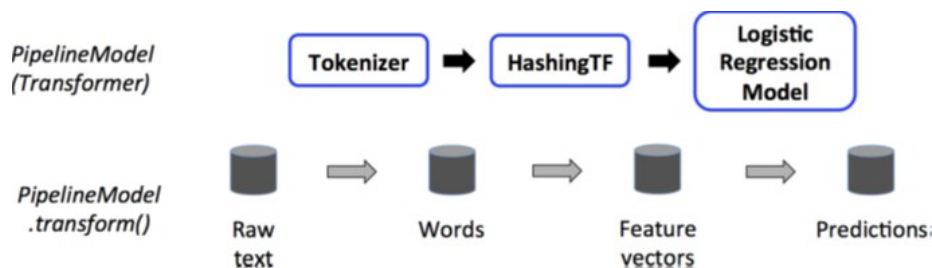


Figura 11. *Pipeline* ajustado (`PipelineModel` , arriba) y procesamiento que ocurre al llamar a `transform` (abajo). Fuente: [Apache Spark](#).

Lo habitual es llamar a `fit` sobre el objeto *pipeline* una sola vez con los datos de entrenamiento. Esto devuelve un objeto `PipelineModel` (modelo ajustado), que es un transformador y sobre el que podemos llamar a `transform` tantas veces como

Tema 4. Computación distribuida.

Computación paralela

queramos, sobre conjuntos de datos nuevos (nunca vistos por el modelo, pero que contengan todas las columnas que esperan cada una las etapas), para realizar predicciones.

El siguiente **ejemplo** lee un conjunto de datos sobre vuelos y tiempo que han llegado retrasados en minutos, los separa en entrenamiento y test, y actúa sobre los datos de entrenamiento: primero, indexa las variables categóricas y las fusiona en una única columna de tipo vector; después, estandariza cada variable por separado, binariza la columna *target* (para convertir el retraso en minutos en una variable binaria: retraso *sí* o *no*) y ajusta un modelo de regresión logística para predecir si un vuelo tendrá o no retraso. Todas las etapas se añaden a un *pipeline* y se efectúa el procesamiento en el momento de llamar a `fit()` sobre los datos de entrenamiento. Una vez que tenemos el *pipeline* entrenado, lo aplicamos para predecir los datos de los test, los cuales seguirán exactamente las mismas etapas.

```
from pyspark.ml.feature import StringIndexer, VectorAssembler,
Binarizer, VectorSlicer, StandardScaler
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression
trainTest = spark.read.parquet("flights.parquet")\
.randomSplit([0.8, 0.2], 12345)
trainingData = trainTest[0] # Dividimos los datos en train y test:
testingData = trainTest[1] # 80 % para entrenar y 20 % para testear.
monthIndexer = StringIndexer().setInputCol("Month")\
.setOutputCol("MonthCat")
dayOfMonthIndexer = StringIndexer().setInputCol("DayOfMonth")\
.setOutputCol("DayOfMonthCat")
dayOfWeekIndexer = StringIndexer().setInputCol("DayOfWeek")\
.setOutputCol("DayOfWeekCat")
uniqueCarrierIndexer = StringIndexer().setInputCol("UniqueCarrier")\
.setOutputCol("UniqueCarrierCat")
originIndexer = StringIndexer().setInputCol("Origin")\
.setOutputCol("OriginCat")
assembler = VectorAssembler()\
.setInputCols([
"MonthCat", "DayOfMonthCat", "DayOfWeekCat",
"UniqueCarrierCat", "OriginCat", "DepTime", "CRSDepTime",
"ArrTime", "CRSArrTime", "ActualElapsedTime", "CRSElapsedTime",
"AirTime", "DepDelay", "Distance"])\
```

Tema 4. Computación distribuida. Computación paralela

```
.setOutputCol("vectorizedFeatures")
# Normalizamos cada variable para tener media 0 y desviación típica 1:
scaler = StandardScaler().setInputCol("vectorizedFeatures")\
.setOutputCol("features")\
.setWithStd(True).setWithMean(True)
binarizer = Binarizer().setInputCol("ArrDelay")\
.setOutputCol("binaryLabel")\
.setThreshold(15.0)
# Algoritmo de machine learning (Estimator) para clasificación:
lr = LogisticRegression().setMaxIter(10)\
.setRegParam(0.3)\
.setElasticNetParam(0.8)\
.setLabelCol("binaryLabel")\
.setFeaturesCol("features")
lrPipeline = Pipeline([
monthIndexer, dayOfMonthIndexer, dayOfWeekIndexer,
uniqueCarrierIndexer, originIndexer, assembler, scaler,
binarizer, lr])
pipelineModel = lrPipeline.fit(trainingData) # ajustar modelos
lrPredictions = pipelineModel.transform(testingData) # predecir
lrPredictions.select("prediction", "binaryLabel", "features").show(20)
```

Spark Structured Streaming

En esta parte, vamos a dar una breve introducción a **Structured Streaming**, el módulo de Spark que ha absorbido al obsoleto Spark Streaming, el cual usaba una estructura de datos llamada DStream, basada en RDD. Actualmente, Spark Structured Streaming utiliza los *streaming dataframes*, que conceptualmente son iguales que un DataFrame, pero a los cuales se les van añadiendo filas automáticamente, en tiempo real, según van llegando. En realidad, no se implementa de esta manera, pero la analogía es válida para razonar sobre *streaming dataframes*. Esto permite disminuir la latencia con respecto a un proceso *batch* que se ejecute periódicamente, ya que es capaz de hacer el cálculo incremental automáticamente en lugar de recalcular siempre todo el resultado partiendo de 0.

El **procesamiento de flujos de datos** (*stream processing*) consiste en incorporar continuamente nuevos datos para actualizar en tiempo real un resultado. Es decir, el cálculo se realiza agregando de alguna forma los datos nuevos a los ya existentes. Esta agregación puede incluir descartar en ciertos momentos los datos más antiguos

Tema 4. Computación distribuida.

Computación paralela

para considerar solo los recibidos en una ventana temporal reciente. Podemos verlo recálculo continuo del resultado, en oposición a lo que ocurre en el procesamiento *batch*, en el que el cálculo se lleva a cabo una sola vez.

Structured Streaming se esfuerza por mantener una API idéntica a los *dataframes*. De hecho, el mismo código que calcula la salida para un *dataframe* convencional (estático) debería funcionar para un *streaming dataframe*. Los conceptos de transformación y acción se mantienen, con una salvedad: la única acción disponible en Structured Streaming es la de comenzar un flujo (`start()`), que iniciará el cálculo y lo ejecutará indefinidamente, actualizando resultados periódicamente.

Fuentes de datos de entradas y salidas permitidas

Spark Structured Streaming permite leer la entrada como flujo de datos desde Kafka; desde un sistema de ficheros como HDFS o Amazon S3, en el que una fuente externa va añadiendo ficheros nuevos a un directorio en tiempo real, y desde una fuente *socket* usable solo para desarrollar y testear. La **lectura** se efectúa con el método `readStream` aplicado al objeto `SparkSession`: `spark.readStream`.

Es importante recordar que, en Structured Streaming, hay que activar explícitamente la opción de inferencia de esquema o bien especificar siempre el **esquema del fichero** de entrada, independientemente del formato. Incluso si es un fichero Parquet, que ya contiene dentro el esquema, es necesario especificar este como argumento si no hemos configurado la propiedad `spark.sql.streaming.schemaInference` a `true` en las opciones de Spark. Si ya se dispone de una versión inicial de los datos guardada, se puede realizar una lectura previa a un *dataframe* convencional (estático), obtener el esquema que se ha leído y usar dicho esquema para el *streaming dataframe*.

La salida puede asimismo escribirse en Kafka, en ficheros y en otras salidas también restringidas para testeo y depuración como, por ejemplo, una salida a «memoria». El **modo de salida** es relevante en estos casos: ¿queremos solamente añadir

Tema 4. Computación distribuida. Computación paralela

información con la salida actualizada o reemplazar completamente el fichero de salida generado en cada actualización? Existen tres modos: añadir, actualizar y reemplazo completo.

Veamos un **ejemplo de código sencillo**. Supongamos que tenemos un directorio de HDFS donde otro proceso externo va creando ficheros en tiempo real, todos con la misma estructura. Cada nuevo fichero incluye información de retrasos sobre un grupo de vuelos que ha aterrizado en diversos aeropuertos recientemente. Asumimos un *dataset* que incluye información sobre los retrasos de vuelos, similar al de ejemplos anteriores. Queremos ver el retraso medio que sufren los vuelos en cada aeropuerto. El código para calcularlo será el mismo que si tuviésemos un *dataframe* estático, pero Spark Structured Streaming irá actualizando automáticamente el fichero de salida agregado en tiempo real.

```
# flights.parquet es una carpeta de archivos, todos con mismo esquema:
staticDF = spark.read.parquet("/path/to/flights/folder")
schema = staticDF.schema
# 1 para que se lea solamente un fichero de la carpeta en cada lectura:
streamingDF = spark.readStream.schema(schema)\
.option("maxFilesPerTrigger", 1) \
.parquet("/path/to/flights.parquet")
# Usamos la API estructurada como haríamos con un DF convencional:
resultDF = streamingDF.where("delay > 15").groupBy("airport").count()
# Invocamos la única acción disponible. Nótese que writeStream no es
query = resultDF.writeStream\ # acción, sino que la acción es start()
.queryName("retrasosPorAeropuerto")\ # ¡un nombre único!
.format("memory")\ # salida a memoria (para pruebas solo)
.outputMode("complete")\ # reemplazo completo
.start() # esto desencadena el cálculo en segundo plano
# IMPRESCINDIBLE para evitar que el driver finalice sin esperarnos:
query.awaitTermination()
```

Tema 4. Computación distribuida. Computación paralela

En el código anterior, **Spark** procesará un fichero, hará la agregación (conteo) y volcará el resultado a la salida indicada, que, en el caso anterior, es memoria. Inmediatamente después de terminar de procesar un fichero, volverá a leer de la carpeta otro distinto. Se irán leyendo de uno en uno los ficheros de la carpeta, debido a que hemos configurado `maxFilesPerTrigger` a 1. Asumimos que existe un proceso externo que está añadiendo en tiempo real ficheros nuevos a esa carpeta y, por este motivo, Spark los va procesando también en tiempo real.

Tema 4. Computación distribuida.

Computación paralela

4.5. MapReduce

Procesamiento de ficheros distribuidos

Como se vio anteriormente, en la **capa de procesamiento** es donde se tratará tanto aquella información que se ha almacenado en la capa de almacenamiento (procesamiento *batch* o lotes) como la información que llega en tiempo real (procesamiento en tiempo real).

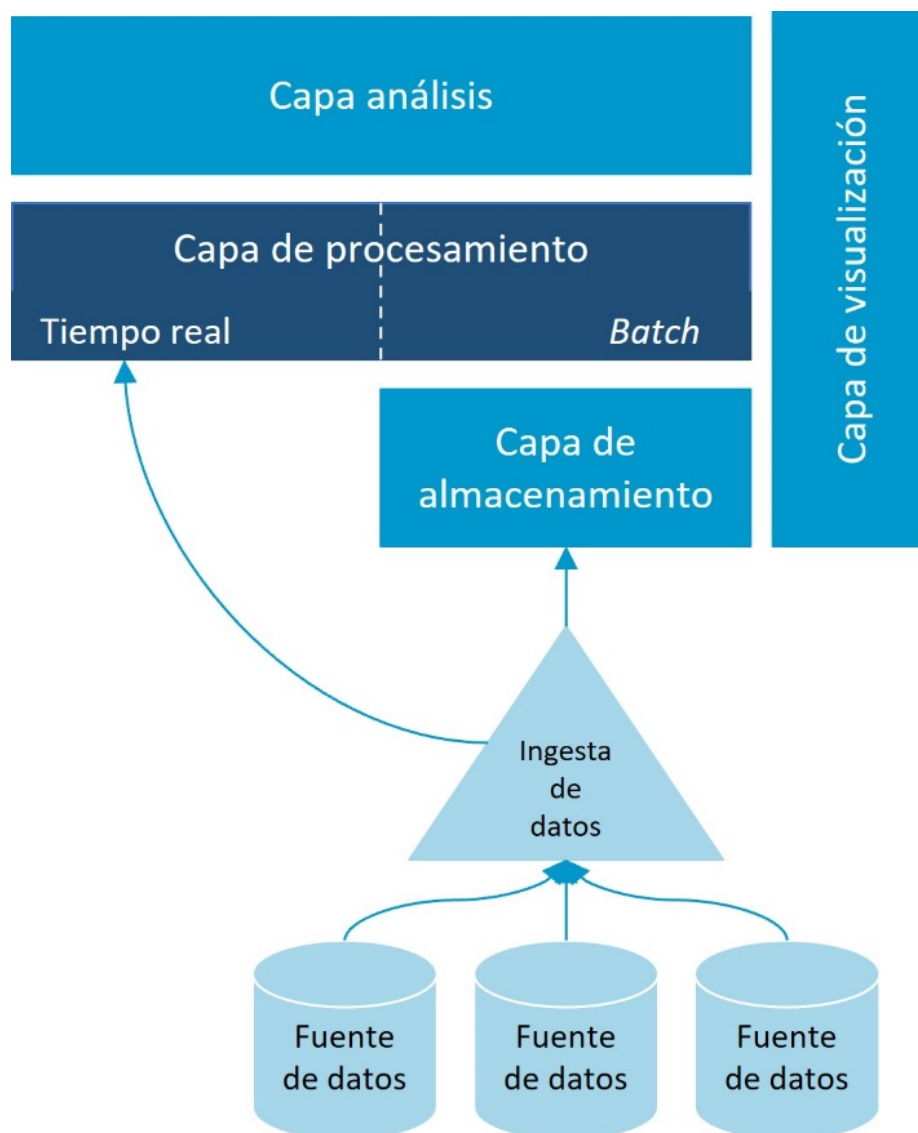


Figura 12. Representación arquitectura *big data*. Capa de procesamiento. Fuente: elaboración propia.

Tema 4. Computación distribuida.

Computación paralela

Dentro de la capa de procesamiento ya se vio que había dos tipos: procesamiento por lotes y procesamiento en tiempo real.

Procesamiento por lotes. MapReduce

MapReduce se encarga de automatizar el procesado de los datos en paralelo a través de distintos nodos de un clúster gestionando las tareas necesarias para la realización del trabajo. También proporciona la capacidad de reconducir el proceso en el momento en que detecta que uno de los nodos falla.

MapReduce fue concebido para una función muy concreta, indexar el contenido de cada página web hasta completar todo el universo de la World Wide Web. A medida que se extendieron las aplicaciones de Hadoop a otras lógicas de negocio diferentes empezaron a aparecer ciertos **problemas** con MapReduce.

Una vez que Google tenía resuelto el problema del almacenamiento de ficheros masivos en el sistema de ficheros distribuido Google File System (GFS), Dean y Ghemawat publicaron un nuevo artículo (2008) sobre cómo aprovechar los DataNodes para procesar estos ficheros que estaban almacenados de forma distribuida, particionados en bloques. Para ello, desarrollaron un **paradigma** de programación llamado MapReduce, que consiste en una manera abstracta de abordar los problemas para que puedan ser implementados y ejecutados sobre un clúster de ordenadores. Junto con el artículo, también liberaron una **biblioteca de programación** en lenguaje Java que implementaba este paradigma.

De forma más precisa, podemos definir MapReduce como un modelo abstracto de programación general e implementación del modelo como bibliotecas de programación, para procesamiento paralelo y distribuido de grandes volúmenes de datos, inspirado en la técnica «divide y vencerás».

Tema 4. Computación distribuida.

Computación paralela

La gran **ventaja** que proporciona tanto el modelo como la implementación que suministraron los autores es que abstrae al programador de todos los detalles relativos a *hardware*, redes y comunicación entre los nodos, con el fin de que pueda centrarse solamente en el desarrollo de *software* para solucionar su problema.

El punto de partida son archivos muy grandes almacenados (por bloques) en HDFS (en aquel momento, aún era GFS). ¿Podríamos aprovechar las CPU de los DataNodes del clúster para procesar en paralelo (simultáneamente) los bloques de un archivo? No queremos mover datos (el tráfico de red es muy lento): llevamos el cómputo (el código fuente de nuestro programa) al lugar donde están almacenados los datos (es decir, a los DataNodes, y no al revés, como ocurría en el modelo tradicional de aplicación). Las **CPU de los DataNodes** van a procesar (preferentemente) los datos que hay en ese DataNode.

En el paradigma MapReduce, el usuario solo necesita escribir dos **funciones** (enfoque «divide y vencerás»):

- ▶ **Mapper.** Es una función escrita por el usuario e invocada por el *framework* en paralelo (simultáneamente) sobre cada bloque de datos de entrada (que generalmente coincide con un bloque de HDFS). De este modo, se generan resultados intermedios, que se presentan siempre en forma de (clave, valor) y son escritos también en el disco local.
- ▶ **Reducer.** Se aplica en paralelo para cada grupo creado por la función Mapper. En concreto, esta función se llama una vez para cada clave única generada de la salida de la función Mapper, junto con la cual se pasan todos los valores asociados que comparte.

Tema 4. Computación distribuida.

Computación paralela

Ejemplo MapReduce: el problema de contar ocurrencias de palabras con MapReduce

Supongamos que queremos resolver un problema que consiste en, dado un texto, saber cuántas veces aparece en él cada palabra. Nuestro punto de partida es un fichero que contiene el texto y que está almacenado en HDFS. Por tanto, está particionado en varios bloques, cada uno de los cuales contiene un fragmento del texto. Asumimos que el texto está almacenado en formato ASCII, es decir, el fichero (y, por extensión, cada uno de sus bloques) contiene líneas de texto, tal como se indica en los rectángulos azules de la **fase input** de la Figura 13.

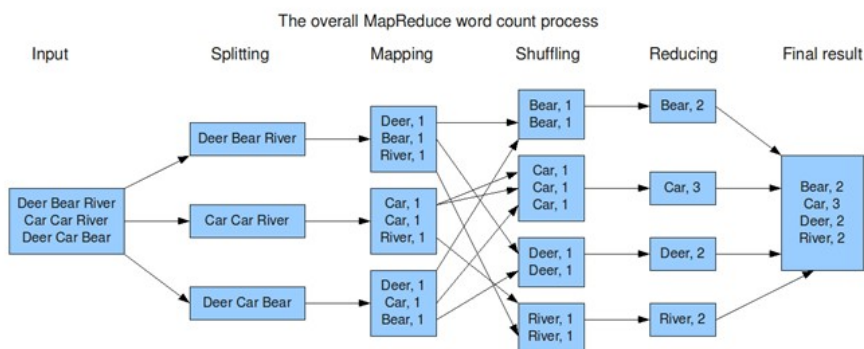


Figura 13. Representación del problema de contar ocurrencias resuelto con MapReduce. Fuente: Lafourcade, 2019.

Tema 4. Computación distribuida.

Computación paralela

En el modelo MapReduce, el programador (usuario de la API) solamente necesita escribir dos **funciones**. En primer lugar, una función llamada `map`, que reciba una línea de texto (cada línea de texto vendrá de la información almacenada en cada bloque de los nodos donde se almacena la información, que representan cada uno un bloque de HDFS. Así de manera paralela se irá leyendo línea a línea en la fase de Mapping, se hará lo que se denomina *splitting* para separar cada palabra de cada frase y con esa información se van generando tuplas (clave/valor) por cada palabra que se recibe de forma que se genere lo siguiente `(palabra_recibida,1)` dando como resultado, las tuplas que se muestran en la **fase de mapping**.

A continuación, las tuplas que genera la fase `map` son enviadas por la red que conecta los nodos del clúster: la **librería de MapReduce**, de forma automática y transparente al programador, lleva a cabo la operación *shuffling*. En esta fase se generan tuplas formadas por una palabra y una lista asociada (más adelante, explicaremos su significado). Lo que está sucediendo es que el propio *framework* está agrupando todas las tuplas que tienen la misma clave y está creando una lista con todos los valores asociados a esa clave común. Esto lo repite por cada clave diferente que encuentre en las tuplas generadas por la función `map` del usuario.

Para cada uno de estos agrupamientos, el *framework* invoca automáticamente a la segunda función que el programador debe escribir: la función `reduce`. Esta es recibida por las tuplas de la fase de *shuffling* (formadas por una palabra y la lista con el número de ocurrencias asociadas a ella) y genera, para cada una de ellas, un resultado como el mostrado en la **fase de reducing**.

La función `map` que implementa el usuario debe recibir como entrada una tupla (`clave_entrada, valor`). En este problema, `clave_entrada` es el número de línea (ignorado) y `valor` representa una línea completa de texto. Es el *framework* el que, de forma automática, invoca, tantas veces como sea necesario, la función que ha implementado el usuario y le pasa los argumentos que hemos indicado. La

Tema 4. Computación distribuida.

Computación paralela

invocación y ejecución de la función `map` se lleva a cabo de manera distribuida, en cada nodo del clúster (en los `DataNodes`, que es donde residen los bloques de HDFS que contienen los fragmentos del texto que actúa como datos de entrada).

La **implementación** de la función `map` del usuario para este problema concreto podría dividir la línea de texto en palabras y, para cada palabra que forma parte de la línea de texto, devolver la tupla `(palabra, 1)`, que indica que «palabra» ha aparecido una vez, por ejemplo, `(hola, 1)`, `(el, 1)`, `(el, 1)`. La palabra es la `out_key` y el valor siempre es 1.

La **función** `reduce` que el usuario ha de implementar es también invocada automáticamente por el *framework*. Para ello, se toman como datos de entrada cada una de las claves y la lista de valores asociados a ellas obtenidas en la etapa anterior. Esta **lista de valores** está formada por el campo valor de todas las tuplas que comparten la misma clave, tal como las ha generado la fase `map`. La implementación de la función `reduce` debe agregar resultados (sumar las ocurrencias de una misma palabra). Por ejemplo, la función `reduce` recibe `(hola, [1, 1, 1, 1, 1, 1, 1])` y da como resultado `(hola, 7)`.

Inconvenientes de MapReduce

MapReduce permite procesar los datos almacenados de manera distribuida en el sistema de archivos (por ejemplo, HDFS) de un clúster de ordenadores. Esto ayuda a resolver todo tipo de problemas, pese a que no siempre es sencillo pensar la solución en términos de operaciones `map` y `reduce` encadenadas.

Tema 4. Computación distribuida.

Computación paralela

En este sentido, se dice que es de propósito general, a diferencia de, por ejemplo, el lenguaje SQL, que está orientado específicamente a realizar consultas sobre los datos. Sin embargo, MapReduce presenta varios **inconvenientes**, algunos muy serios:

- ▶ El **resultado** de la fase `map` (tuplas) se escribe en el disco duro de cada nodo, como resultado intermedio. Los accesos a un disco duro son aproximadamente un orden de magnitud (es decir, diez veces) más lentos que los accesos a la memoria principal (RAM) de cada nodo, por lo que estamos penalizando el rendimiento debido a cómo está estructurado el propio *framework*.
- ▶ Después de la fase `map`, hay **tráfico de red** obligatoriamente (movimiento de datos, conocido como `shuffle`). De hecho, el movimiento de datos forma parte como una etapa obligada en el *framework* de MapReduce. En cualquier aplicación distribuida, el movimiento de datos de un nodo a otro constituye un cuello de botella y es una operación que debe evitarse si es posible.
- ▶ Por otro lado, y relacionado con el punto anterior, para **mover datos** se necesita, en primer lugar, escribirlos temporalmente en el disco duro de la máquina origen, enviarlos por la red y, luego, escribirlos temporalmente en el disco duro de la máquina destino (el `shuffle` siempre va de disco duro a disco duro) para, finalmente, leerlos y pasarlos a la memoria principal de dicho nodo.
- ▶ Estos dos inconvenientes se acentúan especialmente cuando el algoritmo que queremos implementar sobre un clúster es de tipo **iterativo**, es decir, requiere varias pasadas sobre los mismos datos para ir convergiendo. Este tipo de procesamiento es típico en algoritmos de ML, que es uno de los que más se puede beneficiar del procesamiento de grandes conjuntos de datos para extraer conocimiento. Se comprobó que sus implementaciones con MapReduce no eran eficientes debido a la propia concepción del *framework*.

Tema 4. Computación distribuida.

Computación paralela

- ▶ Finalmente, enfocar cualquier **problema** en términos de operaciones `map` y `reduce` encadenadas no siempre es fácil ni intuitivo para un desarrollador, y la solución resultante puede ser difícil de mantener si no está bien documentada. Además, MapReduce trabaja principalmente en Java, aunque tiene alguna librería que permite trabajar con Python también.
- ▶ Además, el **código** que hay que generar es muy complicado y requiere muchas líneas de código para una tarea relativamente simple. Por ejemplo, el código que habría que generar para el ejemplo visto anteriormente sería:

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable>{

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context
            ) throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {
```

Tema 4. Computación distribuida.

Computación paralela

```
private IntWritable result = new IntWritable();
public void reduce(Text key, Iterable<IntWritable> values,
                  Context context
                  ) throws IOException, InterruptedException {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
```

No se pretende que se aprenda que es lo que está haciendo el código, únicamente se busca que se entienda lo complicado que puede llegar a ser además frente a otras tecnologías y que, para hacer exactamente lo mismo, lo pueden ejecutar todo en una única línea.

Arquitectura de MapReduce

Hay dos versiones claramente diferenciadas a la hora de ver la arquitectura de MapReduce, la versión 1.0 y la 2.0 y posteriores. La **diferencia** principal reside en los servicios utilizados en cada caso y en la mejora de rendimiento que hay en la versión 1 con respecto a la versión 2.

Tema 4. Computación distribuida.

Computación paralela

Versión 1 de Hadoop

En la versión original de MapReduce (MapReduce 1) entran en juego dos componentes para realizar la tarea del ejemplo: JobTracker y TaskTracker.

- ▶ **JobTracker**. Es un servicio desarrollado en Java también que se ejecuta en un nodo maestro y es responsable de recoger la petición del cliente y de asignar a los TaskTrackers las tareas concretas que deberán realizar en el proceso. El JobTracker está en el nodo maestro (NameNode), recordemos que es donde se guarda la información de en qué nodo (DataNode) está cada fragmento de cada fichero. Los nodos en los que se van a ejecutar las tareas definidas por el JobTracker serán precisamente los DataNodes, en donde viven los datos. En el caso de que, por cualquier motivo, uno de los nodos fallase el JobTracker se encargaría de redirigir la tarea a otro nodo donde estuviera alojada una réplica de los datos.
- ▶ **TaskTracker**. Se trata de un demonio (*daemon*) que acepta y ejecuta las tareas asignadas: map, reduce y shuffle. Adicionalmente el TaskTracker envía una señal al JobTracker para advertirle de que está vivo y listo para ejecutar la tarea, así como de indicarle como de «ocupado» se encuentra. Una vez iniciada la ejecución de un proceso mantiene un *feedback* continuo de su progreso, llegando incluso el caso de que si una tarea en un nodo se ralentiza demasiado la manda ejecutar en otro nodo que tenga el mismo bloque de fichero sobre el que se está ejecutando.

Tema 4. Computación distribuida. Computación paralela

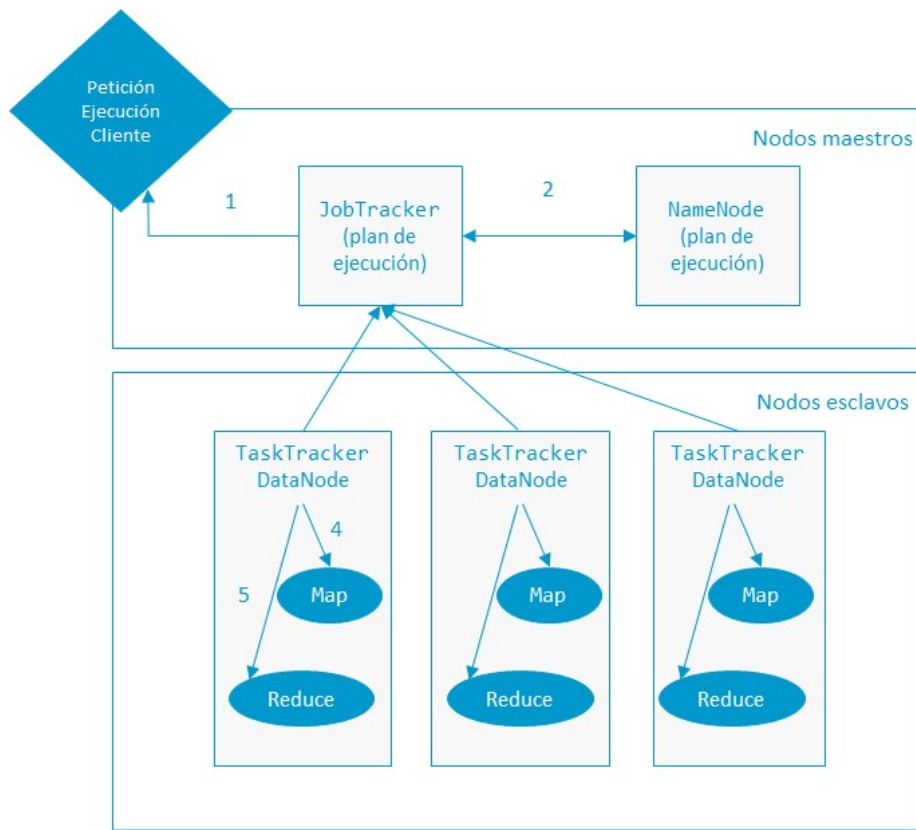


Figura 14. Pasos ejecución MapReduce en versión 1.0 de Hadoop. Fuente: elaboración propia.

- ▶ Un cliente, mediante un programa en MapReduce, envía un trabajo a realizar al JobTracker, en el ejemplo anterior era un conteo de las palabras en una serie de frases.
- ▶ El JobTracker obtiene del NameNode la ubicación de los bloques de los datos en los DataNodes.
- ▶ A partir de ahí el JobTracker manda el programa del cliente en HDFS y asigna las tareas que tienen que realizar los TaskTrackers en los nodos en los que ha localizado los datos.
- ▶ El TaskTracker inicia las tareas Map en cada DataNode trasladando el programa del cliente de la ubicación compartida HDFS al nodo donde se van a ejecutar las tareas (los tres nodos que trabajan con cada una de las frases en el diagrama). Cuando se

Tema 4. Computación distribuida.

Computación paralela

finaliza la tarea de mapear (Map) en todos los nodos se genera un fichero intermedio a nivel local en el nodo del TaskTracker . Este fichero es el que se pasa a la tarea de Reduce.

- El proceso de `reduce` trabaja con todos los datos recibidos de las tareas de mapeo (Map) y, una vez procesado, escribe el *output* final en el sistema de ficheros HDFS. Al finalizar borra los datos intermedios generados por el TaskTracker en los distintos nodos.

Una de las diferencias claves con otros sistemas de gestión de datos es que, como acabamos de ver, en Hadoop, llevamos el programa hasta el nodo en el que se alojan los datos y no al revés. Este cambio de filosofía es precisamente el que permite la **eficiencia del procesamiento distribuido** de los datos en Hadoop.

Pero esta arquitectura tenía un problema en la **escalabilidad** y es que el Jobtracker podía llegar a ser un problema, ya que tenía que ocuparse de saber cómo estaban todos los Tasktrackers de «ocupados» para poder mandarles o no más tareas y a la vez tenía que estar pendiente de todas las ejecuciones que se estuvieran realizando para por ejemplo ver si era necesario relanzarlas. Cuando el número de nodos crecía en el clúster, el rendimiento bajaba bastante debido a lo comentado.

MapReduce fue concebido para una función muy concreta, indexar el contenido de cada página web hasta completar todo el universo de la World Wide Web. A medida que se extendieron las aplicaciones de Hadoop a otras lógicas de negocio diferentes empezaron a aparecer ciertos problemas con MapReduce.

Adicionalmente MapReduce no aportaba ninguna **alternativa** para la gestión de los recursos disponibles, solo permitía usar su *framework*; un *framework* que estaba concebido para procesos en lotes (*batch*). Con la aparición de los nuevos modelos de programación, se hizo patente la necesidad de soportar otros paradigmas de programación (Impala, Spark...) y una mejor forma de gestionar los recursos.

Tema 4. Computación distribuida.

Computación paralela

Versión 2 de Hadoop. YARN

En la versión 2 de MapReduce, se introduce un nuevo gestor de recursos llamado **YARN** (*yet another resource negotiator*). La llegada de YARN sobre MapReduce cambia el funcionamiento de esta arquitectura mejorando bastante su rendimiento.

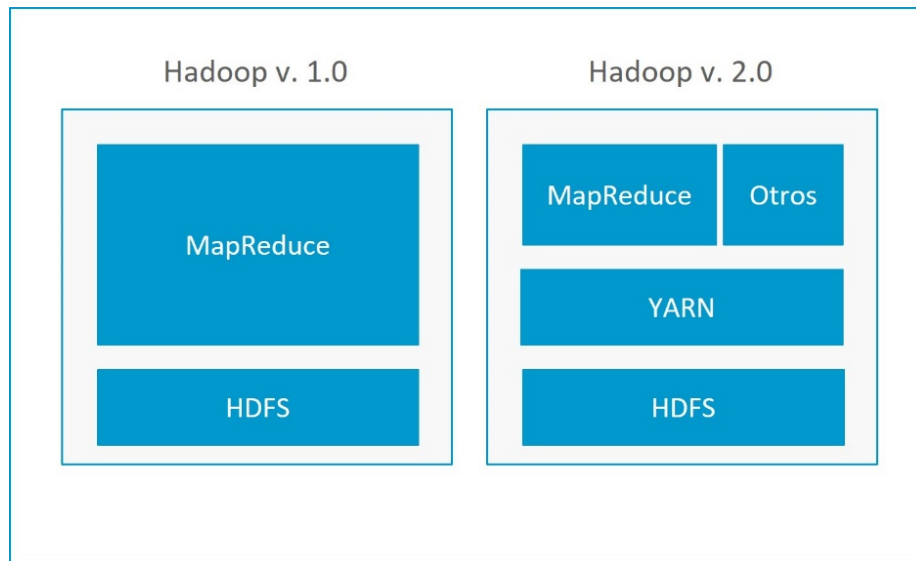


Figura 15. Diferencia arquitectura Hadoop v. 1.0 vs. v. 2.0. Fuente: elaboración propia.

- ▶ YARN (*yet another resource negotiator*): **evolución de MapReduce**.
- ▶ Fue introducido en **Hadoop 2** para mejorar MapReduce y permitir diferentes tipos de procesamiento de datos: Tez, Spark, MapReduce... no solo *batch*.
- ▶ YARN es el encargado de **gestionar un clúster**, lo que quiere decir, que se encarga de gestionar los recursos (memoria, disco, procesos).
- ▶ YARN **expone una serie de API** que son aprovechadas por *frameworks* de cómputo, como MapReduce, Spark, etc. (todos ellos pueden ejecutar sobre YARN), es decir ya no es necesario utilizar MapReduce para acceder a HDFS, lo que aprovecharon muchas aplicaciones.
- ▶ Tiene **procesos distintos** para el proceso de datos y para la gestión del clúster.

Tema 4. Computación distribuida.

Computación paralela

Ejemplo de la vida real que muestra la potencia de Hadoop 2.0 sobre Hadoop 1.0

- ▶ Yahoo fue capaz de ejecutar casi el doble de *jobs* al día con YARN que con Hadoop 1.0.
- ▶ También experimentaron un aumento considerable en la utilización de la CPU.
- ▶ Yahoo incluso llegó a afirmar que la actualización a YARN equivalía a añadir 1000 máquinas a su clúster de 2500 máquinas

En YARN siguen existiendo los dos **procesos**: Map y reduce, sin embargo, la manera en que se gestionan los recursos del ecosistema es completamente diferente. Para realizar un trabajo YARN, proporciona una nueva arquitectura en la que se modifica sustancialmente la forma en que se distribuyen las tareas entre los nodos. Para ello, YARN incorpora los siguiente nuevos elementos a la ecuación:

- ▶ ResourceManager (uno por cada clúster) el responsable de organizar los recursos de los nodos esclavos a la medida de las necesidades de cada tarea. Se encarga de iniciar los trabajos (*jobs*) en el clúster maestro a petición de los usuarios (cliente). Puede sonar parecido a lo que hacíamos con el JobTracker en la versión uno de MapReduce, pero, como veremos más adelante, hay grandes diferencias.
- ▶ NodeManager (uno por cada nodo esclavo) sustituye la función que realizaban los TaskTrackers en la versión anterior. Son los responsables de iniciar los procesos de una aplicación en particular y además se encargan de solicitar los recursos necesarios al ResourceManager garantizando que las tareas asignadas no superen la capacidad de los recursos disponibles en el nodo.
- ▶ Job History Server (uno por clúster) es un proceso opcional cuya función es mantener un archivo de los logs del trabajo (*job*) ejecutado.

Tema 4. Computación distribuida.

Computación paralela

En YARN, el cliente inicia un trabajo (*job*) a través del `ResourceManager` (gestor de recursos), que se encarga de gestionar los recursos necesarios. Cuando el **gestor de recursos** manda una tarea a un nodo esclavo este necesita asignar una serie de sus recursos para la realización de la tarea. El gestor de recursos es el encargado de asignar los recursos a través de la creación de contenedores (*container*).

Un **contenedor** dedica una parte de la memoria y de la capacidad de la CPU del nodo esclavo para realizar la tarea. En un mismo nodo pueden convivir varios contenedores a un mismo tiempo y, por lo tanto, compartir una parte de los recursos del nodo.

YARN introduce también el concepto de ***application master*** (maestro de aplicaciones), un sistema que controla las aplicaciones, que es otra manera de llamar a los trabajos o las consultas que realizan los usuarios. Hay un maestro de aplicaciones por cada trabajo que se realiza. El maestro de aplicaciones también vive en un contenedor, es decir, también se le asignan unos recursos en el nodo esclavo.

El maestro de aplicaciones es el responsable de solicitar los **recursos** de los contenedores al gestor de recursos. Una vez que este asigna los recursos al contenedor de un nodo esclavo es cuando se empiezan a ejecutar las tareas. El `NodeManager` se encarga de supervisar el proceso y de validar el *output* resultante. Como ya se ha mencionado un mismo nodo esclavo puede gestionar varios contenedores a la vez, pero cada tarea se ejecutará siempre en un contenedor distinto.

El **maestro de aplicaciones** corre en uno de los nodos esclavos, sin embargo, recurre al gestor de recursos en el nodo maestro para solicitar recursos no solo para el nodo en que se ejecuta, también se encarga de solicitar los recursos (contenedores) para aquellos nodos donde están alojados los bloques de los datos (`DataNode`) con los que va a trabajar a través del `NodeManager` correspondiente.

Tema 4. Computación distribuida.

Computación paralela

Esta forma de llevar la computación a donde residen los datos es lo que se denomina **data locality** y es lo que permite a YARN minimizar la congestión de la red e incrementar el rendimiento de todo el sistema.

Ejemplo de ejecución de procesos de MapReduce en la versión 2 de Hadoop ejecutándose desde varios clientes

Al empezar tienen que estar arrancados los servicios del ResourceManager como el proceso maestro y el NodeManager en cada máquina esclava.

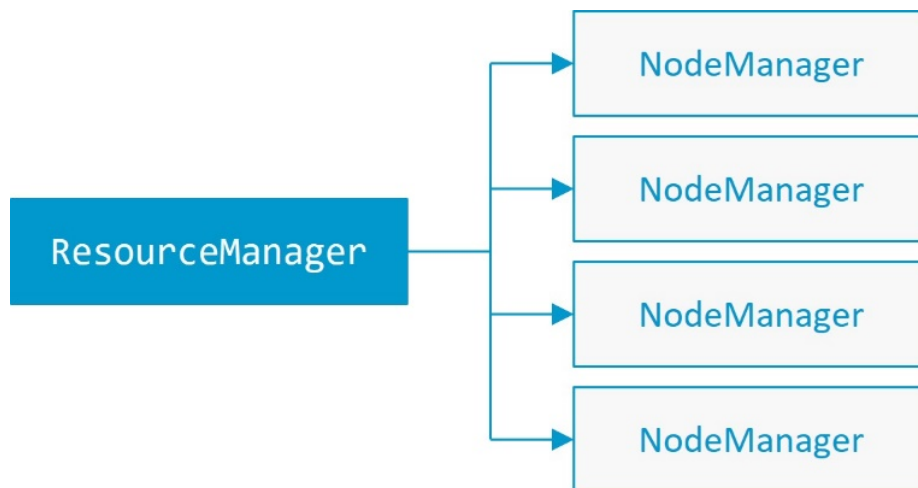


Figura 16. Proceso maestro y máquinas esclavas. Fuente: elaboración propia.

- **Paso 1.** El cliente lanza un proceso de MapReduce, llega la petición al ResourceManager y se crea un nuevo servicio, App Master, en uno de los nodos que será encargado de lanzar el proceso en los nodos disponibles y de controlar la evolución de dichos procesos.

Tema 4. Computación distribuida. Computación paralela

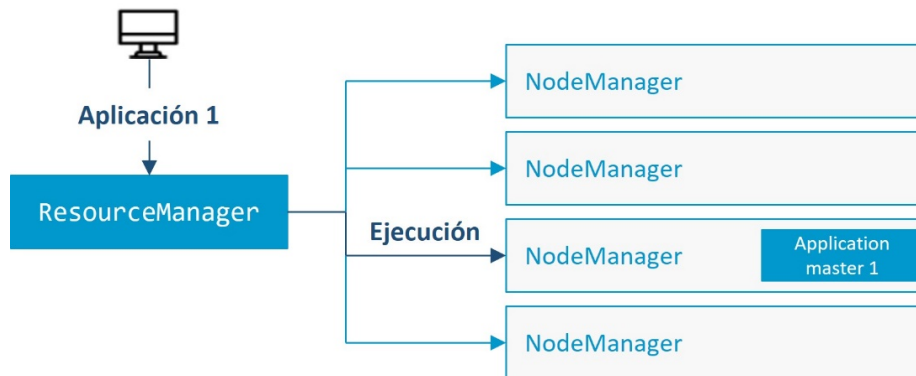


Figura 17. Paso 1. Fuente: elaboración propia.

- **Paso 2.** En nuevo servicio, pregunta al ResourceManager por la disponibilidad de los *slots* de los NodeManager y este le devuelve un listado con los ID de los contenedores donde puede ejecutar el proceso

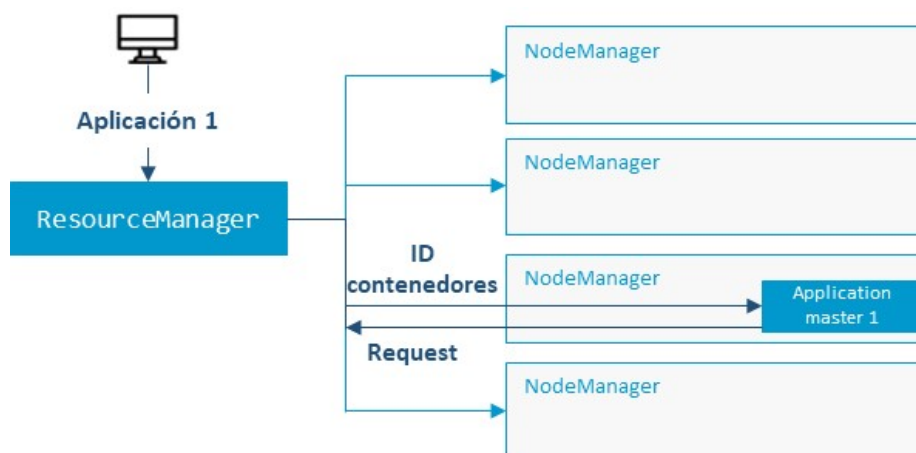


Figura 18. Paso 2. Fuente: elaboración propia.

- **Paso 3.** El servicio App Master lanza el proceso en los *slots* de los contenedores asignados.

Tema 4. Computación distribuida.

Computación paralela

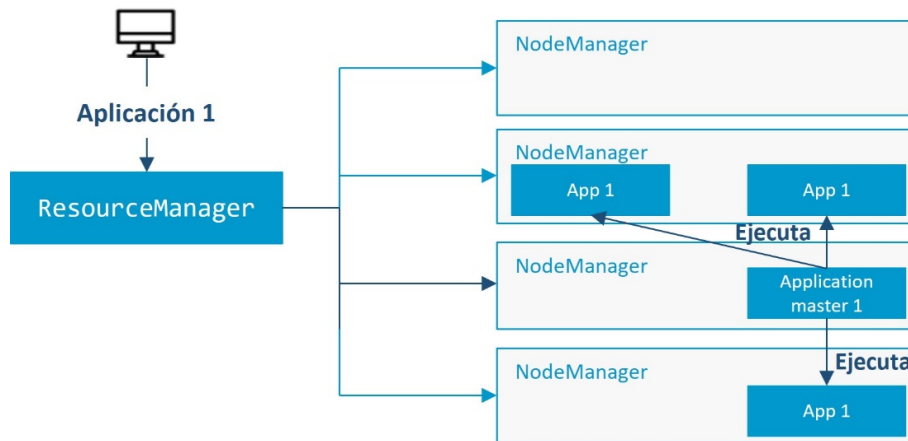


Figura 19. Paso 3. Fuente: elaboración propia.

- **Paso 4.** Otro cliente lanza un nuevo trabajo de MapReduce. Se genera un nuevo App Master que controlará la ejecución de ese nuevo proceso.

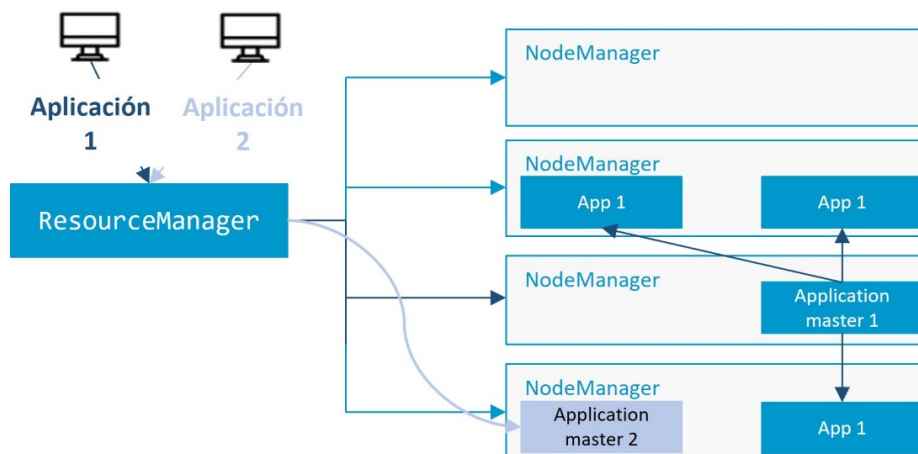


Figura 20. Paso 4. Fuente: elaboración propia.

- **Paso 5.** El App Master solicita los contenedores disponibles al ResourceManager.

Tema 4. Computación distribuida. Computación paralela

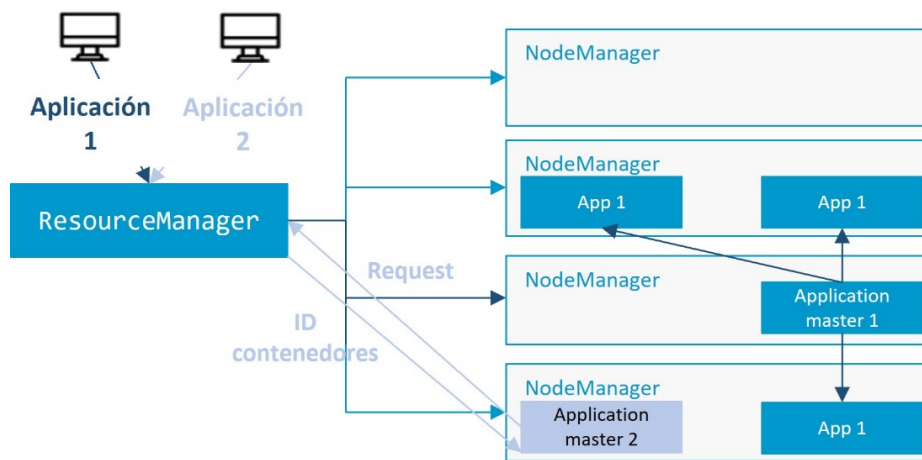


Figura 21. Paso 5. Fuente: elaboración propia.

- **Paso 6.** Con los contenedores obtenidos, lanza de nuevo en los *slots* de esos nodos el nuevo proceso.

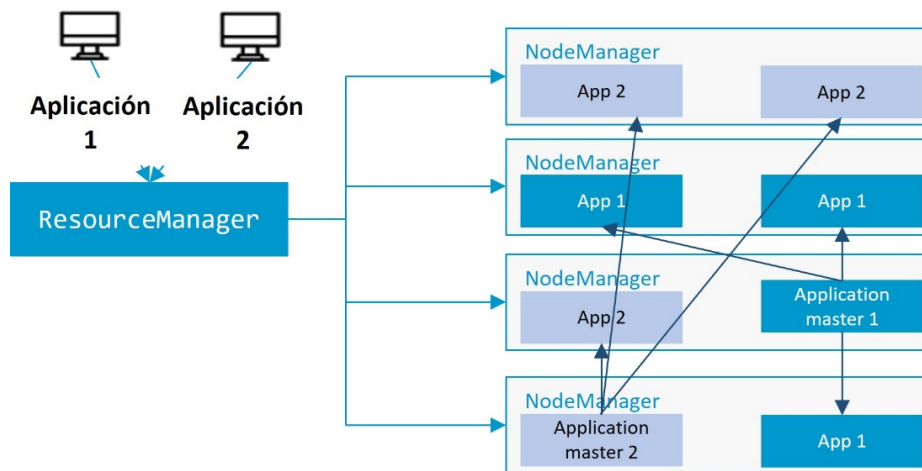


Figura 22. Paso 6. Fuente: elaboración propia.

Cuando todos los procesos finalizan, todos los servicios App Master creados se detienen y únicamente vuelven a quedar el ResourceManager y los NodeManagers (Figura 16).

Tema 4. Computación distribuida.

Computación paralela

Una de las **diferencias** claves con otros sistemas de gestión de datos es que, como acabamos de ver, en Hadoop llevamos el programa hasta el nodo en el que se alojan los datos y no al revés. Este cambio de filosofía es precisamente el que permite la eficiencia del procesamiento distribuido de los datos en Hadoop.

Ecosistema Hadoop

El **ecosistema Hadoop** no es un lenguaje de programación ni un servicio, es una plataforma o marco de trabajo que resuelve problemas de *big data*. Puedes considerarlo como una suite que abarca una serie de servicios (ingesta, almacenamiento, análisis y mantenimiento) en su interior.

A continuación, se muestran algunos de los componentes de Hadoop, que juntos forman lo que se denomina el ecosistema de Hadoop:

- ▶ Facilitar procesamiento o ETL: Pig .
- ▶ *Data warehousing* sobre Hadoop: Hive .
- ▶ NoSQL BigTable sobre Hadoop: HBase .
- ▶ Facilitar ingestión de datos: Sqoop , Flume .
- ▶ ML sobre Hadoop: Mahout .
- ▶ Orquestación: Oozie .
- ▶ Facilitar la coordinación del clúster: Zookeeper .

Tema 4. Computación distribuida. Computación paralela

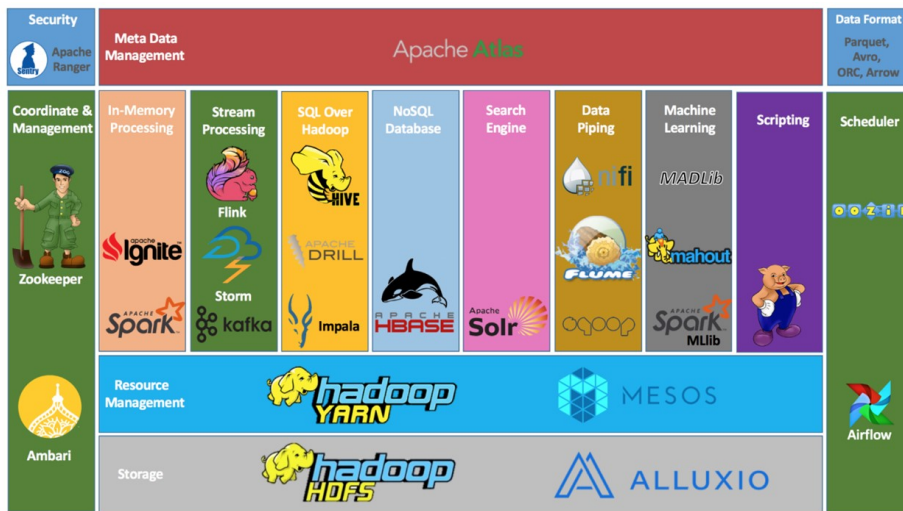


Figura 23. Ecosistema Hadoop. Fuente: Mete, 2019.

Tema 4. Computación distribuida.

Computación paralela

4.6. Referencias bibliográficas

Chambers, B. y Zaharia, M. (2018). *Spark: the definitive guide*. O'Reilly Media.

Datastrophic. (2016, marzo 3). *Apache Spark: core concepts, architecture and internals*. <http://datastrophic.io/core-concepts-architecture-and-internals-of-apache-spark/>

Dean, J. y Ghemawat, S. (2008). MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1), 107-113. <https://doi.org/10.1145/1327452.1327492>

Feregrino. (2018, febrero 20). *¿Qué es MapReduce?* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=b27hWyhq9oo>

Ghemawat, S., Gobioff, H. y Leung, S-T. (2003). The Google File System. *A CM SIGOPS Operating Systems Review*, 37(5), 29-43. <https://doi.org/10.1145/1165389.945450>

Grishchenko, A. (2019, marzo 23). *Hadoop: the end of an era*. Distributed Systems Architecture. <https://0x0fff.com/hadoop-the-end-of-an-era/>

Karau, H. y Warren, R. (2017). *High performance Spark*. O'Reilly Media.

Lafourcade, O. (2019, mayo 17). *Whats up with MapReduce? A concise summary in Spanish*. Medium. <https://octaviocode.medium.com/whats-up-with-mapreduce-a-concise-summary-in-spanish-c9deea5bed0e>

Mete, H. (2019, abril 24). *Example of ETL Application Using Apache Spark and Hive*. <https://dzone.com/articles/example-of-etl-application-using-apache-spark-and>

Página de Apache Spark (<https://spark.apache.org/docs/latest/>).

Tema 4. Computación distribuida. Computación paralela

Página de DATA + AI Summit (<https://databricks.com/sparkaisummit>).

White, T. (2015). *Hadoop: the definitive guide* (4.^a ed.). O'Reilly Media.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

5.1. Introducción y objetivos

En este tema vamos a ver el ciclo de implementación de modelos de ML y sus diferentes etapas. Estudiaremos la plataforma SageMaker de AWS y los diferentes servicios que ofrece para cubrir el *end-to-end* en el desarrollo y despliegue de modelos de ML.

Los **objetivos** que persigue este tema son los siguientes:

- ▶ AWS SageMaker Data Wrangler para preparación de datos.
- ▶ AWS Autopilot para generar modelos de ML de manera automática.
- ▶ AWS JumpStart para facilitar la construcción de modelos de ML.
- ▶ AWS SageMaker Clarify para controlar y mitigar sesgos, y otorgar explicabilidad a los modelos.
- ▶ AWS SageMaker Studio para desarrollar el código de construcción de estos modelos.
- ▶ AWS SageMaker Experiments para ayudar a los científicos de datos y desarrolladores a organizar, rastrear y comparar experimentos de ML.
- ▶ AWS SageMaker Debugger para monitorizar y depurar modelos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

5.2. Machine learning con SageMaker

Relación entre cloud y ML

La relación entre la computación en la nube (*cloud computing*) y el ML es muy estrecha y mutuamente beneficiosa. Así, algunos puntos clave a destacar de este mutuo beneficio serían:

- ▶ **Infraestructura escalable.** La computación en la nube proporciona una infraestructura escalable que es fundamental para el ML. Los algoritmos de ML, especialmente los modelos de *deep learning*, a menudo requieren grandes cantidades de recursos computacionales y de almacenamiento. La capacidad de escalar recursos según sea necesario permite entrenar modelos más grandes y manejar conjuntos de datos masivos de manera eficiente.
- ▶ **Acceso a recursos especializados.** Los proveedores de servicios en la nube, como AWS, Google Cloud Platform y Microsoft Azure, ofrecen una variedad de servicios y recursos especializados diseñados específicamente para ML. Esto incluye unidades de procesamiento gráfico (GPU) y unidades de procesamiento tensorial (TPU) optimizadas para cálculos intensivos requeridos por algoritmos de ML.
- ▶ **Facilita la colaboración y el despliegue.** La computación en la nube facilita la colaboración entre equipos de ML al proporcionar plataformas y herramientas centralizadas para el desarrollo, entrenamiento y despliegue de modelos. Además, permite implementar modelos de manera rápida y escalable, lo que es crucial para aplicaciones en tiempo real.
- ▶ **Costes flexibles.** La naturaleza escalable de la computación en la nube permite a las organizaciones optimizar sus costos de infraestructura. Pueden escalar recursos hacia arriba o hacia abajo según sea necesario y pagar solo por lo que utilizan, lo que puede ser especialmente beneficioso para proyectos de ML que pueden requerir recursos variables durante el desarrollo y el despliegue.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Almacenamiento y gestión de datos.** La computación en la nube ofrece una variedad de servicios de almacenamiento y gestión de datos que son fundamentales para el ML, como bases de datos escalables, sistemas de archivos distribuidos y servicios de almacenamiento en la nube. Estos servicios facilitan la gestión y el acceso a grandes conjuntos de datos utilizados para entrenar modelos de ML.

Por tanto, la **computación en la nube** proporciona la infraestructura, los recursos y las herramientas necesarias para desarrollar, entrenar, desplegar y escalar modelos de ML de manera eficiente y efectiva. Es un componente integral del ecosistema de ML en la actualidad.

AWS y ML

AWS ofrece una amplia gama de servicios y herramientas diseñados específicamente para el ML. A continuación, tienes un resumen de la oferta de AWS en este ámbito:

- ▶ **Amazon SageMaker.** Este es el servicio principal de AWS para el ML. Amazon SageMaker proporciona un entorno completo para construir, entrenar y desplegar modelos de ML de manera rápida y escalable. Incluye características como *notebooks* integrados para el desarrollo, algoritmos de ML preconstruidos, capacidad de entrenamiento distribuido y opciones de implementación automatizada.
- ▶ **Amazon SageMaker Ground Truth.** Este servicio ayuda a etiquetar los conjuntos de datos necesarios para el entrenamiento de modelos de ML. Utiliza la inteligencia artificial para automatizar parte del proceso de etiquetado y proporciona herramientas para la revisión y corrección manual de las etiquetas.
- ▶ **Amazon Comprehend.** Es un servicio de procesamiento de lenguaje natural (NLP) que utiliza ML para analizar texto y extraer información útil, como entidades, frases clave, sentimientos y relaciones. Es útil para tareas como análisis de sentimientos, extracción de información y clasificación de texto.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Amazon Rekognition.** Este servicio utiliza ML para analizar imágenes y vídeos. Puede identificar objetos, personas, texto y emociones en imágenes, así como realizar análisis de contenido inapropiado. Es útil para aplicaciones de reconocimiento de imágenes, seguridad visual y análisis de contenido multimedia.
- ▶ **Amazon Polly.** Es un servicio de conversión de texto a voz que utiliza ML para generar voces sintéticas naturales. Es útil para aplicaciones de asistentes virtuales, narración de texto y accesibilidad.

En resumen, **AWS** ofrece una amplia gama de servicios y herramientas para el ML que abarcan desde la construcción y el entrenamiento de modelos hasta la inferencia en tiempo real y la automatización de tareas relacionadas con el procesamiento de datos y el análisis de contenido. Esto les permite a los desarrolladores y empresas aprovechar el poder del ML de manera eficiente y escalable en la nube.

A continuación, profundizaremos sobre la plataforma de SageMaker, el principal servicio de ML que AWS ofrece.

¿Qué es Amazon Sage Maker?

Amazon SageMaker es un servicio de aprendizaje automático (ML) totalmente gestionado. Con él, SageMaker, los científicos de datos y los desarrolladores pueden crear, entrenar e implementar modelos de aprendizaje automático de forma rápida y segura en un entorno hospedado listo para la producción. Proporciona una experiencia de interfaz de usuario para ejecutar flujos de trabajo de aprendizaje automático que hace que las herramientas de aprendizaje SageMaker automático estén disponibles en varios entornos de desarrollo integrados (IDE).

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Amazon SageMaker es una **plataforma de aprendizaje automático** completamente administrada que permite a los desarrolladores y científicos de datos construir, entrenar y desplegar modelos de aprendizaje automático de manera rápida y eficiente. Desarrollada por AWS, SageMaker simplifica el proceso de construcción y el despliegue de modelos, y elimina la necesidad de administrar la infraestructura subyacente.

Descripción de Amazon SageMaker

Amazon SageMaker proporciona una serie de **características y herramientas** clave para todo el ciclo de vida de los modelos de aprendizaje automático:

- ▶ **Desarrollo ágil.** SageMaker ofrece un entorno integrado para el desarrollo de modelos, que incluye *notebooks* Jupyter preconfigurados y compatibles con una variedad de lenguajes de programación, como Python, R y Scala. Esto permite a los equipos de ciencia de datos colaborar de manera eficiente y experimentar con diferentes enfoques de modelado.
- ▶ **Entrenamiento escalable.** La plataforma permite entrenar modelos utilizando grandes volúmenes de datos de manera rápida y eficiente, porque aprovecha la capacidad de cómputo escalable de AWS. Además, SageMaker ofrece algoritmos de aprendizaje automático predefinidos y optimizados para diversos casos de uso, así como la capacidad de personalizar algoritmos o traer los propios.
- ▶ **Implementación sencilla.** Una vez que se ha entrenado un modelo, SageMaker simplifica su implementación en producción con solo unos pocos clics o líneas de código. La plataforma gestiona automáticamente la infraestructura subyacente y proporciona capacidad de escalamiento automático para manejar cargas de trabajo variables.

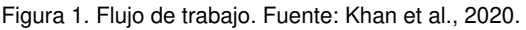
Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Gestión del ciclo de vida.** SageMaker facilita la gestión del ciclo de vida completo de los modelos, desde el desarrollo y entrenamiento hasta el despliegue y monitoreo en producción. Los desarrolladores pueden realizar fácilmente ajustes y actualizaciones en los modelos desplegados, así como monitorear su rendimiento y calidad.
- ▶ **Integración con servicios de AWS.** SageMaker se integra estrechamente con otros servicios de AWS, como S3 para almacenamiento de datos, IAM para la gestión de accesos y políticas, y CloudWatch para el monitoreo y registro de aplicaciones. Esto proporciona una experiencia de desarrollo cohesiva y unificada en el ecosistema de AWS.

Ciclo de ML con AWS Sage Maker

En el aprendizaje automático, le *enseñas* a un ordenador a hacer predicciones o inferencias. En primer lugar, utiliza el algoritmo o los datos de ejemplo para realizar la capacitación de un **modelo**. A continuación, integra el modelo en la aplicación para generar inferencias en tiempo real y a escala.

El Figura 1 ilustra el **flujo de trabajo** típico para crear un modelo de aprendizaje automático. Incluye tres etapas en un flujo circular que trataremos con más detalle a continuación: generar datos de ejemplo, entrenar un modelo e implementar el modelo.



- ▶ **Genere datos de ejemplo.** Para entrenar un modelo, necesita datos de ejemplo. El tipo de datos que necesita depende del problema empresarial que desee que el modelo resuelva (las inferencias que desea que genere el modelo). Por ejemplo, supongamos que desea crear un modelo para predecir un número a partir de una imagen de entrada de un dígito manuscrito. Para realizar la capacitación de ese modelo, necesita ejemplos de imágenes de números escritos a mano. Los científicos de datos suelen dedicar tiempo a explorar y preprocesar datos de ejemplo antes de usarlos para el entrenamiento de modelos. Para preprocesar datos, normalmente realiza la siguiente operación:
- **Adquisición de datos.** Es posible que tenga repositorios de datos de ejemplo internos o que utilice conjuntos de datos que estén disponibles públicamente. Normalmente, se extraen el conjunto o los conjuntos de datos en un repositorio único.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

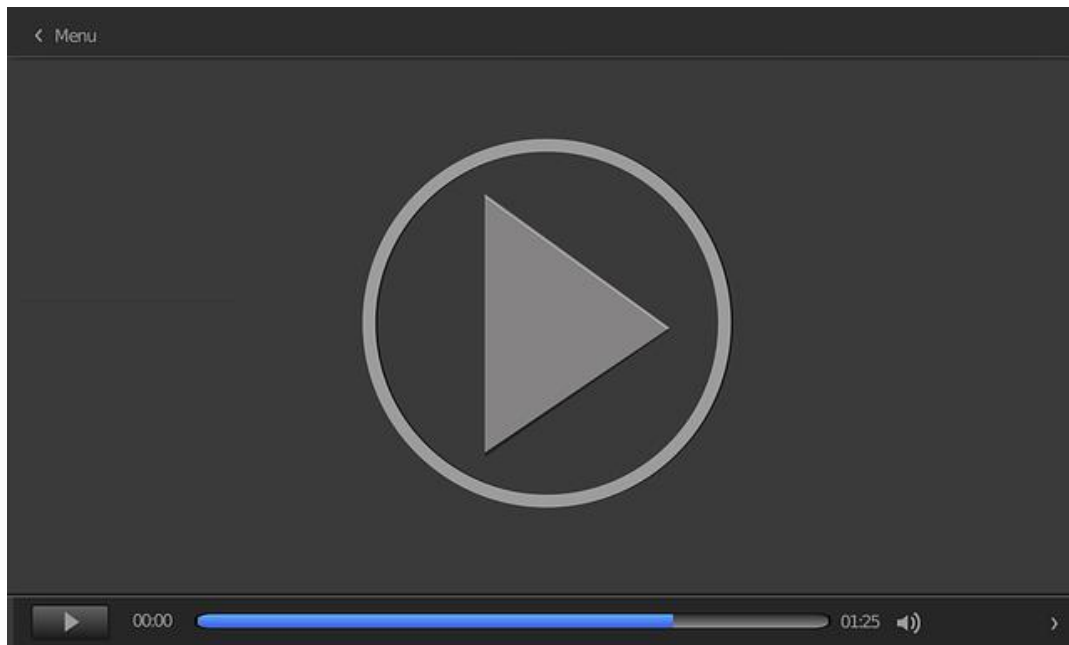
- **Limpieza de datos.** Para mejorar el entrenamiento del modelo, inspeccione los datos y límpielos, según sea necesario. Por ejemplo, si los datos tienen un *country name* atributo con valores *United StatesUS*, puede editarlos para que sean coherentes.
- **Preparación de datos.** Para mejorar el rendimiento, puede realizar transformaciones de datos adicionales. Por ejemplo, puede elegir combinar atributos. Si su modelo predice las condiciones que requieren el deshielo de una aeronave, en lugar de utilizar los atributos de temperatura y humedad por separado, puede combinar esos atributos en un nuevo atributo para obtener un modelo mejor. En SageMaker, puede preprocesar datos de ejemplo mediante SageMaker API con el SDK de SageMaker Python en un entorno de desarrollo integrado (IDE). Con el SDK para Python (Boto3) puedes recuperar, explorar y preparar tus datos para el entrenamiento de modelos. Para obtener información sobre la preparación, el procesamiento y la transformación de los datos, crea, almacena y comparte funciones con Amazon SageMaker Feature Store.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Entrenamiento de un modelo.** El entrenamiento del modelo incluye tanto la capacitación como la evaluación del modelo, de la siguiente manera:
 - **Entrenamiento del modelo.** Para entrenar un modelo, se necesita un algoritmo o un modelo base previamente entrenado. El algoritmo que elija dependerá de un número de factores. Para una solución integrada, puede utilizar uno de los algoritmos que se SageMaker proporcionan. Para obtener una lista de los algoritmos proporcionados por SageMaker y las consideraciones relacionadas, usa los algoritmos SageMaker integrados de Amazon o los modelos previamente entrenados. Para obtener una solución de entrenamiento basada en la interfaz de usuario que proporciona los algoritmos y los modelos, consulta SageMaker JumpStart. También necesita computar los recursos para la **capacitación**. Según el tamaño del conjunto de datos de capacitación y la rapidez con la que necesite los resultados, puede utilizar recursos desde una instancia de propósito general única a un clúster distribuido de instancias de GPU. Para obtener más información, consulta cómo entrenar a un modelo con Amazon SageMaker.
- ▶ **Evaluación del modelo.** Después de entrenar el modelo, se evalúa para determinar si la precisión de las inferencias es aceptable. Para entrenar y evaluar su modelo, puede usar el SDK de SageMaker Python para enviar solicitudes de inferencias al modelo a través de uno de los IDE disponibles.
- ▶ **Implementación del modelo.** Tradicionalmente, se rediseña un modelo antes de integrarlo con la aplicación e implementarlo. Con los servicios de SageMaker alojamiento, puede implementar el modelo de forma independiente, lo que lo desvincula del código de la aplicación. Para obtener más información, consulta cómo implementar modelos para inferencia.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

A continuación, podemos ver el vídeo *Introduction to Amazon SageMaker* (Amazon Web Services, 2021a), que es introductorio de las funcionalidades de AWS Sagemaker en el ciclo de vida de desarrollo de un modelo de ML.



Introduction to Amazon SageMaker.

Accede al vídeo:

https://www.youtube.com/embed/Qv_Tr_BCFCQ

El **aprendizaje automático** es un ciclo continuo. Tras implementar un modelo, se supervisan las inferencias, se recopilan más datos de alta calidad y se evalúa el modelo para identificar desviaciones. A continuación, aumenta la precisión de las inferencias actualizando los datos de entrenamiento para incluir los datos de alta calidad recién recopilados. A medida que haya más datos de ejemplo disponibles, seguirás reentrenando tu modelo para aumentar la precisión.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud



Figura 2. Amazon SageMaker. Fuente: Tomasz Dudek, 2021.

Preparación de datos con AWS Sagemaker

La **preparación de datos** es una fase fundamental en el proceso de desarrollo de modelos de ML. Implica la limpieza, la transformación y la estructuración de los datos crudos para que sean adecuados y útiles para entrenar un modelo de ML. Esta etapa es crucial porque la calidad de los datos tiene un impacto directo en el rendimiento y la eficacia del modelo resultante. A continuación, se muestra una descripción detallada de las principales **tareas** involucradas en la preparación de datos en ML:

- **Recopilación de datos.** El primer paso es recopilar los datos relevantes para tu problema de ML. Estos datos pueden provenir de diversas fuentes, como bases de datos, archivos CSV, datos en tiempo real, API, entre otros.
- **Exploración de datos** (*exploratory data analysis* o EDA). Antes de realizar cualquier modificación en los datos, es importante realizar un análisis exploratorio para comprender la naturaleza y las características de los datos. Esto incluye estadísticas descriptivas, visualización de datos y detección de patrones y correlaciones.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Limpieza de datos.** Esta etapa implica la identificación y corrección de errores, valores atípicos (*outliers*) y valores faltantes en los datos. La limpieza de datos es crucial para garantizar la calidad y la integridad de los datos antes de utilizarlos para entrenar un modelo.
- ▶ **Transformación de datos.** En esta etapa, se aplican transformaciones a los datos para mejorar su calidad y facilitar el aprendizaje del modelo. Esto puede incluir la normalización de datos, la codificación de variables categóricas, la creación de nuevas características (*feature engineering*) y la reducción de dimensionalidad.

Amazon SageMaker Processing es un servicio de Amazon SageMaker que permite ejecutar y escalar fácilmente flujos de trabajo de procesamiento de datos, preprocesamiento y posprocesamiento en la nube. Esta capacidad de procesamiento se integra perfectamente con otros servicios de SageMaker y con los servicios de almacenamiento y procesamiento de datos de AWS, lo que les permite a los desarrolladores y científicos de datos realizar tareas de preparación de datos, transformación de datos y evaluación de modelos de manera eficiente y escalable.

A continuación, tienes una descripción detallada de las **características** y las **funciones** principales de SageMaker Processing:

- ▶ **Orquestación de flujos de trabajo.** SageMaker Processing proporciona una interfaz fácil de usar para definir y ejecutar flujos de trabajo de procesamiento de datos, lo que les permite a los usuarios especificar las tareas que desean realizar, como la ejecución de *scripts* de preprocesamiento, la generación de características o la evaluación de modelos.
- ▶ **Escalabilidad y eficiencia.** El servicio permite escalar automáticamente el poder de cómputo necesario para ejecutar tareas de procesamiento de datos, lo que garantiza que las tareas se completen de manera rápida y eficiente, incluso con grandes volúmenes de datos.

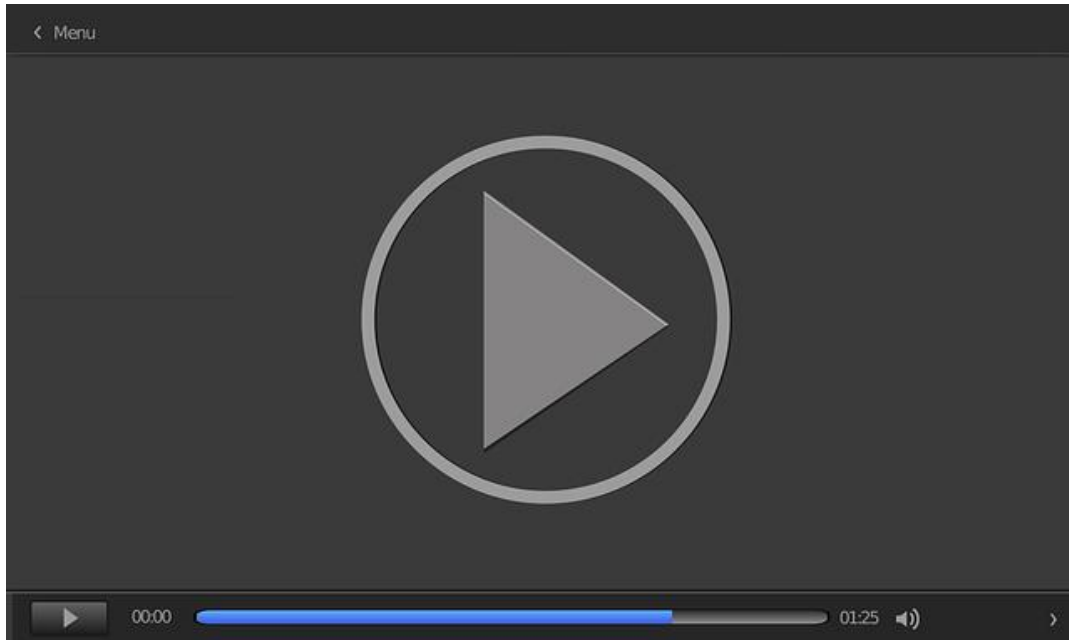
Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Integración con SageMaker Studio y *notebooks*.** SageMaker Processing está integrado con SageMaker Studio y *notebooks*, lo que permite a los usuarios ejecutar flujos de trabajo de procesamiento de datos directamente desde su entorno de desarrollo favorito y colaborar de manera eficiente con otros miembros del equipo.
- ▶ **Compatibilidad con contenedores personalizados.** Los usuarios tienen la flexibilidad de ejecutar código personalizado utilizando contenedores Docker, lo que les permite utilizar sus propias bibliotecas y entornos de desarrollo para realizar tareas de procesamiento de datos personalizadas.
- ▶ **Integración con servicios de AWS.** SageMaker Processing se integra estrechamente con otros servicios de AWS, como Amazon S3 para el almacenamiento de datos, AWS Glue para la preparación de datos y AWS Step Functions para la orquestación de flujos de trabajo complejos, lo que facilita la integración con flujos de trabajo existentes.
- ▶ **Monitorización y registro.** El servicio proporciona capacidades de monitorización y registro que permiten a los usuarios realizar un seguimiento del progreso de las tareas de procesamiento de datos, así como registrar los resultados y los metadatos asociados con cada ejecución de tarea.

En resumen, **Amazon SageMaker Processing** es una herramienta poderosa que facilita la ejecución y el escalado de flujos de trabajo de procesamiento de datos en la nube, lo que les permite a los desarrolladores y científicos de datos realizar tareas de preparación de datos, transformación de datos y evaluación de modelos de manera eficiente y escalable.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Seguidamente, se expone el vídeo *AWS SageMaker Processing API* (Hai Tran, 2022), que explica las posibilidades de SakeMaker para el procesamiento de datos.



AWS SageMaker Processing API.

Accede al vídeo:

<https://www.youtube.com/embed/VZs5yaFDO4E>

Ingeniería de características (*feature selection*)

La selección de **características** es el proceso de identificar y seleccionar las características más relevantes y significativas para el modelo. Esto ayuda a reducir la complejidad del modelo y mejorar su rendimiento al eliminar las características irrelevantes o redundantes.

SageMaker Feature Store es un servicio de Amazon SageMaker diseñado para simplificar y agilizar la gestión de características (*features*) para el desarrollo de modelos de ML. Las características, también conocidas como atributos, son los datos que se utilizan para entrenar y hacer predicciones con modelos de ML. El Feature Store proporciona un repositorio centralizado y escalable para almacenar, compartir y

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

recuperar características de manera eficiente, lo que ayuda a mejorar la consistencia, la reutilización y la colaboración en proyectos de ML.

Seguidamente se muestra una descripción detallada de las **características** y las **funciones** principales del SageMaker Feature Store:

- ▶ **Almacenamiento centralizado.** El Feature Store proporciona un repositorio centralizado y escalable para almacenar características, lo que permite que los equipos de ML accedan fácilmente a las características relevantes para sus proyectos sin tener que preocuparse por la infraestructura subyacente.
- ▶ **Gestión de versiones.** Permite gestionar múltiples versiones de características, lo que facilita la colaboración y la experimentación en proyectos de ML. Los equipos pueden realizar un seguimiento de los cambios en las características a lo largo del tiempo y revertir a versiones anteriores si es necesario.
- ▶ **Búsqueda y recuperación eficientes.** El Feature Store proporciona capacidades de búsqueda y recuperación eficientes que permiten que los equipos encuentren y recuperen rápidamente las características necesarias para sus modelos de ML. Esto ayuda a acelerar el desarrollo y la implementación de modelos.
- ▶ **Integración con SageMaker.** El Feature Store está integrado estrechamente con otros servicios de SageMaker, lo que facilita la incorporación de características almacenadas en el Feature Store en flujos de trabajo de ML. Los equipos pueden acceder a las características almacenadas directamente desde SageMaker Studio o desde *notebooks* de SageMaker.
- ▶ **Consistencia y conformidad.** El Feature Store garantiza la consistencia y la conformidad de las características almacenadas, lo que ayuda a evitar problemas como datos inconsistentes o duplicados que podrían afectar negativamente el rendimiento del modelo.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Seguridad y control de acceso.** Proporciona capacidades de seguridad y control de acceso integradas que permiten a los equipos gestionar quién tiene acceso a qué características y cómo se utilizan en los modelos de ML.
- ▶ **División de datos.** Se divide el conjunto de datos en conjuntos de entrenamiento, validación y prueba. El conjunto de entrenamiento se utiliza para entrenar el modelo, el conjunto de validación se utiliza para ajustar los hiperparámetros del modelo y el conjunto de prueba se utiliza para evaluar el rendimiento final del modelo.
- ▶ **Normalización de datos.** La normalización es el proceso de escalar los datos a un rango específico o a una distribución específica. Esto ayuda a evitar problemas de convergencia en los algoritmos de ML y asegura que todas las características contribuyan de manera equitativa al modelo.
- ▶ **Manejo de desequilibrios de clases.** Si tus datos están desbalanceados, es decir, una clase tiene muchas más instancias que otra, es importante aplicar técnicas de remuestreo, como submuestreo (*undersampling*) o sobremuestreo (*oversampling*), para equilibrar las clases y evitar sesgos en el modelo.
- ▶ **Revisión de sesgos y explicabilidad.**

Puede usar **Amazon SageMaker Data Wrangler** para importar, preparar, transformar, visualizar y analizar datos. Puede integrar Data Wrangler en sus flujos de trabajo de ML para simplificar y agilizar el preprocesamiento de datos y la ingeniería de características con poca o ninguna codificación. También puede añadir sus propios *scripts* y transformaciones de Python para personalizar su flujo de trabajo de preparación de datos.

Importa datos de Amazon S3, Amazon Redshift y Amazon Athena, y utiliza Data Wrangler para crear **flujos de trabajo** sofisticados de preparación de datos de ML con transformaciones y análisis de datos integrados y personalizados, que incluyen el modelado rápido y la fuga de objetivos de características.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Una vez que haya definido un flujo de trabajo de preparación de datos, puede integrarlo con SageMaker Processing, SageMaker Pipelines y SageMaker Feature Store para simplificar la tarea de procesar, compartir y almacenar los **datos de entrenamiento** con aprendizaje automático. También puede exportar el flujo de datos a un *script* de Python y crear una canalización de preparación de datos de ML personalizado.

Limpieza estandarización de datos con SageMaker Data Wrangler

Amazon SageMaker Data Wrangler es una herramienta potente que simplifica y acelera el proceso de preparación de datos para el entrenamiento de modelos de aprendizaje automático. Diseñada para abordar uno de los desafíos más comunes y laboriosos en el desarrollo de modelos de ML, Data Wrangler automatiza tareas de limpieza, transformación y *feature engineering*, permitiendo a los científicos de datos concentrarse en la creación de modelos de mayor calidad en lugar de perder tiempo en la preparación de datos.

Algunas **características** destacadas de SageMaker Data Wrangler son:

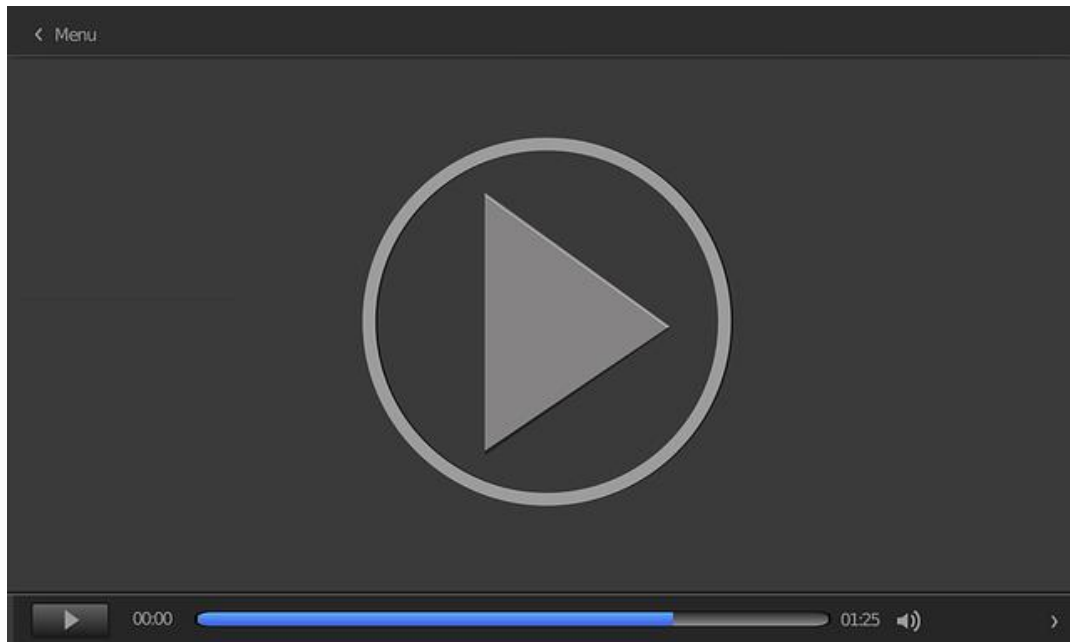
- ▶ **Interfaz visual intuitiva.** Data Wrangler ofrece una interfaz visual fácil de usar que permite a los usuarios explorar, limpiar y transformar datos de manera interactiva. Los científicos de datos pueden visualizar el impacto de las transformaciones en tiempo real y realizar ajustes sobre la marcha.
- ▶ **Automatización de tareas.** La herramienta automatiza tareas repetitivas y tediosas, como la imputación de valores faltantes, la codificación de variables categóricas y la normalización de datos. Esto permite que los equipos de ML aceleren significativamente el proceso de preparación de datos y reduzcan el riesgo de errores.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Integración con SageMaker.** Data Wrangler se integra perfectamente con otras herramientas y servicios de SageMaker, lo que facilita la transición de los datos preparados al entrenamiento de modelos. Los usuarios pueden exportar fácilmente los conjuntos de datos preparados a SageMaker Studio o a flujos de trabajo de entrenamiento personalizados.
- ▶ **Amplia biblioteca de transformaciones.** Data Wrangler ofrece una amplia gama de transformaciones predefinidas para abordar diferentes tipos de datos y escenarios de modelado. Además, los usuarios tienen la flexibilidad de crear sus propias transformaciones personalizadas utilizando código Python, lo que permite adaptar la preparación de datos a necesidades específicas del proyecto.
- ▶ **Reproducibilidad y auditoría.** La herramienta proporciona funcionalidades para garantizar la reproducibilidad y la auditabilidad de los flujos de trabajo de preparación de datos. Los usuarios pueden guardar y compartir fácilmente sus transformaciones, así como rastrear el historial de cambios y revisiones.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

El funcionamiento de SageMaker Data Wrangler se puede ver en el vídeo *Amazon SageMaker Data Wrangler Deep Dive Demo* (Amazon Web Services, 2021b).



Amazon SageMaker Data Wrangler Deep Dive Demo.

Accede al vídeo:

<https://www.youtube.com/embed/b5EIDVPomO4>

Identificar los sesgos y afrontar la explicabilidad

Amazon SageMaker Clarify es una herramienta diseñada para ayudar a los científicos de datos y desarrolladores a identificar y mitigar sesgos y explicar las predicciones de modelos de ML de una manera transparente y ética. Esta herramienta es fundamental para comprender y abordar los posibles sesgos y explicaciones detrás de las decisiones tomadas por los modelos de ML, lo que garantiza que los modelos sean justos, interpretables y confiables.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

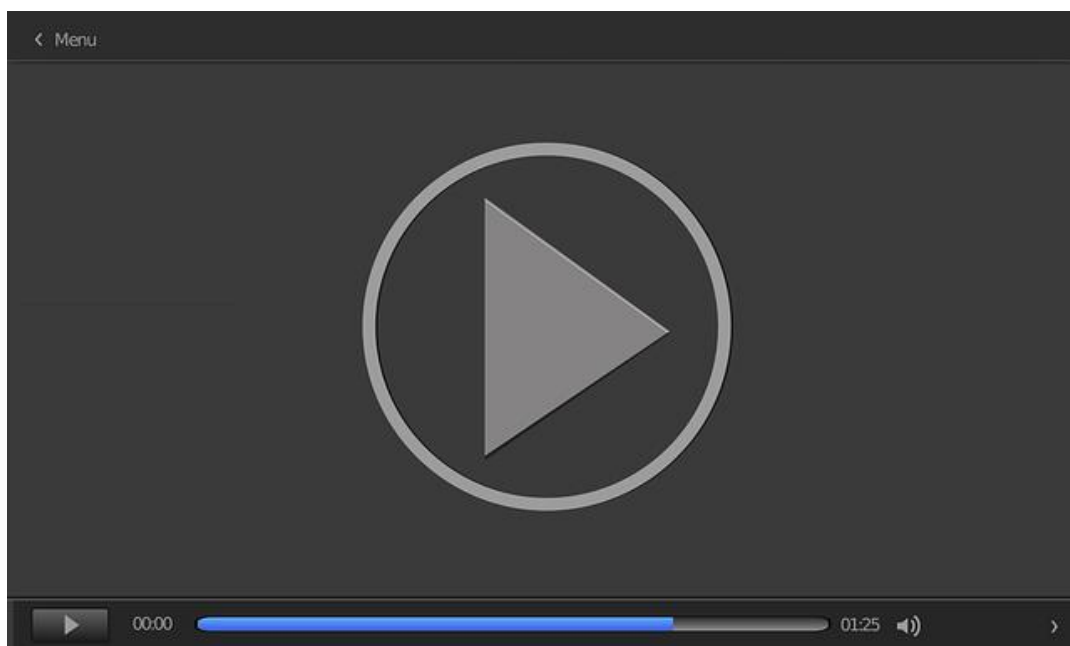
A continuación, hay una descripción detallada de las **características** y las **funciones** principales de SageMaker Clarify:

- ▶ **Detección de sesgos.** SageMaker Clarify permite a los usuarios detectar sesgos en los datos de entrenamiento y en las predicciones de los modelos. Utiliza métricas de equidad para identificar sesgos en función de diferentes atributos protegidos, como raza, género o edad, y proporciona información detallada sobre la magnitud y la dirección del sesgo detectado.
- ▶ **Exploración de sesgos.** La herramienta permite a los usuarios explorar y comprender los sesgos detectados utilizando visualizaciones interactivas y estadísticas descriptivas. Esto ayuda a identificar las causas subyacentes del sesgo y a tomar medidas para mitigarlo de manera efectiva.
- ▶ **Explicabilidad de modelos.** SageMaker Clarify proporciona funcionalidades para explicar las predicciones de los modelos, lo que permite a los usuarios comprender cómo y por qué un modelo toma una determinada decisión. Esto es crucial para mejorar la confianza en el modelo y garantizar su interpretabilidad.
- ▶ **Auditoría de modelos.** La herramienta permite a los usuarios realizar auditorías de modelos para evaluar su rendimiento en términos de equidad, transparencia y confiabilidad. Esto incluye la generación de informes detallados sobre la equidad del modelo y las explicaciones de las predicciones.
- ▶ **Integración con flujos de trabajo de ML.** SageMaker Clarify está integrado con otros servicios de SageMaker, lo que facilita la incorporación de análisis de sesgos y explicaciones en flujos de trabajo de ML. Esto permite a los usuarios realizar evaluaciones éticas y transparentes de sus modelos durante todo el ciclo de vida del desarrollo del modelo.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- **Personalización y ajuste.** Los usuarios tienen la flexibilidad de personalizar y ajustar las métricas de equidad, las técnicas de explicabilidad y otros parámetros según sus necesidades específicas. Esto garantiza que SageMaker Clarify se adapte a una amplia gama de escenarios y requisitos de aplicación.

Profundizamos en AWS Clarify en el vídeo *AWS re:Invent 2023 - Accelerate foundation model evaluation with Amazon SageMaker Clarify (AIM367)* (AWS Events, 2023).



AWS re:Invent 2023 - Accelerate foundation model evaluation with Amazon SageMaker Clarify (AIM367).

Accede al vídeo:

<https://www.youtube.com/embed/9X2oDkOBYyA>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

En resumen, **Amazon SageMaker Clarify** es una herramienta esencial para garantizar que los modelos de ML sean justos, interpretables y confiables. Proporciona capacidades avanzadas para detectar sesgos, explicar predicciones y realizar auditorías éticas de modelos, lo que ayuda a mitigar riesgos y promover la transparencia y la equidad en el desarrollo y despliegue de modelos de ML.

Asimismo, **Amazon SageMaker Data Wrangler** es una herramienta poderosa que simplifica y acelera el proceso de preparación de datos para el aprendizaje automático, permitiendo a los científicos de datos dedicar más tiempo a la creación de modelos de ML de alta calidad y menos tiempo a tareas manuales y repetitivas.

Construcción de modelos con AWS SageMaker

Amazon SageMaker Studio es un entorno de desarrollo integrado (IDE) completamente basado en la nube, diseñado para simplificar y acelerar el desarrollo de proyectos de ML. Proporciona una interfaz unificada y colaborativa que permite a los científicos de datos y desarrolladores trabajar de manera eficiente en todas las etapas del ciclo de vida de ML, desde la preparación de datos y el entrenamiento de modelos hasta el despliegue y la monitorización en producción.

Una descripción detallada de las **características** y las **funciones** principales de SageMaker Studio sería la siguiente:

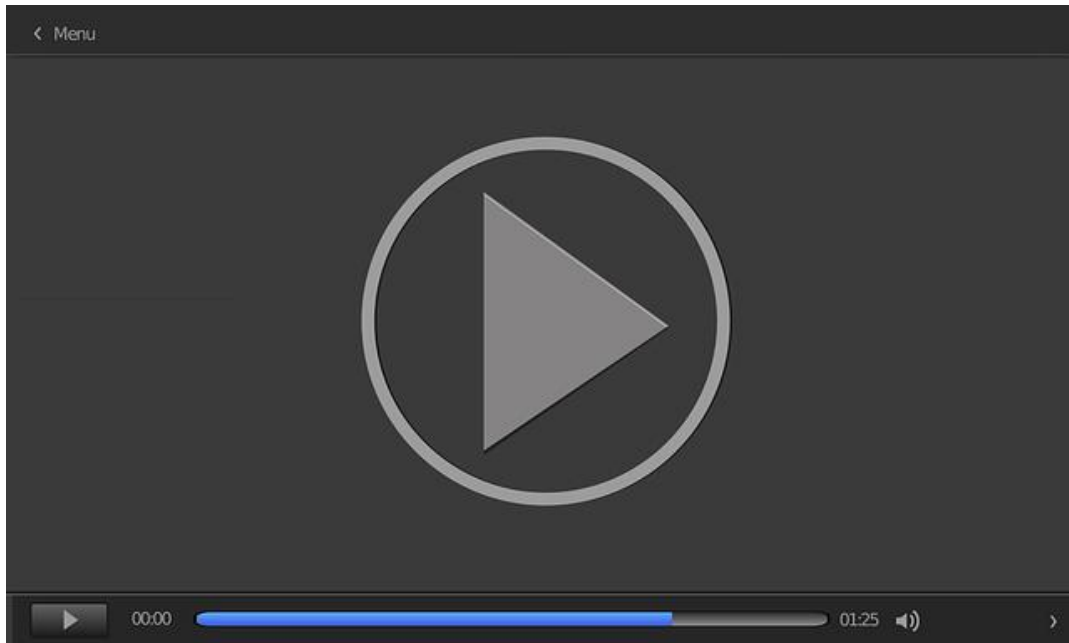
- ▶ **Entorno de desarrollo integrado (IDE).** SageMaker Studio ofrece un entorno de desarrollo unificado que incluye herramientas para escribir y ejecutar código, explorar y visualizar datos, experimentar con modelos de ML, y colaborar con otros miembros del equipo.
- ▶ **Notebooks integrados.** Los usuarios pueden crear y ejecutar *notebooks* Jupyter directamente desde SageMaker Studio, lo que les permite escribir y ejecutar código en lenguajes como Python, R y Scala, y realizar análisis interactivos y experimentos de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Administración de experimentos.** SageMaker Studio proporciona funcionalidades avanzadas para administrar y realizar experimentos de ML, incluyendo la creación de experimentos, el seguimiento de métricas y resultados, y la comparación de modelos entrenados.
- ▶ **Visualización de datos.** La herramienta incluye capacidades de visualización de datos que permiten que los usuarios exploren y visualicen los datos de manera interactiva, lo que facilita la comprensión de la estructura, la distribución y las relaciones en los datos.
- ▶ **Modelo de entrenamiento y despliegue.** SageMaker Studio ofrece capacidades integradas para entrenar y desplegar modelos de ML en la nube, lo que les permite a los usuarios entrenar modelos de forma distribuida y escalar automáticamente la infraestructura de cómputo según sea necesario.
- ▶ **Gestión de recursos.** Los usuarios pueden gestionar y monitorear los recursos de SageMaker, como instancias de entrenamiento y modelos desplegados, desde una interfaz unificada en SageMaker Studio, lo que facilita la administración y el seguimiento de recursos.
- ▶ **Integración con servicios de AWS.** SageMaker Studio está integrado con otros servicios de AWS, como S3 para el almacenamiento de datos, IAM para la gestión de accesos y políticas, y CloudWatch para el monitoreo y el registro de aplicaciones, lo que proporciona una experiencia de desarrollo cohesiva y unificada en el ecosistema de AWS.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

El vídeo *AWS AMER Summit 2020 | Introducción a Amazon SageMaker Studio, un IDE completo para ML* (AWS Events, 2021), sobre las capacidades y las funcionalidades de AWS SageMaker Studio se muestra a continuación.



AWS AMER Summit 2020 | Introducción a Amazon SageMaker Studio, un IDE completo para ML.

Accede al vídeo:

<https://www.youtube.com/embed/Yzxsl2SB0as>

En resumen, **Amazon SageMaker Studio** es una herramienta poderosa que simplifica y acelera el desarrollo de proyectos de ML en la nube, proporcionando un entorno unificado y colaborativo para todas las etapas del ciclo de vida de ML. Con capacidades integradas para la preparación de datos, el entrenamiento de modelos y el despliegue en producción, SageMaker Studio permite a los equipos de ML innovar y experimentar de manera eficiente, y llevar modelos a producción de forma rápida y segura.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Automated Machine Learning con AWS SageMaker Autopilot

Amazon SageMaker Autopilot es un servicio de aprendizaje automático automatizado (AutoML) proporcionado por AWS. Está diseñado para simplificar el proceso de construcción, entrenamiento y despliegue de modelos de aprendizaje automático, permitiendo a los desarrolladores y científicos de datos crear modelos de alta calidad con un esfuerzo mínimo y sin necesidad de experiencia en aprendizaje automático.

Las principales **características** son:

- ▶ **Automatización del proceso.** Autopilot automatiza el proceso completo de construcción de modelos de ML, desde la preparación de datos hasta la optimización de hiperparámetros, eliminando la necesidad de realizar tareas manuales y tediosas.
- ▶ **Preprocesamiento automático de datos.** El servicio realiza automáticamente tareas de preprocesamiento de datos, como la limpieza de datos, la imputación de valores faltantes y la codificación de variables categóricas, lo que simplifica significativamente la preparación de datos.
- ▶ **Selección de algoritmo automática.** Autopilot selecciona automáticamente los algoritmos de ML más adecuados para el conjunto de datos y el tipo de problema, lo que garantiza que se utilice el enfoque más eficaz para cada caso.
- ▶ **Optimización de hiperparámetros automática.** El servicio optimiza automáticamente los hiperparámetros del modelo para maximizar el rendimiento predictivo, lo que ayuda a obtener modelos de alta calidad con la menor cantidad de recursos.
- ▶ **Interpretación de modelos.** Autopilot proporciona capacidades de interpretación de modelos que permiten a los usuarios comprender cómo funciona un modelo y qué características son más importantes para las predicciones.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Despliegue fácil.** Una vez que se ha creado y entrenado un modelo con Autopilot, se puede implementar fácilmente en producción con solo unos pocos clics o líneas de código, lo que acelera significativamente el tiempo de llegada al mercado.

Funciona de este modo:

- ▶ **Preparación de datos.** Los usuarios proporcionan los datos de entrenamiento en formato tabular (CSV) o en Amazon S3. Autopilot realiza automáticamente tareas de preprocesamiento de datos, como la limpieza y la codificación de variables.
- ▶ **Selección y entrenamiento de modelos.** Autopilot selecciona automáticamente los algoritmos de ML y optimiza los hiperparámetros del modelo para maximizar el rendimiento predictivo. Luego, entrena múltiples modelos y evalúa su rendimiento utilizando técnicas avanzadas de validación cruzada.
- ▶ **Evaluación y despliegue.** Una vez que se ha seleccionado el mejor modelo, los usuarios pueden evaluar su rendimiento y comprender su comportamiento utilizando capacidades de interpretación de modelos. Luego, el modelo puede ser implementado fácilmente en producción para su uso en aplicaciones en tiempo real.

Sus **beneficios** son los siguientes:

- ▶ **Ahorro de tiempo y esfuerzo.** Autopilot automatiza muchas tareas tediosas y repetitivas asociadas con la construcción de modelos de ML, lo que les permite a los equipos de ML centrarse en tareas más estratégicas y creativas.
- ▶ **Modelos de alta calidad.** Al utilizar técnicas avanzadas de optimización y selección de modelos, Autopilot ayuda a crear modelos de alta calidad que maximizan la precisión y el rendimiento predictivo.
- ▶ **Accesible para todos.** Autopilot hace que el aprendizaje automático sea accesible para una amplia gama de usuarios, incluso aquellos sin experiencia en ML, lo que democratiza el acceso a la tecnología de aprendizaje automático.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

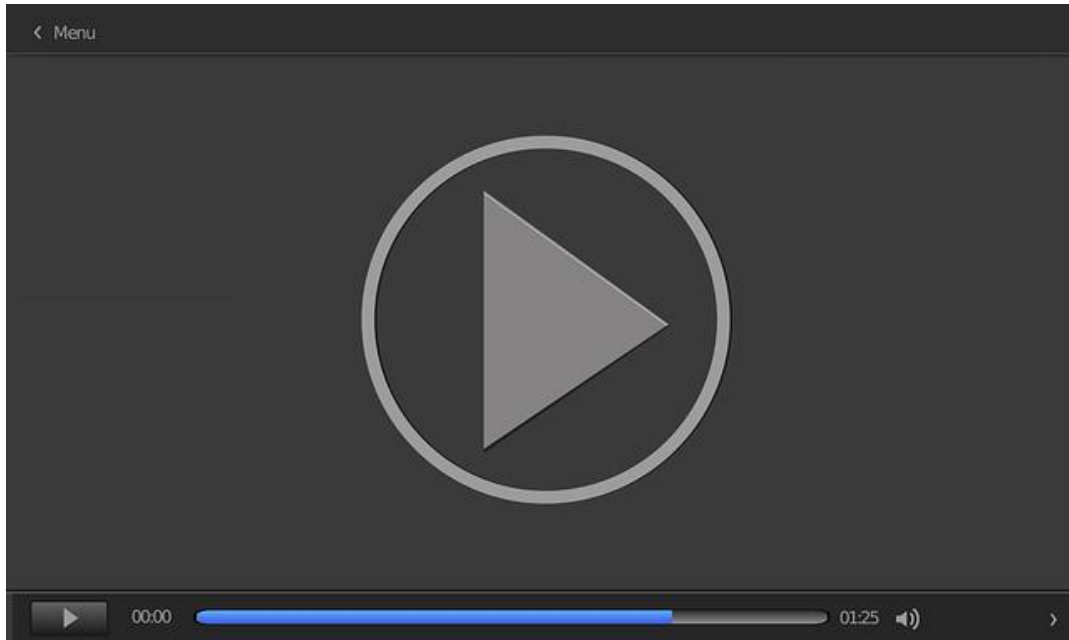
Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Predicción de ingresos y ventas.** Autopilot puede ser utilizado para predecir ingresos futuros o ventas en función de datos históricos y variables relevantes.
- ▶ **Detección de fraude.** El servicio puede ser aplicado para detectar transacciones fraudulentas en tiempo real, lo que ayuda a proteger a las organizaciones contra actividades delictivas.
- ▶ **Personalización de recomendaciones.** Autopilot puede ser utilizado para personalizar recomendaciones de productos o contenido para usuarios individuales, lo que mejora la experiencia del usuario y aumenta la retención.

Por tanto, **Amazon SageMaker Autopilot** es una herramienta poderosa y versátil que simplifica y automatiza el proceso de construcción de modelos de aprendizaje automático. Al proporcionar capacidades avanzadas de automatización, interpretación y despliegue de modelos, Autopilot ayuda a acelerar la innovación y mejorar la precisión de los modelos de ML, lo que les permite a las organizaciones tomar decisiones más informadas y obtener *insights* valiosos a partir de sus datos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Un demo de AWS SageMaker Autopilot se puede ver en el vídeo *Overview of Amazon SageMaker Autopilot* (AWS Developers, 2022a).



Overview of Amazon SageMaker Autopilot.

Accede al vídeo:

<https://www.youtube.com/embed/9pwYlb1Kit0>

AWS Sagemaker Jumpstart para facilitar el desarrollo de ML

Amazon SageMaker JumpStart es un servicio que ofrece una colección de soluciones preconfiguradas de ML diseñadas para abordar una variedad de casos de uso comunes en diferentes industrias. Estas soluciones proporcionan un punto de partida rápido y fácil para que los equipos de ML comiencen a construir, entrenar y desplegar modelos de ML de alta calidad en la nube de AWS.

Las principales **características** son:

- **Soluciones preconfiguradas.** SageMaker JumpStart ofrece una amplia gama de soluciones preconfiguradas para casos de uso comunes en industrias como salud,

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

finanzas, comercio electrónico y más. Estas soluciones incluyen pipelines de datos, modelos preentrenados y *notebooks* de ejemplo, lo que permite a los usuarios comenzar rápidamente con proyectos de ML sin necesidad de configuración adicional.

- ▶ **Compatibilidad con diversos *frameworks*.** Las soluciones de JumpStart son compatibles con una variedad de *frameworks* de ML, incluyendo TensorFlow, PyTorch, XGBoost y más, lo que proporciona flexibilidad para elegir el *framework* que mejor se adapte a las necesidades del proyecto.
- ▶ **Entrenamiento escalable.** Las soluciones de JumpStart están optimizadas para aprovechar la capacidad de cómputo escalable de AWS, lo que permite entrenar modelos en grandes volúmenes de datos de manera rápida y eficiente.
- ▶ **Despliegue en producción.** Una vez que se ha entrenado un modelo utilizando una solución de JumpStart, se puede implementar fácilmente en producción con solo unos pocos clics o líneas de código, lo que acelera el tiempo de llegada al mercado y facilita la adopción de modelos en aplicaciones en tiempo real.
- ▶ **Integración con servicios de AWS.** Las soluciones de JumpStart están integradas con otros servicios de AWS, como Amazon S3 para el almacenamiento de datos, AWS Glue para la preparación de datos, y AWS Lambda para la ejecución de código sin servidor, lo que proporciona una experiencia de desarrollo cohesiva y unificada en el ecosistema de AWS.

Funciona de este modo:

- ▶ **Selección de solución.** Los usuarios seleccionan una solución de JumpStart que coincida con su caso de uso y necesidades específicas. Las soluciones están organizadas en categorías y pueden filtrarse según el dominio o la industria.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Implementación de la solución.** Una vez seleccionada una solución, los usuarios pueden implementarla en su cuenta de AWS con unos pocos clics. La solución incluye todos los recursos necesarios, como *notebooks* preconfigurados, *pipelines* de datos y modelos preentrenados.
- ▶ **Personalización y entrenamiento.** Una vez implementada, los usuarios pueden personalizar la solución según sus necesidades específicas y comenzar a entrenar el modelo utilizando sus propios datos.
- ▶ **Despliegue en producción.** Una vez entrenado el modelo, se puede implementar fácilmente en producción utilizando las capacidades de despliegue integradas en la solución de JumpStart.

Sus **beneficios** son los siguientes:

- ▶ **Rapidez y facilidad de uso.** SageMaker JumpStart proporciona un punto de partida rápido y fácil para comenzar con proyectos de ML, lo que reduce significativamente el tiempo y el esfuerzo requeridos para construir y desplegar modelos de alta calidad.
- ▶ **Flexibilidad y escalabilidad.** Las soluciones de JumpStart son altamente flexibles y escalables, lo que permite adaptarlas a una amplia gama de casos de uso y escenarios de aplicación, desde pequeños proyectos piloto hasta implementaciones a gran escala en producción.
- ▶ **Mejora de la productividad.** Al proporcionar soluciones preconfiguradas y optimizadas, JumpStart permite a los equipos de ML centrarse en la innovación y la experimentación, en lugar de perder tiempo en tareas de configuración y puesta en marcha.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

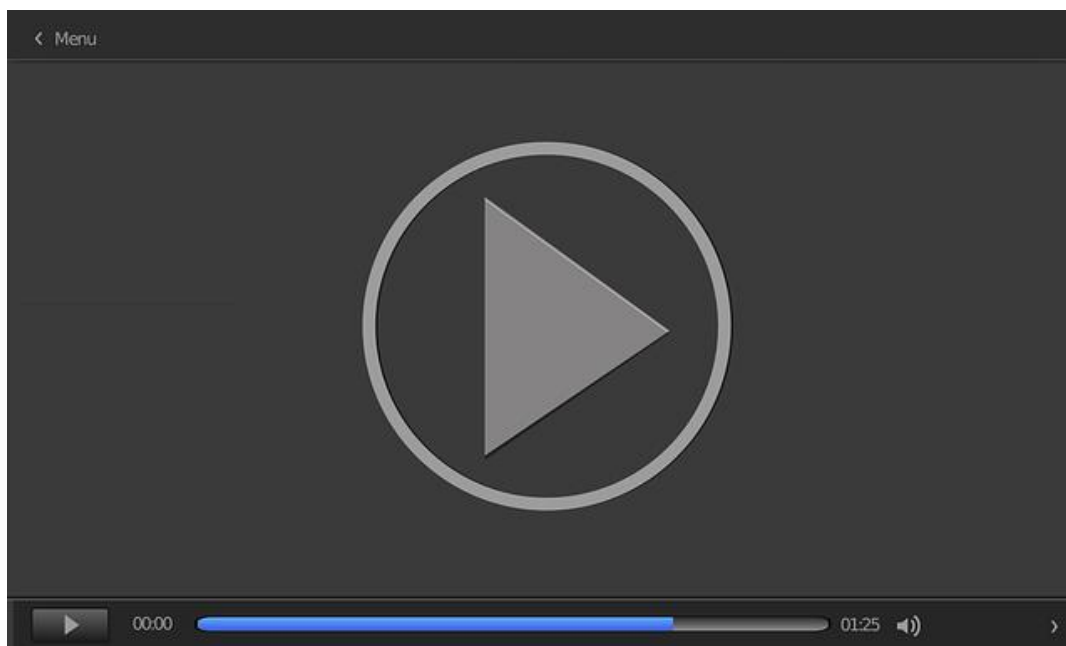
Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Detección de fraude en transacciones financieras.** Utiliza la solución preconfigurada de detección de fraude para identificar y prevenir transacciones fraudulentas en tiempo real.
- ▶ **Predicción de demanda en comercio electrónico.** Utiliza la solución preconfigurada de predicción de demanda para anticipar la demanda de productos y optimizar los niveles de inventario.
- ▶ **Diagnóstico médico de imágenes.** Utiliza la solución preconfigurada de diagnóstico médico para analizar imágenes médicas y ayudar en el diagnóstico de enfermedades.

Amazon SageMaker JumpStart es una herramienta poderosa y versátil que simplifica y acelera el desarrollo de proyectos de ML en la nube. Al proporcionar soluciones preconfiguradas, escalables y flexibles para una amplia gama de casos de uso, JumpStart ayuda a los equipos de ML a comenzar rápidamente, experimentar con modelos de ML y llevar modelos a producción de manera eficiente y confiable.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

En el vídeo *Easily get started with machine learning using Amazon SageMaker JumpStart - AWS Virtual Workshop* (AWS Developers, 2022b), se profundiza sobre las funcionalidades, el uso y las características de AWS JumpStart.



Easily get started with machine learning using Amazon SageMaker JumpStart - AWS Virtual Workshop.

Accede al vídeo:

<https://www.youtube.com/embed/1-AOLoOiuG4>

Entrenamiento de modelos

La **etapa de entrenamiento** del ciclo de vida completo de ML abarca desde el acceso al conjunto de datos de entrenamiento hasta la generación de un modelo final y la selección del modelo con mejor rendimiento para su implementación. A continuación, se ofrece una descripción general de las funciones y los recursos de SageMaker de formación disponible, con información técnica detallada sobre cada uno de ellos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

AWS SageMaker Experiments

Amazon SageMaker Experiments es un servicio de Amazon SageMaker diseñado para ayudar a los científicos de datos y desarrolladores a organizar, rastrear y comparar experimentos de ML de manera eficiente. Proporciona una plataforma centralizada para registrar y gestionar todos los experimentos, así como para analizar y compartir los resultados obtenidos durante el desarrollo de modelos de ML.

Las principales **características** son:

- ▶ **Organización de los experimentos.** SageMaker Experiments les permite a los usuarios organizar y gestionar todos los experimentos de ML en una ubicación centralizada, lo que facilita la búsqueda, la recuperación y el seguimiento de los experimentos realizados en diferentes proyectos.
- ▶ **Registro de metadatos.** El servicio registra automáticamente metadatos detallados de cada experimento, incluyendo los hiperparámetros utilizados, las métricas de rendimiento, los artefactos generados y más, lo que proporciona una trazabilidad completa y una base sólida para el análisis posterior.
- ▶ **Comparación de los experimentos.** SageMaker Experiments proporciona capacidades avanzadas para comparar y analizar múltiples experimentos de ML, lo que les permite a los usuarios identificar patrones, tendencias y mejores prácticas, que pueden conducir a mejoras en el rendimiento del modelo.
- ▶ **Visualización de resultados.** El servicio ofrece herramientas de visualización integradas que les permiten a los usuarios explorar y visualizar los resultados de los experimentos de ML de manera intuitiva, lo que facilita la comprensión y la interpretación de los datos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Colaboración y compartición.** SageMaker Experiments facilita la colaboración y la compartición de resultados entre los miembros del equipo, lo que permite que varios usuarios trabajen juntos en el análisis y la iteración de experimentos de ML.

Funciona de este modo:

- ▶ **Creación de experimentos.** Los usuarios crean experimentos de ML utilizando SageMaker Experiments y registran metadatos detallados, como hiperparámetros, métricas y artefactos asociados con cada experimento.
- ▶ **Ejecución de experimentos.** Una vez creado un experimento, los usuarios pueden ejecutarlo en SageMaker o en su entorno local utilizando los recursos de cómputo y almacenamiento necesarios.
- ▶ **Registro de resultados.** El servicio registra automáticamente los resultados de cada experimento, incluyendo métricas de rendimiento, artefactos generados y cualquier información adicional relevante.
- ▶ **Comparación y análisis.** Una vez completados varios experimentos, los usuarios pueden comparar y analizar los resultados utilizando herramientas de visualización integradas, identificando patrones, tendencias y áreas de mejora.
- ▶ **Iteración y mejora.** Basándose en los resultados obtenidos, los usuarios pueden iterar en el diseño de los modelos, ajustar los hiperparámetros y realizar experimentos adicionales para mejorar el rendimiento del modelo.

Sus **beneficios** son los siguientes:

- ▶ **Organización y trazabilidad.** SageMaker Experiments proporciona una plataforma centralizada para organizar y gestionar todos los experimentos de ML, lo que facilita la trazabilidad y la recuperación de resultados en cualquier momento.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Análisis y mejora continua.** El servicio facilita la comparación y el análisis de múltiples experimentos de ML, lo que permite a los usuarios identificar patrones y tendencias que pueden conducir a mejoras significativas en el rendimiento del modelo.
- ▶ **Colaboración y compartición.** SageMaker Experiments facilita la colaboración y la compartición de resultados entre los miembros del equipo, lo que fomenta la colaboración y el intercambio de ideas en proyectos de ML.

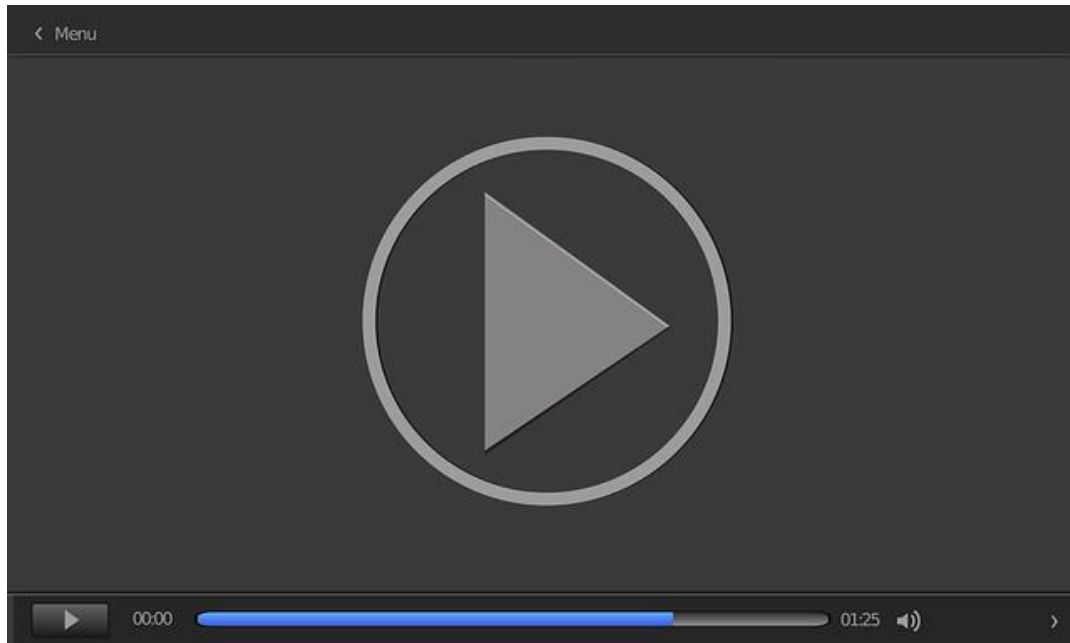
Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Optimización de hiperparámetros.** Utiliza SageMaker Experiments para realizar experimentos de optimización de hiperparámetros y encontrar la combinación óptima de parámetros para maximizar el rendimiento del modelo.
- ▶ **Comparación de modelos.** Utiliza el servicio para comparar múltiples modelos de ML y determinar cuál es el más adecuado para un determinado problema o conjunto de datos.
- ▶ **Análisis de impacto.** Utiliza SageMaker Experiments para analizar el impacto de cambios en el diseño del modelo, los datos de entrenamiento o los hiperparámetros en el rendimiento del modelo.

Amazon SageMaker Experiments es una herramienta poderosa y versátil que simplifica y agiliza la gestión y el análisis de experimentos de ML. Al proporcionar capacidades avanzadas para organizar, registrar, comparar y analizar experimentos de ML, SageMaker Experiments ayuda a los científicos de datos y desarrolladores a tomar decisiones informadas y mejorar continuamente el rendimiento de sus modelos de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

En el vídeo *Managing ML experiments - AWS Machine Learning in 15* (AWS Developers, 2023), podrás aprender sobre la administración y el uso de AWS SageMaker Experiments.



Managing ML experiments - AWS Machine Learning in 15.

Accede al vídeo:

<https://www.youtube.com/embed/kK1ohrpJFC0>

AWS Sagemaker Debugger

Amazon SageMaker Debugger es un servicio de Amazon SageMaker que permite a los desarrolladores y científicos de datos depurar y monitorear modelos de ML durante el entrenamiento. Proporciona herramientas avanzadas para identificar y corregir problemas comunes de entrenamiento de modelos, como sobreajuste, subajuste, gradientes nulos o saturados, y más. Al proporcionar visibilidad en tiempo real sobre el estado interno del modelo durante el entrenamiento, SageMaker Debugger ayuda a mejorar la eficiencia y la calidad del proceso de entrenamiento de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

A continuación, exploraremos en profundidad las **características** y las **funcionalidades** clave de Amazon SageMaker Debugger.

Las principales **características** son:

- ▶ **Monitorización continua.** SageMaker Debugger monitorea continuamente el estado interno del modelo durante el entrenamiento, capturando métricas y datos relevantes, como los valores de los gradientes, los valores de activación, los pesos de las capas y más.
- ▶ **Identificación de problemas en tiempo real.** El servicio identifica automáticamente problemas comunes durante el entrenamiento, como sobreajuste, subajuste, gradientes nulos o saturados, y más, proporcionando alertas en tiempo real y recomendaciones para abordar estos problemas.
- ▶ **Análisis de comportamiento.** SageMaker Debugger permite a los usuarios visualizar y analizar el comportamiento del modelo durante el entrenamiento, ya que identifica patrones y tendencias que pueden conducir a mejoras en el rendimiento y la eficiencia del modelo.
- ▶ **Integración con SageMaker y *frameworks* de ML.** El servicio está integrado con Amazon SageMaker y es compatible con una variedad de *frameworks* de ML, como TensorFlow, PyTorch, MXNet y más, lo que permite a los usuarios depurar modelos contruidos en estos *frameworks* de manera fácil y transparente.
- ▶ **Personalización y extensibilidad.** SageMaker Debugger es altamente personalizable y extensible, lo que permite a los usuarios definir reglas de detección personalizadas, agregar métricas adicionales y adaptar el servicio a las necesidades específicas de su aplicación y entorno de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Optimización de costes.** Al proporcionar visibilidad en tiempo real sobre el estado del modelo y la eficiencia del entrenamiento, SageMaker Debugger ayuda a optimizar los recursos de cómputo utilizados durante el entrenamiento, lo que reduce los costos operativos.

Funciona de este modo:

- ▶ **Configuración del entorno de entrenamiento.** Los usuarios configuran el entorno de entrenamiento en Amazon SageMaker, especificando los algoritmos, los hiperparámetros y otros parámetros relevantes para el entrenamiento del modelo.
- ▶ **Activación de Debugger.** Los usuarios activan SageMaker Debugger en el entorno de entrenamiento al especificar las reglas de detección y las métricas que desean monitorear durante el entrenamiento.
- ▶ **Entrenamiento del modelo.** Una vez configurado y activado, SageMaker Debugger monitorea continuamente el estado del modelo durante el entrenamiento y captura métricas y datos relevantes en tiempo real.
- ▶ **Identificación y corrección de problemas.** El servicio identifica automáticamente problemas durante el entrenamiento, como sobreajuste, subajuste, gradientes nulos o saturados, y más, proporcionando alertas en tiempo real y recomendaciones para abordar estos problemas.
- ▶ **Análisis y mejora continua.** Basándose en las alertas y las recomendaciones proporcionadas por SageMaker Debugger, los usuarios pueden analizar y mejorar continuamente el rendimiento y la eficiencia de sus modelos de M al iterar en el diseño del modelo y ajustar los parámetros de entrenamiento según sea necesario.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Sus **beneficios** son los siguientes:

- ▶ **Mejora del rendimiento del modelo.** SageMaker Debugger ayuda a identificar y corregir problemas comunes durante el entrenamiento del modelo, lo que conduce a modelos de ML más eficientes y precisos.
- ▶ **Reducción de costes.** Al proporcionar visibilidad en tiempo real sobre el estado del modelo y la eficiencia del entrenamiento, SageMaker Debugger ayuda a optimizar los recursos de cómputo utilizados durante el entrenamiento, lo que reduce los costos operativos.
- ▶ **Aceleración del desarrollo.** Al identificar automáticamente problemas durante el entrenamiento y proporcionar recomendaciones para abordarlos, SageMaker Debugger acelera el proceso de desarrollo de modelos de ML, lo que les permite a los usuarios iterar más rápidamente en el diseño del modelo y ajustar los parámetros de entrenamiento según sea necesario.

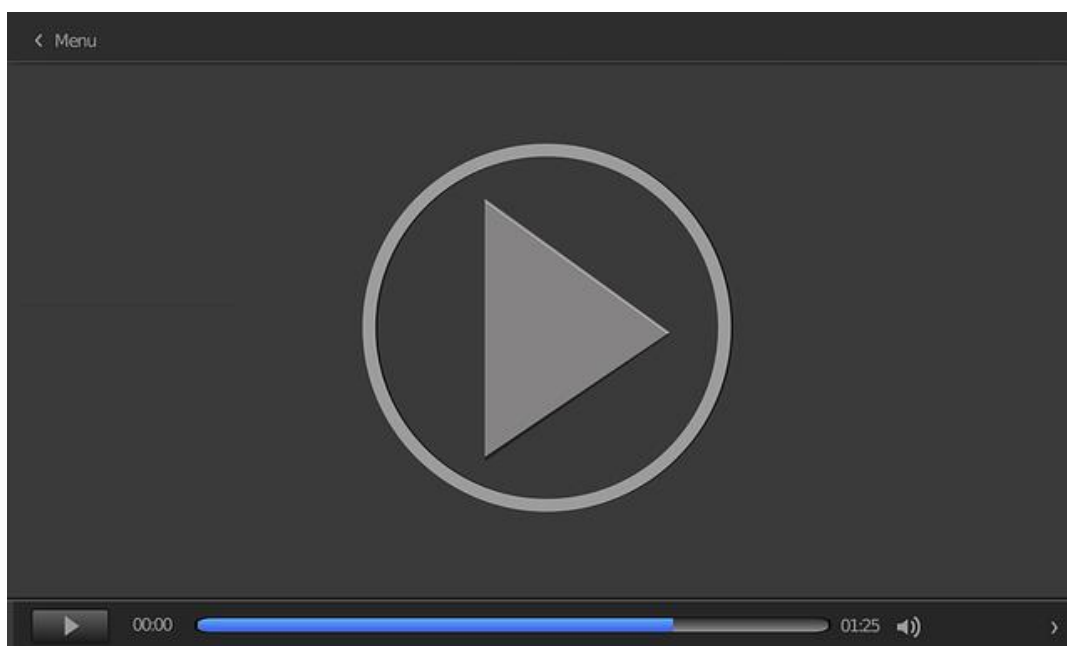
Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Identificación de sobreajuste y subajuste.** Utiliza SageMaker Debugger para identificar y corregir problemas de sobreajuste y subajuste durante el entrenamiento del modelo, lo que mejora la generalización y el rendimiento del modelo en datos nuevos.
- ▶ **Detección de problemas de gradientes.** Utiliza el servicio para identificar y corregir problemas de gradientes nulos o saturados, lo que puede conducir a problemas de convergencia durante el entrenamiento del modelo.
- ▶ **Optimización de recursos de cómputo.** Utiliza SageMaker Debugger para optimizar los recursos de cómputo utilizados durante el entrenamiento del modelo, lo que reduce los costos operativos y mejora la eficiencia del proceso de entrenamiento.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Amazon SageMaker Debugger es una herramienta poderosa y versátil que proporciona capacidades avanzadas para depurar y monitorear modelos de ML durante el entrenamiento. Al identificar automáticamente problemas comunes y proporcionar recomendaciones para abordarlos, SageMaker Debugger ayuda a mejorar la eficiencia y la calidad del proceso de entrenamiento de modelos de ML, lo que les permite a los usuarios construir modelos más precisos y eficientes en menos tiempo y con menos esfuerzo.

Un ejemplo de cómo se puede usar AWS SageMaker para debuggear un modelo se puede ver en el vídeo *Debugging a Customer Churn Model Using SageMaker Debugger* (Amazon Web Services, 2020).



Debugging a Customer Churn Model Using SageMaker Debugger.

Accede al vídeo:

<https://www.youtube.com/embed/8b5-lyRaFgA>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Despliegue e implementación de modelos de ML en SageMaker

AWS SageMaker Pipelines

Amazon SageMaker Pipelines es un servicio que permite a los equipos de ML construir, automatizar y gestionar flujos de trabajo de ML de extremo a extremo de manera eficiente en Amazon SageMaker. Proporciona una interfaz de usuario gráfica y basada en código para definir, ejecutar y monitorizar *pipelines* de ML, desde la preparación de datos y la experimentación con modelos hasta el entrenamiento y el despliegue en producción. A continuación, tienes una descripción detallada.

Las principales **características** son:

- ▶ **Desarrollo de *pipelines* reutilizables.** SageMaker Pipelines permite a los usuarios crear pipelines de ML reutilizables y modulares, lo que facilita la construcción y el mantenimiento de flujos de trabajo complejos y escalables.
- ▶ **Automatización del proceso.** El servicio automatiza muchas tareas comunes de ML, como la preparación de datos, la selección de modelos, la optimización de hiperparámetros y la implementación en producción, lo que acelera significativamente el tiempo de desarrollo y despliegue de modelos de ML.
- ▶ **Integración con SageMaker y servicios de AWS.** SageMaker Pipelines está integrado con otros servicios de SageMaker, como SageMaker Experiments y SageMaker Model Registry, así como con servicios de AWS como Amazon S3, AWS Glue y AWS Step Functions, lo que proporciona una experiencia de desarrollo cohesiva y unificada en el ecosistema de AWS.
- ▶ **Seguimiento y monitorización.** El servicio proporciona capacidades avanzadas para el seguimiento y la monitorización de *pipelines* de ML, incluyendo la visualización de métricas de rendimiento, el registro de artefactos generados durante el entrenamiento y la detección de desviaciones en el comportamiento del modelo.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Gestión de versiones y auditoría.** SageMaker Pipelines facilita la gestión de versiones y la auditoría de pipelines de ML, lo que permite a los usuarios realizar un seguimiento de los cambios realizados en los *pipelines* y revertir a versiones anteriores si es necesario.

Funciona de este modo:

- ▶ **Definición del *pipeline*.** Los usuarios definen el *pipeline* de ML utilizando la interfaz de usuario gráfica o basada en código de SageMaker Pipelines, lo que especifica los pasos del *pipeline*, las dependencias entre ellos y los parámetros relevantes.
- ▶ **Ejecución del *pipeline*.** Una vez definido el *pipeline*, los usuarios pueden ejecutarlo en SageMaker o en su entorno local utilizando los recursos de cómputo y almacenamiento necesarios.
- ▶ **Seguimiento y monitorización.** Durante la ejecución del *pipeline*, SageMaker Pipelines registra las métricas de rendimiento, artefactos generados y cualquier información adicional relevante, lo que proporciona visibilidad en tiempo real sobre el estado y el comportamiento del *pipeline*.
- ▶ **Auditoría y gestión de versiones.** El servicio facilita la gestión de versiones y la auditoría de *pipelines* de ML, permitiendo a los usuarios realizar un seguimiento de los cambios realizados en los *pipelines* y revertir a versiones anteriores si es necesario.

Sus **beneficios** son los siguientes:

- ▶ **Automatización y eficiencia.** SageMaker Pipelines automatiza muchas tareas comunes de ML, lo que acelera significativamente el tiempo de desarrollo y el despliegue de modelos de ML y mejora la eficiencia operativa.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Reusabilidad y modularidad.** El servicio permite a los usuarios crear pipelines de ML reutilizables y modulares, lo que facilita la construcción y el mantenimiento de flujos de trabajo complejos y escalables.
- ▶ **Seguimiento y monitorización avanzados.** SageMaker Pipelines proporciona capacidades avanzadas para el seguimiento y la monitorización de *pipelines* de ML, lo que les permite a los usuarios visualizar métricas de rendimiento, registrar artefactos generados y detectar desviaciones en el comportamiento del modelo.

Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Desarrollo de modelos de ML en producción.** Utiliza SageMaker Pipelines para construir y desplegar pipelines de ML que abarquen todas las etapas del ciclo de vida de ML, desde la preparación de datos hasta el despliegue en producción.
- ▶ **Experimentación y optimización de modelos.** Utiliza el servicio para experimentar con diferentes modelos, algoritmos y parámetros, y optimizar el rendimiento del modelo en función de métricas específicas.
- ▶ **Seguimiento y auditoría de modelos.** Utiliza SageMaker Pipelines para realizar un seguimiento y una auditoría de los modelos de ML en producción, detectar desviaciones en el comportamiento del modelo y tomar medidas correctivas si es necesario.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

5.3. MLOps con SageMaker

MLOps (machine learning operations)

MLOps, también conocida como operaciones de aprendizaje automático (*machine learning operations*), es una práctica que combina los principios de DevOps con el desarrollo y la implementación de modelos de aprendizaje automático. Se centra en gestionar el ciclo de vida completo de los modelos de ML, desde el desarrollo y la formación hasta la implementación, monitorización y mantenimiento en producción.

Las MLOps son un conjunto de prácticas que automatizan y simplifican los flujos de trabajo y las implementaciones de ML. El ML y la inteligencia artificial (IA) son capacidades fundamentales que puede implementar para resolver problemas complejos del mundo real y aportar valor a sus clientes. MLOps es una cultura y una práctica de ML que une el desarrollo de aplicaciones de ML (*dev*) a la implementación y las operaciones (*ops*) de sistemas de ML. Se puede usar las MLOps para automatizar y estandarizar los **procesos** a lo largo del ciclo de vida del ML. Estos procesos incluyen el desarrollo de modelos, las pruebas, la integración, el lanzamiento y la administración de la infraestructura.

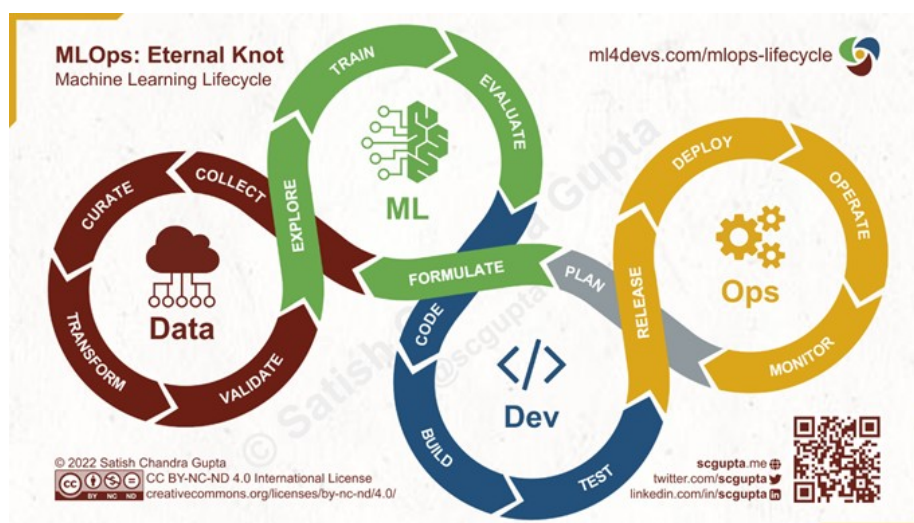


Figura 3. Ciclo de vida del ML. Fuente: ML4Devs, 2022.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

¿Por qué se necesita MLOps?

MLOps es fundamental para administrar de forma sistemática y simultánea el lanzamiento de **nuevos modelos de ML** con cambios en el código de la aplicación y los datos. Una implementación óptima de MLOps trata los activos de ML de manera similar a otros activos de *software* de entornos de integración y entrega continuas (CI/CD). Los modelos de ML se implementan junto con las aplicaciones y los servicios que utilizan y aquellos que los consumen como parte de un proceso de lanzamiento unificado.

MLOps es necesario por varias razones importantes:

- ▶ **Escalabilidad y eficiencia.** Con el crecimiento exponencial de los datos y el aumento en la demanda de modelos de ML, es crucial contar con procesos y herramientas que permitan escalar el desarrollo, implementación y gestión de estos modelos de manera eficiente. MLOps proporciona prácticas y herramientas que permiten gestionar grandes volúmenes de modelos y datos de manera eficiente y escalable.
- ▶ **Agilidad y velocidad.** En un entorno empresarial altamente competitivo, la capacidad de desarrollar y desplegar modelos de ML de manera rápida y ágil es crucial para mantenerse a la vanguardia. MLOps facilita la implementación de prácticas como la integración continua, la entrega continua y el despliegue automatizado, lo que permite a las organizaciones desarrollar y poner en producción modelos de manera rápida y consistente.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Fiabilidad y estabilidad.** Los modelos de ML en producción deben ser confiables y estables para garantizar su utilidad y eficacia en el mundo real. MLOps proporciona prácticas y herramientas para garantizar la calidad, la confiabilidad y la estabilidad de los modelos en producción, incluyendo la monitorización continua del rendimiento y la precisión del modelo, así como la detección y corrección rápida de problemas operativos.
- ▶ **Gestión del riesgo y cumplimiento.** El despliegue de modelos de ML en producción conlleva riesgos potenciales, como la exposición de datos sensibles o la toma de decisiones incorrectas basadas en modelos defectuosos. MLOps ayuda a mitigar estos riesgos mediante la implementación de prácticas de seguridad robustas, la monitorización continua del rendimiento del modelo y el cumplimiento de regulaciones y estándares de seguridad.
- ▶ **Maximización del valor del negocio.** En última instancia, MLOps tiene como objetivo maximizar el valor del negocio derivado de los modelos de ML. Al proporcionar procesos y herramientas que permiten desarrollar, implementar y gestionar modelos de manera eficiente y efectiva, MLOps ayuda a las organizaciones a aprovechar al máximo sus inversiones en inteligencia artificial y ML, generando *insights* valiosos y conduciendo decisiones empresariales informadas.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

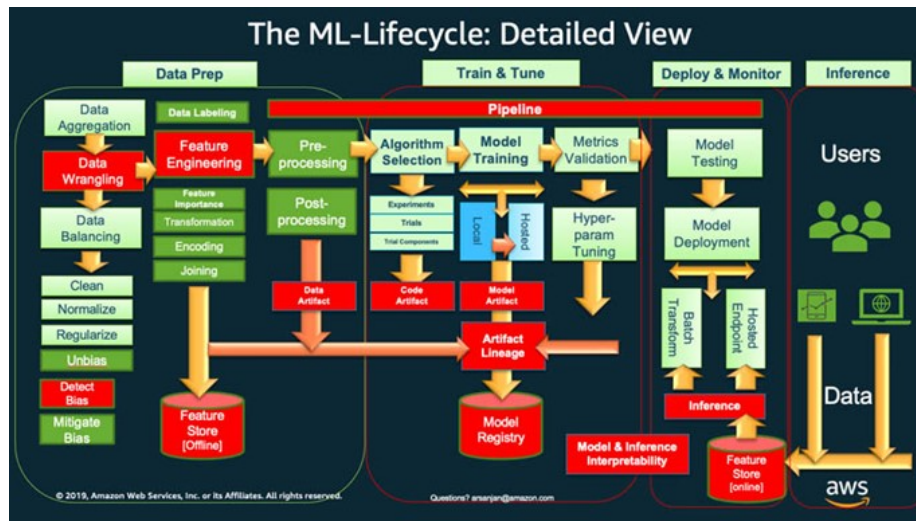


Figura 4. Ciclo de vida del ML. Fuente: Amazon Web Services, s. f.-a.

Principios de MLOps

A continuación, explicamos seis **principios clave** de las MLOps. El primer principio es el del **control de versiones**. Este proceso implica el seguimiento de los cambios en los activos de ML para que pueda reproducir los resultados y volver a usar versiones anteriores si es necesario. Todas las especificaciones de modelos o código de entrenamiento de ML pasan por una fase de revisión del código. Cada una con su número de versión para que el entrenamiento de los modelos de ML se pueda reproducir y auditar.

La **capacidad de replicación** en un flujo de trabajo de ML es importante en todas las fases, desde el procesamiento de datos hasta la implementación del modelo de ML. Implica que cada fase debería producir resultados idénticos con la misma entrada.

El segundo principio es la **automatización**. Se puede automatizar diversas etapas de la canalización de ML para garantizar la repetibilidad, la coherencia y la escalabilidad. Esto incluye etapas que van desde la ingesta de datos, el preprocesamiento, el entrenamiento y la validación del modelo hasta la implementación.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Estos son algunos **factores** que pueden desencadenar el entrenamiento y la implementación automatizados de modelos:

- ▶ Mensajería.
- ▶ Monitoreo de los eventos en el calendario.
- ▶ Cambios en los datos.
- ▶ Cambios en el código de entrenamiento del modelo.
- ▶ Cambios en el código de la aplicación.

Las **pruebas automatizadas** permiten descubrir los problemas de forma temprana para corregir errores y aprender rápidamente. La automatización es más eficiente con la infraestructura como código (IaC). Puede usar herramientas para definir y administrar la infraestructura. De esta manera, se asegura de que puede repetir los procesos e implementar los recursos de manera uniforme en varios entornos.

El **despliegue continuo** es un principio clave en MLOps que promueve la implementación rápida y segura de modelos de ML en producción. Se deben utilizar prácticas como la integración continua (CI), la entrega continua (CD) y el despliegue automatizado para garantizar que los modelos se implementen de manera rápida, segura y consistente en entornos de producción.

El cuarto principio es la **gobernanza de los modelos**. La gobernanza implica administrar todos los aspectos relacionados con los sistemas de ML para lograr una mayor eficiencia. Debe realizar muchas actividades para encargarse de la gobernanza:

- ▶ Fomentar una estrecha **colaboración** entre los científicos de datos, los ingenieros y las partes interesadas de la empresa.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ Utilizar **documentación** clara y **canales de comunicación** efectivos para asegurarse de que todos estén al tanto y de acuerdo con los planes.
- ▶ Establecer mecanismos para recopilar comentarios sobre las **predicciones de los modelos** y entrenarlos aún más.
- ▶ Asegurar que los **datos confidenciales** estén protegidos, que el acceso a los modelos y la infraestructura sea seguro, y que se cumplan los requisitos de conformidad.
- ▶ También es esencial contar con un **proceso** estructurado para revisar, validar y aprobar los modelos antes de que se empiecen a utilizar. Esto puede implicar comprobar la imparcialidad, los prejuicios y las consideraciones éticas.

La **monitorización continua**, quinto principio, del rendimiento y la precisión de los modelos en producción es esencial en MLOps. Esto implica la detección temprana de problemas de rendimiento, la gestión de la deriva de los datos y la actualización periódica de los modelos para mantener su eficacia a lo largo del tiempo. La monitorización continua garantiza que los modelos sigan siendo precisos y relevantes en un entorno en constante cambio.

Por último, la **gestión de la infraestructura**, que es el sexto principio. MLOps incluye la gestión eficiente de la infraestructura necesaria para ejecutar modelos de ML en producción. Esto implica el aprovisionamiento de recursos, la configuración de entornos de ejecución y la optimización del rendimiento y la escalabilidad de la infraestructura subyacente. Una gestión eficiente de la infraestructura garantiza que los modelos se ejecuten de manera óptima en entornos de producción.

En resumen, los **principales principios de MLOps** se centran en la automatización, la colaboración, el despliegue continuo, la monitorización continua y la gestión eficiente de la infraestructura para garantizar que los modelos de ML se desarrollen, implementen y gestionen de manera eficiente y efectiva en producción. Estos

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

principios ayudan a maximizar el valor del negocio derivado de los modelos de ML y a mantener su eficacia a lo largo del tiempo.

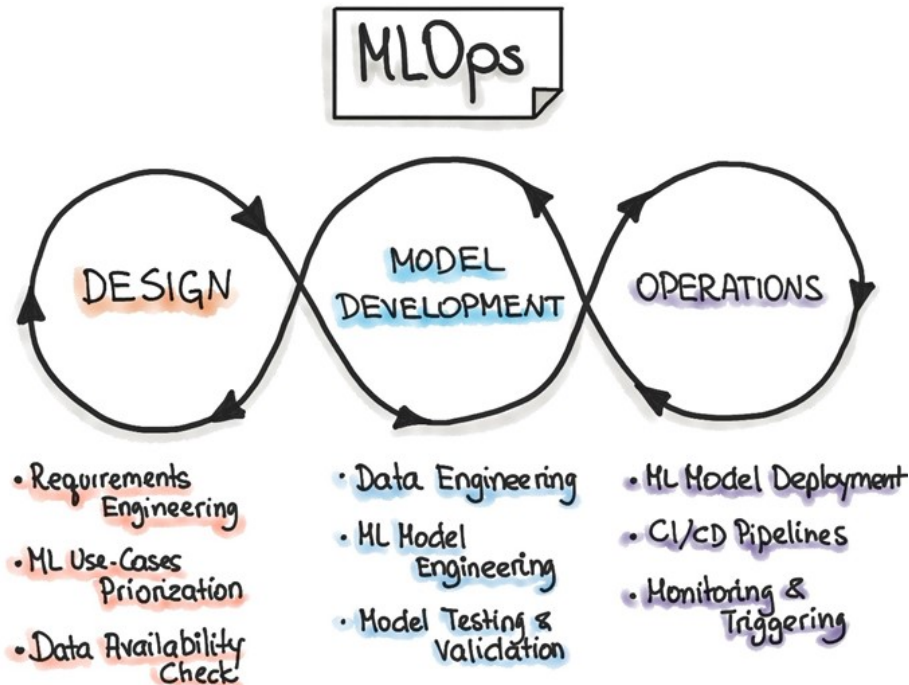


Figura 5. MLOps. Fuente: Jorge Luis, 2023.

Beneficios de MLOps

MLOps aporta una serie de beneficios significativos a una empresa, ayudando a maximizar el valor de sus inversiones en ML y optimizando el rendimiento y la eficiencia de sus modelos en producción. Seguidamente, tienes una descripción detallada de los **beneficios** que MLOps puede ofrecer a una empresa:

- **Velocidad y agilidad en el desarrollo.** MLOps permite un desarrollo más rápido y ágil de modelos de ML al facilitar la automatización de tareas, la integración continua y la entrega continua. Esto reduce el tiempo de comercialización de los modelos y permite que las empresas se adaptan más rápidamente a las cambiantes necesidades del mercado.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Mayor precisión y rendimiento.** Al facilitar la monitorización continua del rendimiento y la precisión de los modelos en producción, MLOps ayuda a garantizar que los modelos sean precisos y efectivos en la toma de decisiones empresariales. La detección temprana de problemas de rendimiento y la gestión de la deriva de los datos permiten mantener la calidad y la relevancia de los modelos a lo largo del tiempo.
- ▶ **Reducción de costes.** MLOps puede ayudar a reducir los costos operativos y de mantenimiento al automatizar tareas repetitivas y minimizar el tiempo de inactividad de los modelos en producción. La automatización de procesos, como la preparación de datos, el entrenamiento de modelos y la monitorización, puede aumentar la eficiencia y reducir la necesidad de recursos humanos y computacionales.
- ▶ **Escalabilidad y flexibilidad.** MLOps permite una mayor escalabilidad y flexibilidad en el desarrollo y la gestión de modelos de ML. Al facilitar el despliegue continuo y la gestión eficiente de la infraestructura, MLOps les permite a las empresas escalar fácilmente el número de modelos en producción y adaptarse a las cambiantes necesidades del negocio.
- ▶ **Gestión de riesgos y cumplimiento.** MLOps ayuda a mitigar los riesgos asociados con la implementación de modelos de ML en producción al proporcionar prácticas de seguridad robustas y garantizar el cumplimiento de regulaciones y estándares de seguridad. Esto ayuda a proteger los datos sensibles y garantiza que los modelos cumplan con los requisitos legales y regulatorios.
- ▶ **Maximización del valor del negocio.** En última instancia, MLOps ayuda a maximizar el valor del negocio derivado de los modelos de ML al garantizar que sean desarrollados, implementados y gestionados de manera eficiente y efectiva en producción. Esto permite a las empresas aprovechar al máximo sus inversiones en ML y generar *insights* valiosos para impulsar decisiones empresariales informadas y estratégicas.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

¿Cómo se implementan las MLOps en la organización?

Hay tres niveles de implementación de MLOps, según la **madurez de la automatización** en su organización.

Nivel 0 de MLOps

El nivel 0 para las organizaciones que recién comienzan a usar sistemas de ML se caracteriza por incluir **flujos de trabajo** de ML manuales y un **proceso** dirigido por científicos de datos.

Cada paso es manual, incluida la preparación de los datos, el entrenamiento de ML y el desempeño y la validación del modelo. Requiere una **transición manual** entre los pasos, y cada paso se ejecuta y administra de forma interactiva. Los científicos de datos suelen entregar modelos entrenados como artefactos que el equipo de ingeniería implementa en la infraestructura de API.

El **proceso** separa a los científicos de datos que crean el modelo de los ingenieros que lo implementan. Los **lanzamientos** poco frecuentes implican que los equipos de ciencia de datos pueden volver a entrenar los modelos solo unas pocas veces al año. No hay consideraciones de CI/CD para los modelos de ML con el resto del código de la aplicación. Del mismo modo, no se monitorea activamente el **rendimiento**.

Nivel 1 de MLOps

Las organizaciones que desean entrenar los mismos modelos con datos nuevos con frecuencia requieren una implementación de nivel 1. El nivel 1 de MLOps tiene como **objetivo** entrenar el modelo de forma continua mediante la automatización de la canalización de ML.

En el nivel 0, se implementa un modelo entrenado en la etapa de producción. Por otro lado, en el nivel 1, se implementa una **canalización de entrenamiento** que se ejecuta de forma recurrente para ofrecer el modelo entrenado a las demás

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

aplicaciones. Como mínimo, se logra una entrega continua del servicio de predicción con modelo.

El nivel 1 tiene las siguientes **características**:

- ▶ **Pasos para experimentar rápidamente con ML** que implican bastante automatización.
- ▶ **Entrenamiento continuo del modelo en producción** con datos nuevos que activan la canalización en funcionamiento.
- ▶ **Implementación de la misma canalización** en todos los entornos de desarrollo, preproducción y producción.

Los equipos de ingeniería trabajan con los científicos de datos para crear **componentes de código en módulos** que se puedan reutilizar, que puedan definirse de acuerdo con la componibilidad y que potencialmente se compartan con todas las canalizaciones de ML. También crea un **almacén de características centralizado** que estandariza el almacenamiento, el acceso y la definición de las características para el entrenamiento y el servicio de ML. Además, puede administrar los **metadatos**, como la información sobre cada ejecución de la canalización y los datos sobre la capacidad de replicación.

Nivel 2 de MLOps

El nivel 2 de MLOps es para las organizaciones que desean experimentar más y crear con frecuencia modelos nuevos que requieran **entrenamiento continuo**. Es adecuado para empresas impulsadas por la tecnología que actualizan sus modelos en cuestión de minutos, los entrenan cada una hora o todos los días y, al mismo tiempo, los vuelven a implementar en miles de servidores.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Como hay varias canalizaciones de ML en juego, una configuración de MLOps de nivel 2 requiere la **configuración** completa de MLOps de nivel 1. También requiere lo siguiente:

- ▶ Un orquestador de canalizaciones de ML.
- ▶ Un registro de modelos para controlar varios modelos.

Las tres **etapas** siguientes se repiten a escala para varias canalizaciones de ML para garantizar la entrega continua del modelo:

- ▶ **Creación de la canalización.** Prueba de forma iterativa nuevos modelos y nuevos algoritmos de ML y, al mismo tiempo, se asegura de que se hayan organizado los pasos de experimentación. Esta etapa genera el código fuente de sus canalizaciones de ML. El código se almacena en un repositorio de código fuente.
- ▶ **Implementación de la canalización.** A continuación, se crea el código fuente y ejecuta las pruebas para obtener los componentes de la canalización para su implementación. El resultado es una canalización desplegada con la implementación del nuevo modelo.
- ▶ **Empleo de la canalización.** Por último, utiliza la canalización como un servicio de predicción para sus aplicaciones. Recopila estadísticas sobre el servicio implementado de predicción con modelo a partir de datos que se obtienen en tiempo real. Con el resultado de esta etapa, se activa la ejecución de la canalización o un nuevo ciclo de experimentación.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

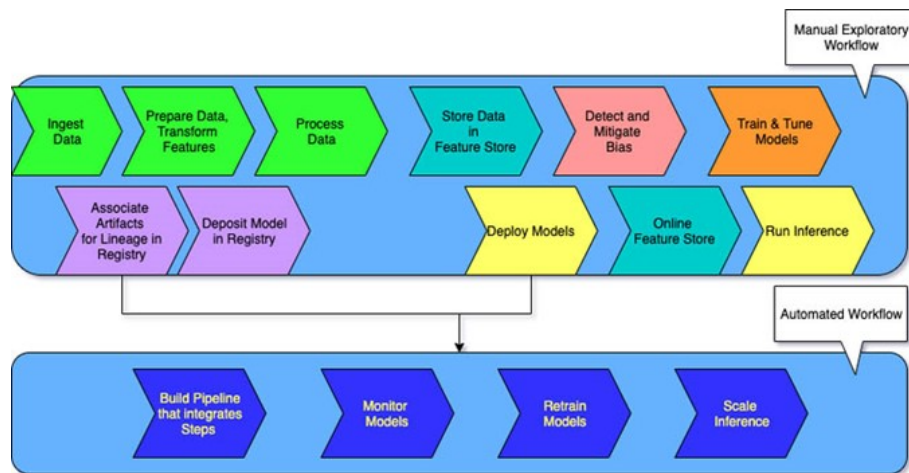


Figura 6. Investigación manual y el flujo de trabajo operativo automatizado. Fuente: Ali Arsanjani, 2021.

¿En qué se diferencia MLOps de DevOps y DataOps?

MLOps, DevOps y DataOps son prácticas relacionadas, pero tienen enfoques y objetivos ligeramente diferentes. Aquí hay una descripción de las diferencias entre MLOps, DevOps y DataOps.

► DevOps.

- DevOps se centra en la integración continua, la entrega continua y la colaboración entre equipos de desarrollo y operaciones para acelerar **el ciclo de vida** del desarrollo de *software*.
- El **objetivo** principal de DevOps es mejorar la velocidad, la calidad y la fiabilidad del desarrollo de *software*, así como fomentar una cultura de colaboración y responsabilidad compartida entre equipos.
- DevOps abarca todo el **ciclo de vida** del desarrollo de *software*, desde la planificación y el desarrollo hasta la implementación, la monitorización y el mantenimiento en producción.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

► DataOps.

- DataOps se centra en la **gestión eficiente de los datos** en toda la organización, desde su adquisición y almacenamiento hasta su procesamiento, análisis y entrega.
- El **objetivo** principal de DataOps es mejorar la calidad, la confiabilidad y la disponibilidad de los datos, así como acelerar el tiempo de comercialización de los proyectos de datos.
- DataOps promueve **prácticas**, como la automatización de la infraestructura de datos, la colaboración entre equipos de datos y de negocio, y la implementación de procesos ágiles, para el desarrollo y la entrega de soluciones de datos.

► MLOps.

- MLOps se centra específicamente en la gestión y el despliegue eficientes de **modelos de ML** en producción.
- El **objetivo** principal de MLOps es garantizar que los modelos de ML sean desarrollados, implementados y gestionados de manera eficiente y efectiva en entornos de producción.
- MLOps combina principios y prácticas de DevOps con herramientas y técnicas específicas para el **desarrollo y gestión de modelos de ML**, como la automatización del entrenamiento y despliegue de modelos, la monitorización continua del rendimiento y la precisión del modelo, y la gestión eficiente de la infraestructura de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

En resumen, mientras que **DevOps** se centra en el desarrollo de *software* en general y **DataOps** se centra en la gestión eficiente de los datos, **MLOps** se centra específicamente en la gestión de modelos de ML en producción. Cada práctica tiene su propio conjunto de principios y objetivos, pero todas comparten el objetivo de mejorar la eficiencia, la calidad y la fiabilidad en el desarrollo y la gestión de sistemas de información.

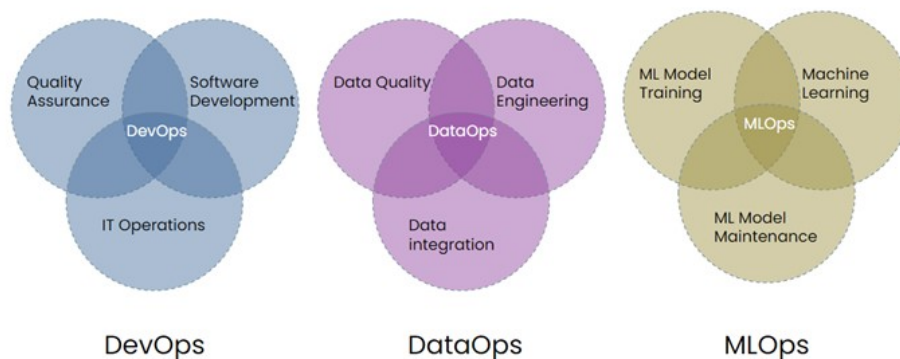


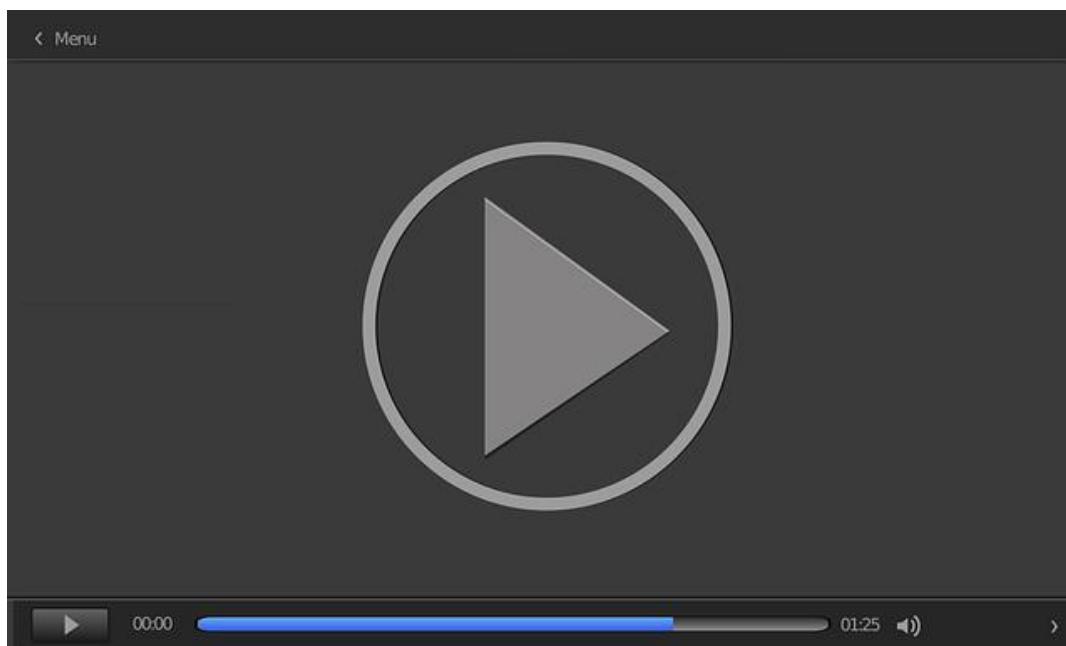
Figura 7. DevOps, DataOps y MLOps. Fuente: Sakarya, s. f.

MLOps en AWS SageMaker

Amazon SageMaker proporciona herramientas creadas expresamente para operaciones de *machine learning* (MLOps) para ayudar a automatizar y estandarizar los procesos en todo el ciclo de vida de ML. Al utilizar las herramientas de MLOps de SageMaker, puede entrenar, probar, solucionar, desplegar y gestionar modelos de ML a escala de forma sencilla para estimular la productividad de los científicos de datos y los ingenieros de ML, mientras mantiene el rendimiento del modelo en producción.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

A continuación, se muestra el vídeo *Amazon SageMaker MLOps | Amazon Web Services* (Amazon Web Services, 2022), que es introductorio al AWS SageMaker MLOps.



Amazon SageMaker MLOps | Amazon Web Services.

Accede al vídeo:

https://www.youtube.com/embed/FSOBmgT_9aQ

Funcionamiento de AWS SageMaker MLOps

Como ya hemos visto, AWS SageMaker es un servicio de aprendizaje automático completamente administrado que proporciona a los desarrolladores e ingenieros de datos las herramientas para crear, entrenar e implementar modelos de ML de manera rápida y sencilla en la nube de AWS. **SageMaker MLOps** es una extensión de SageMaker que se centra en gestionar el ciclo de vida completo de los modelos de ML en producción, desde el desarrollo y entrenamiento hasta la implementación y monitorización continua.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Una **descripción del funcionamiento** de AWS SageMaker MLOps sería la que se desarrolla a continuación. En primer lugar, el **desarrollo y entrenamiento de modelos**. SageMaker proporciona un entorno de desarrollo integrado que permite a los usuarios crear y experimentar con modelos de ML utilizando una variedad de algoritmos y marcos de trabajo populares, como TensorFlow, PyTorch y scikit-learn.

Los desarrolladores pueden utilizar las instancias de SageMaker para entrenar **modelos de manera escalable y eficiente**, aprovechando la potencia de cómputo de AWS para procesar grandes volúmenes de datos y experimentar con diferentes configuraciones de modelo.

En segundo lugar, la **automatización del despliegue y la implementación**. Una vez que se ha desarrollado y entrenado un modelo en SageMaker, MLOps facilita el despliegue y la implementación del modelo en producción.

SageMaker proporciona la **integración con servicios de AWS**, como AWS Lambda, Amazon ECS y Amazon ECR, para permitir la implementación automatizada de modelos en entornos de producción.

Los desarrolladores pueden utilizar *pipelines* de SageMaker para orquestar y automatizar el **flujo de trabajo** completo de implementación, desde la preparación de datos hasta el despliegue y la monitorización del modelo en producción.

Y, en tercer lugar, la **monitorización y optimización continua**. Una vez que un modelo se ha implementado en producción, SageMaker MLOps proporciona herramientas para monitorizar continuamente el rendimiento y la precisión del modelo.

Los **desarrolladores** pueden utilizar métricas y alertas de SageMaker para detectar problemas de rendimiento y desviaciones en el comportamiento del modelo, lo que permite una rápida identificación y resolución de problemas.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

SageMaker también proporciona capacidades de **optimización automática** de modelos, que les permiten a los desarrolladores mejorar continuamente el rendimiento del modelo utilizando técnicas como la búsqueda de hiperparámetros y la optimización de arquitecturas de red neuronal.

En resumen, **AWS SageMaker MLOps** simplifica el ciclo de vida completo de los modelos de ML en producción, desde el desarrollo y entrenamiento hasta la implementación y monitorización continua. Proporciona herramientas y servicios integrados que permiten a los desarrolladores e ingenieros de datos crear, entrenar e implementar modelos de ML de manera rápida, eficiente y escalable en la nube de AWS.

En la Figura 8 podemos ver un diagrama de la **arquitectura del flujo de funcionamiento** de SageMaker MLOps.

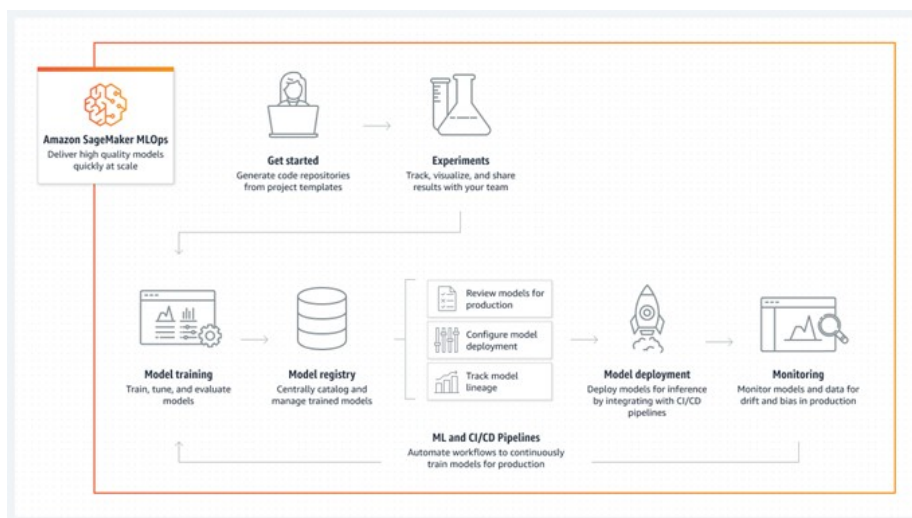


Figura 8. Amazon SageMaker para MLOps. Fuente: Amazon Web Services, s. f.-b.

A continuación, revisaremos los servicios de SageMaker ligados a la implementación de MLOps.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

AWS SageMaker Projects

La estandarización de los entornos de desarrollo de ML aumenta la productividad de los científicos de datos y, finalmente, el ritmo de la innovación mediante la facilidad para lanzar nuevos proyectos, rotar a los científicos de datos en los proyectos e implementar las mejores prácticas de ML. **Amazon SageMaker Projects** ofrece plantillas para el rápido aprovisionamiento de entornos de científicos de datos estandarizados con herramientas y bibliotecas, repositorios de control de origen, código modelo y canales CI/CD actualizados y bien probados.

SageMaker Projects permite gestionar:

- ▶ Creación de imágenes personalizadas para su procesamiento, entrenamiento e inferencia.
- ▶ Preparación de datos e ingeniería de características.
- ▶ Entrenamiento de modelos.
- ▶ Evaluación de modelos.
- ▶ Implementación de modelos.
- ▶ Supervisión y actualización de modelos.

SageMaker Projects ayuda a las organizaciones a configurar y estandarizar **entornos de desarrollo** para científicos de datos y sistemas CI/CD para ingenieros de MLOps. Los **proyectos** también ayudan a las organizaciones a configurar la gestión de dependencias, la gestión de repositorios de códigos, generar reproducibilidad y compartir artefactos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

¿Por qué se deberían utilizar SageMaker Projects? Si bien las *notebooks* son útiles para la experimentación y la creación de modelos, un equipo de científicos de datos e ingenieros de aprendizaje automático que comparten código necesita una forma más escalable de mantener la **coherencia del código y un control estricto de las versiones**.

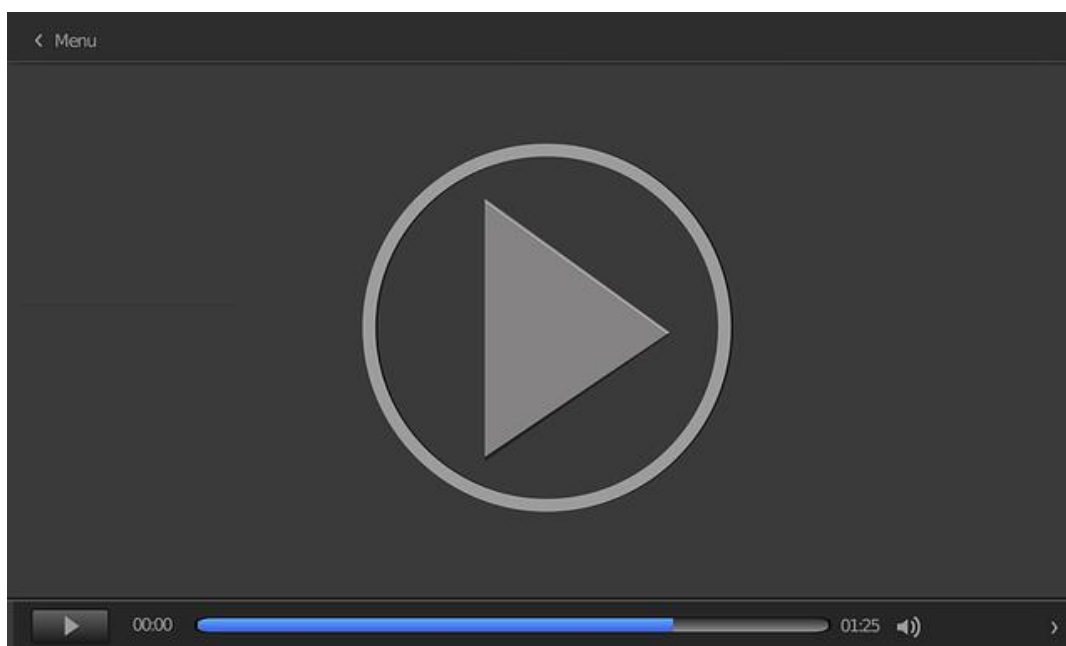
Cada organización tiene su propio conjunto de estándares y prácticas que brindan seguridad y gobernanza para su entorno de AWS. SageMaker proporciona un conjunto de **plantillas** propias para las organizaciones que desean comenzar rápidamente con flujos de trabajo de ML y CI/CD. Las plantillas incluyen proyectos que utilizan servicios nativos de AWS para CI/CD, como AWS CodeBuild, AWS CodePipeline y AWS CodeCommit. Las plantillas también ofrecen la opción de crear proyectos que utilizan herramientas de terceros, como Jenkins y GitHub. Para obtener una lista de las plantillas de proyecto que proporciona SageMaker, consulta cómo usar plantillas de proyecto proporcionadas por SageMaker.

Las organizaciones a menudo necesitan un **control estricto** sobre los recursos MLOps que suministran y administran. Dicha responsabilidad supone ciertas tareas, incluida la configuración de roles y políticas de IAM, la aplicación de etiquetas de recursos, la aplicación del cifrado y el desacoplamiento de recursos en varias cuentas. SageMaker Projects puede admitir todas estas tareas a través de ofertas de plantillas personalizadas donde las organizaciones utilizan plantillas de AWS CloudFormation para definir los recursos necesarios para un flujo de trabajo de aprendizaje automático.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Los científicos de datos pueden elegir una **plantilla** para iniciar y preconfigurar su flujo de trabajo de aprendizaje automático. Estas plantillas personalizadas se crean como productos del catálogo de servicios y puede aprovisionarlas en la interfaz de usuario de Studio o Studio Classic en plantillas de organización. El **catálogo de servicios** es un servicio que ayuda a las organizaciones a crear y administrar catálogos de productos aprobados para su uso en AWS.

Se puede ahondar en el funcionamiento de SageMaker Projects en el vídeo *Automate MLOps with SageMaker Projects | Amazon Web Services* (Amazon Web Services, 2021c).



Automate MLOps with SageMaker Projects | Amazon Web Services.

Accede al vídeo:

https://www.youtube.com/embed/3_cHnk9VSfQ

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

AWS SageMaker Experiments

Amazon SageMaker Experiments es un servicio de Amazon SageMaker diseñado para ayudar a los científicos de datos y desarrolladores a organizar, rastrear y comparar experimentos de ML de manera eficiente. Proporciona una plataforma centralizada para registrar y gestionar todos los experimentos, así como para analizar y compartir los resultados obtenidos durante el desarrollo de modelos de ML.

Las principales **características** son:

- ▶ **Organización de experimentos.** SageMaker Experiments le permite a los usuarios organizar y gestionar todos los experimentos de ML en una ubicación centralizada, lo que facilita la búsqueda, recuperación y seguimiento de los experimentos realizados en diferentes proyectos.
- ▶ **Registro de metadatos.** El servicio registra automáticamente metadatos detallados de cada experimento, incluyendo los hiperparámetros utilizados, las métricas de rendimiento, los artefactos generados y más, lo que proporciona una trazabilidad completa y una base sólida para el análisis posterior.
- ▶ **Comparación de experimentos.** SageMaker Experiments proporciona capacidades avanzadas para comparar y analizar múltiples experimentos de ML, lo que permite a los usuarios identificar patrones, tendencias y mejores prácticas que pueden conducir a mejoras en el rendimiento del modelo.
- ▶ **Visualización de resultados.** El servicio ofrece herramientas de visualización integradas que permiten a los usuarios explorar y visualizar los resultados de los experimentos de ML de manera intuitiva, lo que facilita la comprensión y la interpretación de los datos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Colaboración y compartición.** SageMaker Experiments facilita la colaboración y la compartición de resultados entre los miembros del equipo, lo que les permite a varios usuarios trabajar juntos en el análisis y la iteración de experimentos de ML.

Funciona de este modo:

- ▶ **Creación de experimentos.** Los usuarios crean experimentos de ML utilizando SageMaker Experiments y registran metadatos detallados, como hiperparámetros, métricas y artefactos asociados con cada experimento.
- ▶ **Ejecución de experimentos.** Una vez creado un experimento, los usuarios pueden ejecutarlo en SageMaker o en su entorno local utilizando los recursos de cómputo y almacenamiento necesarios.
- ▶ **Registro de resultados.** El servicio registra automáticamente los resultados de cada experimento, lo que incluye métricas de rendimiento, artefactos generados y cualquier información adicional relevante.
- ▶ **Comparación y análisis.** Una vez completados varios experimentos, los usuarios pueden comparar y analizar los resultados, utilizando herramientas de visualización integradas e identificando patrones, tendencias y áreas de mejora.
- ▶ **Iteración y mejora.** Basándose en los resultados obtenidos, los usuarios pueden iterar en el diseño de los modelos, ajustar los hiperparámetros y realizar experimentos adicionales para mejorar el rendimiento del modelo.

Sus **beneficios** son los siguientes:

- ▶ **Organización y trazabilidad.** SageMaker Experiments proporciona una plataforma centralizada para organizar y gestionar todos los experimentos de ML, lo que facilita la trazabilidad y la recuperación de resultados en cualquier momento.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Análisis y mejora continua.** El servicio facilita la comparación y el análisis de múltiples experimentos de ML, lo que permite a los usuarios identificar patrones y tendencias que pueden conducir a mejoras significativas en el rendimiento del modelo.
- ▶ **Colaboración y compartición.** SageMaker Experiments facilita la colaboración y la compartición de resultados entre los miembros del equipo, lo que fomenta la colaboración y el intercambio de ideas en proyectos de ML.

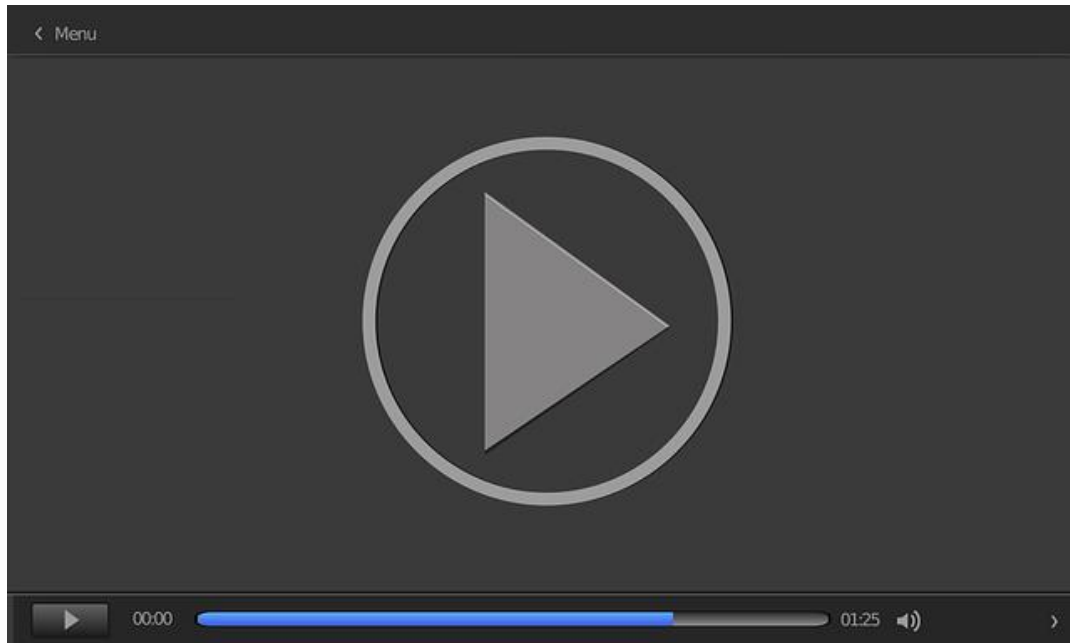
Los siguientes son algunos ejemplos de **casos de uso**:

- ▶ **Optimización de hiperparámetros.** Utiliza SageMaker Experiments para realizar experimentos de optimización de hiperparámetros y encontrar la combinación óptima de parámetros para maximizar el rendimiento del modelo.
- ▶ **Comparación de modelos.** Utiliza el servicio para comparar múltiples modelos de ML y determinar cuál es el más adecuado para un determinado problema o conjunto de datos.
- ▶ **Análisis de impacto.** Utiliza SageMaker Experiments para analizar el impacto de cambios en el diseño del modelo, los datos de entrenamiento o los hiperparámetros en el rendimiento del modelo.

Amazon SageMaker Experiments es una herramienta poderosa y versátil que simplifica y agiliza la gestión y el análisis de experimentos de ML. Al proporcionar capacidades avanzadas para organizar, registrar, comparar y analizar experimentos de ML, SageMaker Experiments ayuda a los científicos de datos y desarrolladores a tomar decisiones informadas y mejorar continuamente el rendimiento de sus modelos de ML.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

En el vídeo *Managing ML experiments - AWS Machine Learning in 15* (AWS Developers, 2023), podrás aprender sobre la administración y el uso de AWS SageMaker Experiments.



Managing ML experiments - AWS Machine Learning in 15.

Accede al vídeo:

<https://www.youtube.com/embed/kK1ohrpJFC0>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

AWS SageMaker Pipelines

Los **flujos de trabajo** de entrenamiento automatizados le ayudan a crear un proceso repetible para organizar pasos de modelo de desarrollo para experimentación rápida y reentrenamiento de modelos. Automatizar todo el flujo de trabajo de creación de modelos, incluidos la preparación de datos, la ingeniería de características, el entrenamiento de modelos, el ajuste de modelos y la validación de modelos con AWS SageMaker Pipelines es un servicio que les permite a los desarrolladores y equipos de ML crear, automatizar y gestionar flujos de trabajo de ML de extremo a extremo de manera sencilla y eficiente en Amazon SageMaker. A continuación, tienes una descripción, las características y las ventajas de AWS SageMaker Pipelines.

Descripción

AWS SageMaker Pipelines es una herramienta que les permite a los equipos de ML definir, orquestar y automatizar flujos de trabajo de ML. Utiliza una interfaz visual basada en el lenguaje de programación Python para definir pasos de flujo de trabajo, gestionar dependencias entre pasos y crear *pipelines* de ML completos que abarcan desde la preparación de datos hasta el despliegue y la monitorización de modelos en producción.

Características de AWS SageMaker Pipelines

- ▶ **Interfaz basada en Python.** SageMaker Pipelines utiliza Python como lenguaje de definición de flujos de trabajo, lo que permite a los desarrolladores definir y personalizar pipelines de manera programática utilizando código Python.
- ▶ **Orquestación automática.** SageMaker Pipelines automatiza la ejecución de flujos de trabajo de ML, gestionando automáticamente la ejecución secuencial de pasos y la gestión de dependencias entre pasos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Gestión de versiones.** SageMaker Pipelines proporciona capacidades integradas para gestionar versiones de flujos de trabajo y artefactos asociados, lo que les permite a los equipos de ML realizar un seguimiento de los cambios en los flujos de trabajo y revertir a versiones anteriores si es necesario.
- ▶ **Integración con SageMaker y AWS.** SageMaker Pipelines está integrado con otros servicios de AWS, como Amazon SageMaker, AWS Step Functions y AWS CodePipeline, lo que permite a los equipos de ML crear flujos de trabajo completos que abarcan desde la preparación de datos hasta la implementación y la monitorización de modelos en producción.
- ▶ **Monitorización y visualización.** SageMaker Pipelines proporciona herramientas para monitorizar y visualizar el progreso de los flujos de trabajo de ML, así como para realizar un seguimiento del rendimiento y la eficiencia de los *pipelines* en tiempo real.

Funcionamiento de AWS SageMaker Pipelines

AWS SageMaker Pipelines proporciona un entorno para crear, automatizar y gestionar flujos de trabajo de ML de extremo a extremo en Amazon SageMaker. El **funcionamiento** de AWS SageMaker Pipelines se describe a continuación:

- ▶ **Definición de *pipelines*.** Los desarrolladores definen los flujos de trabajo de ML utilizando código Python en el SDK de SageMaker, que incluye clases y métodos específicos para crear y configurar *pipelines*. Los *pipelines* se definen como una secuencia de pasos (*steps*) que representan las diferentes etapas del proceso de ML, como la preparación de datos, el entrenamiento de modelos y la implementación en producción.
- ▶ **Configuración de pasos.** Cada paso en el *pipeline* puede ser una tarea independiente, como ejecutar un *script* de Python, entrenar un modelo en SageMaker, ejecutar una transformación de datos, o desplegar un modelo en un *endpoint* de SageMaker. Los desarrolladores configuran los pasos con los

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

parámetros necesarios, como la ubicación de los datos de entrada y salida, los recursos computacionales a utilizar y las dependencias entre pasos.

- ▶ **Orquestación automática.** Una vez definido el *pipeline*, SageMaker Pipelines se encarga de orquestar la ejecución de los pasos de manera automática, gestionando la secuencia de ejecución y la gestión de dependencias entre pasos. Los pasos se ejecutan de manera secuencial, con cada paso esperando a que su predecesor complete su ejecución antes de comenzar.
- ▶ **Monitorización y visualización.** Durante la ejecución del *pipeline*, los desarrolladores pueden monitorizar y visualizar el progreso de los pasos y el estado general del *pipeline* utilizando las herramientas integradas de SageMaker Pipelines. Esto incluye métricas de rendimiento, como el tiempo de ejecución de cada paso y el uso de recursos, así como visualizaciones del flujo de trabajo completo.
- ▶ **Gestión de versiones y reproducibilidad.** SageMaker Pipelines proporciona capacidades integradas para gestionar versiones de flujos de trabajo y artefactos asociados, lo que les permite a los equipos de ML realizar un seguimiento de los cambios en los flujos de trabajo y revertir a versiones anteriores si es necesario. Esto facilita la reproducibilidad de los flujos de trabajo de ML y les permite a los equipos realizar experimentos e iteraciones de manera segura.
- ▶ **Despliegue y monitorización en producción.** Una vez que el *pipeline* se ha completado con éxito, los modelos entrenados pueden ser implementados en producción utilizando los servicios de SageMaker, como SageMaker Endpoints.

SageMaker Pipelines también proporciona herramientas para monitorizar y gestionar modelos en producción, lo que les permite a los equipos realizar un seguimiento del rendimiento y la eficacia de los modelos desplegados.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

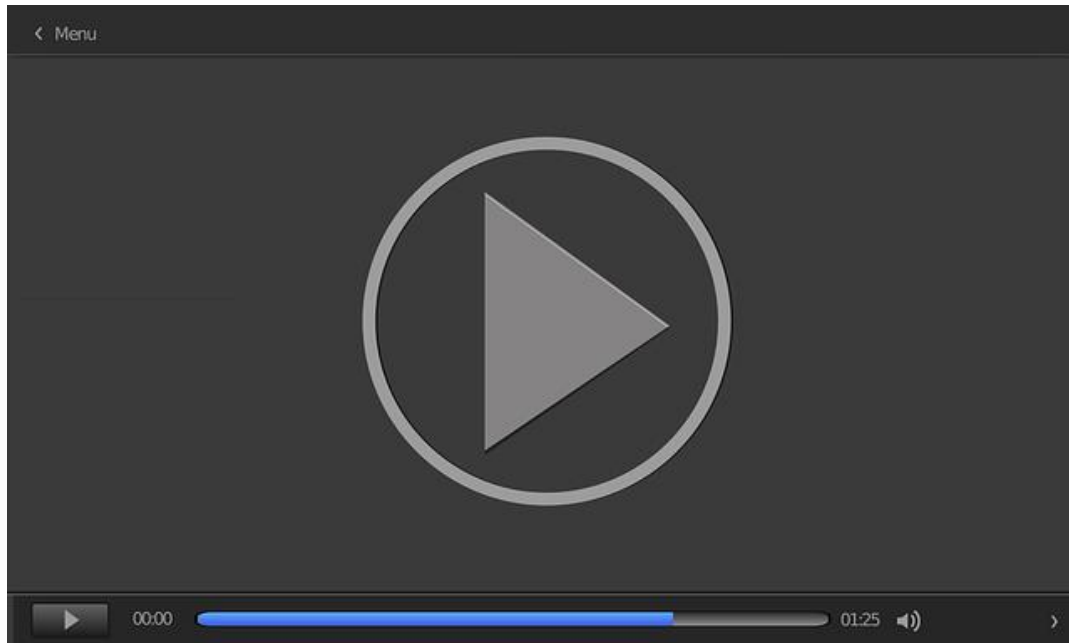
Beneficios de AWS SageMaker Pipelines

Los **beneficios** que proporciona AWS SageMaker son:

- ▶ **Automatización.** SageMaker Pipelines automatiza la ejecución de flujos de trabajo de ML lo que permite a los equipos de ML ahorrar tiempo y esfuerzo al eliminar la necesidad de realizar tareas manuales repetitivas.
- ▶ **Reproducibilidad.** SageMaker Pipelines facilita la reproducibilidad de los flujos de trabajo de ML al permitir a los equipos de ML realizar un seguimiento de los cambios en los flujos de trabajo y revertir a versiones anteriores si es necesario.
- ▶ **Escalabilidad.** SageMaker Pipelines es altamente escalable y puede manejar flujos de trabajo de ML de cualquier tamaño, desde pequeños experimentos hasta flujos de trabajo complejos que abarcan múltiples etapas y recursos computacionales.
- ▶ **Integración con AWS.** SageMaker Pipelines está integrado con otros servicios de AWS, lo que facilita la creación de flujos de trabajo completos de ML que aprovechan la potencia y la escalabilidad de la nube de AWS.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

A continuación, se muestra el vídeo *Introducing Amazon SageMaker Pipelines - AWS re:Invent 2020* (Julien Simon, 2020), que explica el funcionamiento de AWS SageMaker.



Introducing Amazon SageMaker Pipelines - AWS re:Invent 2020.

Accede al vídeo:

<https://www.youtube.com/embed/Hvz2GGU3Z8g>

AWS ML Lineage Tracking

Amazon SageMaker Lineage Tracking es una característica de Amazon SageMaker que permite rastrear y visualizar los datos, los modelos y los recursos utilizados en el proceso de desarrollo de modelos de ML. A continuación, se explica en detalle qué es y cómo funciona.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

¿Qué es SageMaker Lineage Tracking?

SageMaker Lineage Tracking es una función que proporciona **visibilidad** completa sobre el origen y el flujo de los datos y los modelos a lo largo de todo el ciclo de vida de un proyecto de ML. Permite que los desarrolladores y científicos de datos rastreen los datos, los modelos, los artefactos intermedios y los recursos computacionales utilizados en cada paso del proceso de desarrollo de modelos de ML.

Características y funcionamiento

- ▶ **Rastreo automático.** SageMaker Lineage Tracking rastrea automáticamente los recursos utilizados en cada paso del proceso de ML, lo que incluye conjuntos de datos, modelos entrenados, artefactos intermedios y los recursos computacionales utilizados para entrenar y desplegar los modelos.
- ▶ **Metadatos detallados.** Para cada recurso rastreado, SageMaker Lineage Tracking almacena metadatos detallados que incluyen información sobre el origen de los datos, los parámetros de entrenamiento del modelo, las métricas de rendimiento y cualquier otro detalle relevante.
- ▶ **Visualización del flujo de trabajo.** SageMaker Lineage Tracking proporciona herramientas de visualización que les permiten a los usuarios visualizar el flujo de trabajo completo de un proyecto de ML, desde la adquisición y la preparación de datos hasta el entrenamiento y la implementación de modelos en producción. Esto les permite a los equipos de ML comprender fácilmente el proceso de desarrollo de modelos y realizar un seguimiento del impacto de los cambios en los recursos y modelos a lo largo del tiempo.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Gestión de versiones y reproducibilidad.** SageMaker Lineage Tracking proporciona capacidades integradas para gestionar versiones de recursos y modelos, lo que permite a los equipos de ML realizar un seguimiento de los cambios en los recursos y modelos a lo largo del tiempo. Esto facilita la reproducibilidad de los experimentos de ML y les permite a los equipos realizar experimentos e iteraciones de manera segura.
- ▶ **Integración con otros servicios de AWS.** SageMaker Lineage Tracking está integrado con otros servicios de AWS, como Amazon S3, AWS Glue y AWS Step Functions, lo que permite a los equipos de ML rastrear y visualizar los datos y los recursos utilizados en el proceso de desarrollo de modelos, independientemente de dónde se almacenen o procesen los datos.

En resumen, **Amazon SageMaker Lineage Tracking** proporciona una forma fácil y eficiente de rastrear y visualizar los datos, modelos y recursos utilizados en el proceso de desarrollo de modelos de ML. Proporciona visibilidad completa sobre el origen y el flujo de los datos y modelos a lo largo de todo el ciclo de vida de un proyecto de ML, lo que facilita la comprensión del proceso de desarrollo de modelos y la gestión de versiones y reproducibilidad de los experimentos de ML.

La Figura 9 muestra un gráfico de linaje de ejemplo que Amazon SageMaker crea automáticamente en un flujo de trabajo de aprendizaje automático de implementación y entrenamiento de modelos de un extremo a otro.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

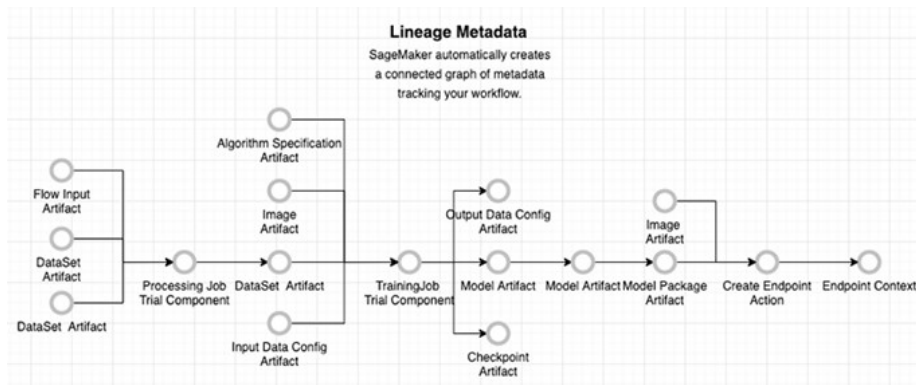


Figura 9. Gráfico de linaje de ejemplo. Fuente: Christensen et al., 2021.

AWS SageMaker Model Registry

Amazon SageMaker Model Registry es un servicio que les permite a los equipos de ML organizar, rastrear, gestionar y compartir modelos de ML en Amazon SageMaker. A continuación, tienes una descripción detallada, las características, el funcionamiento, las ventajas y los casos de uso del AWS SageMaker Model Registry.

Descripción

Amazon SageMaker Model Registry es un servicio de AWS que facilita la gestión de modelos de ML en SageMaker. Proporciona un repositorio centralizado para almacenar y organizar modelos, así como herramientas para rastrear versiones, gestionar permisos de acceso y compartir modelos con otros usuarios.

Características de SageMaker Model Registry

- **Repositorio centralizado.** SageMaker Model Registry proporciona un repositorio centralizado para almacenar y organizar modelos de ML en SageMaker.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Rastreo de versiones.** Permite rastrear diferentes versiones de un modelo a lo largo del tiempo, lo que facilita la gestión de versiones y la reproducción de experimentos.
- ▶ **Gestión de permisos.** Permite gestionar permisos de acceso a los modelos, lo que permite controlar quién tiene acceso a qué modelos y qué acciones pueden realizar.
- ▶ **Integración con SageMaker.** Está integrado con otros servicios de SageMaker, lo que facilita la gestión de modelos dentro del ecosistema de SageMaker.
- ▶ **Historial de versiones.** Proporciona un historial completo de versiones de modelos, incluyendo metadatos como quién creó la versión, cuándo se creó y qué cambios se realizaron.

Funcionamiento de SageMaker Model Registry

Con el Model Registry de SageMaker, se puede hacer lo siguiente:

- ▶ Modelos de catálogo para producción.
- ▶ Gestionar versiones de modelos.
- ▶ Asociar metadatos, como métricas de entrenamiento, con un modelo.
- ▶ Gestionar el estado de aprobación de un modelo.
- ▶ Implementar modelos en producción.
- ▶ Automatizar la implementación de modelos con CI/CD.

Se puede catalogar los modelos creando **grupos de modelos** (paquetes) de registro de modelos de SageMaker que contienen diferentes versiones de un modelo. Puede crear un grupo de modelos que realice un seguimiento de todos los modelos que entrena para resolver un problema en particular. Luego, puede registrar cada modelo que entrene y el «Registro de modelos» lo agrega al «Grupo de modelos» como una nueva versión del modelo. Por último, puede crear categorías de grupos de modelos

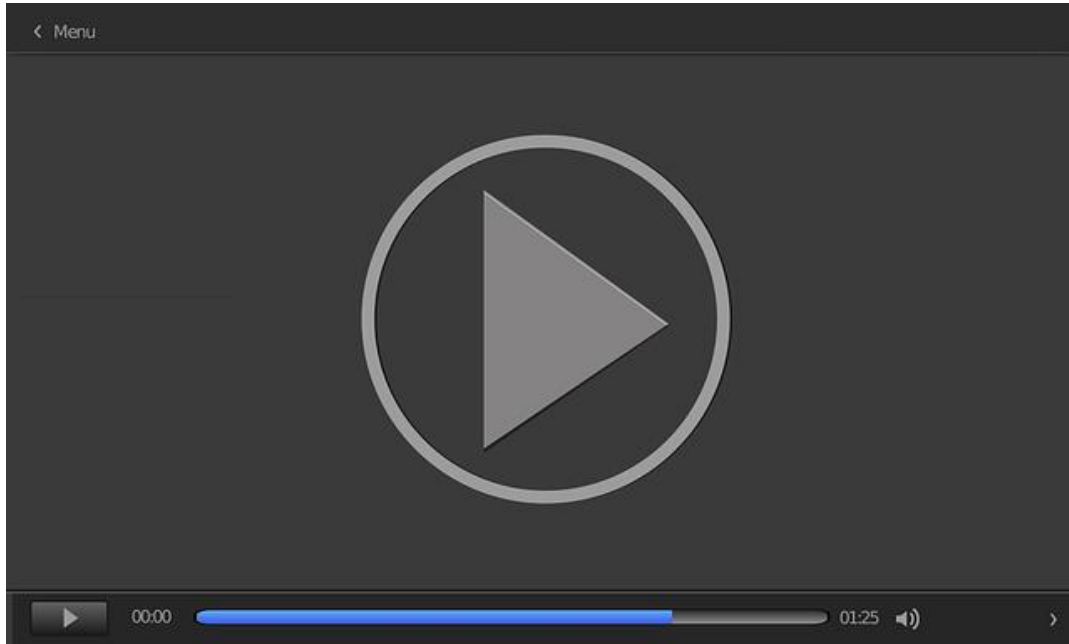
Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

organizándolos aún más en colecciones de registro de modelos de SageMaker. Un **flujo de trabajo** típico podría verse como el siguiente:

- ▶ Cree un **grupo de modelos**.
- ▶ Cree una **canalización de ML que entrene un modelo**. Para obtener información sobre las canalizaciones de SageMaker, consulte «Crear y administrar canalizaciones de SageMaker».
- ▶ Para cada ejecución de la canalización de ML, cree una **versión de modelo** que registre en el grupo de modelos que creó en el primer paso.
- ▶ Agregue su grupo de modelos a una o más colecciones de **registros de modelos**.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Para obtener detalles sobre cómo crear y trabajar con modelos, versiones de modelos y grupos de modelos, se puede ver el vídeo *Simplifying MLOps with Model Registry* (Verta AI, 2021).



Simplifying MLOps with Model Registry.

Accede al vídeo:

<https://www.youtube.com/embed/WrieKPgXZyo>

Ventajas de utilizar Sagemaker Model Registry

Entre sus ventajas, cabe destacar:

- **Gestión centralizada.** Proporciona un repositorio centralizado para almacenar y organizar modelos de ML en SageMaker, lo que simplifica la gestión y la organización de modelos.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- ▶ **Gestión de versiones.** Permite gestionar fácilmente diferentes versiones de un modelo a lo largo del tiempo, lo que facilita la gestión de versiones y la reproducción de experimentos.
- ▶ **Control de acceso.** Permite controlar quién tiene acceso a qué modelos y qué acciones pueden realizar, lo que garantiza la seguridad y la privacidad de los modelos almacenados en el Model Registry.
- ▶ **Facilita la colaboración.** Facilita la colaboración entre equipos de ML, lo que permite compartir modelos entre equipos y colaborar en proyectos de ML.

Casos de uso de aplicabilidad de Model Registry

- ▶ **Desarrollo de modelos en equipo.** Permite a equipos de ML colaborar de manera efectiva en el desarrollo y gestión de modelos de ML.
- ▶ **Reproducción de Experiments.** Facilita la reproducción de experimentos al proporcionar un historial completo de versiones de modelos, lo que permite realizar un seguimiento de los cambios y mejoras realizados en cada versión.
- ▶ **Implementación en producción.** Simplifica la implementación de modelos en producción al proporcionar un repositorio centralizado para almacenar y organizar modelos entrenados.

En resumen, **Amazon SageMaker Model Registry** es un servicio que facilita la gestión de modelos de ML en Amazon SageMaker, que proporciona un repositorio centralizado para almacenar, organizar, rastrear y compartir modelos. Permite gestionar versiones de modelos, controlar el acceso a los modelos y facilita la colaboración entre equipos de ML.

Definir infraestructuras de ML mediante código

En Amazon SageMaker, puedes definir **infraestructuras de ML** mediante código utilizando el SDK de Python de SageMaker. Esto te permite crear y configurar

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

recursos de manera programática, lo que facilita la automatización y la reproducibilidad de tus experimentos de ML. Aquí te explico cómo se hace:

- **Importa las bibliotecas necesarias.** En primer lugar, necesitas importar las bibliotecas necesarias del SDK de SageMaker en tu script de Python. Esto incluye el módulo `boto3` para interactuar con los servicios de AWS y el módulo `sagemaker` para trabajar con SageMaker.

```
import boto3
import sagemaker
```

- **Configura la sesión de SageMaker.** A continuación, configura la sesión de SageMaker utilizando tus credenciales de AWS y la región en la que deseas trabajar.

```
session = sagemaker.Session()
```

- **Define y configura los recursos de ML.** Ahora puedes definir y configurar los recursos de ML que necesitas, como instancias de entrenamiento, instancias de inferencia y *endpoints* de SageMaker.

```
# Define las instancias de entrenamiento
train_instance_type = 'ml.m5.large'
train_volume_size = 30
# Define las instancias de inferencia
deploy_instance_type = 'ml.m5.large'
# Configura el estimador de SageMaker
estimator = sagemaker.estimator.Estimator(image_uri='your_image_uri',
                                           role='your_role',
                                           instance_count=1,
                                           instance_type=train_instance_type,
                                           volume_size=train_volume_size,
                                           output_path='s3://your_output_path',
                                           sagemaker_session=session)
```

- **Entrena y despliega el modelo.** Después de configurar el estimador de SageMaker, puedes usarlo para entrenar un modelo en tus datos de entrenamiento y luego desplegar el modelo entrenado en un *endpoint* de SageMaker para inferencia en tiempo real.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

```
# Entrena el modelo
estimator.fit('s3://your_training_data')
# Despliega el modelo en un endpoint de SageMaker
predictor = estimator.deploy(initial_instance_count=1,
instance_type=deploy_instance_type)
```

- **Utiliza el modelo desplegado.** Una vez que el modelo está desplegado en un *endpoint* de SageMaker, puedes utilizarlo para realizar predicciones en nuevos datos de entrada.

```
# Realiza predicciones en nuevos datos
result = predictor.predict(new_data)
```

Este es un **ejemplo** básico de cómo se pueden definir infraestructuras de ML mediante código en SageMaker utilizando el SDK de Python. Puedes personalizar y ampliar este flujo de trabajo según tus necesidades específicas, como la configuración de hiperparámetros, la evaluación de modelos y la implementación de prácticas de MLOps.

Automatizar flujos de trabajo (CI/CD) de integración y despliegue con SageMaker Projects

Los flujos de trabajo de desarrollo de ML deben integrarse con los flujos de trabajo de integración y despliegue para proporcionar rápidamente nuevos modelos para las aplicaciones de producción. **Amazon SageMaker Projects** lleva las prácticas de CI/CD al ML, como por ejemplo el mantenimiento de la paridad entre los entornos de desarrollo y producción, el control de versiones y orígenes, las pruebas A/B y la automatización integral. Por consiguiente, un modelo pasa a producción en cuanto se aprueba, lo que aumenta la agilidad.

Para automatizar flujos de trabajo (CI/CD) de integración y despliegue con Amazon SageMaker Projects, puedes seguir una serie de **pasos** que te permitirán configurar un entorno de desarrollo eficiente y automatizar la construcción, entrenamiento y despliegue de modelos de ML. Seguidamente se muestra una guía paso a paso.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Pasos para automatizar los flujos de trabajo (CI/CD) con SageMaker Projects

► Configurar el entorno de desarrollo:

- Crea un nuevo proyecto de SageMaker o selecciona un proyecto existente.
- Define el entorno de desarrollo del proyecto, que puede incluir instancias de SageMaker Studio o SageMaker Notebooks.

► Configurar el repositorio de código fuente:

- Configura un repositorio de código fuente utilizando un servicio de control de versiones compatible, como AWS CodeCommit, GitHub o GitLab.
- Conecta el repositorio de código fuente al proyecto de SageMaker.

► Definir la estructura del proyecto:

- Organiza el código fuente y los recursos del proyecto en una estructura de directorios coherente.
- Define archivos de configuración, como `buildspec.yml` para la construcción y `template.yml` para la definición de recursos de SageMaker.

► Configurar los *pipelines* de CI/CD:

- Define los *pipelines* de CI/CD utilizando AWS CodePipeline y AWS CodeBuild.
- Crea un *pipeline* de CI que se active automáticamente cuando se realicen cambios en el repositorio de código fuente.
- Define etapas de construcción, entrenamiento y despliegue en el *pipeline* de CD.

► Automatizar la construcción y el entrenamiento de modelos:

- Define *scripts* de construcción y entrenamiento utilizando el SDK de SageMaker Python o SageMaker Python SDK.

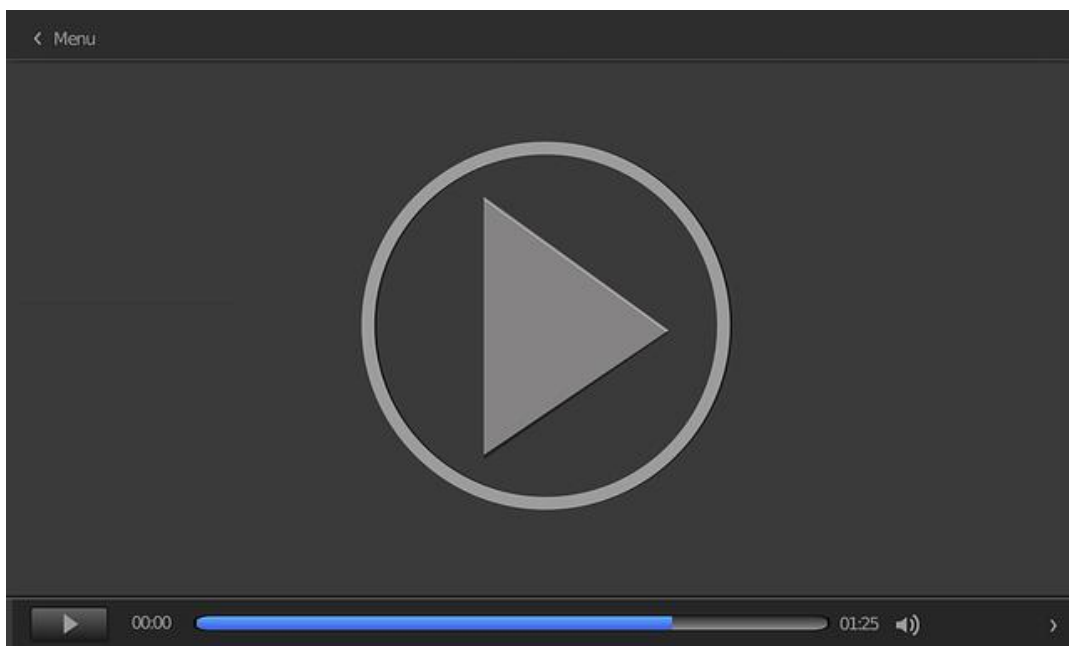
Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

- Automatiza la ejecución de estos *scripts* en la etapa correspondiente del *pipeline* de CI/CD.
- Utiliza funciones de SageMaker Projects para gestionar recursos de SageMaker, como instancias de entrenamiento y *endpoints* de inferencia.
- ▶ **Implementar SageMaker Model Monitor:**
 - Configura SageMaker Model Monitor para monitorear continuamente el rendimiento del modelo en producción y detectar desviaciones en la distribución de datos de entrada o salida.
- ▶ **Configurar alarmas y notificaciones:**
 - Configura alarmas en CloudWatch para alertar sobre problemas de rendimiento o disponibilidad en los *endpoints* de SageMaker.
 - Configura notificaciones por correo electrónico o mensajes de texto para informar a los equipos sobre eventos críticos en el flujo de trabajo de CI/CD.
- ▶ **Gestionar permisos y accesos:**
 - Asegúrate de que los roles de IAM utilizados en tu *pipeline* de CI/CD tengan los permisos adecuados para acceder a los recursos de AWS necesarios, como S3 y CloudWatch.
- ▶ **Monitorear y mejorar el flujo de trabajo:**
 - Utiliza métricas y registros de CloudWatch para monitorear el rendimiento y la eficiencia de tu *pipeline* de CI/CD.
 - Realiza ajustes y mejoras en el flujo de trabajo según sea necesario para optimizar el rendimiento y la eficiencia.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Al seguir estos pasos, podrás establecer un flujo de trabajo de CI/CD eficiente y escalable utilizando SageMaker Projects, lo que te permitirá desarrollar, entrenar y desplegar modelos de ML de manera rápida y confiable en la nube de AWS.

En el vídeo *AWS re:Invent 2018: CI/CD for Your Machine Learning Pipeline with Amazon SageMaker (DVC303)* (Amazon Web Services, 2018), puedes ver con profundidad como implementar flujos de CI/CD en SageMaker



AWS re:Invent 2018: CI/CD for Your Machine Learning Pipeline with Amazon SageMaker (DVC303).

Accede al vídeo:

<https://www.youtube.com/embed/6EYEoAqihPg>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

5.4. Referencias bibliográficas

Ali Arsanjani. (2021, febrero 23). *Architect and build the full machine learning lifecycle with AWS: An end-to-end Amazon SageMaker demo*. Amazon Web Services.

<https://aws.amazon.com/es/blogs/machine-learning/architect-and-build-the-full-machine-learning-lifecycle-with-amazon-sagemaker/>

Amazon Web Services. (2018, noviembre 5). *AWS re:Invent 2018: CI/CD for Your Machine Learning Pipeline with Amazon SageMaker (DVC303)* [Vídeo]. YouTube.

<https://youtu.be/6EYEoAqihPg>

Amazon Web Services. (2020, noviembre 5). *Debugging a Customer Churn Model Using SageMaker Debugger* [Vídeo]. YouTube. <https://youtu.be/8b5-lyRaFgA>

Amazon Web Services. (2021a, febrero 19). *Introduction to Amazon SageMaker* [Vídeo]. YouTube. https://youtu.be/Qv_Tr_BCFCQ

Amazon Web Services. (2021b, junio 15). *Amazon SageMaker Data Wrangler Deep Dive Demo* [Vídeo]. YouTube. <https://youtu.be/b5EIDVPOmO4>

Amazon Web Services. (2021c, noviembre 18). *Automate MLOps with SageMaker Projects | Amazon Web Services* [Vídeo]. YouTube. https://youtu.be/3_cHnk9VSfQ

Amazon Web Services. (2022, agosto 24). *Amazon SageMaker MLOps | Amazon Web Services* [Vídeo]. YouTube. https://youtu.be/FSOBmgT_9aQ

Amazon Web Services. (s. f.-a). *¿Qué son las MLOps?* <https://aws.amazon.com/es/what-is/mlops/>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Amazon Web Services. (s. f.-b.). *Amazon SageMaker para MLOps.*

<https://aws.amazon.com/es/sagemaker/mlops/?sagemaker-data-wrangler-whats-new.sort-by=item.additionalFields.postDateTime&sagemaker-data-wrangler-whats-new.sort-order=desc>

Amazon Web Services. (s. f.-c.). *¿Qué es Amazon SageMaker?*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/whatis.html

Amazon Web Services. (s. f.-d.). *SageMaker Piloto automático.*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/autopilot-automate-model-development.html

Amazon Web Services. (s. f.-e.). *Entrene, implemente y evalúe modelos previamente entrenados con SageMaker JumpStart.*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/studio-jumpstart.html

Amazon Web Services. (s. f.-f.). *Prepare datos de aprendizaje automático con Amazon SageMaker Data Wrangler.*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/data-wrangler.html

Amazon Web Services. (s. f.-g.). *Usa SageMaker Clarify para explicar y detectar sesgos.*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/clarify-configure-processing-jobs.html

Amazon Web Services. (s. f.-h.). *Canalizaciones.*

https://docs.aws.amazon.com/es_es/sagemaker/latest/dg/pipelines.html

AWS Developers. (2022a, noviembre 3). *Overview of Amazon SageMaker Autopilot*

[Vídeo]. YouTube. <https://youtu.be/9pwYIb1Kit0>

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

AWS Developers. (2022b, septiembre 27). *Easily get started with machine learning using Amazon SageMaker JumpStart - AWS Virtual Workshop* [Vídeo]. YouTube.

<https://youtu.be/1-AOLoOiuG4>

AWS Developers. (2023, mayo 19). *Managing ML experiments - AWS Machine Learning in 15* [Vídeo]. YouTube. <https://youtu.be/kK1ohrpJFC0>

AWS Events. (2021, junio 4). *AWS AMER Summit 2020 | Introducción a Amazon SageMaker Studio, un IDE completo para ML* [Vídeo]. YouTube.

<https://youtu.be/Yzxls2SB0as>

AWS Events. (2023, diciembre 1). *AWS re:Invent 2023 - Accelerate foundation model evaluation with Amazon SageMaker Clarify (AIM367)* [Vídeo]. YouTube.

<https://youtu.be/9X2oDkOBYyA>

Christensen, H., Benson, D., Thadaka, K., Trikande, S. y Krishnan, V. (2021, septiembre 21). *Model and data lineage in machine learning experimentation*.

Amazon Web Services. <https://aws.amazon.com/es/blogs/machine-learning/model-and-data-lineage-in-machine-learning-experimentation/>

Hai Tran. (2022, agosto 31). *AWS SageMaker Processing API* [Vídeo]. YouTube.

<https://youtu.be/VZs5yaFDO4E>

Jorge Luis. (2023, julio 19). *Machine Learning Operations*. Medium.

<https://jorgedataengineer.medium.com/machine-learning-operations-99a2bb292b35>

Julien Simon. (2020, diciembre 2). [Vídeo]. YouTube. <https://youtu.be/Hvz2GGU3Z8g>

Khan, M. A, Saqib, S., Alyas, T., Rehman, A., Saeed, Y., Zeb, A., Zareei, M. y Mohamed, E. (2020). Effective Demand Forecasting Model Using Business Intelligence Empowered With Machine Learning. *IEEE Access*, 8(1). 116013-116023. 10.1109/ACCESS.2020.3003790.

Tema 5. Desarrollo, despliegue y monitorización de IA en cloud

Lat, J. A (2021). *Machine Learning with Amazon SageMaker Cookbook: 80 proven recipes for data scientists and developers to perform machine learning experiments and deployments*. Packt Publishing.

ML4Devs. (2022, septiembre 8). MLOps: Machine Learning Life Cycle. <https://www.ml4devs.com/articles/mlops-machine-learning-life-cycle/>

Sakarya, C. (s. f.). *What is DevOps? DEVOPS CONCEPTS*. DataCamp. <https://projector-video-pdf-converter.datacamp.com/29774/chapter1.pdf>

Simon, J. (2021). *Learn Amazon SageMaker: A guide to building, training, and deploying machine learning models for developers and data scientists*. Packt Publishing.

Tomasz Dudek. (2021, agosto 25). *Amazon SageMaker Pipelines — implement MLOps on AWS. A practical introduction to Amazon SageMaker Pipelines, AWS Step Functions and AWS CodePipeline*. Chaos Gear. <https://www.chaosgears.com/blog/amazon-sagemaker-pipelines>

Verta AI. (2021, septiembre 15). *Simplifying MLOps with Model Registry* [Vídeo]. YouTube. <https://youtu.be/WrieKPgXZyo>